Introduzione a Unreal Engine

Lorenzo Vergata

July 11, 2026

Contents

1 Introduzione al Game Development e a Unreal Engine **23**

1.1 Presentazioni . . . 23
1.1.1 Il docente . . . 23
1.1.2 Gli studenti . . . 23
1.2 Che cos'è un game engine . . . 23
1.2.1 Esempi di engine . . . 23
1.3 Fondamenti di rendering 3D . . . 23
1.3.1 Poligoni e vertici . . . 23
1.3.2 La pipeline di rendering . . . 24
1.3.3 Illuminazione e materiali . . . 24
1.4 Rendering in tempo reale e frame rate . . . 24
1.5 Unreal Engine: panoramica . . . 24
1.5.1 Epic Games e installazione . . . 24
1.5.2 Perché Unreal è così diffuso . . . 24
1.5.3 Esempi celebri . . . 24
1.5.4 Oltre i videogiochi: cinema e virtual production . . . 24
1.6 Risorse utili . . . 25
1.7 Game Loop e differenza tra videogioco e cinema . . . 25
1.8 La produzione di un videogioco . . . 25
1.8.1 Pre-produzione . . . 25
1.8.2 Tipologie di produzioni . . . 25
1.8.3 Outsourcing . . . 25
1.9 I ruoli nel team di sviluppo . . . 25
1.9.1 Game Designer . . . 26
1.9.2 Visual Art . . . 26
1.9.3 Animazione . . . 26
1.9.4 Programmatori . . . 26
1.9.5 Altri ruoli e strumenti . . . 26
1.10 Blueprint: la programmazione visuale in Unreal . . . 27
1.11 Attività in classe: gli elementi di un gioco . . . 27
1.11.1 Teoria del gioco e Homo Ludens . . . 27
1.11.2 Motivazioni del giocatore e psicologia . . . 27
1.11.3 Esempio pratico: l'Escape Room . . . 27
1.11.4 Applicazione a un videogioco: Expedition 33 . . . 28
1.11.5 Il tempo come meccanica . . . 28
1.11.6 Bilanciamento e ruolo del QA . . . 28
1.12 Tecniche artistiche e motori grafici . . . 28
1.13 Aspetti di mercato: il tempismo di uscita . . . 29
1.14 Installazione di Unreal Engine 5.5 . . . 29
1.15 Documentazione e supporto . . . 29
1.15.1 Requisiti di sistema e supporto Discord . . . 29
1.16 Il Fab e gli asset . . . 29

1.16.1 Pack gratuiti mensili . . . 29
1.16.2 MetaHuman . . . 29
1.16.3 Materiale didattico del corso . . . 29
1.16.4 Buone pratiche nell'uso degli asset . . . 30
1.17 Teoria: esecuzione nel editor e delta time . . . 30
1.18 Primo avvio e creazione di un progetto . . . 30
1.18.1 Template di progetto . . . 30
1.18.2 Creazione del progetto . . . 30
1.18.3 Compilazione degli shader . . . 31
1.19 Requisiti hardware e prestazioni . . . 31
1.20 L'interfaccia a finestre . . . 31
1.20.1 Il Content Browser e l'organizzazione delle cartelle . . . 31
1.21 Inserimento e manipolazione degli oggetti . . . 31
1.21.1 Place Actors e primi oggetti . . . 31
1.21.2 Trasformazioni: locazione, rotazione, scala . . . 32
1.21.3 Sistema di coordinate: locale e globale . . . 32
1.21.4 Duplicazione e raggruppamento . . . 33
1.21.5 Snapping . . . 33
1.21.6 Scorciatoie utili per la viewport . . . 33
1.22 Static mesh (cenni) . . . 33
1.23 Navigazione nella viewport: regolazione della velocità . . . 33
1.24 Outliner: organizzazione avanzata . . . 34
1.25 Esercitazione pratica: costruzione di una scena . . . 34
1.25.1 Suggerimenti per l'esecuzione . . . 34
1.26 Consigli pratici e risoluzione di problemi comuni . . . 36
1.26.1 Allineamento preciso degli oggetti . . . 36
1.26.2 Selezione, copia e duplicazione di più oggetti . . . 36
1.26.3 Salvataggio e gestione dei file . . . 36
1.27 Comunicazioni finali e prossime lezioni . . . 36

2 Blueprint e Interazione: Actor, Variabili e Input 37
2.1 Introduzione agli Actor . . . 37
2.2 Componenti di un Actor . . . 37
2.3 Blueprint e Classi . . . 37
2.3.1 Concetto di Classe . . . 39
2.4 Tipi di Blueprint . . . 39
2.4.1 Actor . . . 39
2.4.2 Pawn . . . 39
2.4.3 Character . . . 39
2.5 Editor delle Blueprint . . . 39
2.5.1 Viewport . . . 39
2.5.2 Event Graph . . . 39
2.6 Programmazione a Eventi . . . 39
2.6.1 Event Begin Play . . . 39
2.6.2 Print String . . . 40
2.7 Variabili . . . 40
2.7.1 Creare una variabile . . . 40
2.7.2 Get e Set . . . 40
2.8 Collegamenti e Tipi di Pin . . . 40
2.9 Nodi di Esecuzione e Nodi di Calcolo . . . 41
2.10 Operatori Aritmetici . . . 41
2.11 Creare e Modificare Variabili . . . 41
2.12 Proprietà delle Variabili: Pubbliche e Private . . . 42
2.13 Tipi di Variabili . . . 42

2.14 Utilizzo dei Booleani . . . 43
2.14.1 Get e Set di un Bool . . . 43
2.15 Struttura Condizionale: Branch . . . 44
2.16 Operatori Logici: AND, OR, NOT . . . 44
2.17 Esercizio: Combinare Booleani con Branch e Operatori . . . 44
2.18 Esercitazione: Condizioni con Branch e Operatori Booleani . . . 45
2.19 Level Blueprint e Eventi di Input . . . 45
2.19.1 Level Blueprint . . . 45
2.19.2 Evento di pressione tasto . . . 46
2.20 Referenza a un Actor nel Level Blueprint . . . 46
2.21 Rotazione Incrementale di un Actor . . . 46
2.21.1 Preparazione dell'oggetto: Mobility . . . 47
2.21.2 Metodo con GetActorRotation e SetActorRotation . . . 47
2.21.3 Debug con Print String . . . 47
2.22 Troubleshooting: La piattaforma ruota una volta sola . . . 47
2.23 Rotazione nel Mondo con `AddActorWorldRotation` . . . 49
2.23.1 Movimento Avanti/Indietro (W/S) . . . 49
2.23.2 Movimento Destra/Sinistra (A/D) . . . 49
2.24 Fisica: `Simulate Physics` e Opzione `Teleport` . . . 49
2.24.1 Attivare la fisica su un oggetto . . . 49
2.24.2 Interazione piattaforma-oggetto fisico . . . 49
2.25 Troubleshooting: Piattaforma non si muove . . . 51
2.26 Event Tick e Delta Seconds . . . 51
2.26.1 Event Tick . . . 51
2.26.2 Delta Seconds . . . 51
2.27 Input del Mouse per la Rotazione . . . 51
2.28 Clamp dei Valori (Introduzione) . . . 52
2.29 Clamp dei Valori . . . 52
2.30 Rotazione Guidata dal Mouse con Clamp . . . 52
2.30.1 Ottenere i valori del mouse . . . 52
2.30.2 Somma alla rotazione corrente e clamp . . . 52
2.31 Costruzione della Vaschetta (Attach To) . . . 53
2.32 Problemi Comuni e Soluzioni . . . 53
2.33 Compiti e Prossima Lezione . . . 54

3 Introduzione alla Grafica in Unreal Engine 5: Texture, Materiali e Importazione di Mesh 3D 55
3.1 Introduzione alla lezione: il ruolo del grafico in Unreal Engine . . . 55
3.2 Materiale didattico e progetto di riferimento . . . 55
3.3 Le texture: fondamenti teorici . . . 55
3.3.1 Definizione e ruolo . . . 55
3.3.2 Il processo di UV Mapping . . . 55
3.3.3 Tipologie di mappe . . . 56
3.3.4 La logica dei valori e il multi-canale . . . 56
3.4 I materiali in Unreal Engine . . . 56
3.4.1 Dalla texture al materiale . . . 56
3.4.2 L'editor dei materiali . . . 57
3.4.3 Impostazioni del materiale . . . 57
3.5 Material Domain . . . 57
3.6 Blend Mode . . . 57
3.6.1 Effetti della Blend Mode sui pin del nodo risultante . . . 57
3.7 Shading Model . . . 57
3.8 Editor dei materiali in Unreal Engine . . . 58
3.8.1 Creazione di un nuovo materiale . . . 58

3.8.2 Impostazioni nel pannello dei dettagli . . . 58
3.8.3 Material Expression Nodes . . . 58
3.8.4 Parametri . . . 58
3.8.5 Shortcut utili nell'editor dei materiali . . . 58
3.8.6 Utilizzo dei canali di una texture . . . 59
3.8.7 Esempio pratico: Emissive Color e moltiplicazione . . . 59
3.9 Input chiave per la superficie: Metallic, Specular, Roughness . . . 59
3.9.1 Esempio: creazione di uno specchio . . . 59
3.10 Collegare una texture al nodo Texture Sample . . . 59
3.11 Nodo Texture Coordinate (UV) . . . 60
3.11.1 Esempi di tiling . . . 60
3.12 Applicare un materiale a un oggetto nella scena . . . 60
3.13 Material Instance vs Master Material . . . 60
3.13.1 Master Material . . . 60
3.13.2 Material Instance . . . 61
3.13.3 Convertire nodi in parametri . . . 61
3.13.4 Esempio pratico: tiling variabile per istanza . . . 61
3.14 Material Functions . . . 61
3.15 Nodi utili aggiuntivi: Lerp e Panner . . . 61
3.15.1 Linear Interpolation (Lerp) . . . 61
3.15.2 Panner (Movimento texture) . . . 62
3.16 Shortcut riepilogativi . . . 62
3.17 Utilizzo pratico di Lerp e Panner . . . 62
3.17.1 Lerp con texture . . . 62
3.17.2 Panner (movimento texture) . . . 62
3.17.3 Relazione tra Master Material e Material Instance . . . 62
3.18 Lavoro da remoto e versionamento (Git) . . . 64
3.19 Importazione mesh: finestra Import Settings . . . 64
3.19.1 Esempio: importazione della torretta . . . 64
3.20 Mesh: vertici, spigoli, facce . . . 64
3.20.1 Visualizzazione wireframe . . . 64
3.20.2 Accorgimenti per l'importazione . . . 65
3.21 Risoluzione dei problemi di pivot e allineamento . . . 65
3.21.1 Metodo: re-import con trasformazioni . . . 65
3.21.2 Parametri specifici per la torretta . . . 65
3.22 Posizionamento in scena dei pezzi . . . 65
3.23 Reimport di mesh per modificare pivot e trasformazioni . . . 65
3.23.1 Procedura di reimport con dialogo . . . 66
3.23.2 Alternativa con Modeling Mode (Unreal Engine 5) . . . 66
3.24 Organizzazione delle cartelle nel Content Browser . . . 66
3.25 Ispezionare una mesh importata . . . 66
3.25.1 Material Slots . . . 67
3.25.2 Modalità di visualizzazione . . . 67
3.25.3 Visualizzazione vertici e collisioni . . . 67
3.25.4 Collisioni semplici vs complesse . . . 67
3.26 Grouping e Merging di Actor . . . 67
3.26.1 Grouping (CTRL+G) . . . 67
3.26.2 Merge Actors . . . 68
3.27 Creazione materiali da texture importate . . . 68
3.27.1 Passi . . . 68
3.27.2 Preview del materiale su mesh specifica . . . 68
3.27.3 Assegnare i materiali agli slot della mesh . . . 68
3.28 Modifica estetica tramite materiali (esempio con Lerp) . . . 69
3.28.1 Esempio: miscelare colore base con un secondo colore . . . 69

3.29 Consigli pratici . . . 70
3.30 Merge di actor e gestione degli slot materiali . . . 70
3.31 Comportamento dei materiali con trasformazioni degli oggetti . . . 70
3.31.1 Problema di clipping tra superfici coplanari . . . 70
3.32 Materiali World Aligned (allineati al mondo) . . . 70
3.32.1 Nodi World Aligned . . . 70
3.32.2 Comportamento . . . 71
3.32.3 Confronto: UV-based vs World Aligned . . . 71
3.33 Risoluzione di problemi: linee nere tra texture . . . 71
3.34 Esercizio proposto: creare un materiale Lava . . . 71
3.34.1 Caratteristiche richieste . . . 71
3.34.2 Suggerimenti pratici . . . 72
3.34.3 Nodi utili non ancora visti . . . 72
3.35 Consigli finali per il lavoro autonomo . . . 72

4 Introduzione a Unreal Engine 5: Vettori, Rotatori e Comportamenti Interattivi con Blueprint 75
4.1 Risorse per materiali: Quixel Megascans . . . 75
4.1.1 Accesso e utilizzo . . . 75
4.1.2 Creazione di un materiale lava . . . 75
4.2 Trucco: Continuous Collision Detection (CCD) . . . 75
4.3 Progetto odierno: Mulino a vento e torretta . . . 76
4.3.1 Materiale didattico . . . 76
4.3.2 Obiettivi . . . 76
4.4 Elementi di matematica 3D: vettori e rotazioni . . . 76
4.4.1 Definizione di vettore . . . 76
4.4.2 Operazioni principali . . . 76
4.4.3 Vettori normalizzati . . . 76
4.4.4 Rotatori e angoli di Eulero . . . 77
4.4.5 Cenni ai Quaternioni . . . 77
4.5 Creazione di un oggetto rotante: il mulino . . . 77
4.5.1 Struttura del mulino . . . 77
4.5.2 Blueprint del mulino . . . 77
4.5.3 Componente Rotating Movement . . . 77
4.5.4 Dimostrazione rapida . . . 78
4.6 Esercitazione rapida: rotazione di cubi . . . 78
4.7 Costruzione del mulino a vento . . . 78
4.7.1 Blueprint e componenti . . . 78
4.7.2 Limitazione del componente Rotating Movement . . . 78
4.7.3 Rotazione di una pala con Event Tick e Delta Seconds . . . 78
4.7.4 Gerarchia: la seconda pala . . . 80
4.7.5 Nota: risoluzione problemi viewport . . . 81
4.8 Esercitazione: attaccare un cubetto alla pala del mulino . . . 81
4.9 Metodo alternativo per la rotazione: Find Look at Rotation e interpolazione . . . 81
4.9.1 Find Look at Rotation . . . 81
4.9.2 Interpolazione con RInterp . . . 82
4.9.3 Gestione degli assi e offset . . . 82
4.9.4 Limitazione dell'angolo con Clamp . . . 82
4.10 Pausa . . . 82
4.11 Torretta: creazione del Blueprint e composizione . . . 82
4.11.1 Obiettivo . . . 82
4.11.2 Creazione del Blueprint . . . 84
4.11.3 Inserimento delle mesh tramite Browse to Asset . . . 84
4.11.4 Gerarchia consigliata . . . 84

4.11.5 Risoluzione problemi nell'aggiunta di componenti . . . 84
4.12 Torretta: sfera di attivazione e identificazione del giocatore . . . 84
4.12.1 Configurazione del Blueprint della torretta . . . 84
4.12.2 Aggiunta della Sphere Collision . . . 85
4.12.3 Visualizzazione della sfera in gioco . . . 85
4.12.4 Evento OnBeginOverlap . . . 85
4.12.5 Identificare il Player tramite Tag . . . 86
4.12.6 Controllo di validità con `Is Valid` . . . 86
4.12.7 Salvare il riferimento al Player . . . 87
4.12.8 Custom Event e Timer per il tracciamento . . . 87
4.12.9 Continuazione del progetto . . . 87
4.13 Rotazione della torretta verso il giocatore . . . 89
4.13.1 Logica generale . . . 89
4.13.2 Controllo del riferimento al Player . . . 89
4.13.3 Calcolo della direzione: `Find Look at Rotation` . . . 89
4.13.4 Interpolazione della rotazione . . . 89
4.13.5 Applicazione della rotazione al Pivot . . . 89
4.13.6 Gerarchia corretta dei componenti . . . 90
4.13.7 Allineamento delle mesh: offset di rotazione . . . 90
4.13.8 Debug e problemi comuni . . . 90
4.13.9 Test finale . . . 92

5 Sistemi di Torretta in Unreal Engine 5: Spawning, Trace e Collisioni 93
5.1 Problemi di salvataggio e materiali . . . 93
5.1.1 Segnalazione di un errore . . . 93
5.1.2 Buone pratiche di salvataggio . . . 93
5.1.3 Materiali neri . . . 93
5.2 Spawning: teoria e definizione . . . 93
5.2.1 Cos'è lo spawning . . . 93
5.2.2 Esempi di spawning . . . 94
5.2.3 Impatto sulle performance . . . 94
5.2.4 Cenni all'Object Pooling . . . 94
5.3 Destroy Actor . . . 95
5.4 Perfezionamento della torretta: Find Relative Look at Rotation . . . 95
5.4.1 Problema con Find Look at Rotation . . . 95
5.4.2 Soluzione: Find Relative Look at Rotation . . . 95
5.5 Introduzione ai Cicli (Loop) . . . 95
5.5.1 Definizione . . . 95
5.5.2 For Loop in Unreal . . . 96
5.5.3 Esempio di utilizzo: spawning di una griglia . . . 96
5.6 Regolazione della velocità di rotazione (Interpolation Speed) . . . 96
5.6.1 Modifica della variabile . . . 96
5.7 Riepilogo e indicazioni per il proseguimento . . . 97
5.8 Problematiche di salvataggio e materiali . . . 97
5.9 Teoria dello Spawning . . . 97
5.9.1 Definizione . . . 97
5.9.2 Esempi . . . 97
5.9.3 Impatto sulle performance . . . 98
5.9.4 Cenni all'Object Pooling . . . 98
5.10 Destroy Actor . . . 98
5.11 Perfezionamento della torretta . . . 98
5.11.1 Find Relative Look at Rotation . . . 98
5.11.2 Variabile `Interpolation Speed` . . . 98
5.12 Introduzione ai cicli: For Loop . . . 98

5.12.1 Definizione . . . 98
5.12.2 Applicazione: spawning di una griglia . . . 99
5.13 Implementazione del Line Trace (Sphere Trace) . . . 99
5.13.1 Concetto . . . 99
5.13.2 Creazione dell'evento Shoot . . . 99
5.13.3 Punto di partenza: Scene Component . . . 99
5.13.4 Direzione e lunghezza: Forward Vector . . . 100
5.13.5 Nodo Sphere Trace by Channel . . . 100
5.13.6 Debug del tracciamento . . . 100
5.13.7 Creazione di un Trace Channel personalizzato . . . 100
5.14 Note finali e debug . . . 101
5.15 Configurazione del canale di traccia personalizzato per la torretta . . . 101
5.15.1 Verifica del rilevamento . . . 102
5.16 Creazione del proiettile (BluePrint Actor) . . . 102
5.17 Rilevamento delle collisioni del proiettile e autodistruzione . . . 103
5.17.1 Configurazione del Trace sul proiettile . . . 103
5.17.2 Distruzione all'impatto . . . 103
5.18 Canale di traccia per il proiettile . . . 104
5.18.1 Nota sulla risposta del giocatore . . . 104
5.19 Affinità e regolazioni finali . . . 104
5.20 Ignorare il proiettile sulla torretta . . . 105
5.21 Sessione di troubleshooting . . . 105
5.21.1 Il tick dell'evento Shoot non funziona . . . 105
5.21.2 Orientamento del pivot e del trace . . . 105
5.21.3 La torretta non segue il giocatore . . . 105
5.22 Preparazione allo spawn del proiettile . . . 106
5.23 Spawning da due bocche con selezione casuale . . . 106
5.24 Controllo della cadenza di fuoco (Fire Rate) . . . 106
5.25 Arresto del fuoco all'uscita del giocatore . . . 108
5.26 Risoluzione dei problemi più comuni . . . 108
5.26.1 Proiettile che non si muove o si distrugge appena spawnato . . . 108
5.26.2 La torretta spara troppi proiettili o a raffica . . . 108
5.26.3 Torretta non spara pur rilevando il giocatore . . . 108
5.27 Introduzione all'esercizio: matrice di cubi con For Loop . . . 109
5.28 Ciclo For per lo spawn di una fila di cubi . . . 109
5.29 Visualizzazione delle posizioni con Line Trace . . . 111
5.30 Regolazione del puntamento della torretta (approfondimento) . . . 111
5.30.1 Offset manuale sulla posizione del target . . . 111
5.30.2 Uso di un punto di riferimento sul giocatore . . . 112
5.31 Conclusioni e prossima lezione . . . 112

6 Fondamenti di Programmazione e Interattività in Unreal Engine 5: Timeline, Collisioni e Audio 113
6.1 Introduzione e comunicazioni . . . 113
6.2 Riepilogo teorico . . . 113
6.2.1 Ciclo `For Loop` . . . 113
6.2.2 Collisioni: Ignore, Overlap, Block . . . 113
6.2.3 Object Type e Object Responses . . . 114
6.2.4 Eventi di collisione: Begin Overlap e End Overlap . . . 114
6.2.5 Branch (condizione) . . . 114
6.3 Timeline . . . 114
6.3.1 Definizione e caratteristiche . . . 114
6.3.2 Float Track . . . 115
6.4 Approfondimento sui Trace Channels (domanda studente) . . . 115

6.5 Dimostrazione pratica: configurazione delle collisioni . . . 115
6.6 Esercitazioni previste . . . 115
6.7 Collisioni: Overlap, Ignore e Block . . . 116
6.7.1 Dimostrazione pratica . . . 116
6.7.2 Gestione dei conflitti tra risposte . . . 116
6.8 Completamento esercizio: spawn di cubi con `For Loop` . . . 116
6.8.1 Riepilogo della logica . . . 116
6.8.2 Spawn del cubo al punto di impatto . . . 116
6.8.3 Creazione del Blueprint `BP_Cube` . . . 117
6.8.4 Problema: i cubi spawnati interferiscono con i trace successivi . . . 117
6.8.5 Introduzione al secondo ciclo: le righe . . . 117
6.9 Completamento della matrice di cubi con `For Loop` annidati . . . 117
6.9.1 Logica della griglia . . . 117
6.9.2 Realizzazione pratica . . . 118
6.9.3 Ottimizzazione: disabilitare il Tick . . . 120
6.9.4 Risoluzione problemi comuni . . . 120
6.10 Nuovo esercizio: piattaforma mobile con Timeline . . . 120
6.10.1 Obiettivo . . . 120
6.10.2 Creazione del Blueprint `BP_Platform` . . . 120
6.10.3 Rilevamento del giocatore con Overlap . . . 120
6.10.4 La Timeline . . . 121
6.10.5 Integrazione nella logica della piattaforma . . . 121
6.10.6 Audio (cenni) . . . 121
6.11 Riepilogo . . . 121
6.12 Completamento della piattaforma mobile: Timeline e Lerp . . . 122
6.12.1 Riepilogo: movimentazione lungo l'asse Z . . . 122
6.12.2 Creazione e configurazione della Timeline . . . 122
6.12.3 Utilizzo del valore della Timeline: il nodo Lerp (Interpolazione Lineare) . . . 122
6.12.4 Ricostruzione del vettore e applicazione del movimento . . . 123
6.12.5 Attivazione e disattivazione con Overlap . . . 123
6.12.6 Risoluzione dei problemi comuni . . . 123
6.13 Pausa e comunicazioni . . . 123
6.14 Perfezionamento della piattaforma: controllo della velocità e posizionamento preciso . . . 125
6.14.1 Regolazione del Play Rate della Timeline . . . 125
6.14.2 Variabile `Duration` e calcolo automatico del Play Rate . . . 125
6.14.3 Problema del metodo additivo e introduzione del Billboard . . . 126
6.14.4 Salvataggio delle posizioni in variabili . . . 126
6.14.5 Interpolazione vettoriale con `VLerp` . . . 126
6.14.6 Gestione del ritorno con `Reverse` . . . 126
6.15 Integrazione dell'audio (introduzione) . . . 128
6.16 Integrazione dell'audio nella piattaforma mobile . . . 128
6.16.1 Regolazione del Play Rate e risoluzione problemi . . . 128
6.16.2 Importazione del file audio . . . 128
6.16.3 Creazione della Sound Cue . . . 128
6.16.4 Modulazione del pitch e del volume con il Modulator . . . 129
6.16.5 Spawn del suono e controllo della riproduzione . . . 129
6.16.6 Cambiare il pitch in base alla direzione della Timeline . . . 129
6.16.7 Verifica finale . . . 130
6.17 Conclusione e saluti . . . 130

7 Blueprint Avanzati: Trigger, Ereditarietà, Timeline ed Event Dispatcher in UE5 133
7.1 Risoluzione di problemi tecnici pregressi . . . 133
7.1.1 Errore “Sound Wave not found” e accesso nullo . . . 133
7.1.2 Piattaforma che si sotterra o si teletrasporta allo zero del mondo . . . 133
7.1.3 Problema di collisione con la piattaforma e attivazione . . . 134
7.1.4 Bug audio: suono continua dopo interruzione del movimento . . . 134
7.2 Lezione del giorno: Array, Ereditarietà e Pulsante Interattivo . . . 134
7.2.1 Introduzione agli array . . . 134
7.2.2 Ereditarietà (Parent e Child) . . . 134
7.2.3 Esercizio: Pulsante che attiva luce e piattaforma . . . 135
7.2.4 Creazione del Blueprint Pulsante (inizio) . . . 135
7.3 Realizzazione del pulsante con due mesh statiche e Timeline . . . 135
7.3.1 Perché due mesh distinte: frame e bottone . . . 135
7.3.2 Creazione e posizionamento delle mesh . . . 136
7.3.3 Gestione della collisione per evitare blocchi del giocatore . . . 136
7.3.4 Logica di pressione con Timeline . . . 136
7.3.5 Modifica delle curve di interpolazione . . . 137
7.3.6 Verifica e debug . . . 137
7.4 Oggetti attivabili e comunicazione tramite array . . . 138
7.4.1 Creazione della classe base `BP_Activable` . . . 138
7.4.2 Variabile array pubblica nel pulsante . . . 138
7.4.3 Logica di attivazione/disattivazione con Timeline e `For Each Loop` . . . 138
7.4.4 Assegnazione degli oggetti nella scena . . . 140
7.5 Ereditarietà: luce attivabile come classe figlia . . . 140
7.5.1 Creazione del Blueprint `BP_ActivatableLight` . . . 140
7.5.2 Ereditarietà degli eventi e `BeginPlay` . . . 140
7.5.3 Configurazione della `Point Light` . . . 140
7.5.4 Salvataggio dell’intensità iniziale . . . 142
7.5.5 Note sulla timeline: `Play` vs `Play from Start` . . . 142
7.6 Override degli eventi e implementazione della luce attivabile . . . 142
7.6.1 Sovrascrivere gli eventi Activation e Deactivation . . . 142
7.6.2 Logica della luce . . . 142
7.6.3 Debug di problemi comuni . . . 143
7.7 Fix audio per la piattaforma mobile . . . 143
7.7.1 Creazione del suono con `Spawn Sound at Location` . . . 143
7.7.2 Impostazione del Pitch in base alla direzione . . . 143
7.7.3 Interruzione del suono al termine del movimento . . . 144
7.7.4 Bug del Pitch quando il giocatore abbandona la pedana . . . 144
7.8 Event Dispatcher: comunicazione inversa tra pulsante e luce . . . 144
7.8.1 Creazione dell’Event Dispatcher nel pulsante . . . 144
7.8.2 Bind dell’evento nella luce attivabile . . . 144
7.8.3 Chiamata del dispatcher nel pulsante . . . 145
7.8.4 Vantaggi degli Event Dispatcher . . . 145
7.9 Override degli eventi e implementazione della luce attivabile . . . 145
7.9.1 Sovrascrivere gli eventi `Activation` e `Deactivation` . . . 145
7.9.2 Logica della luce . . . 147
7.9.3 Debug di problemi comuni . . . 147
7.10 Fix audio per la piattaforma mobile . . . 147
7.10.1 Creazione del suono con `Spawn Sound at Location` . . . 147
7.10.2 Impostazione del `Pitch` in base alla direzione . . . 148
7.10.3 Interruzione del suono al termine del movimento . . . 148
7.10.4 Correzione del bug del `Pitch` all’uscita anticipata . . . 148
7.11 Event Dispatcher: comunicazione inversa tra pulsante e luce . . . 148
7.11.1 Creazione dell’Event Dispatcher nel pulsante . . . 148

7.11.2 Bind dell'evento nella luce attivabile . . . 148
7.11.3 Chiamata del dispatcher nel pulsante . . . 149
7.11.4 Vantaggi degli Event Dispatcher . . . 149
7.12 Esercizio pratico: oggetto attivato da pulsante con Timeline . . . 149
7.12.1 Opzioni per l'esercizio . . . 149
7.12.2 Implementazione base con Timeline . . . 149
7.12.3 Note specifiche per la piattaforma (variante avanzata) . . . 150
7.12.4 Consegna e suggerimenti . . . 150

8 Gameplay Framework, Dati Globali e Animazioni dei Personaggi in Unreal Engine 5 153
8.1 Problemi tecnici iniziali e raccomandazioni . . . 153
8.2 Mixamo come risorsa gratuita per animazioni . . . 153
8.3 Debug dell'errore `Sound Platform` . . . 153
8.4 Il Gameplay Framework di Unreal Engine . . . 154
8.4.1 Game Instance . . . 154
8.4.2 Game Mode . . . 154
8.4.3 Game State e Player State . . . 154
8.4.4 Pawn e Character . . . 154
8.5 Skeletal Mesh, Scheletro e Animazioni . . . 155
8.5.1 Control Rig e Animation Blueprint . . . 155
8.6 Character Movement Component . . . 155
8.7 Player Controller . . . 155
8.8 Sistema di Input . . . 155
8.9 Fix rapido per l'ascensore (booleano `Activated`) . . . 156
8.10 Game Instance personalizzata per dati globali . . . 156
8.10.1 Creazione della classe Game Instance . . . 156
8.10.2 Aggiunta della variabile globale . . . 156
8.10.3 Impostare la Game Instance di progetto . . . 157
8.11 Creazione del `Collectable` (pallina raccoglibile) . . . 157
8.11.1 Logica di raccolta e comunicazione con la Game Instance . . . 157
8.11.2 Troubleshooting: evento collisione non visibile . . . 158
8.12 Completamento del sistema di raccolta con Game Instance . . . 158
8.12.1 Incremento e stampa dei `Collectable` . . . 158
8.12.2 Visualizzazione del conteggio con un tasto (es. `E`) . . . 158
8.13 Correzione di errori comuni con la Game Instance (troubleshooting) . . . 158
8.14 Volume per azzerare i `Collectable` (`Volume Shop`) . . . 160
8.15 Volume per cambiare livello (`Volume Load Level`) . . . 160
8.15.1 Creazione di un nuovo livello . . . 161
8.16 Test del passaggio dati tra livelli . . . 161
8.17 Introduzione a Mixamo e pausa . . . 162
8.18 Scelta e download del personaggio su Mixamo . . . 162
8.19 Problemi tecnici e debugging durante la pausa . . . 162
8.19.1 Posizionamento del volume a terra . . . 162
8.19.2 Fix della piattaforma con booleano `Activated` . . . 162
8.19.3 Errore `Sound Platform / Audio Component` non valido . . . 163
8.19.4 Errore `Movement Platform` e crash all'uscita . . . 163
8.20 Importazione del personaggio Mixamo in Unreal Engine . . . 163
8.20.1 Analisi dello scheletro . . . 163
8.21 Concetti di animazione e strumenti esterni . . . 164
8.22 Scelta e download delle animazioni su Mixamo . . . 164
8.22.1 Tipologie di animazioni richieste . . . 164
8.22.2 Impostazioni dell'animazione su Mixamo . . . 164
8.22.3 Parametri di download . . . 164
8.23 Importazione delle animazioni in Unreal Engine . . . 165

8.23.1 Impostazioni dell'asset di animazione . . . 165
8.24 Animation Blueprint (logica delle animazioni) . . . 165
8.24.1 Creazione e setup iniziale . . . 165
8.24.2 Inizializzazione e comunicazione con il Character . . . 166
8.25 Creazione del nuovo Character Blueprint . . . 166
8.25.1 Procedura . . . 166
8.25.2 Assegnazione della Skeletal Mesh . . . 166
8.25.3 Collegamento con l'Animation Blueprint . . . 166
8.26 Configurazione del nuovo Character Blueprint . . . 166
8.26.1 Assegnazione dell'Animation Blueprint al Character . . . 167
8.27 Logica dell'Animation Blueprint . . . 167
8.27.1 Inizializzazione e referenze (Event Graph) . . . 167
8.27.2 Aggiornamento a ogni frame (Event Blueprint Update Animation) . . . 167
8.28 Animation Graph e Blend Space (introduzione) . . . 167
8.28.1 Collegamento diretto di un'animazione . . . 167
8.28.2 Blend Space 1D (anteprima) . . . 169
8.29 Preparazione per la lezione successiva . . . 169
8.30 Risoluzione problema materiali personaggio . . . 169
8.31 Chiusura e compiti . . . 169

9 Introduzione ai Sistemi di Animazione del Personaggio in Unreal Engine 5 171
9.1 Introduzione e obiettivi della lezione . . . 171
9.2 Animation Blueprint: eventi fondamentali . . . 171
9.2.1 Initialize Animation . . . 171
9.2.2 Update Animation (Event Blueprint Update Animation) . . . 171
9.3 Ottenere i dati dal personaggio . . . 171
9.3.1 Cast al Pawn Owner . . . 171
9.3.2 Velocità (Ground Speed) . . . 172
9.3.3 Verifica dello stato di caduta . . . 172
9.4 State Machine per la locomozione . . . 172
9.4.1 Creazione della State Machine . . . 172
9.4.2 Entry e stati . . . 172
9.4.3 Condizioni di transizione . . . 172
9.5 Blend Space 1D per camminata e corsa . . . 172
9.5.1 Scopo . . . 172
9.5.2 Creazione . . . 174
9.5.3 Posizionamento delle animazioni . . . 174
9.5.4 Collegamento con l'Animation Blueprint . . . 174
9.6 Regolazione della velocità di movimento . . . 174
9.6.1 Modifica della Max Walk Speed . . . 174
9.6.2 Pianificazione dello sprint . . . 174
9.7 Problemi tecnici e soluzioni . . . 175
9.7.1 Errore "Questo asset non ha schede ancorate" . . . 175
9.7.2 Cambio lingua dell'editor . . . 175
9.8 Collegamento del Blend Space nell'Animation Blueprint . . . 175
9.8.1 Utilizzo del Blend Space Player . . . 175
9.8.2 Scorciatoie per variabili nei Blueprint . . . 175
9.8.3 Ordinamento dei nodi . . . 175
9.9 Test del Blend Space e anteprima . . . 175
9.10 Aggiunta della camera al personaggio . . . 176
9.10.1 Assenza di camera nei nuovi Character . . . 176
9.10.2 Componenti necessari . . . 176
9.10.3 Configurazione dello Spring Arm . . . 176
9.10.4 Abilitazione della rotazione della camera . . . 176

9.11 Movimento del personaggio . . . 176
9.11.1 Copia dell'input dal Third Person Character di esempio . . . 176
9.11.2 Spiegazione dell'Input Movement . . . 176
9.12 Orientamento del personaggio al movimento . . . 178
9.13 Suggerimento sul Field of View (FOV) . . . 178
9.14 Retargeting delle animazioni di salto e caduta . . . 178
9.14.1 Perché il retargeting . . . 178
9.14.2 Procedura . . . 178
9.14.3 Note sul retargeting . . . 178
9.15 Aggiornamento della State Machine . . . 178
9.15.1 Assegnazione delle animazioni . . . 178
9.15.2 Configurazione delle transizioni . . . 179
9.16 Test finale . . . 179
9.17 Perfezionamento delle transizioni di salto e caduta . . . 179
9.17.1 Differenziazione tra salto e caduta tramite velocità verticale . . . 179
9.17.2 Calcolo della Jump Z Velocity nell'Animation Blueprint . . . 179
9.17.3 Aggiornamento delle condizioni di transizione . . . 180
9.18 Risoluzione del problema di rotazione del personaggio . . . 180
9.18.1 Sintomo . . . 180
9.18.2 Soluzione . . . 181
9.19 Implementazione della corsa (Sprint) . . . 181
9.19.1 Creazione dell'Input Action IA_Run . . . 181
9.19.2 Aggiunta del mapping al contesto di input (IMC_Default) . . . 181
9.19.3 Logica nel Blueprint del personaggio (BP_Knight) . . . 181
9.19.4 Transizione morbida con FInterpTo . . . 181
9.20 Scaricamento delle animazioni per Blend Space 2D e Montaggi . . . 182
9.20.1 Obiettivi . . . 182
9.20.2 Animazione di attacco con Root Motion . . . 183
9.20.3 Animazioni direzionali in-place per il Blend Space 2D . . . 183
9.20.4 Importazione e preparazione . . . 183
9.21 Discussione e feedback sul corso . . . 183
9.22 Preparazione per il Blend Space 2D . . . 184
9.22.1 Animazioni necessarie . . . 184
9.23 Calcolo della direzione nell'Animation Blueprint . . . 184
9.23.1 Estensione della Sequence . . . 184
9.23.2 Nodo Calculate Direction . . . 184
9.24 Creazione del Blend Space 2D . . . 184
9.24.1 Impostazioni iniziali . . . 184
9.24.2 Posizionamento delle animazioni . . . 185
9.25 Integrazione nell'Animation Blueprint . . . 185
9.25.1 Sostituzione nello stato WalkRun . . . 185
9.25.2 Configurazione del Character . . . 186
9.26 Test e problemi di interpolazione . . . 186
9.27 Riepilogo del Blend Space 2D e regolazione parametri . . . 186
9.27.1 Impostazione dello Smoothing . . . 186
9.27.2 Spiegazione del sistema di direzione . . . 186
9.28 Animation Montages . . . 187
9.28.1 Scopo e descrizione . . . 187
9.28.2 Creazione del Montage . . . 187
9.28.3 Sezioni del Montage . . . 187
9.28.4 Notify . . . 187
9.28.5 Slot di animazione . . . 187
9.29 Integrazione del Montage nell'Animation Blueprint . . . 188
9.29.1 Cache della posa di locomozione . . . 188

9.29.2 Ramo del montage . . . 188
9.30 Attivazione del Montage da Blueprint . . . 188
9.30.1 Input e chiamata a Play Montage . . . 188
9.30.2 Gestione del movimento e input durante l'attacco . . . 188
9.30.3 Blocco della rotazione e della camera . . . 189
9.31 Task per la prossima lezione . . . 189

10 Sviluppo di un Bot in Unreal Engine: Animazioni e Physical Materials 191
10.1 Risoluzione dei problemi riscontrati nel progetto . . . 191
10.1.1 Problemi con l'Input Action e l'Input Mapping Context . . . 191
10.1.2 Correzione della telecamera (Spring Arm) . . . 191
10.1.3 Problemi con le animazioni e l'interazione . . . 192
10.2 Lezione del giorno: Suoni dei passi con Physic Material . . . 192
10.2.1 Introduzione ai Physic Material . . . 192
10.2.2 Creazione del nuovo personaggio (Bot2) . . . 192
10.2.3 Materiale didattico . . . 193
10.2.4 Creazione del nuovo Character Blueprint (Bot2) . . . 193
10.2.5 Blend Space e animazioni Lean . . . 193
10.2.6 Calcolo della Lean nell'Animation Blueprint . . . 195
10.2.7 Note e chiarimenti . . . 195
10.2.8 State Machine della Locomozione (Anim Graph) . . . 197
10.3 Testing e correzione del personaggio . . . 198
10.3.1 Preparazione del Character Blueprint per il test . . . 198
10.3.2 Correzione degli errori comuni . . . 198
10.4 Suoni dei passi con Anim Notify e Physic Material . . . 200
10.4.1 Creazione dell'Anim Notify . . . 200
10.4.2 Logica del Line Trace per il rilevamento del terreno . . . 200
10.4.3 Note . . . 201
10.4.4 Inserimento dell'Anim Notify nelle animazioni . . . 201
10.4.5 Creazione e assegnazione dei Physic Material . . . 202
10.4.6 Debug e verifica . . . 202
10.4.7 Problema residuo e prossimi argomenti . . . 202

11 Sviluppo dell'HUD in Unreal Engine: Vita, Stamina e Logiche di Gioco 203
11.1 Risoluzione dei problemi del personaggio . . . 203
11.1.1 Input non funzionante (personaggio fermo) . . . 203
11.1.2 Problemi con l'Animation Blueprint . . . 203
11.1.3 Correzione dell'interazione con la torretta . . . 204
11.2 Introduzione all'User Interface (UI) . . . 204
11.2.1 Definizione e componenti . . . 204
11.2.2 Obiettivi della lezione . . . 204
11.2.3 Utilizzo degli Event Dispatcher . . . 205
11.2.4 Prossimi passi . . . 205
11.3 Implementazione del danno e della vita del personaggio . . . 205
11.3.1 Configurazione del proiettile per infliggere danno . . . 205
11.3.2 Variabili e logica della vita nel Character Blueprint . . . 205
11.3.3 Creazione e utilizzo dell'Event Dispatcher per la vita . . . 206
11.4 Creazione del Widget per la UI . . . 206
11.4.1 Creazione del Widget Blueprint e design della barra della vita . . . 206
11.4.2 Logica del Widget e collegamento con il personaggio . . . 206
11.4.3 Prossimi passi (preannunciati) . . . 208
11.5 Visualizzazione e aggiornamento della barra della vita . . . 208
11.5.1 Creazione e visualizzazione del Widget nel Character Blueprint . . . 208
11.5.2 Binding dell'Event Dispatcher e aggiornamento grafico . . . 208

11.5.3 Allineamento preciso della Progress Bar . . . 209
11.6 Preparazione della barra della stamina . . . 209
11.6.1 Design nel Widget Blueprint . . . 209
11.7 Implementazione della corsa (Stamina) . . . 209
11.7.1 Configurazione dell'Input Action per la corsa . . . 209
11.7.2 Logica della velocità e della stamina nel Character Blueprint . . . 209
11.7.3 Problemi tecnici segnalati . . . 210
11.8 Gestione della User Interface (UI) e del Personaggio . . . 210
11.8.1 Creazione e visualizzazione del Widget HUD . . . 210
11.8.2 Design e logica della barra della vita nel Widget . . . 210
11.8.3 Preparazione della barra della stamina . . . 211
11.9 Implementazione della corsa e della velocità . . . 211
11.10 Correzioni e troubleshooting . . . 211
11.10.1 Problemi con la torretta . . . 211
11.10.2 Problemi con l'Animation Blueprint . . . 213
11.10.3 Altri problemi . . . 213
11.11 Correzioni e completamento della logica di danno e UI . . . 214
11.11.1 Proiettile: applicazione del danno e autodistruzione . . . 214
11.11.2 Correzione della barra della vita e della morte . . . 214
11.11.3 Ripristino della torretta . . . 214
11.12 Implementazione del sistema di Stamina . . . 215
11.12.1 Variabili e timer principali . . . 215
11.12.2 Logica di consumo della stamina . . . 215
11.12.3 Logica di recupero della stamina . . . 215
11.12.4 Gestione dell'input e stop forzato della corsa . . . 215
11.12.5 Note aggiuntive . . . 216
11.13 Debugging del sistema di stamina e corsa . . . 216
11.13.1 Correzione dell'aggiornamento della barra della stamina . . . 216
11.13.2 Risoluzione del problema della corsa (Shift non funzionante) . . . 216
11.13.3 Consumo della stamina durante la caduta . . . 216
11.14 Compito per casa: Porta automatica con Timeline . . . 218
11.15 Conclusioni e consigli . . . 218

12 Sviluppo del Menu di Gioco e Interazioni tramite Blueprint Interface e Component 219
12.1 Introduzione e organizzazione del corso . . . 219
12.1.1 Recupero delle lezioni . . . 219
12.1.2 Supporto e comunicazione . . . 219
12.1.3 Esame e percorso didattico . . . 219
12.2 Pianificazione della lezione odierna . . . 220
12.3 Creazione del menu principale . . . 220
12.3.1 Creazione del Widget Blueprint . . . 220
12.3.2 Preparazione del livello vuoto per il menu . . . 220
12.3.3 Design dei pulsanti nel Widget . . . 220
12.4 Creazione del Menu Principale . . . 221
12.4.1 Preparazione del livello vuoto . . . 221
12.4.2 Widget Blueprint del Menu . . . 221
12.4.3 Logica del Menu nel Level Blueprint . . . 222
12.4.4 Funzionalità dei pulsanti nel Widget Graph . . . 222
12.4.5 Risultato finale . . . 223
12.5 Componente per il Tracciamento dell'Interazione . . . 223
12.5.1 Creazione del Blueprint Component (Scene Component) . . . 223
12.5.2 Logica della funzione `LookForInteractions` . . . 223
12.5.3 Inizializzazione del componente (Begin Play) . . . 225
12.6 Sistema di Interazione . . . 225

12.6.1 Blueprint Interface BPI_InteractionRay . . . 225
12.6.2 Componente BPC_RayInteract: logica di tracciamento e comunicazione . . . 226
12.6.3 Completamento del Componente BPC_RayInteract . . . 226
12.6.4 Creazione dell'Actor interagibile di test . . . 228
12.6.5 Input per l'interazione: IA_Interact . . . 229
12.6.6 Logica di interazione nel Character Blueprint . . . 229
12.7 Completamento del Sistema di Interazione . . . 230
12.7.1 Reset del DoOnce e completamento dell'interazione . . . 230
12.8 Widget dell'interazione (press prompt) . . . 230
12.8.1 Creazione del Widget VB_InteractUI . . . 230
12.8.2 Visualizzazione e nascondimento del Widget nell'Actor Interactable . . . 232
12.9 Esempio di interazione: Attivazione/Disattivazione di una Luce . . . 232
12.9.1 Preparazione dell'Actor Interactable con Luce . . . 232
12.10 Conclusioni del corso e saluti . . . 232
12.10.1 Riepilogo e utilizzo del sistema di interazione . . . 232
12.10.2 Feedback e supporto futuro . . . 234
12.10.3 Saluti finali . . . 234

13 Fondamenti di Level Design in Unreal Engine: Dal Concept al Blackout 235
13.1 Introduzione e panoramica del corso . . . 235
13.1.1 Saluti e comunicazioni iniziali . . . 235
13.1.2 Struttura delle lezioni successive . . . 235
13.2 Principi fondamentali del Level Design . . . 235
13.2.1 Ispirazione: realtà vs altri videogiochi . . . 235
13.2.2 Documentazione e Layout . . . 236
13.2.3 La Gym (Test Level) . . . 236
13.3 Principi di Level Design . . . 236
13.3.1 La Gym (Test Level) – continuazione . . . 236
13.4 Il Block Out . . . 237
13.4.1 Definizione e scopo . . . 237
13.4.2 Esempio: Case Study del Block Out di God of War (2018) . . . 237
13.5 Utilizzo pratico della Gym e del Block Out in Unreal Engine . . . 238
13.5.1 La griglia come strumento di misura . . . 238
13.5.2 Misurazione dello sprint . . . 238
13.5.3 Test delle altezze di salto e valutazione del *feeling* . . . 238
13.5.4 Modifica dei parametri del Character Movement . . . 238
13.6 Lezione di Level Design – Davide (seconda parte) . . . 239
13.6.1 Struttura e obiettivi delle lezioni . . . 239
13.6.2 Principi di Level Design . . . 239
13.6.3 Il Block Out . . . 240
13.6.4 Lavorare con le misure: griglia e snap . . . 240
13.6.5 Test e regolazione dei parametri di movimento . . . 241
13.6.6 Geometry Brush . . . 241
13.7 Illuminazione in Unreal Engine . . . 242
13.7.1 Tipi di luce disponibili . . . 242
13.7.2 Il parametro Mobility . . . 242
13.7.3 Comportamento di Directional Light e Skylight . . . 243
13.7.4 Point Light e Spotlight: parametri e utilizzo . . . 243
13.7.5 Interazione tra luci e Post Process (cenni) . . . 244
13.8 Lezione di Level Design – Davide . . . 244
13.8.1 Introduzione e organizzazione . . . 244
13.8.2 Principi di Level Design . . . 245
13.8.3 Il Block Out . . . 245
13.8.4 Lavorare con le misure: griglia e snap . . . 246

13.8.5 Test e regolazione dei parametri del Character Movement . . . 246
13.8.6 Geometry Brush . . . 246
13.9 Illuminazione in Unreal Engine . . . 247
13.9.1 Tipi di luci disponibili . . . 247
13.9.2 Il parametro `Mobility` . . . 248
13.9.3 Comportamento delle luci ambientali . . . 248
13.9.4 Point Light e Spotlight: parametri e buone pratiche . . . 248
13.9.5 Rect Light e riflessi . . . 249
13.9.6 Post Process (cenni) . . . 249
13.9.7 Build dell'illuminazione . . . 249
13.10 Assegnazione per la prossima lezione . . . 249

14 Creazione di Paesaggi in Unreal Engine: Strumenti di Scultura e Materiali Landscape 251
14.1 Discussione iniziale e aggiornamenti . . . 251
14.2 Riepilogo della lezione precedente . . . 251
14.2.1 Documento di layout . . . 251
14.2.2 Blockout . . . 251
14.2.3 Gym: definizione e scopo . . . 252
14.3 Analisi della gym di Federico . . . 252
14.3.1 Test con torretta e coperture . . . 252
14.3.2 Piattaforme mobili e strumenti di debug . . . 252
14.3.3 Gestione della telecamera . . . 252
14.3.4 Esperimenti sul salto . . . 252
14.4 Principi di level design emersi . . . 253
14.4.1 Progettare a partire dalle capacità del personaggio . . . 253
14.4.2 Less is more e versatilità delle meccaniche . . . 253
14.5 Aspetti organizzativi . . . 253
14.6 Risultati della gym di Federico (continuazione) . . . 253
14.6.1 Metriche dei salti e leggibilità degli ostacoli . . . 253
14.6.2 Esposizione di parametri in editor . . . 253
14.7 Concetti di design emersi dalla gym . . . 254
14.7.1 Forgiveness e bilanciamento dei salti . . . 254
14.7.2 Valore delle informazioni raccolte nella gym . . . 254
14.7.3 Pivot degli asset e trucchi di editor . . . 254
14.8 Livello di Filippo . . . 254
14.8.1 Struttura generale . . . 254
14.8.2 Caratteristiche delle sezioni . . . 254
14.8.3 Aspetti di level design osservabili . . . 255
14.9 Lezione da trarre: *less is more* e apprendimento attivo . . . 255
14.10 Landscape Mode . . . 255
14.10.1 Cos'è il Landscape . . . 255
14.10.2 Creazione del Landscape . . . 255
14.10.3 Gestione (Manage) . . . 256
14.10.4 Il pennello (Brush) . . . 256
14.10.5 Visualizzazione della risoluzione poligonale . . . 257
14.10.6 Creare cavità (Visibility Mask) . . . 257
14.10.7 Pennello Visibility e creazione di buche . . . 257
14.10.8 Sculpt, Erase e Smooth . . . 257
14.10.9 Flatten (Appiattire) . . . 258
14.10.10 Rampe (Ramp Tool) . . . 258
14.10.11 LOD (Level of Detail) e ottimizzazione . . . 258
14.10.12 Erosion e Hydro . . . 258
14.11 Materiale del Landscape e pittura . . . 259
14.11.1 Creazione e assegnazione dei layer . . . 259

14.11.2 Struttura del materiale: Material Function e nodo di output . . . 259
14.11.3 Nodo Landscape Layer Blend . . . 259
14.11.4 Altri nodi utili . . . 260
14.11.5 Pittura dei layer . . . 260
14.12 Configurazione del materiale del Landscape e spawn automatico di foliage . . . 260
14.12.1 Struttura del materiale `M_Landscape` . . . 260
14.12.2 Nodo `Landscape Layer Blend` . . . 260
14.12.3 Creazione dei layer nel Landscape . . . 261
14.12.4 Spawn automatico di erba tramite `Landscape Grass Output` . . . 261
14.13 Creazione di un materiale personalizzato per il Landscape . . . 262
14.13.1 Rimozione del materiale preesistente e pulizia dei layer . . . 262
14.13.2 Creazione di un nuovo materiale . . . 262
14.13.3 Creazione dei Layer Info e pittura . . . 263
14.13.4 Strumenti di pittura: Smooth, Flatten, Noise . . . 263
14.13.5 Abilitazione della Visibility Mask per scavare buchi . . . 263
14.14 Note finali e argomenti per la prossima lezione . . . 264
14.14.1 Automateriale basato sulla pendenza . . . 264
14.14.2 Misurare l'inclinazione del Landscape per il gameplay . . . 264
14.14.3 Compiti per casa e conclusione . . . 264

15 Auto-Materiali e Foliage in Unreal Engine: Tecniche per la Creazione di Ambienti Naturali 265
15.1 Materiale automatico per il Landscape . . . 265
15.1.1 Introduzione all'automateriale . . . 265
15.1.2 Struttura del materiale `M_Autolandscape_Base` . . . 265
15.1.3 Il nodo LERP come strumento di interpolazione . . . 266
15.1.4 Il criterio di interpolazione (Alpha) . . . 266
15.1.5 Creazione ed esposizione dei parametri . . . 266
15.2 Miglioramento dell'automateriale . . . 266
15.2.1 Aggiunta di un terzo layer: erba, sabbia e roccia . . . 266
15.2.2 Calcolo dell'Alpha basato sull'altezza . . . 267
15.2.3 Dimostrazione dell'effetto . . . 267
15.3 Discussione e organizzazione . . . 267
15.3.1 Recupero lezioni e argomenti futuri . . . 267
15.4 Automateriale avanzato per il Landscape . . . 268
15.4.1 Riepilogo e obiettivo . . . 268
15.4.2 Materiale base: pendenza e altezza . . . 268
15.4.3 Interpolazione e attributi del materiale . . . 268
15.4.4 Aggiunta del terzo layer: erba . . . 268
15.4.5 Combinare automateriale e layer dipingibili . . . 269
15.5 Correzioni e approfondimenti sull'illuminazione . . . 269
15.5.1 Lumen e illuminazione dinamica . . . 269
15.5.2 God Rays (Light Shaft) con Directional Light . . . 269
15.6 Modalità Foliage . . . 270
15.6.1 Introduzione e concetto . . . 270
15.6.2 Static Mesh Foliage vs. Static Mesh . . . 270
15.6.3 Gestione del Foliage generato dal Landscape Grass Output . . . 270
15.6.4 Utilizzo pratico del Foliage Mode . . . 271
15.7 Modalità Foliage . . . 271
15.7.1 Gestione dei parametri di piazzamento . . . 271
15.7.2 Attenzione alle performance . . . 272
15.8 Correzioni e approfondimenti sull'illuminazione . . . 272
15.8.1 Lumen e Global Illumination . . . 272
15.8.2 God Rays (Light Shaft) tramite Directional Light . . . 272

15.9 Gestione del Foliage generato dal Landscape . . . 273
15.10 Modalità Foliage (Pittura manuale) . . . 273
15.10.1 Concetti fondamentali . . . 273
15.10.2 Tipi di mesh utilizzabili . . . 273
15.10.3 Strumenti di pittura . . . 274
15.10.4 Parametri di piazzamento per tipo di foliage . . . 274
15.10.5 Gestione manuale delle istanze . . . 275
15.10.6 Consigli sull'uso e performance . . . 275
15.11 Assegnazione per la prossima lezione . . . 275
15.12 Introduzione al Post-Process Volume . . . 276
15.12.1 Cos'è e dove si trova . . . 276
15.12.2 Proprietà fondamentale: `Infinite Extent (Unbound)` . . . 276
15.12.3 Utilizzo di più volumi e transizioni . . . 276
15.12.4 Esempi di effetti e lenti . . . 277
15.13 Domande e approfondimenti . . . 277
15.13.1 God Rays e finestre colorate . . . 277
15.13.2 Conclusioni e saluti . . . 277

16 Introduzione al Game Audio: Ruoli, Tecniche ed Evoluzione del Suono nei Videogiochi 279
16.1 Introduzione alla masterclass di Game Audio . . . 279
16.2 Percorso ed esperienza del docente . . . 279
16.2.1 Formazione e ingresso nell'industria . . . 279
16.2.2 Consigli su branding e presenza online . . . 280
16.2.3 Specializzazione verticale e portfolio . . . 280
16.3 Il ruolo dell'audio nel videogioco: lineare vs interattivo . . . 280
16.3.1 Il problema dell'audio lineare applicato ai giochi . . . 280
16.3.2 Parallelismo con il cinema e differenza fondamentale . . . 280
16.3.3 Flusso di lavoro tipico del Game Audio . . . 281
16.4 Conclusioni e prossimi argomenti . . . 281
16.5 Masterclass di Game Audio – Davide Pensato (DP Studios) . . . 281
16.5.1 Introduzione e presentazione del docente . . . 281
16.5.2 Il percorso del docente . . . 281
16.5.3 Consigli su branding e presenza online . . . 282
16.5.4 Il portfolio di DP Studios e la specializzazione . . . 282
16.5.5 Contrazione del settore gaming . . . 282
16.5.6 L'importanza dell'audio nel videogioco . . . 282
16.5.7 Esempi video . . . 283
16.5.8 Ruoli e competenze nel Game Audio . . . 283
16.5.9 Pipeline di produzione e integrazione . . . 283
16.5.10 Consigli pratici per iniziare . . . 284
16.6 Approfondimenti sul potere evocativo: esempi di audio oppressivo . . . 284
16.6.1 Manhunt (Rockstar Games) . . . 284
16.6.2 Altri esempi citati . . . 284
16.7 La caratterizzazione . . . 284
16.7.1 Definizione e complessità . . . 284
16.7.2 Esempio: Hitman (missione a Kinshasa) . . . 285
16.7.3 Esempio: Assassin's Creed (con musica diegetica) . . . 285
16.8 Supporto al gameplay . . . 285
16.8.1 Guida e informazione tramite l'audio . . . 285
16.8.2 Sincronizzazione musicale con le meccaniche . . . 285
16.8.3 Sfruttamento intelligente della musica da parte del giocatore . . . 286
16.9 Conclusioni parziali . . . 286
16.10 Storia dell'audio nei videogiochi . . . 286
16.10.1 Gli albori: anni '80 . . . 286

16.10.2 Il Voice Stealing . . . 286
16.10.3 La nascita del Tracker . . . 287
16.10.4 Creazione dei suoni . . . 287
16.11 Il supporto al gameplay tramite l'audio (conclusione) . . . 287
16.11.1 Sfruttamento della musica da parte del giocatore . . . 287
16.11.2 Il passaggio al digitale e il CD (anni '90) . . . 287
16.11.3 Il presente (2000–oggi) . . . 288
16.11.4 I ruoli nel team audio . . . 288
16.11.5 Limitazioni e buone pratiche nella produzione audio . . . 288
16.12 Storia dell'audio nei videogiochi (continuazione) . . . 289
16.12.1 Il passaggio al digitale e il CD (anni '90) . . . 289
16.12.2 Il presente (2000–oggi) . . . 289
16.13 I ruoli nel team audio . . . 289
16.13.1 Audio Director . . . 289
16.13.2 Sound Designer . . . 290
16.13.3 Compositore . . . 290
16.13.4 Dialogues Director e doppiaggio . . . 290
16.13.5 Audio Programmer . . . 291
16.14 Il processo di sviluppo e il ruolo dell'audio . . . 291
16.14.1 L'importanza dell'implementazione . . . 291
16.14.2 Pre-produzione . . . 291
16.14.3 Produzione . . . 292
16.14.4 Post-produzione: Beta, Candidate Master, Gold Master . . . 292
16.15 Categorie di suoni nel videogioco . . . 293
16.15.1 Musica . . . 293
16.15.2 Effetti sonori (SFX) . . . 293
16.15.3 Dialoghi (Voices) . . . 293
16.15.4 Ambience (Ambienti) . . . 293
16.16 Musica non lineare (adattiva/interattiva) . . . 293
16.16.1 Tipologie di reattività . . . 293
16.16.2 Esempio: Mod di Far Cry (musica mood-driven) . . . 293
16.17 Effetti sonori (SFX) . . . 294
16.18 Suoni d'ambiente (Ambience) . . . 294
16.19 Dialoghi . . . 295
16.19.1 Tipologie . . . 295
16.19.2 Criticità e gestione . . . 295
16.20 Domande e risposte . . . 295
16.20.1 Sincronizzazione labiale e adattamento lingue . . . 295
16.20.2 Interazione in tempo reale (esempio televisione e passi) . . . 295
16.21 Conclusioni e contatti . . . 296

Chapter 1

Introduzione al Game Development e a Unreal Engine

1.1 Presentazioni

1.1.1 Il docente

Igor Ripanti, ex studente di Hive nel corso di Game Design, ha lavorato come level designer al titolo *Steel Seed* in un contesto indie, occupandosi anche di programmazione e scripting con Unreal Engine.

1.1.2 Gli studenti

I partecipanti si presentano brevemente: provengono dai corsi di Game Design, Character Animation, Narrative Design, 3D Art e Programmazione, tutti al primo anno. La maggior parte ha già avuto qualche esperienza con engine come Unity o Godot, spesso realizzando semplici prototipi (platform, Flappy Bird); qualcuno è completamente alle prime armi.

1.2 Che cos'è un game engine

Un software – cioè un insieme di programmi e dati – che mette a disposizione funzionalità specifiche per creare applicazioni interattive. Il motore grafico è il "calderone" che riunisce 3D art, programmazione e design in un unico ambiente di sviluppo.

1.2.1 Esempi di engine

Unreal Engine, Unity, Godot, Game Maker. Ogni engine integra motore fisico, gestione delle collisioni, audio, animazioni, IA. La logica di gioco viene implementata tramite *scripting*, che può sfruttare diversi linguaggi: C++, C#, Blueprint. Il corso si concentrerà sul sistema Blueprint di Unreal Engine, un linguaggio di programmazione visuale che richiede soprattutto logica e capacità di adattarsi a nuovi strumenti.

1.3 Fondamenti di rendering 3D

1.3.1 Poligoni e vertici

I modelli 3D sono composti da poligoni, principalmente triangoli. Un alto numero di poligoni garantisce maggiore dettaglio ma anche un maggiore peso computazionale: per questo l'ottimizzazione è fondamentale per evitare rallentamenti (lag).

1.3.2 La pipeline di rendering

Tre passaggi principali:

- **Vertex shading**: elaborazione dei vertici che definiscono la forma dei triangoli.
- **Rasterizzazione**: trasformazione del modello vettoriale in pixel (immagine raster), necessaria per la visualizzazione sullo schermo. A differenza delle immagini vettoriali, quelle raster sono soggette a sgranatura se ingrandite, ma per la resa su display è indispensabile rasterizzare.
- **Fragment shading**: applicazione di colori, luci, ombre e riflessi in base alla posizione della camera. È una fase dal forte impatto artistico, perché determina l'aspetto visivo finale.

1.3.3 Illuminazione e materiali

L'illuminazione può essere calcolata per vertice (effetto a sfaccettature visibili) o per frammento (luci più morbide e diffuse). Le mappe UV, create dagli artisti 3D, definiscono come luce e materiali interagiscono con la superficie dei modelli.

1.4 Rendering in tempo reale e frame rate

Unreal Engine produce rendering in tempo reale, mostrando la scena a una velocità compresa tra 20 e 120 fotogrammi al secondo (fps). L'animazione si basa su una sequenza di frame: nel cinema lo standard è di 24 fps, nei videogiochi si va dai 30 ai 60 fps, fino a 120/144 fps per un movimento più fluido.

1.5 Unreal Engine: panoramica

1.5.1 Epic Games e installazione

Unreal Engine è sviluppato da Epic Games. Occorre scaricare l'Epic Games Launcher, all'interno del quale sono disponibili diverse versioni del motore (5.5, 5.6, 5.7), costantemente aggiornate con nuove funzionalità (ad esempio Control Rig per animazioni procedurali). Sul canale YouTube ufficiale vengono presentate le novità di ogni release.

1.5.2 Perché Unreal è così diffuso

Utilizzato sia in ambito AAA che indie, l'engine fornisce un linguaggio comune che semplifica la collaborazione tra professionisti. I designer lo adoperano per la prototipazione rapida: un'idea può essere immediatamente testata nell'engine per verificarne la validità, grazie anche alla presenza di funzionalità predefinite (movimento, salto, ecc.). In passato, invece, i team erano costretti a sviluppare engine proprietari con tempi di realizzazione di 2-3 anni, e spesso tali motori erano poco riutilizzabili.

1.5.3 Esempi celebri

Numerosi giochi sono realizzati in Unreal: *A Way Out*, *ARK*, *Crash Bandicoot 4*, *Fortnite*. Quest'ultimo ha introdotto l'Unreal Editor per Fortnite, che consente di creare esperienze personalizzate (similmente a Roblox). *Expedition 33* è un caso emblematico: nato da persone che hanno appreso tramite tutorial online, il primo prototipo era essenziale ma funzionante, dimostrando come l'engine permetta di concentrarsi subito sul gameplay.

1.5.4 Oltre i videogiochi: cinema e virtual production

Unreal Engine è impiegato anche in ambito cinematografico e video. Ambienti virtuali renderizzati in tempo reale su schermi LED sostituiscono il classico green screen (come nello spot con il motociclista). In queste produzioni la priorità è il dettaglio visivo, a differenza dei videogiochi dove i modelli devono essere ottimizzati e spesso mantenuti in low-poly per garantire buone prestazioni.

1.6 Risorse utili

Il Fab (marketplace di Unreal Engine) mette a disposizione asset gratuiti, scontati o "free for the month", ideali per prototipare e apprendere senza dover creare ogni elemento da zero.

1.7 Game Loop e differenza tra videogioco e cinema

La differenza sostanziale tra un film e un videogioco è la presenza del *game loop*. In un'applicazione interattiva il programma:

1. Inizializza lo stato iniziale.
2. Rimane in attesa di input (es. pressione di un tasto).
3. Aggiorna la logica (es. il personaggio salta).
4. Renderizza l'immagine.
5. Ripete il ciclo finché non si esce dal gioco.

Questo ciclo continuo di input, update e rendering è assente nel video-making tradizionale.

1.8 La produzione di un videogioco

1.8.1 Pre-produzione

Prima dello sviluppo vero e proprio si definisce l'idea. Ogni buona idea deve essere messa su carta e compresa da tutto il team. Strumenti come lo *storyboard* aiutano a visualizzare la narrazione. I designer producono documentazione dettagliata per spiegare ogni meccanica.

1.8.2 Tipologie di produzioni

- **Indie**: team ristretti (fino a 20-30 persone), ruoli meno specializzati e grande flessibilità. Chi fa design spesso programma e viceversa.
- **AA**: media impresa, budget significativo ma inferiore ai blockbuster. Può permettersi outsourcing di alcune parti (es. animazioni di *Expedition 33* realizzate in Corea).
- **AAA**: grandi produzioni con budget elevatissimi. Ruoli ultra-specializzati (es. *leaf artist* che si occupa solo delle foglie).

1.8.3 Outsourcing

L'outsourcing è l'affidamento di una parte della produzione (animazioni, level design, ecc.) a studi esterni. Va distinto dalla cessione dell'intera proprietà intellettuale: l'azienda X commissiona all'azienda Y un lavoro specifico, mantenendo il controllo del progetto.

1.9 I ruoli nel team di sviluppo

Tutte le figure professionali si interfacciano in Unreal Engine, l'ambiente comune.

1.9.1 Game Designer

- **Combat Designer**: prototipa situazioni di combattimento, utilizzando parametri esposti (es. danno) creati dal programmatore.
- **Level Designer**: definisce gli spazi di gioco attraverso *blockout* (o grey box), la forma primordiale del livello in cui si testano metriche, distanze e visibilità, prima che gli artisti creino l'ambiente definitivo. In Unreal sono utili i *Modeling Tool* per forme rapide.
- **Narrative Designer**: inserisce dialoghi, momenti scriptati (es. monologhi interiori), interazioni con NPC e cutscene, collaborando con i programmatori per realizzare sistemi narrativi.

1.9.2 Visual Art

- **Materiali e texture**: in Unreal è possibile animare i materiali (es. lava che si muove) e usare effetti di tassellatura.
- **Concept Artist**: fornisce riferimenti visivi; può lavorare su progetti 2D, realizzabili anche in Unreal.
- **VFX Artist**: utilizza il *Niagara System* per effetti particellari.
- **Technical Artist**: figura ponte tra arte e programmazione.

1.9.3 Animazione

Il *Sequencer* di Unreal Engine consente di realizzare cutscene ed eventi cinematografici scriptati in tempo reale (es. astronavi di passaggio in *Jedi Fallen Order*).

1.9.4 Programmatori

I programmatori finalizzano le idee dei designer. Più si sale di livello produttivo, più i ruoli si specializzano:

- **Combat Programmer**: logica di combattimento.
- **3C Programmer**: gestione di camera, character e controlli.
- **Rendering/Engine Programmer**: ottimizzazione della pipeline di rendering (es. la nebbia in *Silent Hill 2* Remake).
- **IA Programmer**: comportamenti dei nemici.
- **Network Programmer**: implementazione del multiplayer (client-server).

Consiglio ai designer: trattare bene i programmatori, perché sono loro che trasformano le idee in realtà.

1.9.5 Altri ruoli e strumenti

- **Audio**: gestione e modulazione dei suoni direttamente in Unreal.
- **QA e tester**: segnalazione bug, utilizzo di build di debug (spesso fornite con cheat per la stampa specializzata).
- **Unreal Insights**: strumento di profiling per analizzare il carico della scena e le performance.
- **Producer** e figure business: lato gestionale, meno legato all'eseguibile dell'engine.

1.10 Blueprint: la programmazione visuale in Unreal

I Blueprint sono un linguaggio a nodi che permette di creare logica di gioco senza scrivere codice C++. Sono lo standard anche per molti programmatori, specialmente in ambito indie: moltissimi giochi sono realizzati esclusivamente con Blueprint, senza toccare una riga di C++. L'ottimizzazione spinta può richiedere C++, ma l'interfaccia principale rimane quella visuale.

1.11 Attività in classe: gli elementi di un gioco

Domanda: da cosa è formato un gioco? Quali sono gli elementi che lo rendono tale? Prendendo come esempio un gioco tradizionale come acchiapparello (nascondino), emergono:

- **Regole**: una struttura che definisce come si gioca.
- **Interazione**: la possibilità di agire all'interno del sistema.
- **Giocatori**: senza partecipanti il gioco non esiste; sono essi stessi *game object*.
- **Divertimento**: non è un elemento strutturale, ma una conseguenza dell'esperienza.
- **Socialità**: in giochi come *D&D*, MMO o party game, l'interazione sociale è essa stessa gameplay.

1.11.1 Teoria del gioco e Homo Ludens

Un partecipante cita il concetto di *Homo Ludens* di Huizinga: il gioco come componente culturale fondamentale. La discussione si arricchisce di definizioni complesse e opinioni diverse. Si aggiungono:

- **Obiettivo**: il movente che spinge a giocare.
- **Spazio di gioco (game space)**: il campo fisico o virtuale che delimita l'azione (es. il parco in acchiapparello, il corridoio in un escape room, la mappa in un videogioco).
- **Tempo**: può essere una regola (tempo limite) o una risorsa da gestire.

1.11.2 Motivazioni del giocatore e psicologia

Ognuno porta nel gioco una ricerca personale: c'è chi gioca per la trama, chi per la competizione sociale, chi per il senso di potenza (es. il "numerone" dei danni). Citando Arrowhead Game Studios, "un gioco per tutti è un gioco per nessuno": voler accontentare tutti rende l'esperienza dispersiva. Le motivazioni possono essere riassunte nel framework *MDA* (Meccanica, Dinamica, Estetica):

- **Meccanica**: ciò che il giocatore può fare (saltare, attaccare).
- **Dinamica**: come si sceglie di agire (attaccare subito o aspettare, evitare).
- **Estetica**: come ci si sente nel farlo (sensazione di controllo, impatto, intelligenza).

1.11.3 Esempio pratico: l'Escape Room

Definendo gli elementi per un escape room:

- **Regole**: stabilire confini invalicabili per non rovinare la scenografia; evitare comportamenti dannosi.
- **Ruoli e socialità**: si crea una dinamica di gruppo (il veloce, il riflessivo) che porta a collaborare per l'obiettivo comune.
- **Gestione emotiva**: controllare ansia e focalizzarsi sugli indizi importanti; la capacità di scelta del giocatore è gameplay.

1.11.4 Applicazione a un videogioco: Expedition 33

Scelto dagli studenti come esempio:

- **Attori**: i personaggi giocabili.
- **Meccaniche**: combattimento a turni rinnovato, con elementi action che hanno attirato anche chi odiava i giochi a turni.
- **Spazi**: livelli esplorabili connessi alla narrazione.
- **Obiettivi**: scoprire la storia, superare le sfide, vivere un'esperienza emotivamente intensa.
- **Regole**: il sistema di combattimento, la progressione del personaggio.

Perché si gioca a Expedition? Per la qualità della motion capture, la trama straziante, l'arte ricca e un sistema di combattimento che ha saputo modernizzare una formula datata.

1.11.5 Il tempo come meccanica

Il tempo può essere:

- **Regola vincolante**: *Majora's Mask* (la luna si abbatte dopo tre giorni), *Lightning Returns: Final Fantasy XIII* (game over allo scadere del tempo), *Persona 5* (calendario da rispettare).
- **Gestione strategica**: il ciclo giorno/notte in *Blood of the Walker* influisce sulle abilità del personaggio e sulle interazioni.
- **Ritmo**: giochi ritmici come *Hi-Fi Rush*, dove il tempo va seguito.
- **Manipolazione**: *Life is Strange* permette di riavvolgere il tempo per cambiare le scelte, trasformando il tempo in una meccanica di gameplay.
- **Relazione sociale**: accumulare ore di gioco (es. 4000 ore su un titolo) può diventare elemento di identità sociale.

1.11.6 Bilanciamento e ruolo del QA

Il bilanciamento è frutto della collaborazione tra Game Designer e QA tester. Il QA non si limita a giocare, ma gioca in modo metodico per scovare problemi: ad esempio, una tester scoprì come scalare tutte le superfici sfruttando una spadata in diagonale. Le build di debug permettono di saltare livelli per test rapidi. Nei team piccoli è il designer a testare, ma la cosa migliore è far provare il gioco a persone nuove, perché il loro feedback è più puro.

1.12 Tecniche artistiche e motori grafici

Alla domanda su come viene realizzato un gioco in stop-motion come *The Midnight Walk* o *Harold Halibut*:

- Le animazioni in stop-motion vengono scannerizzate frame per frame e importate nell'engine come sequenze di immagini o come mesh 3D animate.
- L'engine funge da compositore: unisce gli asset provenienti da software esterni (Blender, Maya, Cinema 4D) e vi applica le regole di gioco.
- La qualità visiva risiede nella tecnica artigianale dell'animazione, mentre l'engine gestisce logica e interattività.

Esempi come *Cuphead* hanno usato un approccio simile, con disegni a mano digitalizzati.

1.13 Aspetti di mercato: il tempismo di uscita

Il successo di un gioco può dipendere anche dal momento del lancio. Un'uscita in concomitanza con grandi titoli (es. il "shadow drop" di *Oblivion*) può influenzare le vendite. *Expedition 33* ha beneficiato del fatto di essere arrivato al momento giusto, con un combat system a turni che mancava da tempo, nonostante titoli affini come *Sea of Stars* fossero usciti l'anno prima.

1.14 Installazione di Unreal Engine 5.5

- Scaricare l'Epic Games Launcher dal sito ufficiale.
- Nella sezione "Versioni motore" cliccare sul "+" e selezionare la versione **5.5** (l'ultimo numero, ad esempio 5.5.4, indica solo l'aggiornamento minore; i progetti creati con 5.5.x sono compatibili tra loro).
- Procedere con il download e l'installazione.

1.15 Documentazione e supporto

Oltre a YouTube, la risorsa principale è la **documentazione ufficiale di Unreal Engine**, molto esaustiva e ben strutturata. In essa si trovano spiegazioni su qualsiasi argomento (es. i branch, il costrutto `if/else`), guide introduttive e una sezione *Learning* con corsi gratuiti su diversi ambiti: programmazione, arte 3D (es. Environmental Basic, Scatter on Mesh), effetti visivi (Niagara). Sono inoltre disponibili forum e community (es. Reddit) dove ritrovare problemi già risolti in passato.

1.15.1 Requisiti di sistema e supporto Discord

I requisiti minimi verranno condivisi nella chat generale del corso su Discord. In generale, un PC portatile o fisso di fascia media recente dovrebbe reggere Unreal; se si è leggermente sotto i requisiti minimi è comunque consigliabile provare, perché le prestazioni possono variare molto da sistema a sistema.

1.16 Il Fab e gli asset

Il **Fab** è il marketplace integrato che funge da "magazzino dei desideri". Contiene *asset* di ogni tipo: modelli 3D, animazioni, texture, decal, pennelli, sistemi di gioco, suoni, sprite e altro. Gli asset sono disponibili per Unreal Engine, Unity e altri motori.

1.16.1 Pack gratuiti mensili

All'inizio di ogni mese viene messa in evidenza una sezione di **pack gratuiti** (es. scan di ruderi, vetri rotti, elementi modulari). È importante riscattarli sempre: rimangono per sempre nella propria libreria.

1.16.2 MetaHuman

Per chi si occupa di character design, **MetaHuman** è lo strumento integrato per la creazione di volti umani realistici. Sul Fab sono presenti anche asset correlati a personaggi, abbigliamento e volti.

1.16.3 Materiale didattico del corso

Verrà fornito un link a un drive con il materiale didattico, che include asset e un progetto Unreal preconfigurato da utilizzare durante il corso. È fondamentale scaricare la versione **5.5** (l'ultimo numero, es. 5.5.4, indica solo l'aggiornamento minore; la compatibilità è garantita all'interno della stessa versione principale). Passare da 5.5 a 5.3, ad esempio, può causare errori.

1.16.4 Buone pratiche nell'uso degli asset

Prima di inserire asset decorativi, è meglio costruire la funzionalità di base. Evitare di appesantire il progetto con elementi inutilizzati (es. un progetto da 70 GB di cui solo 3 GB realmente sfruttati). Scaricare solo ciò che serve.

1.17 Teoria: esecuzione nel editor e delta time

Quando si avvia il gioco in editor (*Play in Editor*) avvengono diversi processi:

- Gli **attori** (game object) vengono inizializzati e duplicati nello spazio di gioco.
- Vengono caricate le mappe e tutti gli attori, tra cui il *character* (esistono diverse classi: `Actor`, `Pawn`, `Character`).

Un concetto fondamentale è il **delta time** (delta seconds): ogni azione che produce movimento o cambiamento nel tempo deve essere moltiplicata per il tempo di gioco, in modo da essere indipendente dal frame rate. Se un movimento è definito senza tenere conto del delta time, a fps bassi impiegherà più tempo del previsto, perché il tempo dell'editor e il tempo reale non coincidono.

1.18 Primo avvio e creazione di un progetto

All'apertura di Unreal Engine si accede alla schermata dei progetti recenti, suddivisi per categorie: Games, Film, Video, Live Events.

1.18.1 Template di progetto

Nella sezione **Games** sono disponibili diversi template (preset):

- Blank (vuoto, senza Starter Content)
- First Person
- Third Person
- Top Down
- Vehicle
- Handheld AR
- Virtual Reality

Ogni template fornisce una base già configurata secondo il paradigma delle **3C** (Camera, Controller, Character). Il template Blank, selezionato con l'opzione *Starter Content*, include asset di base (forme geometriche, materiali semplici) utili per sperimentare.

1.18.2 Creazione del progetto

- Scegliere una *Project Location* (cartella dedicata, es. `UnrealEngineProjects`).
- Assegnare un *Project Name* **senza spazi** (usare trattini o underscore, es. `Prova-1` o `Prova_1`).
- Selezionare **Blueprint** (non C++), Target Platform **Desktop**, Quality Preset **Maximum**.
- Cliccare *Create*.

1.18.3 Compilazione degli shader

Al primo avvio di un progetto (e spesso dopo aggiornamenti) Unreal compila gli **shader**, ovvero i programmi che determinano la visualizzazione grafica. Questa operazione può richiedere molto tempo (es. 15 minuti per giochi pesanti come Final Fantasy XVI). È una fase inevitabile; nei giochi commerciali viene spesso eseguita una pre-compilazione al primo lancio.

1.19 Requisiti hardware e prestazioni

Il componente più sollecitato da Unreal Engine è la **scheda grafica**, analogamente a quanto avviene con i videogiochi. Su sistemi di fascia media la versione 5.5 è generalmente supportata, ma l'upgrade della GPU può diventare obbligatorio per evitare rallentamenti o crash. La compilazione iniziale degli shader rappresenta uno dei momenti più pesanti: è normale attendere diversi minuti (anche oltre il 40% di progressione) prima di poter interagire con l'editor.

1.20 L'interfaccia a finestre

Unreal Engine è organizzato a **finestre** ridimensionabili e spostabili, che a prima vista possono disorientare. I pannelli principali sono:

- **Viewport**: la visualizzazione centrale dello spazio 3D.
- **Content Browser**: il sistema di navigazione dei file interni al progetto, posizionato in basso di default.
- **Details**: mostra i parametri dell'oggetto selezionato (locazione, rotazione, scala, ecc.).
- **Place Actors**: elenco degli attori (elementi di gioco) trascinabili nella scena.
- **World Outliner**: elenco gerarchico di tutti gli attori presenti nel livello.

Se un pannello manca, è possibile riaprirlo dal menu `Window` in alto a destra. Le finestre possono essere agganciate tra loro, spostate, ridimensionate a piacere; è persino possibile avere due viewport contemporaneamente.

1.20.1 Il Content Browser e l'organizzazione delle cartelle

Il Content Browser visualizza tutti gli asset del progetto. Con **tasto destro** si creano cartelle (`New Folder`) per organizzare il lavoro. È fortemente consigliato:

- Creare una struttura di cartelle e sottocartelle chiara.
- Usare `Set Color` per assegnare colori alle cartelle e velocizzare la navigazione visiva.
- Mantenere l'ordine non solo per sé, ma anche per eventuali collaboratori, che ringrazieranno.

La visualizzazione può essere a cascata (struttura ad albero sulla sinistra) o in stile explorer.

1.21 Inserimento e manipolazione degli oggetti

1.21.1 Place Actors e primi oggetti

Dal pannello `Place Actors`, nella sezione `Shapes`, si possono trascinare forme primitive (cubi, cilindri, coni) direttamente nella viewport. Ogni oggetto inserito è un **attore** e appare con un **pivot** a tre assi colorati:

- **Rosso**: asse X
- **Verde**: asse Y

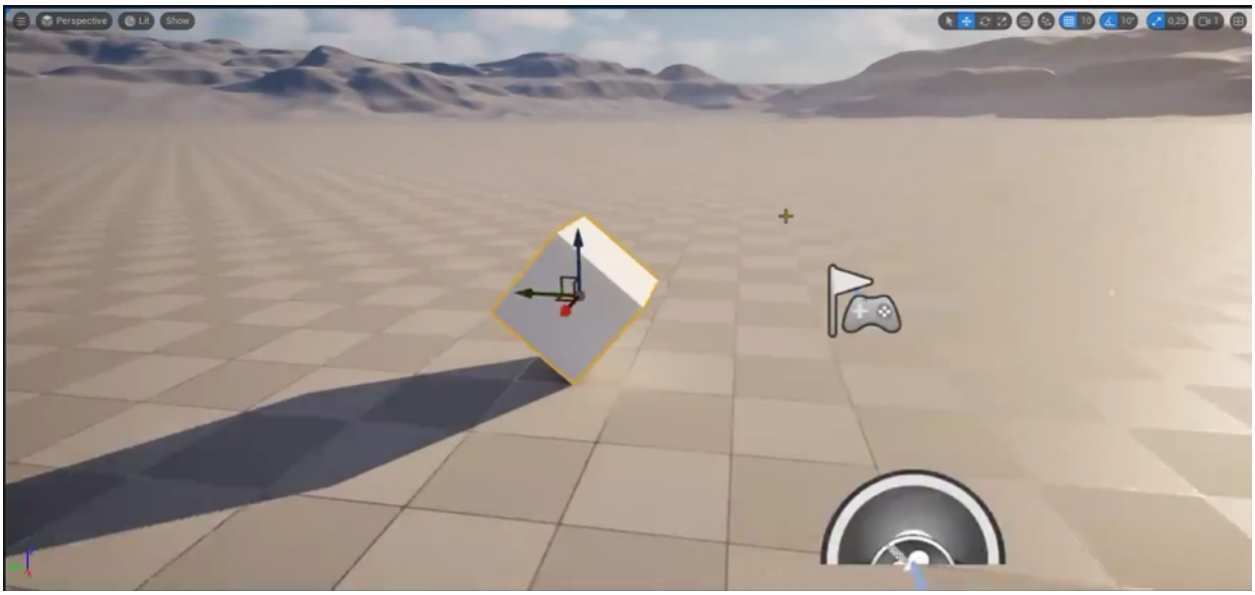


Figure 1.1: In alto a destra, impostazioni per il pivot (locale o globale)

- **Blu**: asse Z

L'unità di misura di Unreal è il **centimetro**. La griglia visibile nella viewport rappresenta un metro: un cubo standard equivale a 1 metro. Ci si muove con WASD(QE).

1.21.2 Trasformazioni: locazione, rotazione, scala

Le tre operazioni fondamentali sul **transform** di un oggetto si attivano con le shortcut:

- **W**: locazione (spostamento)
- **E**: rotazione
- **R**: scala

In alternativa, si possono usare i pulsanti in alto nella viewport. Durante lo spostamento, i valori numerici nel pannello `Details` si aggiornano in tempo reale: la locazione è espressa in centimetri, la rotazione in gradi, la scala in unità.

1.21.3 Sistema di coordinate: locale e globale

Il pivot può operare in due modalità, selezionabili tramite l'icona del cubetto/mondo in alto:

- **Relativa (locale)**: gli assi ruotano insieme all'oggetto. Utile per muovere un oggetto lungo il suo orientamento.
- **Globale (mondo)**: gli assi restano fissi rispetto al mondo. Utile per alzare un oggetto ruotato senza movimenti composti.

La scala è sempre **locale**: scalare lungo assi globali deformerebbe l'oggetto in modo innaturale, perciò Unreal lo impedisce. Una visualizzazione del menù è visibile in Fig. 1.1. Nota: il pivot è al centro dell'oggetto considerato.

1.21.4 Duplicazione e raggruppamento

- **Duplicare**: tenere premuto `Alt` e trascinare l'oggetto con il mouse. Funziona anche durante la rotazione. È una delle operazioni più frequenti.
- **Selezionare più oggetti**: tenere premuto `Shift` e cliccare sugli attori desiderati.
- **Gruppare**: selezionati gli oggetti, premere `Ctrl+G` (o tasto destro >`Group`). Il gruppo appare con un reticolo verde e si comporta come un tutt'uno per locazione e rotazione.
- **Sciogliere il gruppo**: `Shift+G` o tasto destro >`Groups` >`Ungroup`.

È possibile creare gruppi di gruppi. Durante lo **scaling**, però, ogni oggetto mantiene la propria asse locale: un gruppo scalato non si comporta come un unico solido. Per ottenere un unico oggetto scalabile uniformemente si dovrà creare una **static mesh**.

1.21.5 Snapping

Lo **snap** vincola i movimenti a intervalli predefiniti, evitando decimali indesiderati. Si attiva/disattiva cliccando sulle icone dei valori in alto. Si può impostare:

- **Locazione**: scatti di 1, 5, 10, 50, 100 unità (centimetri).
- **Rotazione**: scatti di 5, 10, 15, 30, 45, 60, 90, 120 gradi.
- **Scala**: scatti configurabili (es. 0.25, 1).

Quando lo snap è disattivato, i movimenti sono liberi al decimale. Il `Transform` racchiude queste 3 entità.

1.21.6 Scorciatoie utili per la viewport

- **Movimento camera**: tenere premuto `tasto destro` nella viewport e usare `WASD` per spostarsi, `Q` per scendere, `E` per salire.
- **Allineare a terra**: selezionare un oggetto e premere `End` per farlo cadere sulla prima superficie sottostante.
- **Inseguimento**: tenere premuto `Shift` mentre si sposta un oggetto per far sì che la camera lo segua.
- **Undo**: `Ctrl+Z` annulla l'ultima operazione.
- **Scaling uniforme**: afferrare il centro bianco del pivot per scalare su tutti gli assi contemporaneamente (esterno per allargare, interno per stringere).

1.22 Static mesh (cenni)

Una **mesh** è un modello 3D formato da poligoni. La **static mesh** è una tipologia di mesh non animata, il "modello solido" di un oggetto di gioco (es. il modello di un tavolo). Quando si lavora con forme primitive, gli oggetti restano indipendenti anche se gruppati; per fonderli in un unico oggetto occorre creare una static mesh, argomento che verrà approfondito in seguito.

1.23 Navigazione nella viewport: regolazione della velocità

Per regolare la velocità di spostamento della camera all'interno della viewport si utilizza la **rotellina del mouse**:

- Ruotando in **avanti** si aumenta la velocità (utile per coprire rapidamente grandi distanze, ad esempio per raggiungere una montagna lontana).

- Ruotando **indietro** si diminuisce la velocità fino a valori molto bassi (es. 0.03), ideale per osservare un oggetto da vicino con precisione millimetrica.

Il valore corrente della velocità è visibile in alto a destra nella viewport.

1.24 Outliner: organizzazione avanzata

Il **World Outliner** (attivabile da `Window >World Outliner`) mostra l'elenco gerarchico di tutti gli attori presenti nella scena. Di norma si apre a sinistra. Oltre alla semplice selezione, offre funzionalità fondamentali per gestire scene complesse:

- **Ridenominazione**: ogni nuovo attore riceve un nome progressivo (Cubo_1, Cubo_2...); selezionandolo si può rinominare.
- **Creazione di cartelle**: tasto destro nell'Outliner >`Create Folder`. Le cartelle aiutano a raggruppare logicamente gli oggetti (es. "Coni", "Tavolo_da_3").
- **Spostamento in cartella**: trascinare gli attori selezionati all'interno della cartella. Per selezionare rapidamente un intervallo di oggetti si tiene premuto `Shift` e si clicca sul primo e sull'ultimo della lista.
- **Select All Descendants**: tasto destro su una cartella >`Select All Descendants` seleziona all'istante tutti gli oggetti contenuti, risparmiando tempo quando si devono spostare o modificare interi gruppi. Vedi Fig. 1.2.

Una buona organizzazione nell'Outliner è cruciale quanto quella nel Content Browser e viene molto apprezzata in team.

1.25 Esercitazione pratica: costruzione di una scena

Per mettere in pratica i concetti di manipolazione (duplicazione, scala, rotazione, gruppi, snapping) viene proposto un esercizio di **level design primitivo**: allestire tre tavoli per una cena.

1. **Tavolo da tre ospiti**: forma libera, con tre sedie.
2. **Tavolo quadrato da cinque ospiti**: tavolo quadrato con cinque sedie.
3. **Cena romantica**: tavolo con due sedie e, a piacere, una candela (luce).

1.25.1 Suggerimenti per l'esecuzione

- Non è richiesta precisione artistica: l'obiettivo è prendere dimestichezza con i comandi. Si può usare una reference semplice (es. sedia della nonna) e riprodurla con forme primitive.
- Per riportare un oggetto ai valori di default di scala o locazione, usare la freccia `Reset to its default value` accanto al parametro nei Details.
- Se il **pivot** scompare dalla vista, premere il tasto **G** per mostrare/nascondere gli elementi invisibili (utile anche per visualizzare luci, suoni ecc.).
- Per la candela: trascinare una luce (Point Light o Spot Light) e posizionarla sopra il tavolo; per evitare ombre indesiderate, nelle proprietà della luce deselezionare `Cast Shadow`.
- Organizzare subito la scena creando una cartella nell'Outliner per ogni tavolo e trascinandovi dentro gli oggetti corrispondenti.

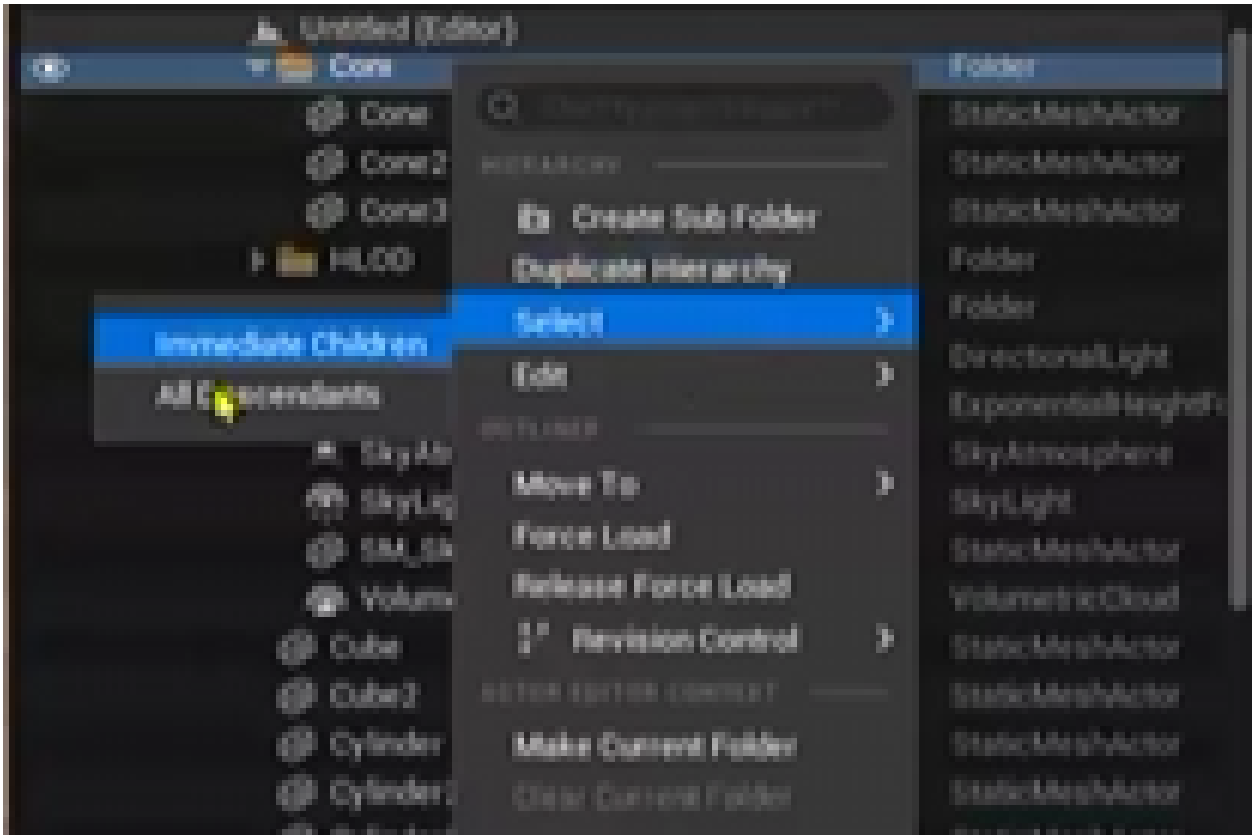


Figure 1.2: Selezionando il comando `All Descendants` di una cartella, possiamo compiere delle azioni su tutti gli elementi della cartella.

1.26 Consigli pratici e risoluzione di problemi comuni

1.26.1 Allineamento preciso degli oggetti

Nell'editor base di Unreal non esiste un allineamento automatico a un altro oggetto come in software di modellazione (es. Blender). Per posizionamenti accurati si può:

- Disattivare momentaneamente lo **snap** per movimenti liberi.
- Usare il tasto `Home` per ruotare un oggetto di 90° netti.
- In futuro sarà introdotta la **Modeling Mode**, che offre strumenti di modellazione più precisi all'interno di Unreal.

Nota: ottimo tool per modellare mappe di dungeon è https://www.dungeonscrawl.com/.

1.26.2 Selezione, copia e duplicazione di più oggetti

- **Selezionare più oggetti**: tenere premuto `Shift` e cliccare su ciascun attore (nell'Outliner o nella viewport). Nell'Outliner, per selezionare un intervallo contiguo, cliccare sul primo, tenere `Shift` e cliccare sull'ultimo.
- **Copiare e incollare**: dopo la selezione, `Ctrl+C` e `Ctrl+V`. Gli oggetti incollati appariranno nella stessa posizione; vanno poi spostati.
- **Duplicare con Alt**: tenere premuto `Alt`, quindi trascinare uno degli assi del pivot. Attenzione: bisogna mantenere `Alt` premuto mentre si trascina, non un semplice tap. Se non funziona, controllare di aver rilasciato `Shift` dopo la selezione.

1.26.3 Salvataggio e gestione dei file

- `Ctrl+S` salva la mappa corrente. Se la mappa non è ancora stata salvata, Unreal chiede di assegnarle un nome e una posizione all'interno della cartella del progetto (es. `/Content/Prova`).
- Il pulsante `Save Current` nella barra degli strumenti ha lo stesso effetto.
- All'atto del salvataggio di una mappa che contiene un **Landscape** (terreno modellabile, visibile come sfondo montuoso), può comparire la richiesta di salvare anche il file `HLOD layer`: si tratta del sistema di livello di dettaglio gerarchico per il paesaggio, che verrà approfondito più avanti.

1.27 Comunicazioni finali e prossime lezioni

- **Calendario**: la prossima lezione con Igor si terrà giovedì; il martedì successivo gli studenti avranno una sessione con Davide, dedicata alla parte più artistica di Unreal. Le lezioni si alterneranno settimanalmente.
- **Materiale didattico**: gli asset e il progetto Unreal saranno forniti tramite drive. Le registrazioni delle lezioni vengono caricate il giorno dopo; eventuali variazioni verranno comunicate su Discord.
- **Supporto e condivisione**: gli studenti possono contattare Igor su Discord per domande o per mostrare i propri lavori. Si valuterà la creazione di un canale dedicato per condividere screenshot ed esercizi, in modo da mantenere ordine.

Chapter 2

Blueprint e Interazione: Actor, Variabili e Input

2.1 Introduzione agli Actor

Gli **actor** sono gli elementi di gioco che possono essere posizionati nella scena (o viewport). Ogni actor possiede una **transform**, composta da:

- Locazione (posizione)
- Rotazione
- Scala

Questi tre fattori sono sempre sottintesi quando si parla di transform. Inoltre, nel content browser è possibile filtrare ciò che si vede, come in Fig. 2.1.

2.2 Componenti di un Actor

Un actor può essere composto da diversi **componenti**. Un esempio è la **mesh** (modello 3D solido formato da poligoni). I componenti possono essere:

- Audio
- Particle effect
- Point light (luce puntiforme)
- Skeletal mesh (mesh dotata di scheletro, per personaggi umanoidi, su cui si basa l'animazione tramite rotazione delle ossa)
- Static mesh (mesh senza scheletro)

I componenti possono essere organizzati in una **gerarchia**: un componente può essere figlio di un altro (imparentato).

2.3 Blueprint e Classi

Unreal Engine utilizza i **Blueprint**, un linguaggio di programmazione visuale a blocchi.

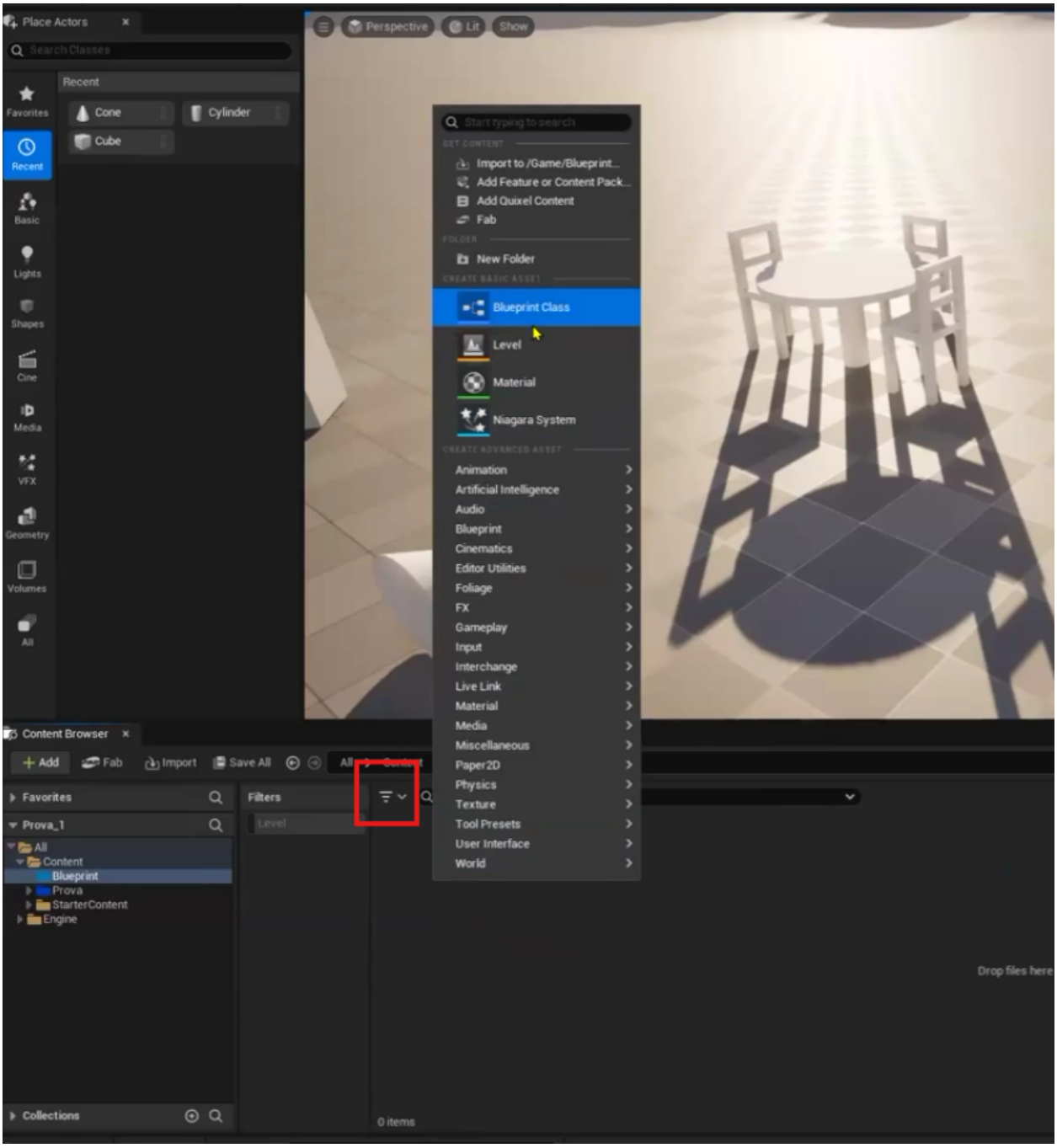


Figure 2.1: Nel quadro rosso, il tasto per poter filtrare, mentre è possibile vedere la struttura usuale per creare il Blueprint file.

2.3.1 Concetto di Classe

Una **classe** è un insieme di oggetti che condividono caratteristiche simili (analogia: reparto del supermercato). Creare una `Blueprint Class` equivale a scegliere una tipologia di classe. Le principali sono:

- Actor
- Pawn
- Character
- Player Controller
- Game Mode Base
- Actor Component
- Scene Component

2.4 Tipi di Blueprint

2.4.1 Actor

Un actor vuoto, privo di componenti predefiniti. Nell'editor appare solo un indicatore del centro; nel gioco non è visibile, ma può contenere logica.

2.4.2 Pawn

Un actor che può essere **posseduto** (controllato) da un giocatore e ricevere input. Non ha mesh di default, ma è possibile aggiungerla manualmente (es. Skeletal Mesh).

2.4.3 Character

Un tipo di Pawn che include già una **Skeletal Mesh** e altri componenti. Anch'esso è possedibile dal giocatore (ogni Character è anche un Pawn).

2.5 Editor delle Blueprint

All'interno di una Blueprint si trovano diverse schede, come in Fig. 2.2:

2.5.1 Viewport

Visualizzazione 3D del singolo oggetto, utile per modifiche grafiche.

2.5.2 Event Graph

Il nucleo della programmazione dell'oggetto. Qui si definisce il comportamento tramite **nodi** collegati.

2.6 Programmazione a Eventi

In Blueprint si lavora con **eventi**: momenti con una specifica condizione che attivano l'esecuzione. Un evento fondamentale è `Event Begin Play`.

2.6.1 Event Begin Play

Si attiva quando un actor **inizia a esistere** nel gioco (all'avvio o quando viene generato). Collegando un nodo al suo pin di esecuzione si definisce cosa accade in quel momento.

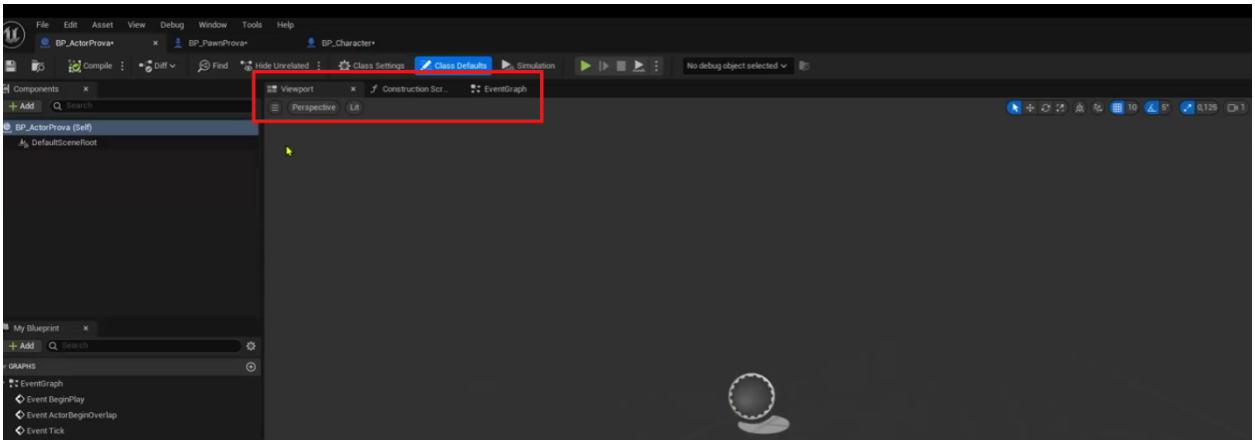


Figure 2.2: Nel quadro rosso, delle cartelle visibili nell'editor.

2.6.2 Print String

Nodo che stampa un messaggio a schermo. Funziona solo in modalità sviluppo (*development*), non in una build distribuita. Parametro importante: `Duration` (durata del messaggio in secondi).

Per testare: trascinare il pin di esecuzione di `Event Begin Play` verso l'esterno, cercare `Print String`, collegare. Compilare (tasto `Compile`) e avviare il gioco (`Play`); il messaggio apparirà in alto a sinistra.
Nota: compilare il blueprint con l'opzione in alto a sinistra.

2.7 Variabili

Le variabili sono contenitori per valori. Tipi comuni:

- **Boolean** (vero/falso) – colore del pin: rosso
- **Integer** (interi) – non mostrato nell'esempio
- **Float** (numeri con virgola) – colore del pin: verde

2.7.1 Creare una variabile

Nel pannello `Variables` (a sinistra), premere il tasto `+`. Assegnare un nome (es. `durationTimeText`) e scegliere il tipo (es. `Float`).

2.7.2 Get e Set

Trascinando la variabile nell'Event Graph senza premere tasti, si può scegliere:

- **Get**: legge il valore attuale della variabile.
- **Set**: imposta un nuovo valore alla variabile.

2.8 Collegamenti e Tipi di Pin

- I **blocchi di esecuzione** (colore blu) possiedono pin di esecuzione (forma a freccia) che determinano il flusso del programma.
- I **blocchi di calcolo** (es. operazioni matematiche, get di variabili) non hanno pin di esecuzione e restituiscono un valore tramite il proprio pin colorato.

Per assegnare un valore a un parametro (es. `Duration` di `Print String`), la variabile e il pin devono essere dello stesso tipo (colore). Se il tipo non corrisponde, occorre modificarlo dalla finestra delle variabili (`Variable Type`). Dopo aver creato o modificato variabili, bisogna sempre **compilare** il Blueprint.

2.9 Nodi di Esecuzione e Nodi di Calcolo

All'interno dell'Event Graph esistono due tipologie di nodi:

- **Nodi di esecuzione**: possiedono il *pin di esecuzione* (la freccetta). Definiscono il flusso del programma e possono contenere azioni come `Print String`, `Branch`, `Set` di variabili, ecc.
- **Nodi di calcolo**: *non* hanno pin di esecuzione. Si limitano a calcolare un valore a partire da uno o più input e restituiscono il risultato tramite un pin colorato. Esempi: operazioni matematiche, `Get` di variabili, `Find Look at Rotation`.

2.10 Operatori Aritmetici

I principali operatori matematici disponibili sono:

- `Subtract` ($-$)
- `Multiply` ($\times$, indicato con l'asterisco)
- `Divide` ($\div$)

Tutti accettano in ingresso pin dello stesso tipo (es. `float`) e restituiscono un risultato dello stesso tipo.
Esempio: Sottrarre 2 secondi alla durata di un `Print String`.

1. Ottenere il valore di una variabile `float` (es. `durationTimeText`) tramite `Get`.
2. Collegare il pin di uscita del `Get` a uno degli ingressi di un nodo `Subtract` (float – float).
3. Impostare il secondo ingresso a `2.0` (float).
4. Collegare il risultato del `Subtract` al pin `Duration` del `Print String`.
5. Per visualizzare anche il valore numerico, collegare il risultato del `Subtract` al pin `In String` del `Print String`: verrà automaticamente convertito in stringa (pin rosa).

Se `durationTimeText` vale `10`, a schermo apparirà `8` e il messaggio resterà visibile per 8 secondi.

2.11 Creare e Modificare Variabili

Per creare una nuova variabile:

- Nel pannello `Variables` a sinistra premere il tasto `+`.
- Assegnare un nome (es. `timeToPrint`).
- Scegliere il tipo (es. `Float`) dal menu a tendina `Variable Type`.

Per utilizzarla:

- Trascinare la variabile nell'Event Graph tenendo premuto `Ctrl` per creare un nodo `Get` (legge il valore), oppure `Alt` per un nodo `Set` (modifica il valore).
- Per staccare un collegamento: tenere premuto `Alt` e cliccare sul pin.

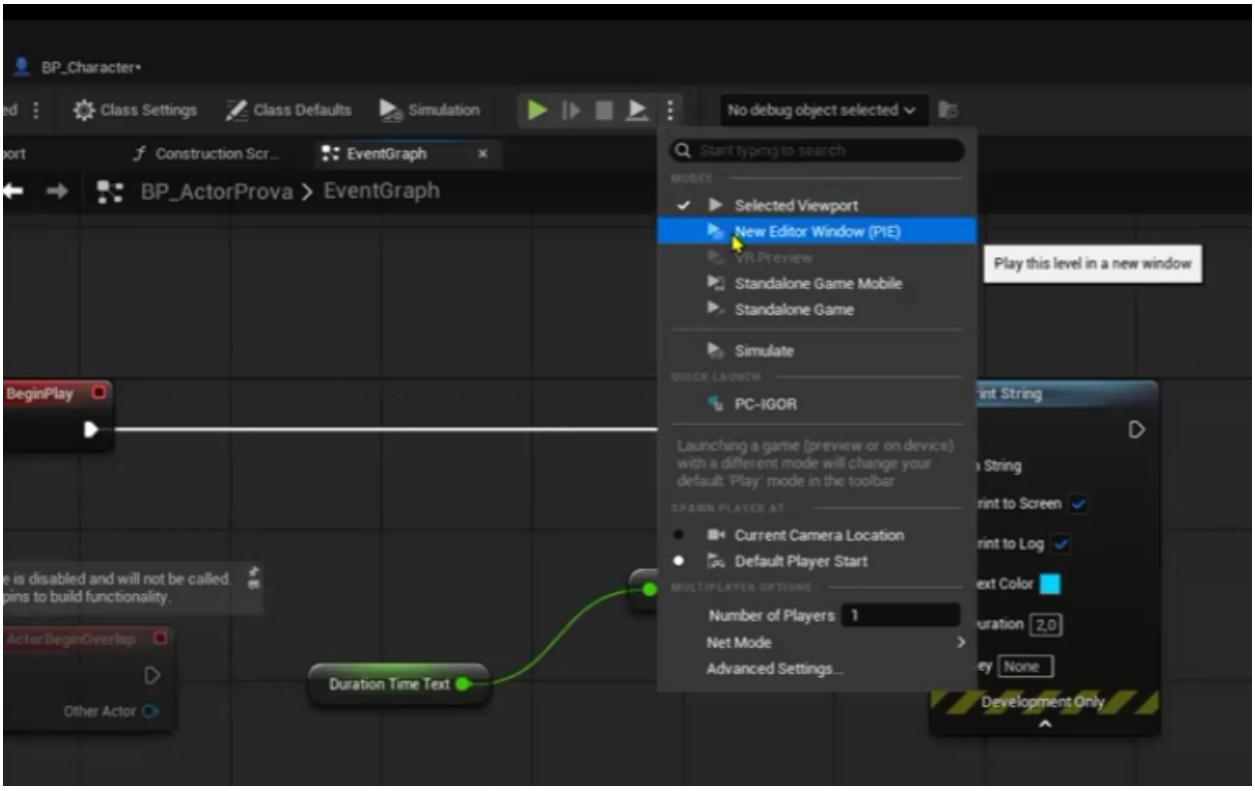


Figure 2.3: Caption

Impostare il valore di default: dopo aver compilato, selezionare la variabile nel pannello `Variables` e modificarne il valore nel pannello `Details` (a destra). In alternativa, se la variabile è pubblica, appare direttamente nei dettagli dell'actor nella Viewport.
Nota: è possibile compilare il blueprint e creare una nuova finestra per testare quanto fatto premendo in alto in centro i 3 puntini e selezionare quanto mostrato in Fig. 2.3.
Nota: i Details possono essere modificati sulla colonna di destra.

2.12 Proprietà delle Variabili: Pubbliche e Private

Una variabile può essere:

- **Privata**: visibile e modificabile solo all'interno del Blueprint.
- **Pubblica**: resa tale spuntando l'icona a forma di occhio (*Instance Editable*) o la voce `Instance Editable` nei dettagli. Appare nella sezione `Default` dell'actor quando lo si seleziona nella Viewport, permettendo di modificarla direttamente dall'editor.

Questo meccanismo è utile per il **tuning** (regolazione fine dei valori) da parte dei designer senza dover aprire il Blueprint.

2.13 Tipi di Variabili

I tipi di variabile più comuni in Unreal sono:

- **Boolean** (`bool`): vero (`true`) o falso (`false`). Il pin è di colore rosso.
- **Byte**: valore intero tra 0 e 255.

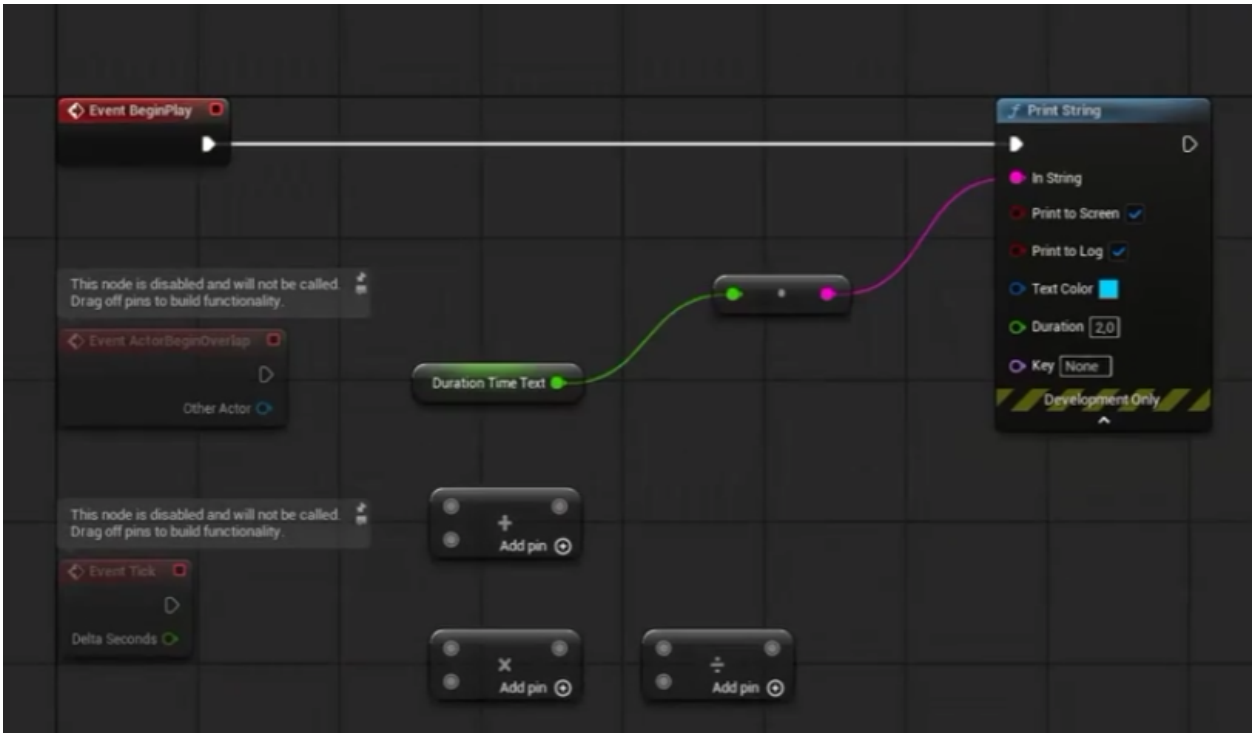


Figure 2.4: Primo codice Blueprint per stampare una stringa.

- **Integer** (`int`): numero intero senza virgola. Range: $-2^{31} \dots 2^{31} - 1$ (per `int32`) o $-2^{63} \dots 2^{63} - 1$ (per `int64`).
- **Float**: numero con virgola (decimale). Il pin è di colore verde.
- **Name**: stringa usata come *etichetta* per identificare asset, livelli, ecc. (simile a un tag).
- **String**: testo alfanumerico libero, usato principalmente per l'interfaccia utente (UI) e per messaggi.
- **Vector**: insieme di tre valori `X`, `Y`, `Z` (es. per la posizione).
- **Rotator**: insieme di tre valori `Roll`, `Pitch`, `Yaw` (es. per la rotazione).
- **Transform**: raggruppa `Location` (Vector), `Rotation` (Rotator) e `Scale` (Vector).

È inoltre possibile creare variabili del tipo di un **actor** specifico (es. un riferimento a un altro Blueprint).

2.14 Utilizzo dei Booleani

2.14.1 Get e Set di un Bool

- `Get` (trascinamento con `Ctrl`): restituisce il valore attuale (`true`/`false`), senza pin di esecuzione.
- `Set` (trascinamento con `Alt`): possiede un pin di esecuzione e un tick che imposta il valore a `true` o `false` manualmente.

Per stampare il valore booleano come testo: collegare il pin `bool` di un `Get` al pin `In String` di un `Print String` (conversione automatica a stringa). A schermo apparirà `true` o `false`.

2.15 Struttura Condizionale: Branch

Il nodo **Branch** (si crea premendo `B` e cliccando nell'Event Graph) rappresenta un bivio (if-else). Prende in ingresso:

- Un pin di esecuzione.
- Una condizione booleana (pin rosso).

Due uscite di esecuzione:

- **True**: eseguita se la condizione è `true`.
- **False**: eseguita se la condizione è `false`.

Esempio: collegare un `Get` di una variabile `bool` al pin `Condition` del Branch. Sul ramo `True` inserire un `Print String` con messaggio `"la variabile è vera"`, sul ramo `False` uno con `"la variabile è falsa"`. Modificando il valore di default della variabile nei dettagli e mandando in Play, si vedrà il messaggio corrispondente.
Nota: selezionando più blocchi e premendo Q, si allineeranno. Utile per l'ordine.

2.16 Operatori Logici: AND, OR, NOT

Per combinare più condizioni booleane si usano gli operatori logici (reperibili cercando `and`, `or`, `not`):

- **AND**: restituisce `true` **solo se** entrambi gli ingressi sono `true`. Se anche uno solo è `false`, il risultato è `false`.
- **OR**: restituisce `true` se **almeno uno** degli ingressi è `true`. Solo se entrambi sono `false` il risultato è `false`.
- **NOT**: ha un solo ingresso e inverte il valore: `true` diventa `false`, `false` diventa `true`.

2.17 Esercizio: Combinare Booleani con Branch e Operatori

Obiettivo: utilizzare due variabili booleane (`bool` e `bool2`) per testare gli operatori logici.

1. Creare una seconda variabile booleana chiamata `bool2`.
2. In `Event BeginPlay`, inserire un `Set` della prima variabile `bool` (ad esempio impostandola a `true`).
3. Collegare l'esecuzione del `Set` a un nodo `Branch`.
4. Collegare il `Get` di `bool` al pin `Condition` del Branch.
5. Sul ramo `True` del Branch:
   - Inserire un nodo `AND` booleano. Collegare i `Get` di `bool` e `bool2` ai suoi ingressi.
   - Prendere un secondo `Branch` e collegare l'uscita dell'`AND` al suo pin `Condition`.
   - Sul ramo `True` di questo secondo Branch, mettere un `Print String` che mostri, ad esempio, il valore di `bool2` convertito in stringa (o un messaggio a piacere).
6. Ripetere lo stesso schema per l'operatore `OR`: sul ramo `True` del primo Branch (o in parallelo) aggiungere un nodo `OR` e un ulteriore Branch per testare il risultato.
7. Compilare, impostare i valori di default desiderati per `bool2` nei dettagli, e avviare il Play per verificare il comportamento.

Suggerimento: per mantenere ordinato il grafico, selezionare i nodi e premere `Q` per allinearli automaticamente.

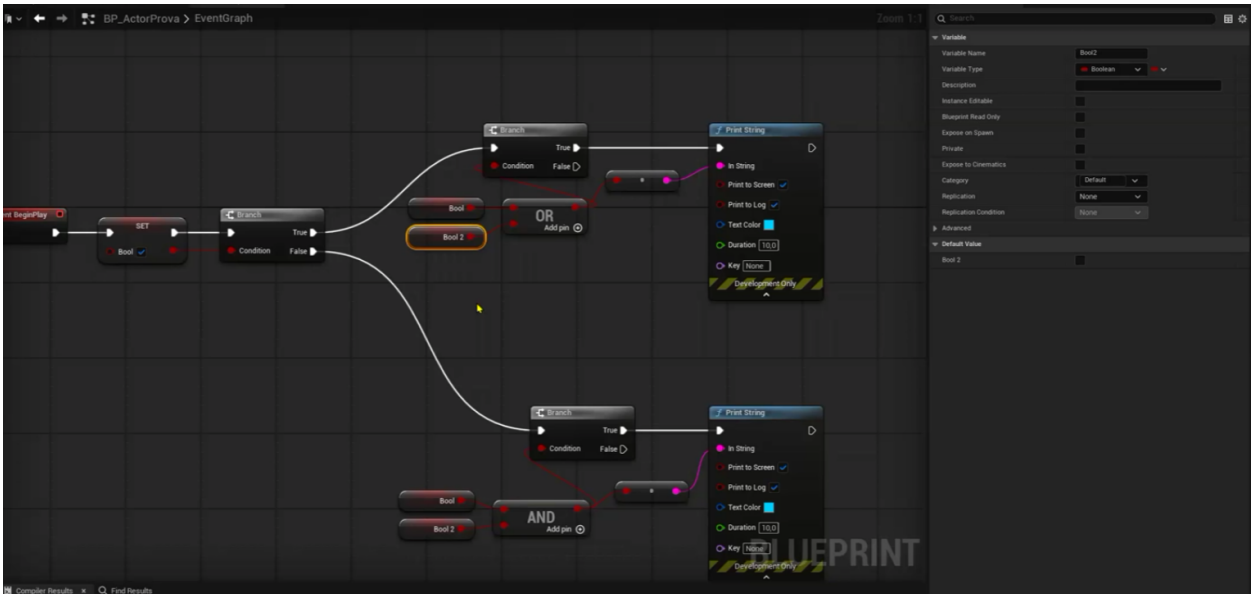


Figure 2.5: Codice considerato.

2.18 Esercitazione: Condizioni con Branch e Operatori Booleani

Nel precedente esercizio, dopo aver creato le variabili `bool` e `bool2` e aver collegato `Set`, `Branch` e operatori logici, è importante collegare correttamente i pin booleani:

- Il nodo `Set` di un `bool`, oltre al pin di esecuzione, espone anche un pin booleano (rosso) in uscita: esso rappresenta il **valore attuale** della variabile dopo l'impostazione.
- Tale pin va collegato al **pin di condizione** del nodo `Branch`, in modo che il bivio dipenda proprio dal valore appena impostato (e non da un altro `Get` scollegato).
- Se si utilizza solo un `Get` o una costante, il `Branch` valuterà quella condizione, ma potrebbe non riflettere il valore aggiornato se non collegato al flusso corretto.

Esempio di analisi del flusso:

- Con `bool` = `true` e `bool2` = `false`:
  - Il primo `Branch` (condizione = `bool`) va sul ramo `True`.
  - Sul ramo `True` è presente un operatore `OR` tra `bool` e `bool2`. Poiché `bool` è `true`, l'`OR` restituisce `true` e il successivo `Branch` esegue il ramo `True` (stampa il valore).
- Con `bool` = `false` e `bool2` = `false`:
  - Il primo `Branch` va sul ramo `False`.
  - Qui si trova un operatore `AND` tra `bool` e `bool2`. Entrambi sono `false`, quindi l'`AND` restituisce `false`: il `Branch` associato non stampa nulla.

Il codice considerato è visibile in Fig. 2.5.

2.19 Level Blueprint e Eventi di Input

2.19.1 Level Blueprint

Oltre ai `Blueprint Class` per i singoli actor, Unreal Engine mette a disposizione il **Level Blueprint**, una blueprint associata all'intero livello. Si apre dal menù `Blueprints` → `Open Level Blueprint`. Qui si

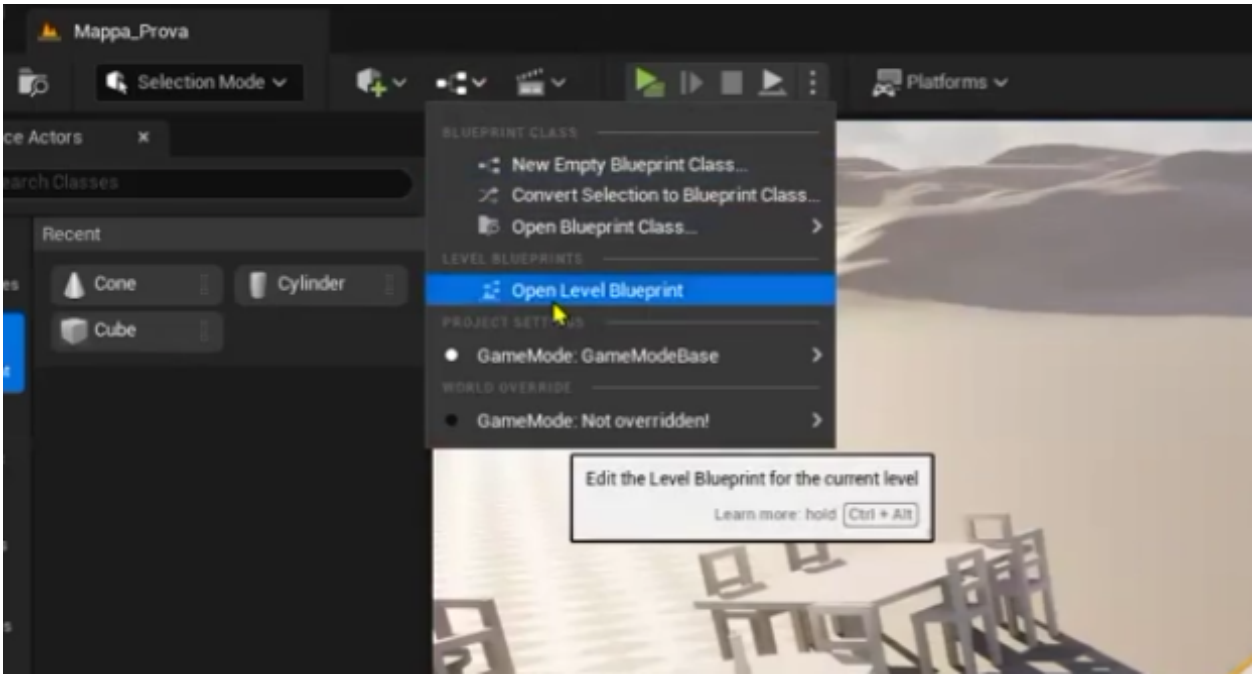


Figure 2.6: Visualizzazione della schermata per selezionare il Level Blueprint.

possono definire regole globali, spawnare oggetti e gestire input.

2.19.2 Evento di pressione tasto

- Tasto destro nell'Event Graph del Level Blueprint → cercare il tasto desiderato (es. `W`).
- Viene creato un nodo evento con l'icona della tastiera, che si attiva quando quel tasto è premuto.
- La proprietà `Consume Input` (tick) se attiva, **consuma** l'input, impedendo che venga gestito anche da altri sistemi (ad esempio il movimento del giocatore). Se la si disabilita, il giocatore potrà ancora muoversi con lo stesso tasto.

2.20 Referenza a un Actor nel Level Blueprint

Per interagire con un oggetto specifico (es. un cubo) dal Level Blueprint:

1. Selezionare l'actor nella Viewport.
2. Nel Level Blueprint, fare tasto destro e scegliere `Create a Reference to <nome actor>`.
3. Appare un nodo con il pin blu (riferimento all'actor) che può essere collegato al parametro `Target` di funzioni come `SetActorRotation`.

2.21 Rotazione Incrementale di un Actor

Obiettivo: far ruotare una piattaforma (un cubo) di un angolo costante ogni volta che si preme il tasto `W`.

2.21.1 Preparazione dell'oggetto: Mobility

L'actor deve essere impostato come **Movable** (nel pannello `Transform` → `Mobility`, Fig. 2.7). Le tre opzioni sono:

- **Static**: oggetto completamente immobile; luci/ombre precalcolate.
- **Stationary**: non può muoversi, ma può cambiare proprietà (es. colore).
- **Movable**: completamente dinamico, può essere spostato durante il gioco.

2.21.2 Metodo con GetActorRotation e SetActorRotation

1. Ottenere la rotazione attuale dell'oggetto tramite `GetActorRotation` (o `Get Actor Rotation` se si usa l'inglese). Collegare il riferimento al `Target`.
2. Il pin in uscita (lilla) è un `Rotator`. Per modificarne solo una componente, fare tasto destro sul pin e scegliere `Split Struct Pin`. Il nodo si espanderà nei tre pin `X`, `Y`, `Z` (tipo `float`).
3. Prendere il valore di `X` (Roll) e sommarlo a un offset (es. `5.0`) usando un nodo `Float + Float` (o `Add`).
4. Creare un nuovo `Rotator` con `X` modificata e `Y`, `Z` inalterati: si può usare `Make Rotator` oppure, dopo aver aggiunto l'offset, passare il float a `SetActorRotation` collegandolo al pin `New Rotation` e utilizzare un nodo `Break Rotator`/`Make Rotator` con split pin.

Nota: In alternativa, per semplicità didattica, si può usare `SetActorWorldRotation` con un nuovo `Rotator` creato manualmente (es. `X=5, Y=0, Z=0` per la pressione di `W` e `X=-5` per `S`), evitando di dover ottenere la rotazione corrente. Questo metodo però non è incrementale, ma imposta un valore assoluto; per ottenere un incremento occorrerebbe comunque sommare il valore corrente.

2.21.3 Debug con Print String

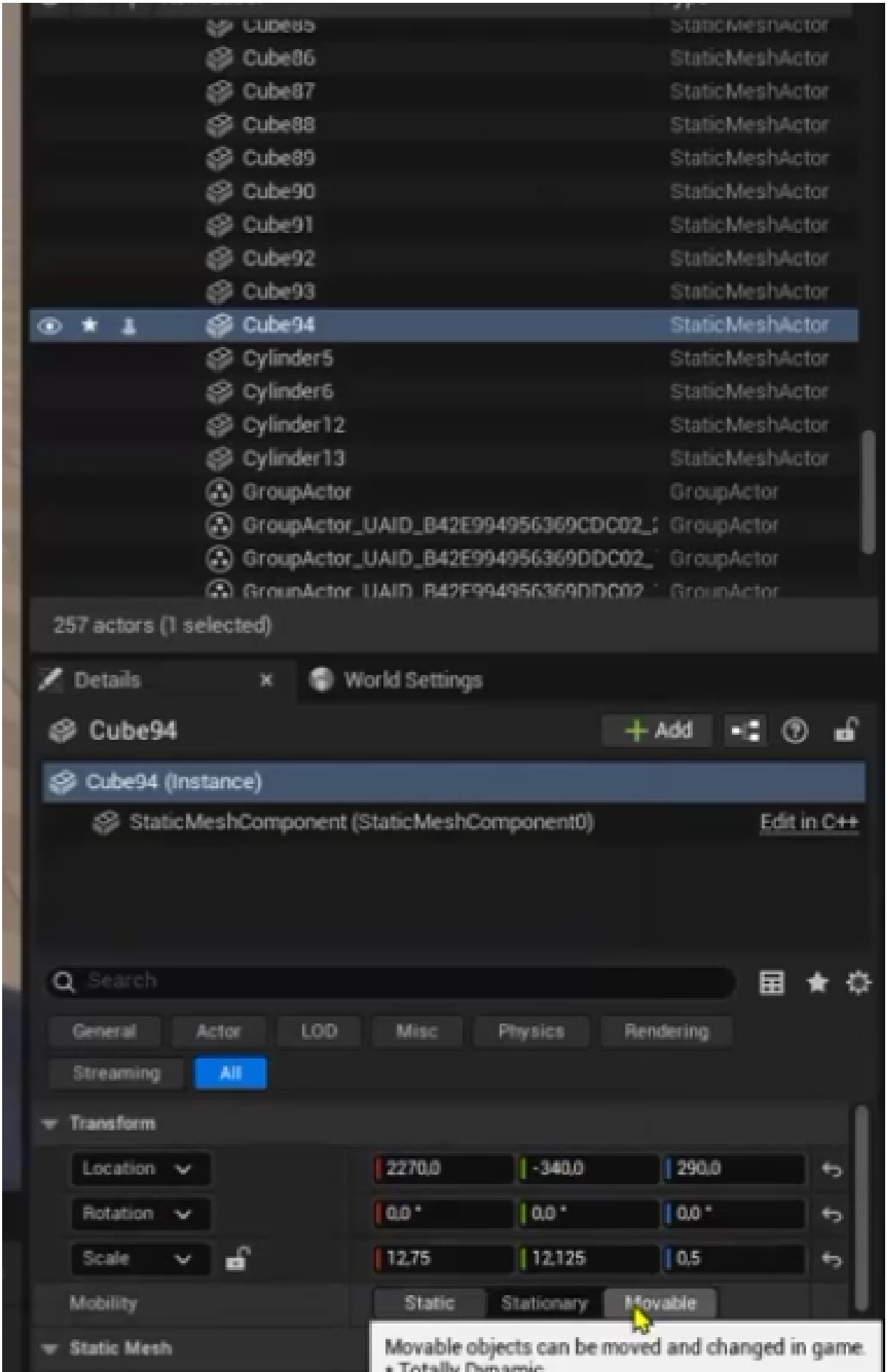
Per verificare che la rotazione aumenti a ogni pressione, si può collegare un `Print String` che mostri il valore di `X` ottenuto da `GetActorRotation` (dopo split). Premendo più volte `W`, si vedrà la sequenza 0, 5, 10, 15...

2.22 Troubleshooting: La piattaforma ruota una volta sola

Se la piattaforma si muove solo al primo input e poi resta ferma, verificare:

- Che l'oggetto sia **Movable**.
- Che non vi siano problemi di collisione che impediscono il movimento (sollevare la piattaforma da terra).
- Che la logica stia effettivamente recuperando la rotazione **corrente** e non stia impostando sempre lo stesso valore fisso. Se si usa `SetActorRotation` con una costante, il movimento è assoluto e non incrementale.
- Usare un nodo `Print String` per stampare il valore prima di impostarlo, per capire se la rotazione viene letta correttamente.
- Impostare un breakpoint (tasto `F9` sul nodo `SetActorRotation`) per vedere se l'esecuzione arriva correttamente a ogni pressione del tasto.

Approccio semplificato risolutivo: Invece di `GetActorRotation`, usare `SetActorWorldRotation` con un `Rotator` costruito manualmente (es. `Roll = 5` per `W`, `Roll = -5` per `S`). Sebbene non incrementale, permette di testare immediatamente il collegamento input-rotazione senza problemi di accumulo.


Cube86 StaticMeshActor
Cube87 StaticMeshActor
Cube88 StaticMeshActor
Cube89 StaticMeshActor
Cube90 StaticMeshActor
Cube91 StaticMeshActor
Cube92 StaticMeshActor
Cube93 StaticMeshActor
Cube94 StaticMeshActor
Cylinder5 StaticMeshActor
Cylinder6 StaticMeshActor
Cylinder12 StaticMeshActor
Cylinder13 StaticMeshActor
GroupActor GroupActor
257 actors (1 selected)
Details
World Settings
Cube94
Add
Cube94 (Instance)
StaticMeshComponent (StaticMeshComponent0)
Edit in C++
Search
General
Actor
LOD
Misc
Physics
Rendering
Streaming
All
Transform
Location
2270.0
-340.0
290.0
Rotation
0.0 °
0.0 °
0.0 °
Scale
12.75
12.125
0.5
Mobility
Static
Stationary
Movable
Static Mesh
Movable objects can be moved and changed in game.

2.23 Rotazione nel Mondo con AddActorWorldRotation

Per semplificare l'esercizio precedente, si utilizza il nodo `AddActorWorldRotation` (o `Add Actor World Rotation`) che modifica la rotazione dell'actor nello spazio **globale** (mondo), senza dipendere dall'orientamento locale.

2.23.1 Movimento Avanti/Indietro (W/S)

- Creare due eventi di pressione tasto: uno per `W` e uno per `S`.
- In ciascuno, collegare un nodo `AddActorWorldRotation` con il target (riferimento al cubo).
- Nel nodo, splittare il pin `Delta Rotation` (tasto destro → `Split Struct Pin`) per esporre i tre assi `X`, `Y`, `Z`.
- Per `W`: impostare `X = 5`, `Y = 0`, `Z = 0` (rotazione in avanti).
- Per `S`: impostare `X = -5`, `Y = 0`, `Z = 0` (rotazione all'indietro).

2.23.2 Movimento Destra/Sinistra (A/D)

Per la rotazione laterale, l'asse da modificare è `Y` (rollio laterale).

- Creare eventi per i tasti `A` e `D`.
- Usare `AddActorWorldRotation` con split pin.
- Per `D` (destra): `X = 0`, `Y = -5`, `Z = 0` (osservando l'orientamento nella viewport, la rotazione a destra corrisponde a un valore negativo sull'asse `Y` globale).
- Per `A` (sinistra): `X = 0`, `Y = 5`, `Z = 0`.

Nota: verificare sempre il corretto collegamento del riferimento all'actor nel campo `Target`. Script presente in Fig. 2.8.

2.24 Fisica: Simulate Physics e Opzione Teleport

2.24.1 Attivare la fisica su un oggetto

- Selezionare un actor (es. una sfera) nella viewport.
- Nel pannello `Details`, sotto la sezione `Physics`, spuntare `Simulate Physics`.
- L'oggetto ora risponde alla gravità e alle collisioni.

2.24.2 Interazione piattaforma-oggetto fisico

Se una piattaforma in movimento colpisce un oggetto fisico (es. una sfera) senza ulteriori accorgimenti, la fisica potrebbe causare reazioni violente (la sfera viene scagliata via). Per evitare ciò, si può abilitare il **teleport** nel menù (outliner) dell'oggetto considerato, e.g. una sfera.

- Nei nodi `AddActorWorldRotation` (o qualsiasi nodo di spostamento) è presente una freccia in basso che espone parametri avanzati.
- Spuntare l'opzione `Teleport` (o `Teleport Physics`).
- Con il teleport attivo, lo spostamento viene considerato istantaneo e non genera forze fisiche sull'oggetto in movimento, impedendo di scagliare via altri corpi.

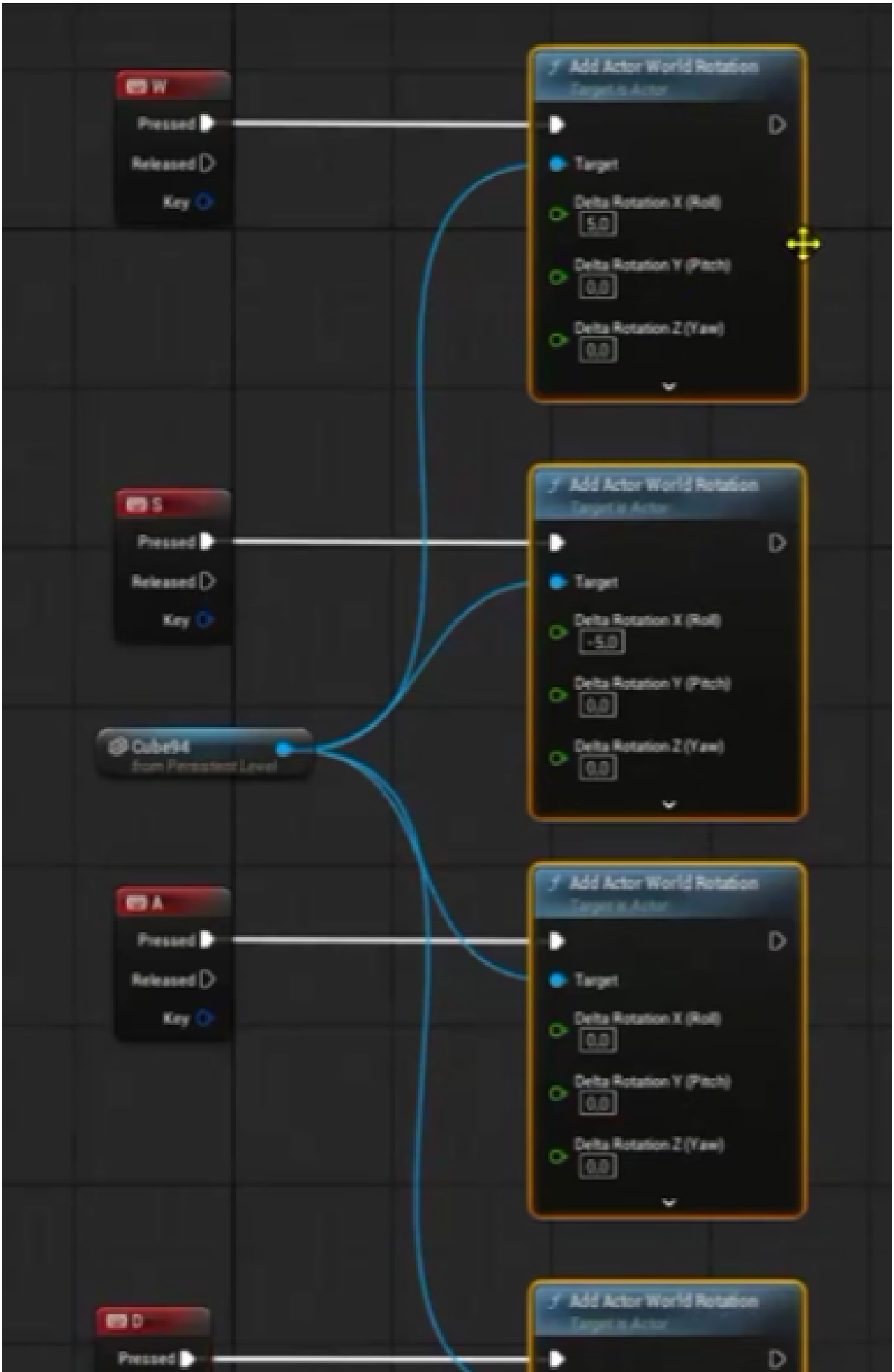

W
Pressed
Released
Key
Add Actor World Rotation
Target is Actor
Target
Delta Rotation X (Roll)
5.0
Delta Rotation Y (Pitch)
0.0
Delta Rotation Z (Yaw)
0.0
S
Pressed
Released
Key
Add Actor World Rotation
Target is Actor
Target
Delta Rotation X (Roll)
-5.0
Delta Rotation Y (Pitch)
0.0
Delta Rotation Z (Yaw)
0.0
Cube94
from Persistent Level
A
Pressed
Released
Key
Add Actor World Rotation
Target is Actor
Target
Delta Rotation X (Roll)
0.0
Delta Rotation Y (Pitch)
0.0
Delta Rotation Z (Yaw)
0.0
D
Pressed
Add Actor World Rotation
Target is Actor

Figure 2.9: Per visualizzare il Teleport, estendere il blueprint

2.25 Troubleshooting: Piattaforma non si muove

Se la piattaforma non ruota nonostante i collegamenti siano corretti:

- Verificare che l'oggetto abbia `Mobility` impostata su **Movable** (nel pannello `Transform`).
- Se impostato su `Static` o `Stationary`, l'actor non può essere spostato a runtime.
- Dopo aver modificato la mobilità, compilare e riavviare il Play.

2.26 Event Tick e Delta Seconds

2.26.1 Event Tick

Il nodo `Event Tick` viene eseguito **ad ogni frame**. Ciò che è collegato alla sua uscita di esecuzione viene processato continuamente.

2.26.2 Delta Seconds

Il pin `Delta Seconds` fornisce l'intervallo di tempo (in secondi) dall'ultimo frame. Per ottenere un movimento **frame-rate independent**, moltiplicare ogni variazione per `Delta Seconds`:

$$\text{Nuova velocità} = \text{Valore} \times \Delta t$$

Così la velocità percepita rimane costante sia a 30 FPS che a 144 FPS. Se non si moltiplica, l'oggetto si muoverà più velocemente su macchine con frame rate più alto.

2.27 Input del Mouse per la Rotazione

Invece di usare i tasti WASD, è possibile collegare la rotazione al movimento del mouse.

- `Get Mouse X` e `Get Mouse Y` (trovabili cercando `mouse x`) restituiscono il movimento orizzontale e verticale del mouse.
- Si possono collegare direttamente ai pin `X` e `Y` di un `AddActorWorldRotation` (dopo split) per muovere la piattaforma con il mouse.
- Se la direzione del movimento risulta invertita, moltiplicare il valore per `-1` (nodo `Multiply` con `-1`) per invertire il senso.

Osservazione: i movimenti sugli assi potrebbero apparire scambiati a seconda dell'orientamento della camera. In tal caso, invertire i collegamenti `X` e `Y`.

2.28 Clamp dei Valori (Introduzione)

Il nodo `Clamp Float` (cercare `clamp float`) permette di limitare un valore all'interno di un intervallo definito. Prende in ingresso:

- `Value` (il valore da limitare)
- `Min` e `Max` (i limiti inferiore e superiore)

E restituisce il valore *clampato*: se `Value` è inferiore a `Min`, restituisce `Min`; se superiore a `Max`, restituisce `Max`; altrimenti restituisce `Value`. Utile per impedire che la rotazione superi certi angoli, rendendo il controllo più stabile. L'argomento sarà approfondito successivamente.

2.29 Clamp dei Valori

Il nodo `Clamp Float` (o `Clamp`) serve a limitare un valore all'interno di un intervallo chiuso $[\min, \max]$.

- **Parametri:** `Value` (il valore da limitare), `Min`, `Max`.
- **Funzionamento:** se Value $<$ Min restituisce `Min`; se Value $>$ Max restituisce `Max`; altrimenti restituisce `Value`.

Nel nostro caso viene usato per impedire alla piattaforma di ruotare oltre un angolo prestabilito (es. $-20°$ e $+20°$).

2.30 Rotazione Guidata dal Mouse con Clamp

Obiettivo: controllare l'inclinazione della piattaforma con il movimento del mouse, limitandone l'escursione.

2.30.1 Ottenere i valori del mouse

- `Get Mouse Y` (movimento verticale) e `Get Mouse X` (movimento orizzontale) restituiscono uno spostamento scalare (tipo `float`).
- Se il verso di controllo risulta invertito, moltiplicare per -1 tramite un nodo `Multiply`.

2.30.2 Somma alla rotazione corrente e clamp

1. Prendere la rotazione attuale del cubo: nodo `GetActorRotation` → `Break Rotator` (oppure `Split Struct Pin`) per esporre gli assi `X`, `Y`, `Z`.
2. Sommare il valore del mouse all'asse desiderato:
   - Per l'inclinazione avanti/indietro: `Get Mouse Y` sommato all'asse `Y` (Roll).
   - Per l'inclinazione laterale: `Get Mouse X` sommato all'asse `X` (Pitch).

Figure 2.10: Codice finale per far roteare un rettangolo col mouse.

3. Applicare `Clamp Float` al risultato della somma, con min $= -20$, max $= 20$ (o valori desiderati).
4. Ricostruire il `Rotator` con i nuovi valori clampati usando `Make Rotator` (pin: `X`, `Y`, `Z`).
5. Impostare la rotazione finale con `SetActorRotation` (non `SetActorWorldRotation`), collegando il nuovo `Rotator` e il `Target` (referenza al cubo). Abilitare `Teleport Physics` per evitare impulsi indesiderati sugli oggetti fisici.

Nota: `SetActorWorldRotation` tendeva a far scattare la piattaforma alla posizione iniziale dopo il movimento; `SetActorRotation` risolve il problema.
Il codice finale è visibile in Fig. 2.10

2.31 Costruzione della Vaschetta (Attach To)

Per realizzare un piano inclinabile con bordi che seguano il movimento:

- Creare dei cubi (muretti) attorno alla piattaforma principale.
- Selezionare un muretto, poi nel menù contestuale (tasto destro) scegliere `Attach To` e con il contagocce selezionare il cubo principale (la piattaforma). In alternativa, nel pannello `Details` cercare l'opzione `Attach to Actor`.
- Ripetere per ogni muretto. Ora tutti i cubi figli si muoveranno solidalmente al genitore.
- Per il soffitto invisibile (o un muro superiore), è sufficiente aggiungere un cubo sopra, attaccarlo, e impostare `Hidden in Game` (nel pannello `Details` → `Rendering`) affinché la mesh non sia visibile ma la collisione rimanga attiva.

2.32 Problemi Comuni e Soluzioni

- **La piattaforma non si muove:** verificare che il `Target` di `SetActorRotation` sia effettivamente la referenza al cubo (trascinare il nodo di riferimento sul pin `Target`).
- **Scatto al rilascio del mouse / ritorno in piano:** accertarsi di usare `SetActorRotation` e non `SetActorWorldRotation`.
- **La palla scompare o compenetra le pareti:** può accadere a causa della fisica real-time; sollevare leggermente la palla o ridurre la velocità di rotazione.

2.33 Compiti e Prossima Lezione

- Provare a costruire un piccolo percorso / labirinto con la vaschetta, aggiungendo ostacoli interni (sempre attaccati alla piattaforma).
- La prossima lezione con Davide introdurrà i **materiali**, per dare colore e aspetto grafico agli oggetti.
- Scaricare il materiale didattico fornito in modo da poter seguire insieme la lezione.

Chapter 3

Introduzione alla Grafica in Unreal Engine 5: Texture, Materiali e Importazione di Mesh 3D

3.1 Introduzione alla lezione: il ruolo del grafico in Unreal Engine

La sessione odierna si concentra sugli aspetti visivi e sul lavoro che un artista 3D svolge all'interno di Unreal Engine. È importante sottolineare che la maggior parte della produzione artistica (modellazione, UV mapping, creazione delle texture) avviene in software esterni come Blender o Maya. In Unreal il grafico si occupa principalmente di importare gli asset e creare gli shader tramite l'editor dei materiali.

3.2 Materiale didattico e progetto di riferimento

Sul server Discord è disponibile il materiale didattico, che include un piccolo progetto Unreal preconfigurato e diversi asset. La lezione si basa su uno di questi asset, una **torretta**, per esplorare i concetti di *mesh*, *texture* e *materiali*. Il progetto di esempio verrà utilizzato anche nelle lezioni successive per integrare altri elementi come l'audio.

3.3 Le texture: fondamenti teorici

3.3.1 Definizione e ruolo

Una texture può essere immaginata come la **carta regalo** che avvolge un oggetto 3D, cioè una serie di mappe che, applicate a una mesh, forniscono informazioni specifiche sull'aspetto della superficie. Nell'immagine di esempio, la texture della torretta appare scomposta in varie sezioni che corrispondono a diverse parti del modello.

3.3.2 Il processo di UV Mapping

Perché una texture funzioni correttamente, deve essere progettata sulle **coordinate UV** del modello. L'UV mapping è la tecnica che assegna a ogni vertice della geometria 3D una posizione su una mappa 2D, permettendo alla texture di avvolgersi sul modello senza distorsioni. Se si applica una texture non mappata per le UV di quell'oggetto, il risultato apparirà stirato o errato.

3.3.3 Tipologie di mappe

Una singola torretta può richiedere diverse texture, ciascuna con un compito preciso. Ogni tipo di mappa viene convenzionalmente indicato con una nomenclatura specifica, ad esempio `T_NomeOggetto_Lettera` (dove la lettera indica il tipo: `_A` per Albedo, `_D` per Diffuse, ecc.).

Diffuse o Albedo

Fornisce le informazioni sul **colore** di base. Contiene tutti i dettagli cromatici dell'oggetto, come l'arancione della base della torretta o il nero delle parti meccaniche.

Normal Map

Simula la **tridimensionalità** senza alterare la geometria reale. Riconoscibile per il caratteristico colore blu con sfumature rosse e verdi, inganna l'occhio facendo credere che esistano rilievi e incavi (come i mattoni di un muro). Quando il motore calcola luci e ombre, interpreta la normal map per proiettare le ombre in modo da rendere l'illusione di profondità. È fondamentale per aggiungere dettagli come graffi, rughe o bulloni senza appesantire il modello con poligoni extra.

Specular

Definisce la **riflettività** e la brillantezza della superficie. Viene rappresentata in scala di grigi: il **bianco** equivale al valore 1 (massima riflessione), il **nero** allo 0 (nessuna riflessione), mentre i grigi sono valori intermedi. Determina come l'oggetto reagisce alla luce, differenziando ad esempio il metallo dalla pietra.

Light Map (Height Map)

A differenza della normal map, modifica **effettivamente** la geometria della mesh durante il rendering, sollevando o abbassando alcune zone (displacement). Le aree bianche indicano protuberanze, quelle nere rientranze. Aggiunge profondità reale ma è computazionalmente più costosa.

Procedural Texturing

Non si tratta di una mappa specifica ma di un **metodo** per generare texture e materiali tramite **algoritmi matematici**. In Unreal Engine questo processo è gestito dall'editor dei materiali, che opera attraverso nodi, in modo analogo ai Blueprint.

3.3.4 La logica dei valori e il multi-canale

Tutte le mappe in scala di grigi operano su un intervallo tra 0 (nero) e 1 (bianco). Questo principio consente di compattare più mappe in un'unica texture sfruttando i canali di colore **RGB**. Ogni canale (Rosso, Verde, Blu) può trasportare un'informazione indipendente in bianco e nero (0–1). Una pratica comune nel settore è l'uso di una mappa combinata chiamata **AORM** (Ambient Occlusion, Roughness, Metallic), che riunisce in un solo file tre informazioni fondamentali, riducendo il numero di asset da gestire e migliorando l'efficienza del progetto.

3.4 I materiali in Unreal Engine

3.4.1 Dalla texture al materiale

All'interno di Unreal Engine non si applicano direttamente le texture agli oggetti, ma si creano dei **materiali**. Il materiale può essere immaginato come la **vernice** definitiva che riveste la mesh. Esso combina le diverse texture (diffuse, normal, specular, ecc.) e fornisce al motore di rendering tutte le istruzioni su come l'oggetto interagisce con la luce e appare visivamente.

3.4.2 L'editor dei materiali

L'editor dei materiali utilizza un sistema a **nodi**, proprio come l'editor dei Blueprint. I singoli nodi (che possono rappresentare texture, costanti numeriche, operazioni matematiche) vengono collegati tra loro per convogliare le informazioni fino al **nodo risultante** del materiale. La struttura a nodi determina il comportamento finale dello shader.

3.4.3 Impostazioni del materiale

Ogni materiale possiede un pannello `Details` con diverse proprietà. Le impostazioni più importanti sono quelle che definiscono la **struttura fondamentale** del materiale stesso, ovvero le sue capacità e il modo in cui viene interpretato dal motore (ad esempio, se è un materiale per superfici opache, trasparenti, per effetti speciali, ecc.). Queste impostazioni influenzano direttamente le possibilità espressive e tecniche del materiale e verranno approfondite nella parte pratica.

3.5 Material Domain

Il **material domain** definisce lo scopo d'uso del materiale. Ad esempio:

- Superfici
- Interfacce (UI)
- Post-processing
- Effetti visivi

3.6 Blend Mode

La **blend mode** stabilisce come il materiale si fonde con i pixel già calcolati nella scena.

- **Opaco (default)**: occlude completamente ciò che sta dietro.
- **Translucent**: permette di aggiungere parametri di opacità. Utile per effetti come fuoco o erba che sfuma verso l'alto.
- **Masked**: crea una maschera che si comporta diversamente dal resto della superficie.

3.6.1 Effetti della Blend Mode sui pin del nodo risultante

- Con blend mode **opaca** non è presente il pin `Opacity` (l'oggetto è sempre opaco).
- Passando a **translucent** vengono rimossi i pin `Metallic`, `Specular`, `Roughness`, `Anisotropy` (non ha senso calcolare la riflessione della luce su un materiale potenzialmente trasparente). Compare invece il pin `Opacity`.

3.7 Shading Model

Lo **shading model** definisce come il materiale reagisce alla luce.

- `Default Lit`: segue le regole base dell'illuminazione realistica.
- `Unlit`: la luce non influenza la resa dell'effetto (utile per effetti speciali).

Esistono diverse varianti di shading model applicabili al nodo risultante.

3.8 Editor dei materiali in Unreal Engine

3.8.1 Creazione di un nuovo materiale

1. Nel `Content Browser`, fare tasto destro → `New Material`.
2. Rinominare (es. `New Material`).
3. Doppio clic per aprire l'editor del materiale.

3.8.2 Impostazioni nel pannello dei dettagli

Le impostazioni discusse si trovano nella parte inferiore dell'editor del materiale (Material Property Overrides).

3.8.3 Material Expression Nodes

I **Material Expression Nodes** sono piccoli set di istruzioni preprogrammate, analoghi ai nodi dei Blueprint.

- `Texture Sample`: comunica l'informazione di una texture (es. base color).
- `Texture Coordinate` (rosso): determina le coordinate UV e il tiling della texture.
- `Multiply`: moltiplica due valori (A × B).
- `Constant` (quadratino): valore float singolo.

Combinando questi nodi si ottengono risultati diversi: moltiplicazioni, interpolazioni, movimenti di texture per simulare animazioni, ecc.

3.8.4 Parametri

Qualsiasi nodo (non solo le costanti) può essere convertito in **parametro**:

- Tasto destro sul nodo → `Convert to Parameter`.
- I parametri permettono di modificare i valori nelle *Material Instances* senza dover ricompilare il materiale originale.

3.8.5 Shortcut utili nell'editor dei materiali

- `T` + clic sinistro: `Texture Sample`
- `1` clic sinistro: costante a 1 vettore (float)
- `2` clic sinistro: costante a 2 vettori
- `3` clic sinistro: costante a 3 vettori (colore RGB)
- `A` + clic sinistro: `Add`
- `M` + clic sinistro: `Multiply`
- `D` + clic sinistro: `Divide`
- `Alt + clic sinistro` su una connessione: rompe il collegamento

3.8.6 Utilizzo dei canali di una texture

Aprendo una texture (la si ha in intro UE di AIV, in Content del Content Browser) è possibile visualizzare i singoli canali (R, G, B). Nel nodo `Texture Sample` sono disponibili output separati per ogni canale, utili per trasportare informazioni specifiche (es. collegare il canale rosso al pin `Specular` per controllare quali parti riflettono più luce).

3.8.7 Esempio pratico: Emissive Color e moltiplicazione

- Il pin `Emissive Color` determina quali parti del materiale emettono luce (glow).
- Per aumentare l'intensità dell'emissivo, si moltiplica il valore per una costante:

```
[Colore] -> [Multiply] -> Emissive Color
             |
             [Constant (es. 100)]
```

- Convertendo la costante in parametro (es. `Glow Intensity`) sarà possibile modificarla direttamente nelle Material Instanze.

3.9 Input chiave per la superficie: Metallic, Specular, Roughness

Questi input determinano l'aspetto della superficie del materiale secondo il modello **PBR** (Physically Based Rendering).

- `Roughness`: valore da 0 a 1.
  - **1**: superficie molto ruvida (es. roccia) – non riflette la luce.
  - **0**: superficie liscia (es. specchio) – massima riflettività.
- `Specular`: determina quanta luce viene riflessa e l'angolazione della riflessione.
- `Metallic`: definisce il comportamento metallico del materiale.
- `Emissive Color`: fa emettere luce al materiale (già visto).
- `Normal`: richiede una **normal map** per creare l'illusione del dettaglio tridimensionale su una superficie 2D.

3.9.1 Esempio: creazione di uno specchio

Per uno specchio servono:

```
Roughness = 0
Specular = 1 (o valori alti)
```

3.10 Collegare una texture al nodo Texture Sample

Nel pannello dei `Details` del nodo `Texture Sample`:

- Cliccare sulla freccia accanto all'anteprima della texture.
- `Use selected asset from content browser`: se si ha già selezionata una texture nel Content Browser, viene automaticamente assegnata.
- Cliccare sul nome della texture per fare `Browse` e cercare l'asset desiderato (es. `T_Bark`).

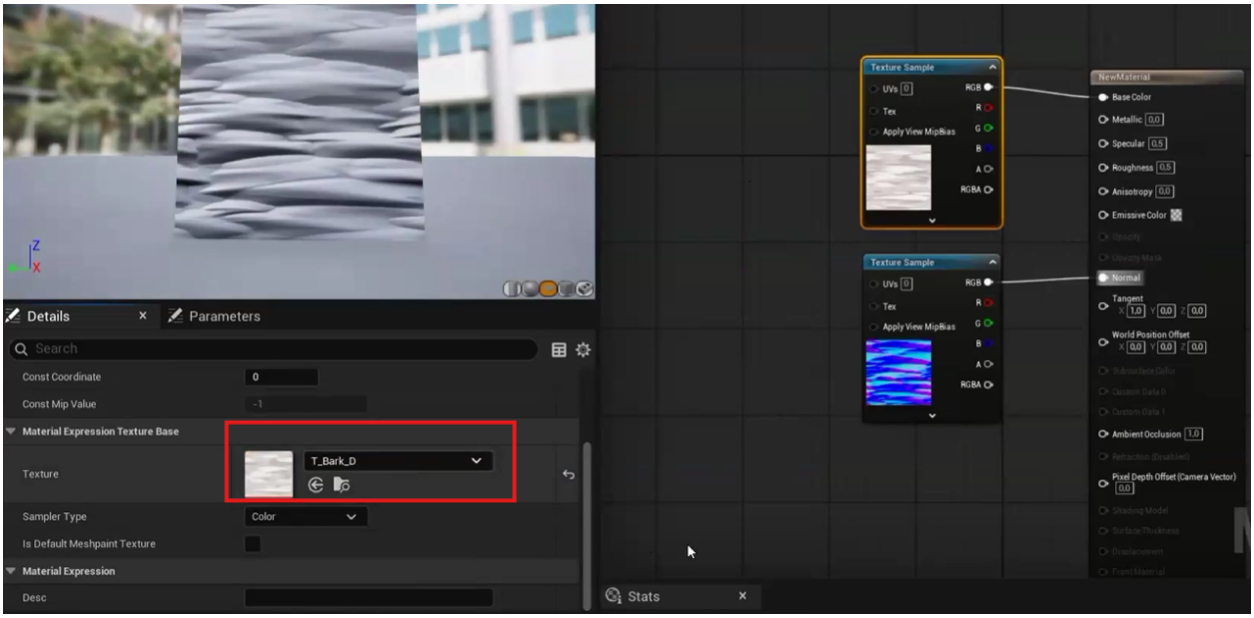


Figure 3.1: Come selezionare la texture voluta. Si può anche scrivere il nome, filtrando i risultati.

3.11 Nodo Texture Coordinate (UV)

- **Shortcut**: `U` + clic sinistro (per ricordare che lavora sulle UV).
- Parametri principali: `U Tiling` e `V Tiling`.
- Modificando questi valori si controlla la ripetizione della texture sulla superficie.

3.11.1 Esempi di tiling

Nei details di un materiale è possibile selezionare il tiling.

- `U Tiling = 2, V Tiling = 1`: la texture si ripete 2 volte sull'orizzontale, 1 volta sul verticale.
- `U Tiling = 2, V Tiling = 2`: ripetizione 2×2 (4 volte).
- `U Tiling = 0.5`: la texture viene "zoomata" (appare meno volte, come se fosse ingrandita).

3.12 Applicare un materiale a un oggetto nella scena

1. Selezionare la mesh (es. un pavimento).
2. Nel pannello `Details`, sotto `Materials`, trascinare il materiale desiderato nel campo `Element 0`.
3. Il materiale viene applicato immediatamente.

3.13 Material Instance vs Master Material

3.13.1 Master Material

È l'intero grafo dei nodi che definisce il materiale. Modificarlo richiede ricompilazione.

3.13.2 Material Instance

- **Creazione**: tasto destro sul Master Material → `Create Material Instance`.
- Non contiene un grafo modificabile, ma espone i parametri (texture, valori float, colori, etc.) precedentemente convertiti a *parameter*.
- **Vantaggio prestazionale**: se più oggetti condividono la stessa Material Instance, l'engine carica una sola volta il Master Material sottostante. Per 10 000 oggetti con lo stesso materiale, si ha un solo caricamento del materiale invece di 10 000.
- Permette di personalizzare velocemente le superfici senza ricompilare.

3.13.3 Convertire nodi in parametri

1. Nel grafo del Master Material, tasto destro sul nodo (es. `Texture Sample`, costante, moltiplicatore) → `Convert to Parameter`.
2. Assegnare un nome (es. `BaseColorTex`, `NormalMap`, `TilingMultiplier`).
3. Salvare il Master Material.
4. Aprire la Material Instance creata: i parametri compaiono nel pannello `Details` e possono essere modificati (es. cambiare texture, alterare il tiling).

Ovviamente il materiale è da inserire in Material di un oggetto selezionato.

3.13.4 Esempio pratico: tiling variabile per istanza

- Nel Master Material: `Texture Coordinate` → `Multiply` → collegato agli UV delle texture.
- Una costante (es. 1) viene moltiplicata e convertita in parametro `TilingMultiplier`.
- Nella Material Instance si può impostare `TilingMultiplier = 10` per una superficie e `= 3` per un'altra, usando due istanze diverse dello stesso Master Material.

3.14 Material Functions

- Raccolgono blocchi di calcoli complessi o ripetuti.
- Possono essere riutilizzate in più materiali e condivise tra team.
- Non verranno usate in questa lezione, ma è utile conoscerle.

3.15 Nodi utili aggiuntivi: Lerp e Panner

3.15.1 Linear Interpolation (Lerp)

- **Shortcut**: `L` + clic sinistro.
- Sintassi: `Lerp(A, B, Alpha)` dove `Alpha` $\in [0,1]$.
  - Alpha = 0 → output = A
  - Alpha = 1 → output = B
  - Valori intermedi → interpolazione lineare.
- Può lavorare su colori, float, texture.

Esempio con due colori

```
A = Blu (RGB 0,0,1)
B = Rosso (1,0,0)
Alpha = 0.5 → Viola
```

Utile per mischiare texture: l'Alpha può provenire da un canale di una texture (es. canale rosso come maschera).

3.15.2 Panner (Movimento texture)

- **Shortcut**: `P` + clic sinistro.
- Fa muovere le texture nel tempo (utile per acqua, fuoco, etc.).

3.16 Shortcut riepilogativi

`L` + clic sinistro	Lerp
`P` + clic sinistro	Panner
`S` + clic sinistro	Parameter (utile per sottrazione)
`1,2,3` clic sinistro	Costanti (1=float, 2=2 vettori, 3=colore RGB)
`U` + clic sinistro	Texture Coordinate
`T` + clic sinistro	Texture Sample
`A` + clic sinistro	Add
`M` + clic sinistro	Multiply
`D` + clic sinistro	Divide

3.17 Utilizzo pratico di Lerp e Panner

3.17.1 Lerp con texture

Oltre a utilizzare colori costanti, il nodo `Lerp` può essere alimentato con due texture diverse. L'alpha (valore o canale di una terza texture) determina la prevalenza dell'una o dell'altra. Ad esempio, collegando una texture al posto del colore blu e un'altra al posto del rosso, l'alpha decide in quali zone della superficie prevale la prima o la seconda texture (Fig. 3.2.

3.17.2 Panner (movimento texture)

- **Shortcut**: `P` + clic sinistro.
- Il nodo `Panner` modifica le coordinate UV nel tempo, creando l'illusione del movimento.
- Parametri principali: `Speed X` e `Speed Y` (es. 0.3 su X, 0.1 su Y).
- Di default utilizza il *game time* (movimento continuo).
- Tipiche applicazioni: acqua, fluidi, qualsiasi superficie animata.

Vedi Fig. 3.3.

3.17.3 Relazione tra Master Material e Material Instance

Modificando un parametro nel Master Material (es. il valore di tiling o le speed del Panner), tutte le Material Instance che ne derivano si aggiornano automaticamente in tempo reale, senza bisogno di ricompilazione.

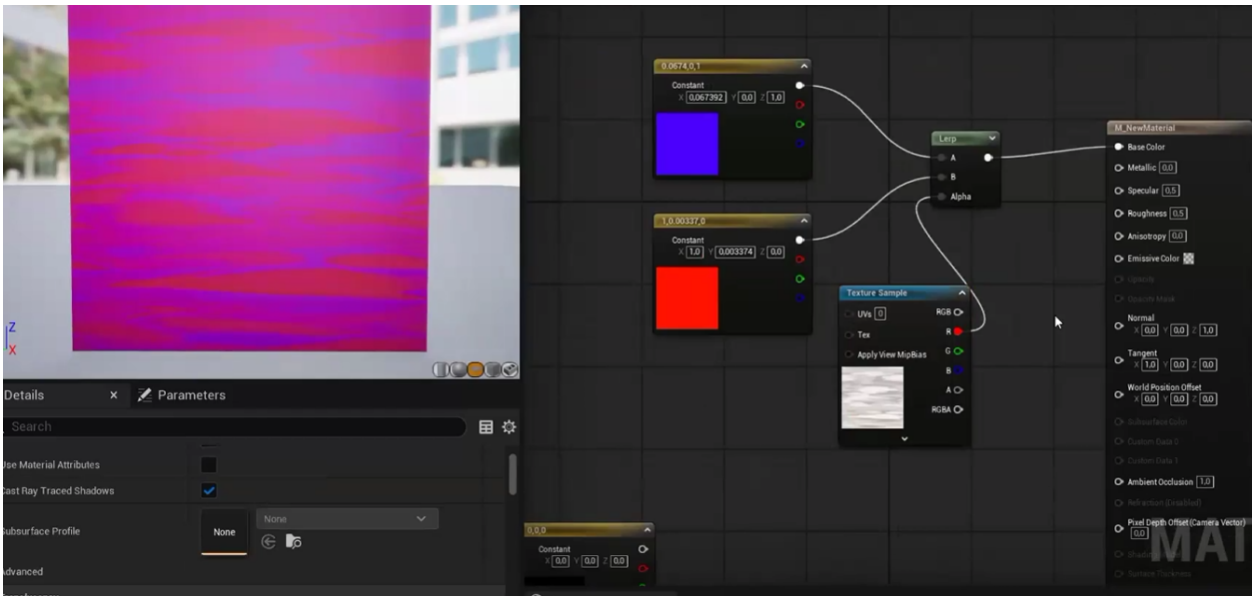


Figure 3.2: Bluepring per LERP

Figure 3.3: Bluepring per panner

3.18 Lavoro da remoto e versionamento (Git)

- Per lavorare in team su Unreal Engine si consiglia l'uso di repository Git (o altri sistemi di source control).
- Git è lo strumento più comune anche per Unity; con Unreal richiede attenzione in più ma è perfettamente supportato.
- Esistono plugin e soluzioni VPN/LAN per la modifica collaborativa in tempo reale dello stesso livello, ma l'approccio standard è lavorare su branch separati e unire le modifiche.

3.19 Importazione mesh: finestra Import Settings

Quando si importa un file FBX (contenente mesh statiche o scheletriche. FBX import Options), si apre una finestra con numerosi parametri:

- **Collisioni**: generazione automatica se non presenti nell'FBX.
- **Transform**: modifica di posizione, rotazione e scala del pivot.
- **Conversione unità**: adattamento da unità del software di origine a centimetri (unità di Unreal).
- **Force Front X-Axis**: cambio dell'asse considerato come "davanti".
- Per **skeletal mesh**: possibilità di associare uno scheletro o importare solo animazioni.
- Sezioni separate per `Import Materials` e `Import Textures` (in questa esercitazione si consiglia di deselezionarle per ricreare i materiali a mano).

3.19.1 Esempio: importazione della torretta

1. Nel Content Browser, premere `Import` e selezionare i tre file FBX: `SM_TurretBase`, `SM_TurretCannon`, `SM_TurretPivot`.
2. Nella finestra di import, deselezionare `Import Materials` e `Import Textures`.
3. Premere `Import`. Le tre mesh vengono caricate nel progetto.

3.20 Mesh: vertici, spigoli, facce

Una mesh è composta da:

- **Vertici**: punti nello spazio (angoli dell'oggetto).
- **Spigoli**: segmenti che collegano due vertici.
- **Facce**: superfici piatte delimitate da spigoli (tipicamente triangoli, forma più efficiente per il calcolo).

L'insieme dei triangoli forma poligoni, che a loro volta costituiscono la superficie della mesh.

3.20.1 Visualizzazione wireframe

Nell'editor di Unreal è possibile visualizzare la struttura a filo di ferro (wireframe) per osservare vertici, spigoli e facce.

3.20.2 Accorgimenti per l'importazione

- **Unità di misura**: Unreal utilizza i centimetri. Software come Blender usano altre unità; è necessario scalare correttamente la mesh prima dell'export.
- **Formato consigliato**: FBX (supporto nativo e affidabile). OBJ e altri formati possono dare problemi.
- **Texture**: formati supportati: JPEG, PNG, TGA, BMP, etc.

3.21 Risoluzione dei problemi di pivot e allineamento

Dopo l'importazione, i pezzi della torretta potrebbero avere pivot non ideali per le rotazioni (es. il cannone ruota intorno a un punto lontano dal suo perno reale). Per modificare il pivot:

3.21.1 Metodo: re-import con trasformazioni

1. Selezionare la mesh nel Content Browser.
2. Tasto destro → `Reimport Mesh With Dialog` (o `Reimport` con opzioni).
3. Nella finestra di import, modificare i campi:
   - `Translation` (es. sull'asse Z): per spostare il pivot.
   - `Rotation` (es. riallineare l'asse di rotazione).
4. Applicare e sovrascrivere la mesh.

3.21.2 Parametri specifici per la torretta

Per riallineare il cannone della torretta:

- Traslazione sull'asse Z: `-135` (avvicina il pivot al perno corretto).
- Rotazione: `113` (riallinea l'orientamento).

Questi valori possono essere inseriti nella finestra di re-import per correggere la mesh senza uscire da Unreal.

3.22 Posizionamento in scena dei pezzi

- Trascinare base, cannone e pivot nella viewport.
- Copiare i valori di X e Y della base nelle altre due mesh per sovrapporle perfettamente (lasciando libera la rotazione).
- In questo modo si ottiene un assemblaggio pronto per essere trasformato in Blueprint (es. torretta che traccia bersagli ruotando su se stessa e inclinando il cannone).

3.23 Reimport di mesh per modificare pivot e trasformazioni

Quando una mesh importata ha il pivot in una posizione scomoda (es. alla base invece che all'altezza del perno di rotazione), è possibile correggerla agendo direttamente sull'import.

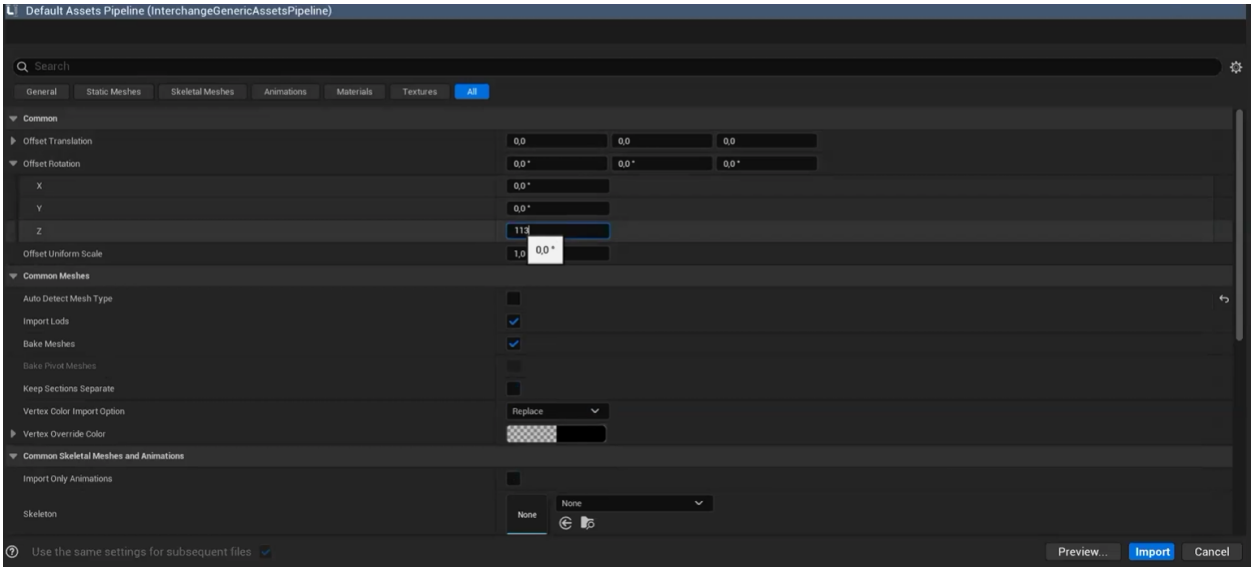


Figure 3.4: Come selezionare l'offset di import di una mash.

3.23.1 Procedura di reimport con dialogo

1. Selezionare la mesh nel `Content Browser`.
2. Tasto destro → `Reimport Mesh With Dialog`.
3. Nella finestra che si riapre, modificare i parametri di trasformazione:
   - `Translation` (es. asse Z): per spostare il pivot verticalmente.
   - `Rotation` (es. 113° su un asse): per riallineare l'orientamento.
4. Confermare per sovrascrivere la mesh originale.

3.23.2 Alternativa con Modeling Mode (Unreal Engine 5)

- Entrare in `Modeling Mode`.
- Selezionare `Edit Pivot`.
- Spostare il pivot manualmente nella posizione desiderata.
- Questo metodo è più diretto e non richiede reimport.

3.24 Organizzazione delle cartelle nel Content Browser

- Creare una cartella principale (es. `Turret`).
- All'interno, una sottocartella `Textures` per organizzare le texture importate.
- Best practice: mantenere una struttura ordinata per facilitare il lavoro in team e la manutenzione.

3.25 Ispezionare una mesh importata

Dopo l'import, aprire la mesh per visualizzarne le proprietà.

3.25.1 Material Slots

Nel pannello dei dettagli a destra, sotto `Material Slots`, sono elencati i materiali assegnati ai diversi elementi della mesh. Alcune mesh possono avere più slot (es. base e cannone della torretta).

3.25.2 Modalità di visualizzazione

Utilizzare il menu in alto a sinistra nella viewport della mesh per cambiare modalità:

- `Lit`: illuminazione standard.
- `Unlit`: senza luce (mostra solo i colori base).
- `Wireframe`: visualizza la struttura a filo di ferro (vertici, spigoli, facce).
- `Lighting Only`, `Reflections`, etc.

3.25.3 Visualizzazione vertici e collisioni

- `Show → Vertices`: evidenzia i vertici della mesh.
- `Show → Collision`: mostra le **Simple Collision** e **Complex Collision**.

3.25.4 Collisioni semplici vs complesse

- **Simple Collision**: forme primitive (scatole, sfere, capsule, convesse). Sono leggere e modificabili.
- **Complex Collision**: utilizza la geometria reale della mesh (vertice per vertice). È più pesante, utile solo per casi specifici.
- Per modificare le collisioni:
  - Selezionare la mesh → `Collision` → `Remove Collision` (per eliminare quelle esistenti).
  - Aggiungere nuove collisioni: `Collision → Add Sphere/Capsule/Box/Capsule/Convex Collision`.
  - Regolare dimensioni e posizione delle primitive.
- Una `Box Collision` è spesso più ottimizzata di una collisione convessa generata automaticamente.

3.26 Grouping e Merging di Actor

Se si hanno più pezzi (base, cannone, pivot) posizionati in scena, è possibile raggrupparli o fonderli.

3.26.1 Grouping (CTRL+G)

- Selezionare più actor → `CTRL+G` per creare un gruppo.
- Muovendo un qualsiasi membro del gruppo, tutti si muovono insieme.
- `SHIFT+G` per sciogliere il gruppo.

3.26.2 Merge Actors

- Selezionare tutti gli actor da fondere.
- Menu `Actor` → `Merge Actors` (o `Unreal Engine Tools` → `Merge Actors`).
- Scegliere `Merge` (o `Simple Merge`).
- Specificare nome e percorso per la nuova mesh statica risultante.
- La nuova mesh unisce geometricamente i pezzi; i materiali originali vengono mantenuti se presenti.
- Utile per ridurre il numero di actor nella scena, ma attenzione: si perde la possibilità di animare i singoli pezzi (es. rotazione indipendente del cannone).

3.27 Creazione materiali da texture importate

Dopo aver importato le texture (diffuse, metallic, roughness, normal), creare un materiale per ogni parte della torretta.

3.27.1 Passi

1. Nel Content Browser, tasto destro → `Material`. Rinominare (es. `M_TurretBase`, `M_TurretCannon`, `M_TurretPivot`).
2. Aprire il materiale. Inserire quattro nodi `Texture Sample` (shortcut `T` + clic sinistro).
3. Assegnare a ciascuno la texture corrispondente:
   - Base Color (es. `T_TurretBase_Color`)
   - Metallic
   - Roughness
   - Normal
4. Collegare gli output ai pin del nodo risultante:
   - Colore → `Base Color`
   - Metallic → `Metallic`
   - Roughness → `Roughness`
   - Normal → `Normal`
5. Salvare e compilare.

3.27.2 Preview del materiale su mesh specifica

Nel pannello di anteprima del materiale, selezionare la mesh desiderata (es. `SM_TurretBase`) dal menu a tendina accanto alla sfera/cubo per visualizzare il materiale applicato direttamente su quell'oggetto.

3.27.3 Assegnare i materiali agli slot della mesh

1. Aprire la mesh statica (es. `SM_TurretBase`).
2. Nella sezione `Material Slots`, trascinare il materiale creato nello slot appropriato.
3. Se la mesh ha più slot (es. due), assegnare materiali diversi a ciascuno (es. base e TV).

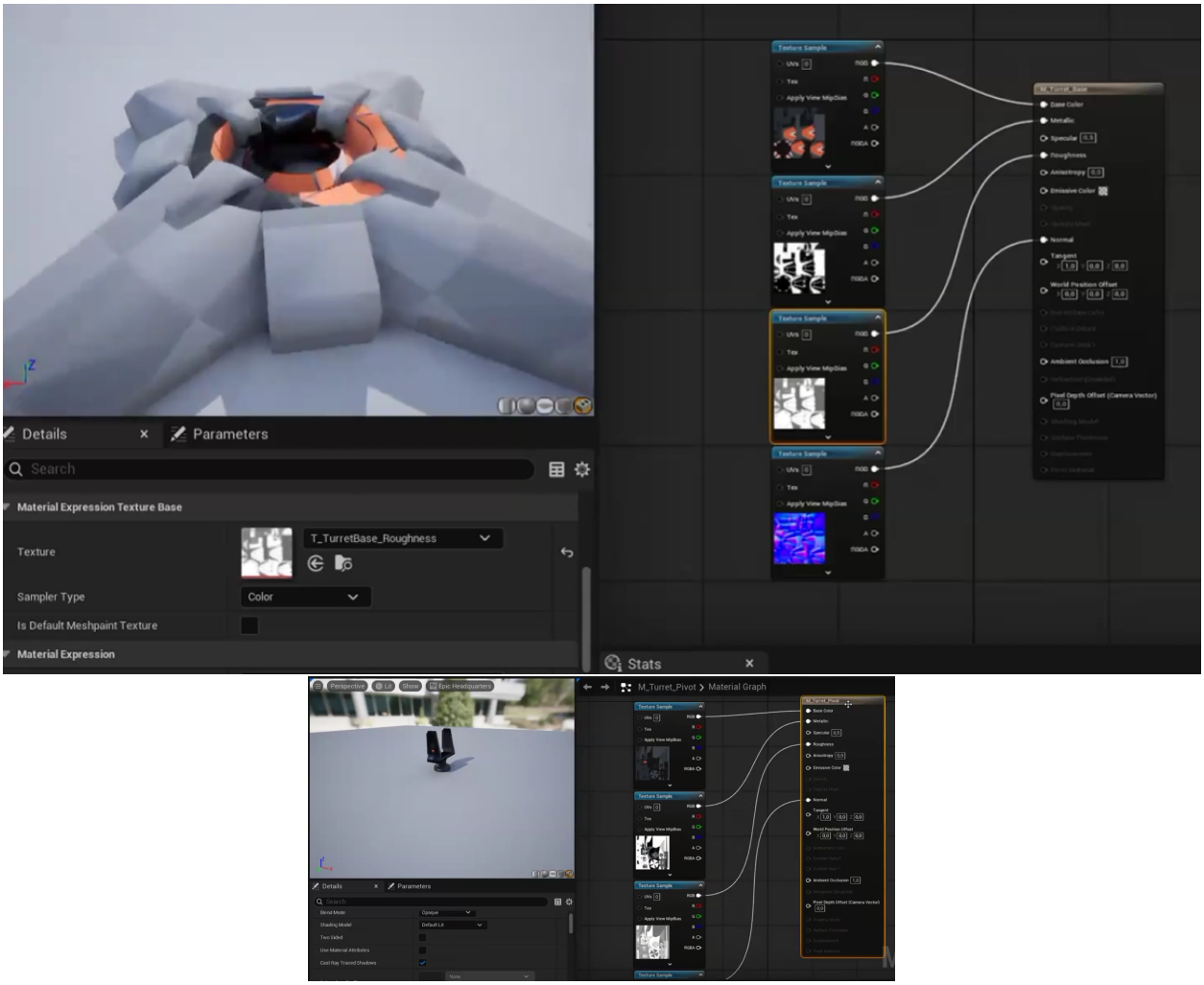


Figure 3.5: Blueprints per le texture di ciascuna parte della torretta. Il cannone è mancante per motivi di voglia, ma per estensione è facile da fare.

3.28 Modifica estetica tramite materiali (esempio con Lerp)

Per variare l'aspetto (es. scurire o schiarire il colore), è possibile utilizzare un nodo `Lerp` nel grafo del materiale.

3.28.1 Esempio: miscelare colore base con un secondo colore

```
Texture Sample (Base Color) --+
                              +-- Lerp --> Base Color
Constant3Vector (Nero/Verde) -+
          |
          +-- Alpha (es. 0.6, convertibile a parametro)
```

- L'Alpha da 0 a 1 decide la prevalenza del primo o del secondo colore.
- Convertendo l'Alpha a parametro, si può creare una Material Instance per regolare l'intensità senza ricompilare.

3.29 Consigli pratici

- **Salvare frequentemente** il lavoro (Unreal può crashare).
- Usare shortcut di sistema: `Ctrl+C`, `Ctrl+V`, `Ctrl+Z`, `Ctrl+Y`, `Ctrl+A` funzionano anche in Unreal.
- Quando si importano texture, mantenere nomi chiari e coerenti.
- Per mesh fornite da grafici esterni, verificare che i pivot siano posizionati correttamente prima di iniziare ad animare o a creare Blueprint.

3.30 Merge di actor e gestione degli slot materiali

Quando si esegue il merge di più actor (es. base, cannone, pivot) tramite `Merge Actors`, la mesh risultante eredita tutti gli **slot materiali** dei componenti originali.

- Esempio: se la torretta originale aveva tre slot (cannone, pivot, base), la mesh fusa avrà tre slot.
- Ogni slot può essere assegnato a un materiale diverso.
- Modificando il materiale di uno slot (es. cambiandolo in nero), si osserva immediatamente l'effetto sulla porzione di mesh corrispondente.

3.31 Comportamento dei materiali con trasformazioni degli oggetti

I materiali sono applicati in base alle **coordinate UV** dell'oggetto. Pertanto:

- Se si **scala** un oggetto (ingrandimento o stiramento), il materiale si deforma insieme alla mesh.
- Esempio: allungando un piano su cui è applicata una texture di corteccia (`Bark`), la texture appare stirata.

3.31.1 Problema di clipping tra superfici coplanari

Se due superfici si trovano esattamente sulla stessa posizione (es. due piani sovrapposti), si verifica **clipping** (l'engine non sa quale rendering privilegiare). Soluzione:

- Separare leggermente gli oggetti lungo l'asse Z (es. spostare uno di 0,1 unità).
- Lo spostamento è impercettibile ma elimina lo z-fighting.

3.32 Materiali World Aligned (allineati al mondo)

Per evitare che il materiale si deformi quando l'oggetto viene stirato, si possono usare **World Aligned Materiali**. Questi materiali non usano le UV dell'oggetto, ma le coordinate del mondo.

3.32.1 Nodi World Aligned

- `WorldAlignedTexture` (da cercare nel menu dei nodi, non ha shortcut).
- `WorldAlignedNormal`.
- Ingressi principali:
  - `Texture Object` (non un Texture Sample – convertire una texture in `Texture Object` con tasto destro).
  - `Texture Size` (simile al tiling, valore vettoriale 3D).

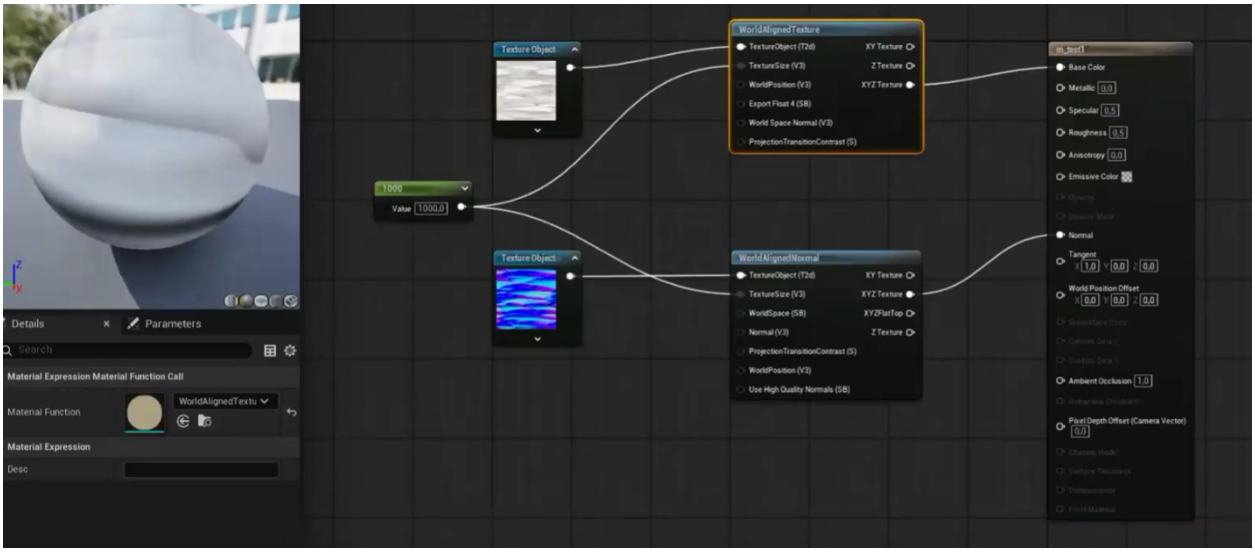


Figure 3.6: Blueprint per world aligned.

3.32.2 Comportamento

- Se si stira l'oggetto, il materiale mantiene le sue proporzioni (le texture non si deformano).
- Tuttavia, il materiale è **ancorato al mondo**. Se l'oggetto si muove o ruota, il materiale rimane fermo (come se "scorresse" sulla superficie).
- Esempio: una porta con materiale world aligned, quando si apre, il motivo sembra scivolare sulla porta.

3.32.3 Confronto: UV-based vs World Aligned

UV-based	**World Aligned**
Si deforma con l'oggetto	Non si deforma
Segue le trasformazioni dell'oggetto	Ancorato al mondo
Ideale per oggetti fissi o non deforming	Ideale per superfici continue (terreni, pavimenti ampi)

3.33 Risoluzione di problemi: linee nere tra texture

Possibili cause di linee di separazione visibili tra tile di una texture:

- Uso errato degli assi nel nodo `WorldAlignedTexture` (es. collegare solo XY invece di XYZ).
- Soluzione: assicurarsi che l'output `XYZ` sia collegato all'input `UV` del nodo.

3.34 Esercizio proposto: creare un materiale Lava

Obiettivo: realizzare un materiale che simuli la lava, utilizzando i concetti visti.

3.34.1 Caratteristiche richieste

- Movimento (usare `Panner` sulle UV).
- Base color (rosso/arancione/giallo).
- Emissive color (la lava deve emettere luce).

- Possibile utilizzo di `Lerp` per alternare zone scure e luminose.
- Opzionale: nodo `Noise` (es. `Perlin Noise` o `Voronoi`) per creare variazioni casuali.

3.34.2 Suggerimenti pratici

1. Creare un nuovo materiale.
2. Aggiungere un `Panner` per muovere le UV.
3. Usare `Lerp` con due colori (rosso scuro e giallo brillante) e un alpha ricavato da una texture o da un nodo `Noise`.
4. Collegare il risultato del `Lerp` al pin `Emissive Color` (e eventualmente anche al `Base Color`).
5. Moltiplicare l'emissivo per un valore alto (es. costante 5 o 10) per ottenere un bagliore intenso.
6. Sperimentare con texture alternative (non solo `Bark`) per l'alpha o per il colore di base.

3.34.3 Nodi utili non ancora visti

- `Noise`: genera pattern casuali (bianco/nero) utili come maschere o per distorsioni.
- I nodi `Add`, `Multiply` e `Lerp` sono i più utilizzati in combinazione.
- `Texture Coordinate` può essere usato anche con `Panner`.

3.35 Consigli finali per il lavoro autonomo

- Salvare frequentemente.
- Provare a combinare texture diverse per l'alpha del `Lerp`.
- Per l'emissivo, valori piccoli (es. 0.5) danno un effetto debole; per una lava convincente usare valori ¿ 2.
- Se il materiale non appare luminoso, verificare che `Emissive Color` sia collegato e che la scena abbia il rendering impostato su `Lit`.

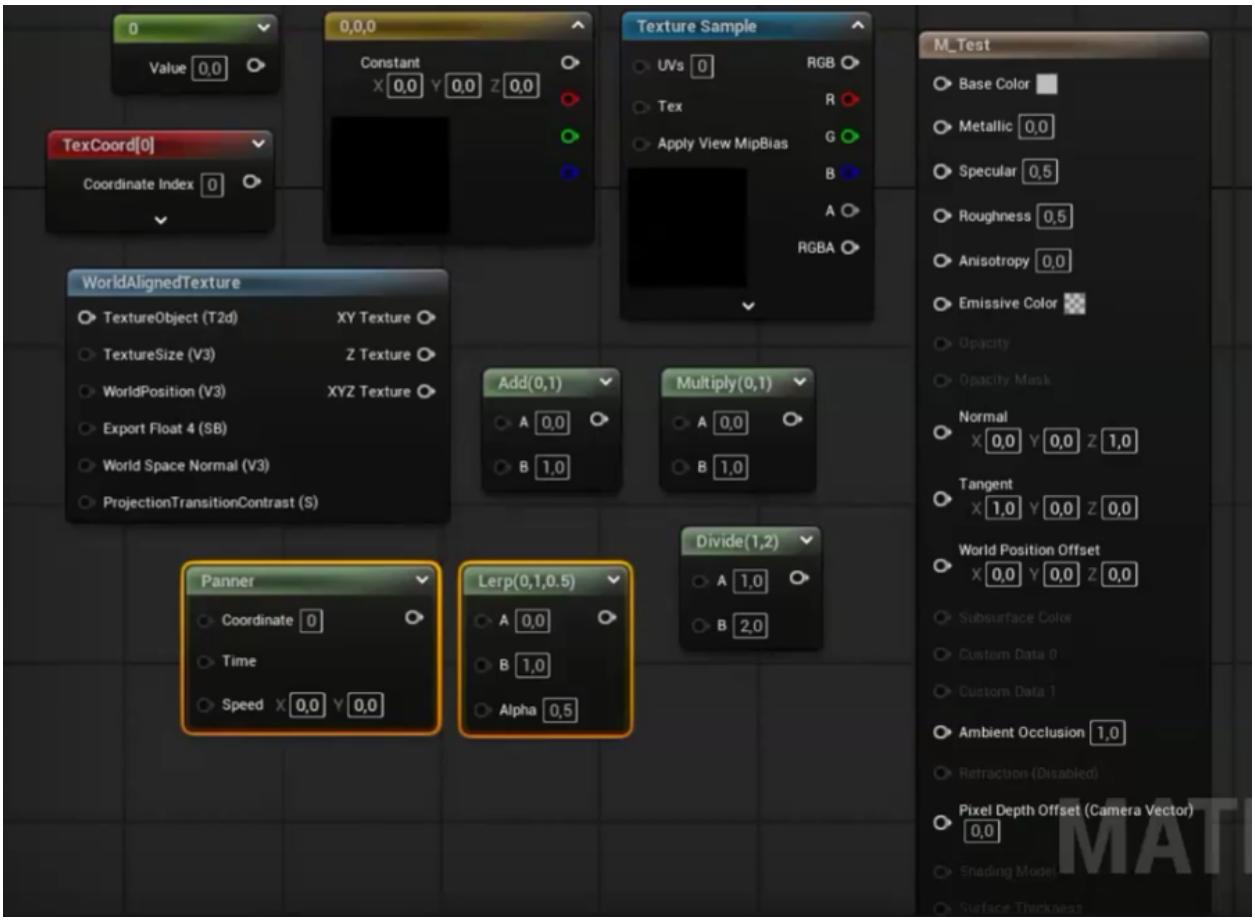


Figure 3.7: Blueprint con il sunto dei comandi usati.

Chapter 4

Introduzione a Unreal Engine 5: Vettori, Rotatori e Comportamenti Interattivi con Blueprint

4.1 Risorse per materiali: Quixel Megascans

4.1.1 Accesso e utilizzo

- Quixel (ora integrato con Unreal Engine) fornisce una vasta libreria di materiali, texture e mesh scansionati ad alta qualità.
- È necessario effettuare l'accesso con l'account di Unreal Engine per scaricare le risorse.
- I materiali sono veri e propri *scan* realizzati con apparecchiature professionali; la qualità è molto elevata.

4.1.2 Creazione di un materiale lava

Per realizzare un materiale lava a partire da una texture di roccia:

1. Scaricare una texture di roccia da Quixel Megascans (ad esempio `Iceland Quarry Rock`).
2. Utilizzare i nodi matematici **Multiply** e **Add** per inserire il colore rosso della lava nelle zone desiderate (es. crepe).
3. Impiegare il nodo **Lerp** (interpolazione lineare) pilotato da un canale **alpha**: l'alpha definisce la miscelazione tra roccia e lava.
4. Testare sempre il materiale nel viewport per verificare che le operazioni (Multiply, Lerp) siano impostate correttamente; eventuali errori producono risultati visibili direttamente in viewport.

Nota: La ricerca su YouTube con parole chiave come `"make a lava material"` può fornire tutorial e, in alcuni casi, texture già predisposte.

4.2 Trucco: Continuous Collision Detection (CCD)

Per il progetto del *gioco della pallina*, per evitare che la sfera penetri all'interno del piano:

- Abilitare l'opzione **CCD** (Continuous Collision Detection) sull'asset (menù a sinistra).
- Con CCD attivo, Unreal calcola le collisioni in modo continuo, rendendo la rilevazione più precisa e riducendo drasticamente l'intersezione.

- È sufficiente attivare il flag nell'apposita sezione dei dettagli dell'asset.

4.3 Progetto odierno: Mulino a vento e torretta

4.3.1 Materiale didattico

- Scaricare il progetto `Hive UE5 Intro Start`.
- Estrarre il contenuto e aprire l'eseguibile di Unreal.
- Attendere la compilazione degli shader prima di iniziare.

4.3.2 Obiettivi

- Far ruotare le pale del mulino a vento presente nella scena.
- Correggere il bug della torretta che dovrebbe seguire il giocatore (verrà risolto durante la lezione).

4.4 Elementi di matematica 3D: vettori e rotazioni

4.4.1 Definizione di vettore

Un vettore in uno spazio tridimensionale può essere pensato come una **freccia** dotata di:

- **Direzione** (retta lungo la quale giace).
- **Verso** (orientamento lungo la direzione).
- **Magnitudine** (lunghezza o intensità).

In Unreal Engine, le coordinate `(x, y, z)` di una `Location` definiscono un vettore.

4.4.2 Operazioni principali

- **Addizione:** $A + B = C$. Il vettore risultante è la diagonale (ipotenusa) del parallelogramma formato da A e B. Es.: $A = (5, 0)$, $B = (0, 4)$ dà $C = (5, 4)$.
- **Sottrazione:** $A - B$ restituisce il vettore che da B va ad A; $B - A$ restituisce il vettore opposto. La magnitudine del risultato è la distanza tra i due punti.
- **Moltiplicazione:** $k \cdot V$ moltiplica le componenti (x, y, z) per lo scalare k, allungando o accorciando il vettore lungo la sua direzione.
- **Dot product (prodotto scalare):** $A \cdot B$ produce un **valore scalare** (float). Interpretazione:
  - > 0: i vettori puntano nella stessa direzione (acuto).
  - $= 0$: i vettori sono perpendicolari.
  - < 0: i vettori puntano in direzioni opposte (ottuso).
  - $= 1$ (per vettori normalizzati): vettori esattamente uguali come direzione.

4.4.3 Vettori normalizzati

Un vettore **normalizzato** ha lunghezza unitaria (*magnitudine* $= 1$). Si ottiene dividendo ciascun componente per la magnitudine originale del vettore. Viene spesso usato per rappresentare direzioni pure.

4.4.4 Rotatori e angoli di Eulero

Un **rotator** in Unreal è composto da tre angoli:

- **X** (roll, asse rosso).
- **Y** (pitch, asse verde).
- **Z** (yaw, asse blu).

Concettualmente sono anelli (giroscopi) che ruotano attorno a ciascun asse. Tuttavia, con gli angoli di Eulero può verificarsi il **Gimbal Lock**: due assi si allineano causando la perdita di un grado di libertà.

4.4.5 Cenni ai Quaternioni

Per evitare il Gimbal Lock si usano i **quaternioni**, un sistema matematico a quattro dimensioni che rappresenta le rotazioni senza singolarità. In Unreal vengono impiegati internamente ma non è necessario manipolarli direttamente per le operazioni comuni.

4.5 Creazione di un oggetto rotante: il mulino

4.5.1 Struttura del mulino

Il mulino è un **Actor** composto da più **Componenti**:

- `Body`: mesh principale del corpo del mulino.
- Due `Pale`: mesh separate per le pale rotanti.

4.5.2 Blueprint del mulino

1. Creare un nuovo Blueprint di tipo `Actor`, chiamarlo ad esempio `BP_Windmill`.
2. L'Actor fa da contenitore per tutti i componenti; non è direttamente utilizzabile dal giocatore ma funge da elemento di scena.

4.5.3 Componente Rotating Movement

Unreal fornisce il componente **Rotating Movement** per far ruotare un Actor senza scrivere codice complesso:

- Aggiungere il componente tramite il pulsante `Add` → `Rotating Movement`.
- Nei dettagli del componente compaiono i parametri **Rotation Rate** (velocità di rotazione) per gli assi X, Y, Z (in gradi al secondo). Il valore predefinito è 180 su uno degli assi.
- **Attenzione:** la mobilità dell'Actor (o del componente mesh) deve essere impostata su **Movable**. Se lasciata `Static`, durante l'esecuzione il componente non ruoterà e verrà generato un errore:

```
Static mesh component has to be movable if you'd like
to move it with rotating movement component
```

4.5.4 Dimostrazione rapida

- Trascinare un cubo nella scena.
- Aggiungere un componente `Rotating Movement` dai dettagli.
- Impostare `Mobility` → `Movable`.
- Avviare il gioco: il cubo ruoterà continuamente con la velocità specificata.

Questo metodo può essere applicato direttamente al Blueprint del mulino per far ruotare l'intero Actor (in seguito si affronterà la rotazione delle sole pale, che richiede un approccio più articolato).

4.6 Esercitazione rapida: rotazione di cubi

Consegna: creare due cubi; il primo deve ruotare attorno all'asse X, il secondo attorno all'asse Y.

- Copiare un cubo già presente nella scena (tasto `Alt` e trascinamento, oppure `Ctrl+C`, `Ctrl+V`).
- Su ciascun cubo aggiungere un componente `Rotating Movement` (dalla finestra *Details* → `Add Component`).
- Impostare la mobilità del cubo su **Movable**.
- Nel componente `Rotating Movement` configurare:
  - Per il cubo che ruota in X: `Rotation Rate` → `X = 180`, `Y = 0`, `Z = 0`.
  - Per il cubo che ruota in Y: `X = 0`, `Y = 180`, `Z = 0`.
- Avviare il gioco per verificare le rotazioni desiderate.

4.7 Costruzione del mulino a vento

4.7.1 Blueprint e componenti

- Aprire il Blueprint `BP_Windmill` (creato in precedenza come Actor).
- Aggiungere tre componenti `Static Mesh`:
  - **Body**: corpo centrale del mulino. Impostare `Static Mesh` → `SM_Cube` (o cubo con pivot centrato). Posizione $Z = 40$ (sollevato). Scala: $X = 3$, $Y = 2$, $Z = 5$ (larghezza, profondità, altezza). Assegnare un materiale (es. `PrototypeGrid`).
  - **Pala_1** e **Pala_2**: componenti copiati da `Body` (o creati nuovi) e rinominati. Posizionarli sopra il corpo, scalati opportunamente (es. assi lunghi verticali). Assegnare un materiale differente per distinguerle (es. `M_Grid` violacea).
- Per organizzare la gerarchia: trascinare `Pala_2` su `Pala_1` nella finestra *Components*, rendendola figlia. In questo modo `Pala_2` seguirà automaticamente le rotazioni di `Pala_1` senza bisogno di ulteriore programmazione.

4.7.2 Limitazione del componente Rotating Movement

Il `Rotating Movement` applicato a un componente ruota **l'intero Actor** e non è in grado di muovere un singolo componente (ad esempio una sola pala) indipendentemente dal resto. Se si prova a usarlo, il corpo del mulino ruoterebbe insieme alle pale. Per ruotare solo una pala è necessario procedere via Blueprint.

4.7.3 Rotazione di una pala con Event Tick e Delta Seconds

La rotazione deve essere aggiornata a ogni fotogramma, per cui si utilizza l'evento `Event Tick`.

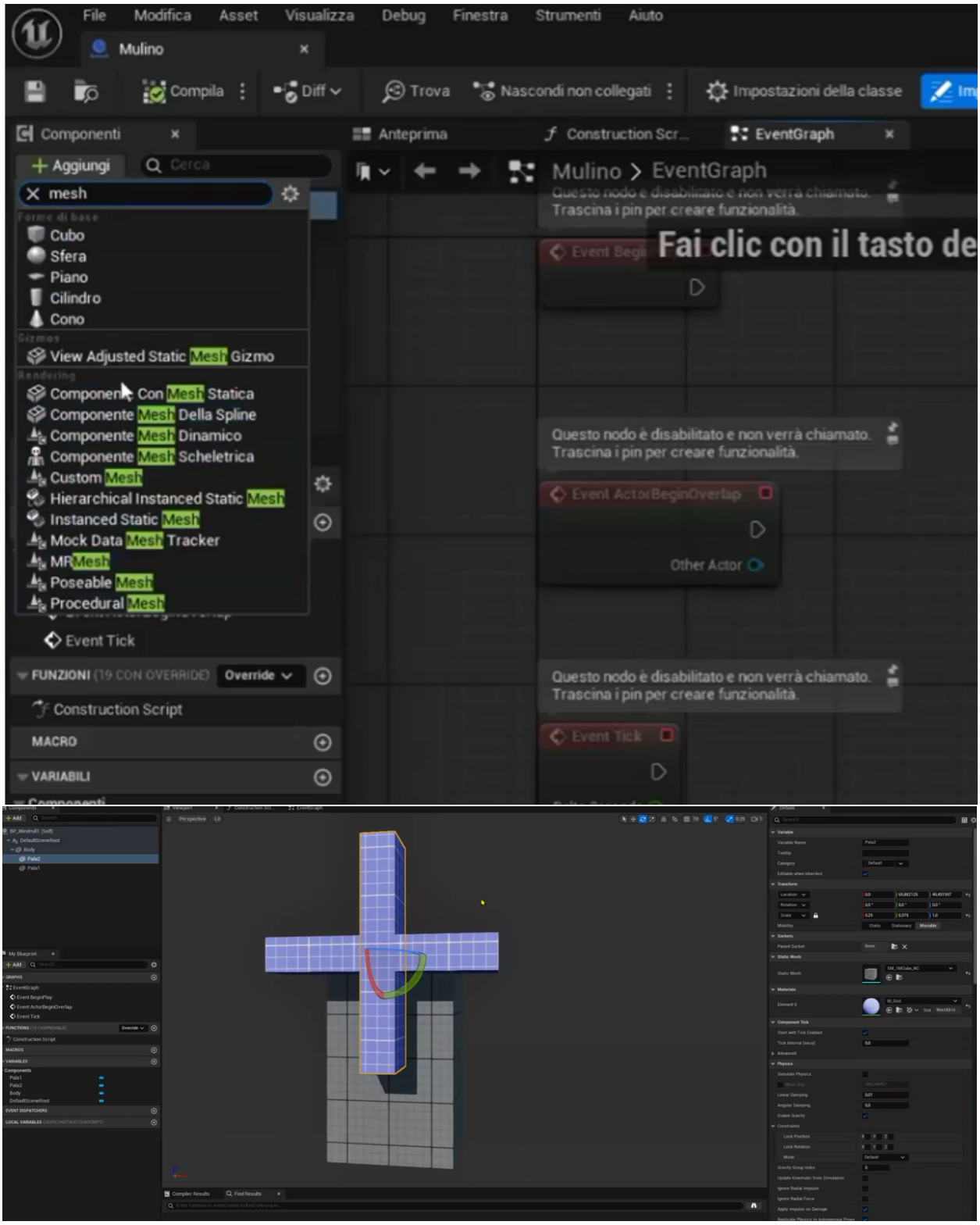


Figure 4.1: Creazione della mesh del mulino a vento.

Delta Seconds

`Delta Seconds` è il tempo (in secondi) intercorso tra il frame precedente e quello corrente. Moltiplicare una quantità per `Delta Seconds` rende il movimento indipendente dal frame rate: un oggetto si sposta alla stessa velocità percepita sia a 30 fps che a 120 fps.

Nodi coinvolti:

- Trascinare il pin `Delta Seconds` dell'`Event Tick` e collegarlo a un nodo `Multiply` (moltiplicazione per uno scalare).
- Il secondo ingresso del `Multiply` è un valore `float` che definisce la velocità angolare base (es. `90.0`).

Creazione del Rotator

Un `Rotator` è un insieme di tre valori `float` (pitch Y, yaw Z, roll X). Per far ruotare la pala solo attorno al proprio asse Y (pitch) occorre comporre un rotatore con $X = 0$, $Y =$ risultato del Multiply, $Z = 0$.

- Utilizzare il nodo `Make Rotator` (o `Make Rot from X, Y, Z`): collegare il float calcolato al pin `Y (Pitch)`.

Recupero e impostazione della rotazione relativa

Poiché la pala è un componente figlio del `Body`, il suo movimento deve essere espresso nella rotazione *relativa* al proprio sistema di riferimento locale, non nella rotazione mondiale.

- Ottenere la rotazione relativa corrente della pala: trascinare il componente `Pala_1` nel `Event Graph` → `Get Relative Rotation`.
- Combinare questa rotazione corrente con la nuova rotazione generata dal `Make Rotator`. Si può usare il nodo `Combine Rotators`: ingresso A = rotazione attuale, ingresso B = rotazione incrementale (appena calcolata); l'uscita è la somma delle due rotazioni.
- Applicare la nuova rotazione: trascinare `Pala_1` e chiamare `Set Relative Rotation`, collegando l'uscita di `Combine Rotators` al pin `New Rotation`.

Sequenza dei nodi:

```
Event Tick (Delta Seconds) -> Multiply (float * speed)
Multiply -> Make Rotator (Y)
Pala_1 -> Get Relative Rotation -> Combine Rotators (A)
Combine Rotators (B) <- Make Rotator
Combine Rotators -> Pala_1 -> Set Relative Rotation
```

Verifica e accorgimenti

- Assicurarsi che la mobilità del componente `Pala_1` sia `Movable` (nel pannello *Details* del componente).
- Posizionare il Blueprint del mulino nella scena, avviare il gioco: la pala girerà attorno al proprio asse Y.
- Se la rotazione appare innaturale, verificare che il pivot della mesh sia centrato. In caso contrario, sostituire la `Static Mesh` con una che abbia il pivot posizionato correttamente (ad esempio `SM_StaticMesh` con pivot centrale).

4.7.4 Gerarchia: la seconda pala

Poiché `Pala_2` è stata resa figlia di `Pala_1` (trascinata sotto di essa nella gerarchia dei componenti), qualsiasi rotazione applicata a `Pala_1` si propaga automaticamente a `Pala_2`. Non è necessario scrivere codice aggiuntivo per la seconda pala; essa segue il movimento della prima, mantenendo la disposizione angolare (ad esempio opposte) definita nello spazio locale.

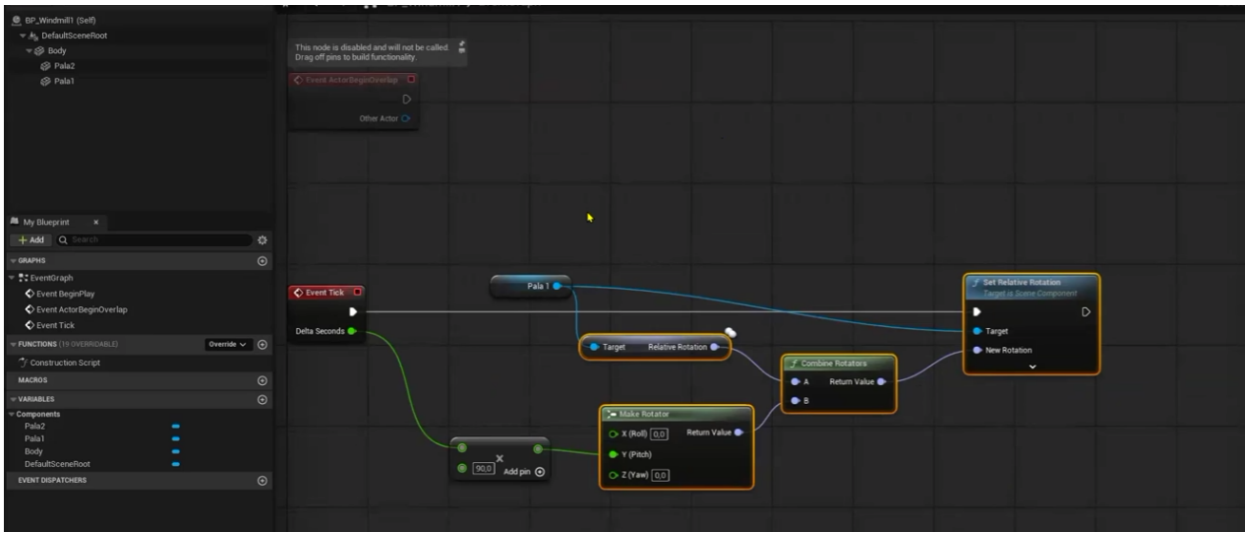


Figure 4.2: Creazione del blueprint del mulino a vento. Per far ruotare un oggetto seguendo la medesima rotazione di Pala1, bisogna rendere figlio di Pala1 questo oggetto.

4.7.5 Nota: risoluzione problemi viewport

Se la finestra *Viewport* del Blueprint scompare:

- Controllare che non sia stata ancorata in un'altra parte dell'interfaccia (può separarsi come finestra indipendente; usare `Alt+Tab` per trovarla).
- Dal *Content Browser*, selezionare il Blueprint e fare doppio clic per aprirlo; poi dal menu `Window` → `Viewport` per ripristinarla.

4.8 Esercitazione: attaccare un cubetto alla pala del mulino

Obiettivo: aggiungere un cubo all'estremità di `Pala_2` e verificare che segua il movimento.

- Nel Blueprint del mulino, aggiungere un nuovo componente `Static Mesh` e assegnare un cubo.
- Renderlo **figlio** di `Pala_2` trascinandolo sotto di essa nella gerarchia dei componenti.
- Spostare il cubo nella posizione desiderata (es. estremità della pala).
- Il cubo mantiene la posizione relativa a `Pala_2` e quindi ruota con essa senza bisogno di codice aggiuntivo.
- Nota: il pivot del cubo rimane al centro del cubo stesso; non occorre spostarlo sul centro di rotazione perché la rotazione viene ereditata dalla pala genitore.

4.9 Metodo alternativo per la rotazione: Find Look at Rotation e interpolazione

4.9.1 Find Look at Rotation

Il nodo `Find Look at Rotation` calcola la rotazione necessaria affinché un punto di partenza (`Start`) guardi verso un punto di arrivo (`Target`).

- `Start`: posizione dell'oggetto che deve ruotare (es. `Get Actor Location` del mulino).
- `Target`: posizione verso cui guardare (es. `Get Actor Location` del `Player Character`).
- Il nodo restituisce un `Rotator` che orienta il vettore dallo Start al Target.

Esercizio: fare in modo che il mulino ruoti (solo sull'asse Z) per seguire il personaggio.

4.9.2 Interpolazione con RInterp

Per ottenere un movimento fluido si utilizza l'interpolazione:

- `RInterp` (o `RInterp To`) permette di interpolare tra due rotatori A (rotazione corrente) e B (rotazione desiderata) in base a un fattore `Delta Time` e una velocità di interpolazione (`Interp Speed`).
- `A`: `Get Actor Rotation` (o `Get Relative Rotation` del componente da ruotare).
- `B`: `Find Look at Rotation`.
- `Delta Time`: connettere il `World Delta Seconds` (o il `Delta Seconds` dell'Event Tick).
- `Interp Speed`: valore float che regola la velocità di rotazione (es. 2.0).
- Se il movimento appare bloccato, verificare che `Delta Time` e `Interp Speed` siano valorizzati.

4.9.3 Gestione degli assi e offset

- Per limitare la rotazione a un solo asse (es. Z), splittare il `Rotator` in uscita da `RInterp` (tasto destro sul pin $\rightarrow$ `Split Struct Pin`) e utilizzare solo la componente desiderata.
- Ricostruire un `Make Rotator` con le altre componenti a zero (o con valori fissi), inserendo la componente Z calcolata.
- Eventuali offset (es. aggiungere 90°) si realizzano con un nodo `Add` sul float prima di inserirlo nel `Make Rotator`.

4.9.4 Limitazione dell'angolo con Clamp

- Per restringere il range di rotazione si impiega il nodo `Clamp (float)`.
- Splittare il pin Z del rotatore ottenuto, collegarlo a `Clamp` con un minimo e un massimo.
- Determinare i limiti: si può usare un `Print String` temporaneamente collegato al valore Z per leggere a schermo gli angoli raggiunti durante il movimento, quindi impostare i valori di clamp (es. -160 e 160).
- Ricordarsi di rimuovere o disconnettere il `Print String` dopo il debug.

4.10 Pausa

Pausa di 20 minuti prima di affrontare la costruzione della torretta.

4.11 Torretta: creazione del Blueprint e composizione

4.11.1 Obiettivo

Realizzare una torretta in grado di ruotare verso il giocatore e, successivamente, sparare proiettili. Utilizzerà anche il *Line Trace* per rilevare ostacoli.

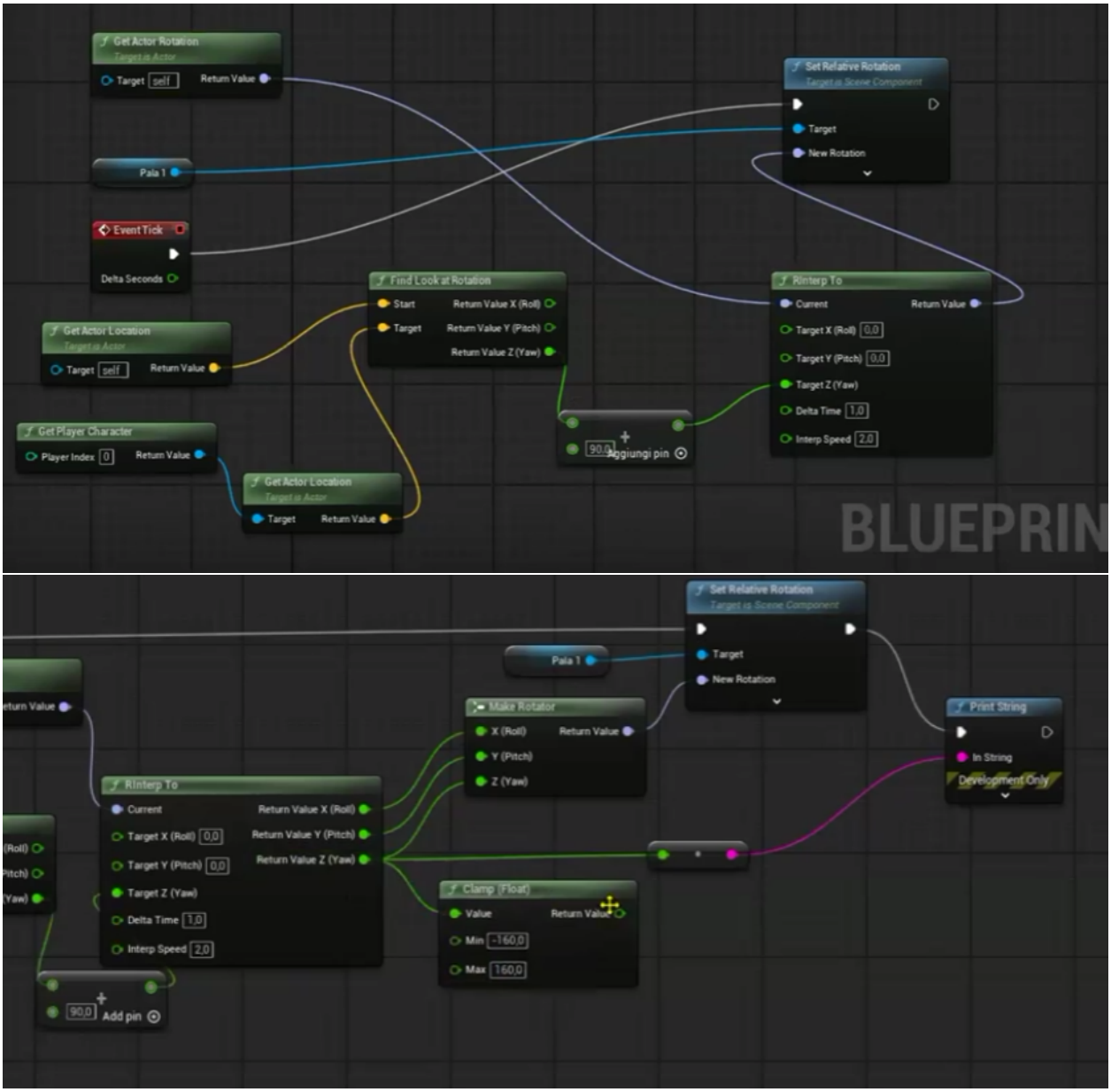


Figure 4.3: Metodo la rotazione del mulino seguendo i movimenti del giocatore (sopra), con clamping (sotto).

4.11.2 Creazione del Blueprint

- Creare un nuovo Blueprint di tipo `Actor` e chiamarlo `BP_Turret`.
- La torretta sarà composta da tre componenti `Static Mesh` separati:
  1. **Base** (supporto inferiore, non ruota).
  2. **Holder** (supporto brandeggiabile, ruota sull'asse Z per seguire il bersaglio).
  3. **Gun** (canna, eventualmente ruota sull'asse Y per l'elevazione).
- Se in precedenza si era eseguito il *Merge Actor* unendo i tre pezzi, ora occorre invece utilizzarli singolarmente per consentire rotazioni indipendenti.

4.11.3 Inserimento delle mesh tramite Browse to Asset

Per individuare rapidamente gli asset nel `Content Browser`:

- Posizionare l'asset (es. la base) nella scena.
- Tasto destro sull'asset → `Browse to Asset`: il `Content Browser` si posiziona automaticamente sulla risorsa.
- Trascinare la `Static Mesh` dal `Content Browser` direttamente sul componente corrispondente nel Blueprint (o assegnarla nel pannello `Details`).

4.11.4 Gerarchia consigliata

- `Base` (root o componente principale).
- Sotto `Base`, aggiungere `Holder`.
- Sotto `Holder`, aggiungere `Gun`.
- In questo modo, ruotando `Holder` si sposta anche la `Gun`; per l'elevazione si potrà ruotare solo `Gun`.

4.11.5 Risoluzione problemi nell'aggiunta di componenti

- Se il menù `Add Component` mostra solo la mesh intera invece dei tre pezzi, verificare di non aver importato o selezionato l'asset unito; è necessario avere i tre file separati.
- Se la finestra di aggiunta non appare, controllare che l'interfaccia non sia stata accidentalmente chiusa o nascosta (menu `Window` → `Components`).
- Eventuali disallineamenti vanno corretti regolando posizione e rotazione di ciascun componente nella `Viewport` del Blueprint.

4.12 Torretta: sfera di attivazione e identificazione del giocatore

4.12.1 Configurazione del Blueprint della torretta

Il Blueprint `BP_Turret` è già stato composto con i tre componenti `Static Mesh`:

- `Base` (supporto inferiore, non ruota).
- `Holder` (supporto brandeggiabile, ruota sull'asse Z).
- `Gun` (canna, eventuale elevazione).

La gerarchia prevede `Gun` figlio di `Holder`, a sua volta figlio di `Base`.

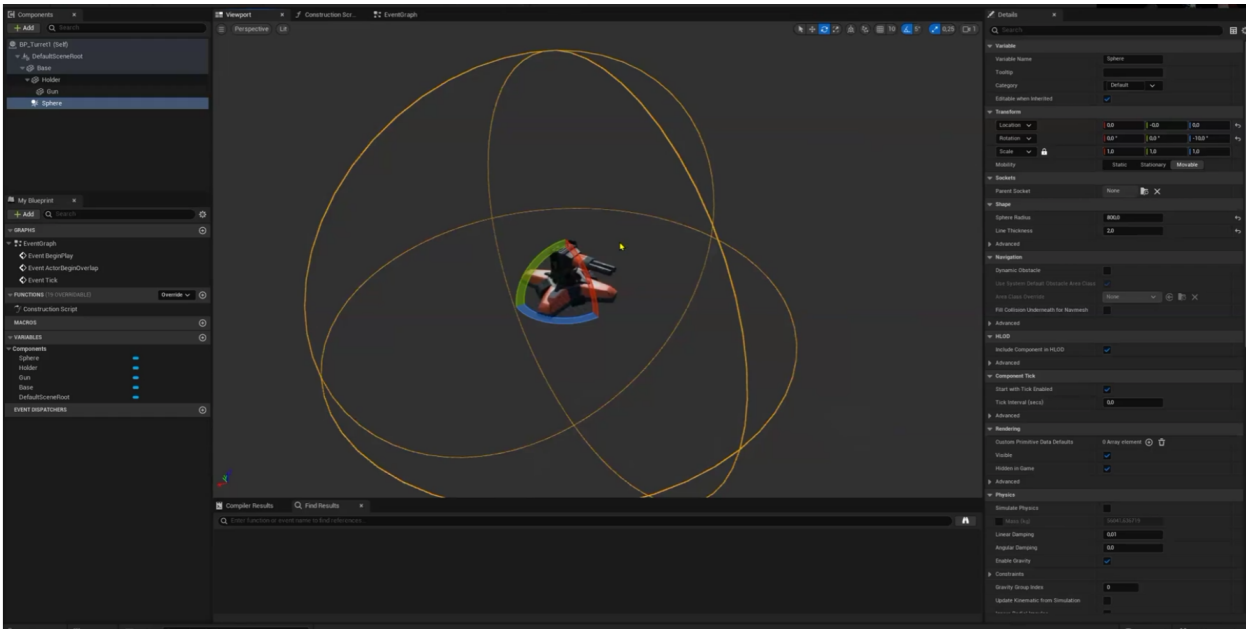


Figure 4.4: Creazione mesh torretta. Non dimenticare di nascondere la sfera facendo un tick a hidden in game.

4.12.2 Aggiunta della Sphere Collision

Per definire il raggio entro cui la torretta si attiva:

- Nel Blueprint, aggiungere un componente **Sphere Collision** (`Add Component` → `Sphere Collision`).
- Posizionarlo centrato rispetto alla torretta.
- Impostare il parametro `Sphere Radius` al valore desiderato (es. `800`).

4.12.3 Visualizzazione della sfera in gioco

Di default le collisioni non sono renderizzate durante l'esecuzione. Per vederla in play e facilitare il debug:

- Selezionare il componente `Sphere Collision` nella gerarchia.
- Nel pannello *Details*, sotto la categoria *Rendering*, disabilitare l'opzione **Hidden in Game** (impostare `false`).
- Opzionalmente aumentare `Shape` → `Line Thickness` per rendere i contorni più visibili.

Vedi Fig. 4.4.

4.12.4 Evento OnBeginOverlap

L'evento `On Begin Overlap` viene attivato quando un altro `Actor` (o un suo componente) entra in contatto con la sfera di collisione.

- Selezionare il componente `Sphere Collision`, scendere nella sezione *Events* e cliccare `On Begin Overlap` per creare il nodo nell'`Event Graph`.
- Il nodo espone diversi pin di output; i più importanti sono:
  - `Other Actor`: l'Actor che ha iniziato la sovrapposizione.
  - `Other Component`: il componente specifico che ha colliso.

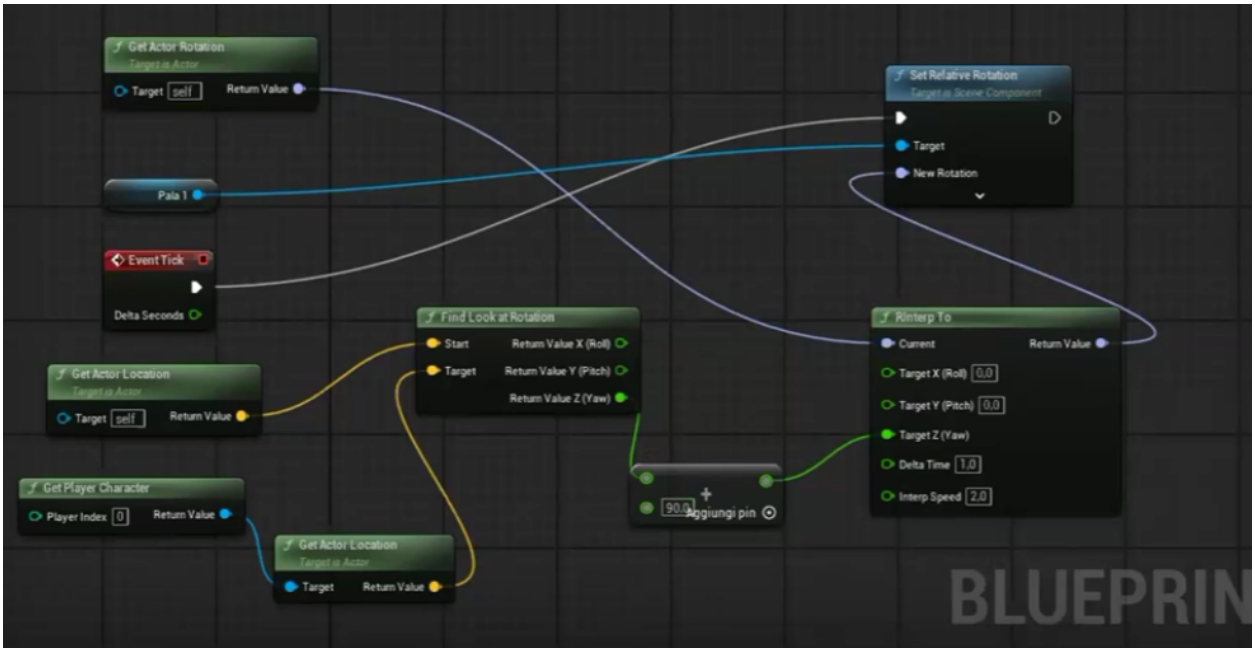


Figure 4.5: Menù creazione tag giocatore.

4.12.5 Identificare il Player tramite Tag

Per assicurarsi che l'attore entrato nella sfera sia effettivamente il personaggio giocante, si utilizza un sistema di **tag**.

Assegnare un tag al Character

- Aprire il Blueprint `ThirdPersonCharacter`.
- Nel pannello *Details*, cercare `Tags` (categoria *Actor* → *Advanced* → *Tags*).
- Aggiungere un elemento e inserire il nome `Player` (attenzione: è case-sensitive).

Vedi Fig. 4.5.

Controllo del tag con `Actor Has Tag`

All'interno dell'evento `On Begin Overlap`:

1. Trascinare il pin `Other Actor` e creare un nodo `Actor Has Tag`.
2. Collegare `Other Actor` al pin `Target` del nodo.
3. Impostare il campo `Tag` con la stringa esatta `"Player"`.
4. Il nodo restituisce un `Boolean`: `True` se l'attore possiede quel tag.

4.12.6 Controllo di validità con `Is Valid`

Prima di effettuare qualsiasi operazione sull'`Other Actor`, è buona norma verificare che esso esista (che la collisione non sia avvenuta con un riferimento nullo):

- Dal pin `Other Actor`, creare un nodo `Is Valid` (quello con il punto interrogativo).
- Il ramo `Is Valid` proseguirà solo se l'attore è effettivamente valido.

- Il controllo può essere combinato con il risultato di `Actor Has Tag` tramite una `Branch` (se entrambi sono `True`, allora procedere).

Nota: il nodo `Is Valid` non serve per riconoscere il player, ma per evitare errori nel caso in cui la collisione avvenga con attori eliminati o non inizializzati.

4.12.7 Salvare il riferimento al Player

Per utilizzare il player in seguito (es. per orientare la torretta), è comodo conservare il suo riferimento in una variabile:

- Dal pin `Other Actor`, scegliere `Promote to Variable`.
- Verrà creato automaticamente un nodo `Set` e una variabile di tipo `Actor` (rinominabile ad es. `TargetPlayer`).
- Collegare il flusso di esecuzione dopo i controlli (tag e validità) al nodo `Set`, in modo che la variabile venga popolata solo quando il player entra nella sfera.

4.12.8 Custom Event e Timer per il tracciamento

Invece di utilizzare l'`Event Tick` (che esegue operazioni anche quando non necessario), si può creare un meccanismo a **timer** per attivare il tracciamento solo dopo l'overlap.

Creazione di un Custom Event

- Nel grafo, fare tasto destro → `Add Custom Event` e nominarlo `TrackTargetTimer`.
- Un `Custom Event` è un evento definito dall'utente, richiamabile in qualsiasi momento tramite il suo nome. Può essere usato come destinazione per un `Set Timer by Event`.

Avviare il timer con `Set Timer by Event`

- A valle del nodo `Set` (dopo aver salvato il player), chiamare `Set Timer by Event`.
- **Event:** collegare il pin rosso (tipo `Delegate`) a un riferimento al `Custom Event TrackTargetTimer`. Per crearlo, trascinare un filo dal pin rosso → `Create Event` → `TrackTargetTimer` (oppure usare un `Custom Event` già esistente).
- **Time:** utilizzare `Get World Delta Seconds` per eseguire il timer ad ogni frame, con lo stesso intervallo di un `Tick`.
- **Looping:** spuntare `Looping` affinché il timer venga ripetuto continuamente.
- **Max Calls Per Frame:** impostare a `1` per limitare l'esecuzione a una volta per frame, migliorando le prestazioni.

Con questa configurazione, dal momento in cui il player entra nella sfera, il `Custom Event TrackTargetTimer` verrà chiamato ogni frame esattamente come un `Event Tick`, ma solo dopo l'attivazione.

Esempio dimostrativo con `Print String`

Per verificare il funzionamento del timer, all'interno del `Custom Event TrackTargetTimer` si può inserire temporaneamente un `Print String` (es. "Hello"). Avviando il gioco, il messaggio apparirà a schermo ripetutamente solo dopo essere entrati nella sfera.

4.12.9 Continuazione del progetto

La logica di movimento della torretta (rotazione verso il player) verrà inserita all'interno del `Custom Event TrackTargetTimer` nella lezione successiva, sfruttando il riferimento al player memorizzato nella variabile.

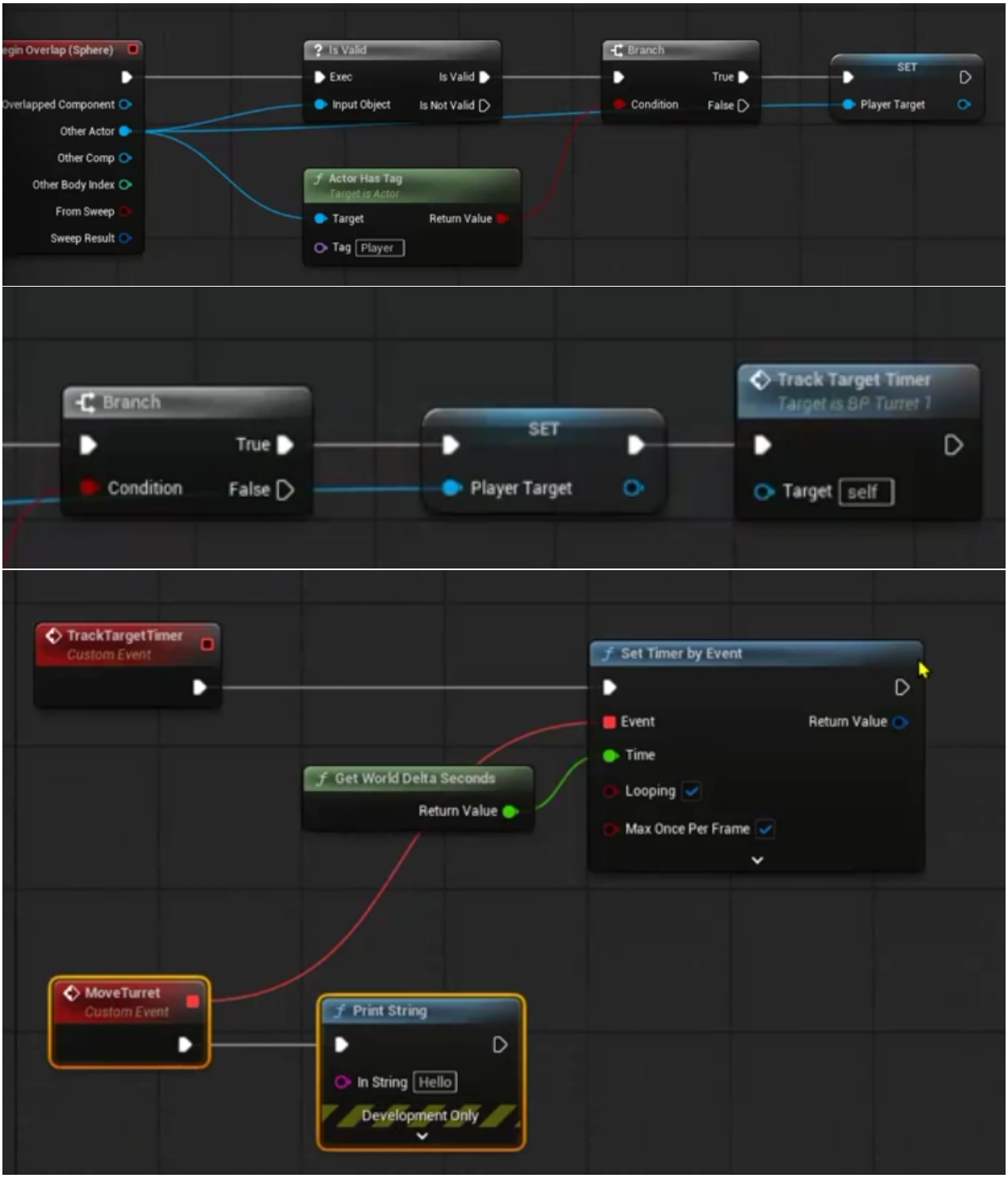


Figure 4.6: Blueprint per richiamare il giocatore (sopra); richiamo al custom event (in mezzo); custom event (sotto).

4.13 Rotazione della torretta verso il giocatore

4.13.1 Logica generale

Una volta entrati nella sfera di collisione e salvato il riferimento al `Player` nella variabile `TargetPlayer`, il `Custom Event` (ad esempio `MoveTurret`) invocato dal `Timer` esegue ad ogni frame:

1. Verificare che `TargetPlayer` sia ancora valido (`IsValid`).
2. Calcolare la rotazione necessaria per puntare verso il giocatore.
3. Interpolare la rotazione attuale del `Pivot` verso quella desiderata.
4. Applicare la nuova rotazione al componente `Pivot`.

4.13.2 Controllo del riferimento al Player

- Trascinare la variabile `TargetPlayer` nel grafo e collegarla a un nodo `IsValid`.
- Il ramo `Is Valid` prosegue solo se il giocatore è ancora presente (non è uscito dalla sfera o non è stato distrutto). Se non valido, non eseguire la rotazione.

4.13.3 Calcolo della direzione: `Find Look at Rotation`

- **Start**: `Get Actor Location` della torretta (il `Self`).
- **Target**: `Get Actor Location` della variabile `TargetPlayer` (trascinare il pin della variabile e chiamare `Get Actor Location`).
- Il nodo restituisce un `Rotator` che orienta l'oggetto dallo Start al Target.

4.13.4 Interpolazione della rotazione

Per un movimento fluido si utilizza `RInterp` (o `RInterp To`):

- **Current**: rotazione attuale del componente `Pivot`. Ottenere con `Get Relative Rotation` (trascinando il componente `Pivot` nel grafo e chiamando la funzione). Splittare il pin di uscita (`Split Struct Pin`) e utilizzare solo la componente Z.
- **Target**: componente Z del `Rotator` proveniente da `Find Look at Rotation` (anch'essa splittata).
- **Delta Time**: collegare `Get World Delta Seconds`.
- **Interp Speed**: valore `float` che regola la velocità di rotazione. Valori tipici: 0.02–0.05 per un movimento fluido; valori più alti (es. 1–2) rendono la rotazione quasi istantanea (snap). È possibile promuoverlo a variabile per regolarlo dall'esterno.
- L'uscita dell'interpolazione è un singolo `float` Z interpolato.

4.13.5 Applicazione della rotazione al Pivot

- Creare un nuovo `Rotator` con `Make Rotator`: $X = 0$, $Y = 0$, $Z =$ valore interpolato.
- Trascinare il componente `Pivot` e chiamare `Set Relative Rotation` (**non** `Set World Rotation`), collegando il `Make Rotator` al pin `New Rotation`.

4.13.6 Gerarchia corretta dei componenti

Perché solo il supporto ruoti e non l'intera torretta, la struttura nel Blueprint deve essere:

```
Base (Static Mesh, root)
  |-- Pivot (Static Mesh)
        |-- Gun (Static Mesh)
```

- `Base` non ruota.
- `Pivot` è figlio di `Base`; applicando `Set Relative Rotation` solo a lui, ruoterà attorno all'asse Z locale.
- `Gun`, figlio di `Pivot`, seguirà automaticamente.
- Se accidentalmente si ruota la `Base` o l'intero Actor, verificare che il `Set Relative Rotation` sia chiamato esclusivamente sul componente `Pivot` e che la gerarchia sia quella indicata.

4.13.7 Allineamento delle mesh: offset di rotazione

I modelli importati potrebbero avere un orientamento predefinito non allineato con l'asse desiderato. In tal caso la torretta punterà in una direzione errata.

- Verificare l'orientamento della mesh del `Pivot` e, se necessario, reimportare l'asset con una rotazione correttiva.
- Per il progetto didattico, i valori di correzione sono:
  - `Pivot`: rotazione $Z = -113°$ (oppure $+113°$ a seconda dell'orientamento iniziale).
  - `Gun`: rotazione $Z = -271°$ (o il valore che la allinea correttamente al `Pivot`).
- La reimportazione si esegue dal `Content Browser`, cliccando con il tasto destro sull'asset → `Reimport` (o modificando il file sorgente e aggiornandolo).
- Questi offset vanno applicati **solo** alle mesh che ne hanno bisogno; la `Base` di solito non richiede modifiche.

4.13.8 Debug e problemi comuni

La torretta non ruota affatto

- Controllare che la variabile `TargetPlayer` sia stata effettivamente popolata. Nell'evento `OnBeginOverlap`, dopo aver verificato il tag, il nodo `Set` della variabile deve ricevere `Other Actor` e il flusso di esecuzione deve attraversarlo.
- Verificare che il `Timer` sia stato avviato correttamente (`Set Timer by Event`, con `Looping = True`, `Max Calls Per Frame = 1`, `Time = Get World Delta Seconds`).
- Assicurarsi che il `Custom Event` chiamato dal timer contenga la logica di rotazione e che non ci siano errori di compilazione.

La torretta ruota a scatti o in modo innaturale

- Regolare `Interp Speed`: diminuendolo si ottiene un movimento più fluido, aumentandolo si ha uno snap immediato.
- Se la velocità non cambia, verificare di aver collegato `Get World Delta Seconds` al pin `Delta Time` di `RInterp` e non un valore costante.

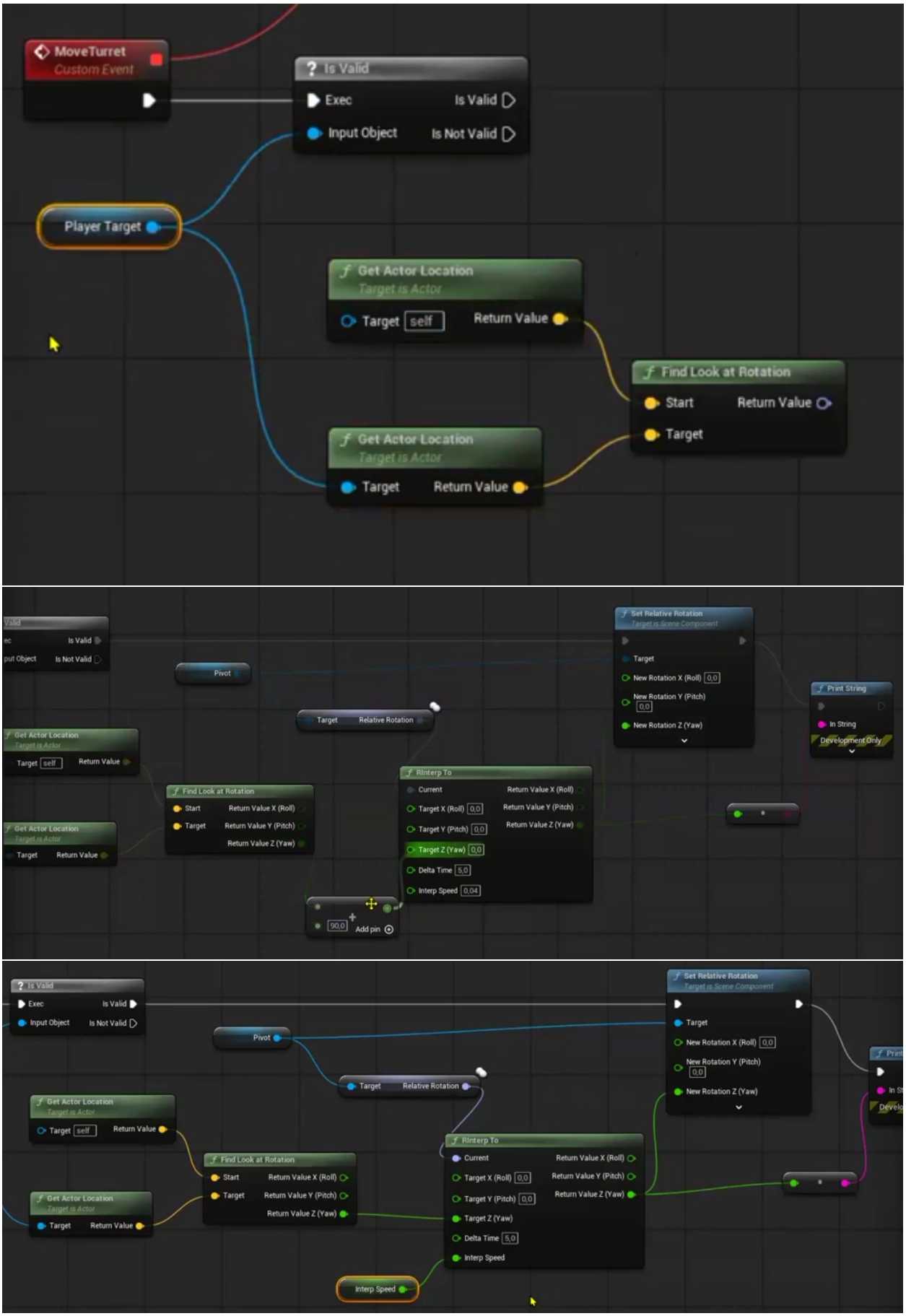


Figure 4.7: Blueprint del custom event della torretta completo. La figura in basso è riferita a quando si importa la mesh con offset di rotazione già perfezionato.

La sfera di collisione non si vede in gioco

- Selezionare il componente `Sphere Collision`, nella scheda *Details* → *Rendering*, impostare `Hidden in Game = False`.
- Aumentare il parametro `Sphere Radius` (es. 800) per coprire l'area desiderata.

4.13.9 Test finale

Posizionare il Blueprint `BP_Turret` nella scena (non la sola mesh statica), entrare con il personaggio nell'area della sfera e verificare che:

- La torretta ruoti fluidamente per seguire il giocatore.
- Solo il `Pivot` (e la `Gun`) ruoti, mentre la `Base` rimane ferma.
- Uscendo dal raggio la rotazione si fermi (grazie al controllo `IsValid` su `TargetPlayer`).

Chapter 5

Sistemi di Torretta in Unreal Engine 5: Spawning, Trace e Collisioni

5.1 Problemi di salvataggio e materiali

5.1.1 Segnalazione di un errore

Un allievo ha riscontrato la perdita dei materiali assegnati alla torretta e di parte della logica realizzata nelle lezioni precedenti, nonostante avesse salvato sia il Blueprint sia la mappa. In alcuni casi Unreal Engine può manifestare comportamenti anomali: alla chiusura non sempre richiede il salvataggio di tutti gli asset modificati, oppure può capitare che un file di riferimento venga spostato accidentalmente.

5.1.2 Buone pratiche di salvataggio

- Utilizzare sempre il pulsante **Save All** (in basso a destra) che mostra l'elenco di *tutti* gli asset modificati e permette di salvarli in un'unica operazione.
- Evitare di spostare o rinominare file dall'esterno dell'editor; eventuali riorganizzazioni vanno fatte tramite il **Content Browser**.

5.1.3 Materiali neri

Se un materiale appare di colore nero significa che è presente ma non ha una texture associata, oppure la texture non è stata caricata correttamente. Per rimediare:

- Aprire il materiale e verificare il nodo collegato al pin `Base Color`; se manca una texture, assegnarne una.
- In alternativa, usare un colore solido: fare doppio clic sul nodo costante (es. nero) per aprire la ruota dei colori e scegliere una tinta.
- È possibile duplicare un materiale esistente e modificarlo, per poi applicarlo alla static mesh della torretta.

5.2 Spawning: teoria e definizione

5.2.1 Cos'è lo spawning

Lo **spawning** è l'azione con cui l'engine *alloca in memoria* un nuovo elemento di gioco (actor) e lo rende visibile e interagibile nel mondo. A livello hardware corrisponde alla prenotazione di uno spazio di memoria RAM per quell'oggetto.

Figure 5.1: Codice per la decisione di colori della texture. Con un doppio click sarà poi possibile selezionare i colori con la palette colori.

5.2.2 Esempi di spawning

- **Minecraft**: il punto di spawn è il luogo in cui il giocatore compare all'inizio o dopo la morte.
- **Dark Souls**: il falò funge da checkpoint; morendo si ritorna alle coordinate del falò (respawn).
- **Bullet hell**: lo spawn continuo di proiettili (bullet) da parte dei nemici.

In tutti i casi, lo spawning richiede di specificare **dove** (una posizione nello spazio) e **cosa** (la classe dell'oggetto da creare).

5.2.3 Impatto sulle performance

Poiché ogni spawn alloca nuova memoria, un numero eccessivo di oggetti simultanei può causare:

- Calo dei fotogrammi al secondo (FPS).
- *Lag* percettivo.
- Saturazione della memoria disponibile.

Per questo motivo lo spawning va gestito con **parsimonia**.

5.2.4 Cenni all'Object Pooling

Una tecnica di ottimizzazione molto usata è l'**object pooling** (o *pool*): si crea un insieme (piscina) di oggetti identici che vengono attivati e disattivati all'occorrenza, anziché crearne e distruggerne di nuovi ogni volta. In questo modo si ricicla la memoria già allocata, riducendo il carico sul *garbage collector* (il sistema che si occupa di liberare la memoria dagli oggetti distrutti). L'argomento sarà approfondito più avanti.

5.3 Destroy Actor

L'operazione inversa allo spawn è la distruzione di un actor.

- Nodo `Destroy Actor`: richiede un `Target` (l'actor da eliminare).
- Quando un actor viene distrutto, la memoria che occupava viene marcata come spazzatura e, periodicamente, il **garbage collector** di Unreal la recupera.

5.4 Perfezionamento della torretta: Find Relative Look at Rotation

5.4.1 Problema con Find Look at Rotation

Nella lezione precedente si era usato il nodo `Find Look at Rotation` per orientare la torretta verso il giocatore. Questo nodo lavora nello spazio **mondo** (globale), causando un offset di 90° quando il componente `Pivot` ha una rotazione relativa diversa da zero. L'offset era stato corretto manualmente aggiungendo 90°, ma la soluzione non era robusta.

5.4.2 Soluzione: Find Relative Look at Rotation

Unreal Engine mette a disposizione il nodo `Find Relative Look at Rotation` (o `Find Look at Rotation Relative`), che calcola la rotazione **relativa** a un transform di partenza.

- **Target**: posizione del giocatore (`Get Actor Location` del `TargetPlayer`).
- **Transform**: transform del componente stesso (`Pivot` → `Get Component Transform`), che definisce il sistema di riferimento locale.

In questo modo il calcolo della direzione avviene nello spazio locale del componente, eliminando la necessità di correzioni manuali dell'angolo.

Modifica da applicare nel Blueprint:

1. Sostituire il nodo `Find Look at Rotation` con `Find Relative Look at Rotation`.
2. Collegare `Get Actor Location` del player al pin `Target`.
3. Ottenere il transform del `Pivot` con `Get Component Transform` e collegarlo al pin `Transform`.
4. Il resto del flusso (`RInterp`, `Set Relative Rotation`, ecc.) rimane invariato.

La torretta ora seguirà correttamente il giocatore indipendentemente dall'orientamento iniziale del componente `Pivot`.

5.5 Introduzione ai Cicli (Loop)

5.5.1 Definizione

Un **ciclo** (o *loop*) è una struttura di controllo che esegue ripetutamente un blocco di istruzioni **finché** una condizione rimane vera. Nei linguaggi di programmazione classici (C, C++, C#) si usano `while`, `do while` e `for`. In Blueprint il nodo principale è il `For Loop`.

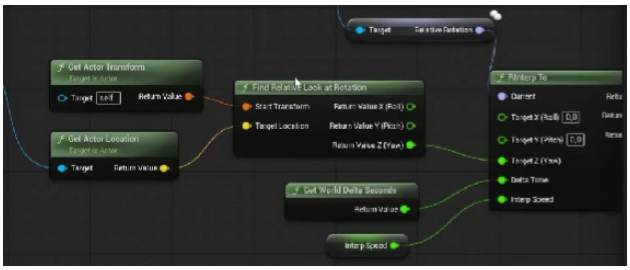


Figure 5.2: Sua maestà: Find Relative Look at Rotation.

5.5.2 For Loop in Unreal

Il nodo `For Loop` ripete un'operazione un numero determinato di volte, utilizzando un **indice** che parte da 0 e viene incrementato automaticamente a ogni iterazione.

- **First Index**: valore iniziale dell'indice (di default 0).
- **Last Index**: valore finale; il ciclo si interrompe quando l'indice lo supera.
- **Loop Body**: pin di esecuzione che viene attivato a ogni iterazione.
- **Index**: pin di uscita che fornisce il valore corrente dell'indice.
- **Completed**: pin di esecuzione attivato al termine del ciclo.

5.5.3 Esempio di utilizzo: spawning di una griglia

L'obiettivo è disporre una serie di cubi in una matrice (righe × colonne) senza posizionarli manualmente uno a uno. Per farlo si annidano due `For Loop`: uno esterno per le righe e uno interno per le colonne.

- Il `For Loop` esterno itera sulle righe (es. 0–4).
- All'interno, un secondo `For Loop` itera sulle colonne (es. 0–4).
- A ogni iterazione interna si calcola la posizione:

$$\begin{aligned} X &= \text{indice_colonna} \times \text{spaziatura}, \\ Y &= \text{indice_riga} \times \text{spaziatura}, \\ Z &= 0 \quad \text{(o altezza fissa)}. \end{aligned}$$

- Si utilizza il nodo `Spawn Actor` con:
  - **Class**: la classe dell'actor da spawnare (es. un cubo).
  - **Transform**: un `Make Transform` costruito con la posizione calcolata e rotazione/scala di default.

Questo metodo iterativo evita di scrivere lo stesso codice per ogni cubo e rende la scena parametrica.

5.6 Regolazione della velocità di rotazione (Interpolation Speed)

5.6.1 Modifica della variabile

Nella lezione precedente era stata creata una variabile `Interpolation Speed` per regolare la reattività della torretta. Per modificarla:

- Nel Blueprint della torretta, nel pannello `Variables` a sinistra, selezionare `Interpolation Speed`.

- Nel pannello `Details` (a destra) cambiare il `Default Value`.
- Valori tipici:
  - `0.01`–`0.05`: movimento molto fluido ma lento.
  - `1`–`2`: movimento rapido e reattivo, quasi istantaneo.
- Dopo la modifica, compilare e testare.

Con `Find Relative Look at Rotation` e un valore di `2.0`, la torretta risulta più reattiva e segue il giocatore in modo naturale.

5.7 Riepilogo e indicazioni per il proseguimento

- È stato risolto il problema di orientamento della torretta grazie al nodo `Find Relative Look at Rotation`.
- È stata introdotta la teoria dello spawning, distinguendo tra allocazione di un nuovo oggetto e riciclo tramite pool.
- È stato presentato il `For Loop` come strumento per operazioni ripetitive (es. creazione di una griglia).
- La prossima lezione si concentrerà sull'implementazione pratica dello spawning di proiettili dalla torretta e sulla loro distruzione all'impatto.

5.8 Problematiche di salvataggio e materiali

- **Segnalazione**: alcuni studenti hanno perso i materiali e la logica della torretta nonostante i salvataggi. Possibili cause: chiusura dell'editor senza la richiesta completa di salvataggio, spostamento involontario di file.
- **Pratica consigliata**: usare sempre il pulsante `Save All` (in basso a destra) per salvare tutto l'elenco degli asset modificati; evitare di spostare file manualmente fuori dall'editor.
- **Materiali neri**: indicano che il materiale è presente ma manca la texture collegata al `Base Color`. Si può aprire il materiale e assegnare una texture oppure un colore solido (doppio clic sul nodo costante → ruota dei colori). In alternativa, duplicare un materiale esistente e modificarlo.

5.9 Teoria dello Spawning

5.9.1 Definizione

Lo **spawning** è l'azione con cui l'engine *alloca in memoria* un nuovo oggetto di gioco (Actor) e lo rende visibile e interagibile nel mondo. Richiede di specificare:

- **Classe** (che tipo di oggetto creare, ad es. un proiettile).
- **Transform** (posizione, rotazione e scala iniziali).

5.9.2 Esempi

- **Minecraft**: il punto di spawn dove il giocatore compare all'inizio o dopo la morte.
- **Dark Souls**: il falò come checkpoint (respawn).
- **Bullet hell**: generazione continua di proiettili.

5.9.3 Impatto sulle performance

Ogni spawn occupa memoria. Un numero eccessivo di oggetti simultanei causa:

- Calo di frame rate (FPS).
- *Lag* e possibile saturazione della RAM.

Per questo lo spawning va usato con **parsimonia**.

5.9.4 Cenni all'Object Pooling

Tecnica di ottimizzazione: si crea una *pool* (piscina) di oggetti identici che vengono attivati/disattivati all'occorrenza, invece di crearne e distruggerne continuamente. In questo modo si riutilizza la memoria allocata e si riduce il carico sul **garbage collector**. L'argomento sarà approfondito in futuro.

5.10 Destroy Actor

L'operazione inversa allo spawn. Il nodo `Destroy Actor` elimina l'attore specificato come `Target`. La memoria viene poi recuperata dal garbage collector.

5.11 Perfezionamento della torretta

5.11.1 Find Relative Look at Rotation

Nella lezione precedente si era usato `Find Look at Rotation`, che lavora nello spazio *mondo*, causando offset quando il componente ha una rotazione relativa. Per risolvere:

- Sostituire con **Find Relative Look at Rotation** (o `Find Look at Rotation Relative`).
- **Target**: posizione del giocatore (`Get Actor Location`).
- **Transform**: transform del componente `Pivot` (`Get Component Transform`).

In questo modo il calcolo avviene nello spazio locale del componente, eliminando correzioni manuali dell'angolo. Il resto della logica (`RInterp`, `Set Relative Rotation`) rimane invariato.

5.11.2 Variabile `Interpolation Speed`

Per regolare la velocità di rotazione:

- La variabile `Interpolation Speed` (tipo `Float`) è stata creata con `Promote to Variable` a partire dal nodo `RInterp`.
- Dopo la compilazione, nel pannello `Details` si imposta il `Default Value`: valori bassi (es. 0.01) rendono il movimento fluido ma lento; valori alti (es. 2.0) lo rendono più reattivo.
- Collegare sempre `Delta Time` al nodo `RInterp` usando `Get World Delta Seconds` per garantire l'indipendenza dal frame rate.

5.12 Introduzione ai cicli: For Loop

5.12.1 Definizione

Un **ciclo** (loop) esegue ripetutamente un blocco di istruzioni finché una condizione è vera. In Blueprint il nodo principale è **For Loop**, che itera un numero determinato di volte usando un **indice** intero.

- `First Index`: valore iniziale (di default 0).

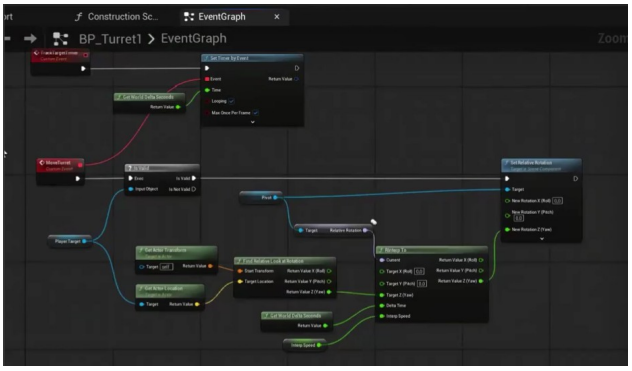


Figure 5.3: Codice finale prima di implementare il Line Trace.

- `Last Index`: valore finale (incluso).
- `Loop Body`: pin di esecuzione attivato a ogni iterazione.
- `Index`: fornisce il valore corrente dell'indice.
- `Completed`: esecuzione al termine del ciclo.

5.12.2 Applicazione: spawning di una griglia

Per disporre automaticamente una matrice di oggetti (es. cubi) si possono annidare due `For Loop`:

- Ciclo esterno per le righe, ciclo interno per le colonne.
- A ogni iterazione si calcola la posizione:

$$X = \text{indice_colonna} \times \text{spaziatura}, \quad Y = \text{indice_riga} \times \text{spaziatura}, \quad Z = 0$$

- Si usa `Spawn Actor` con classe e transform (`Make Transform`) costruiti con i valori calcolati.

Questo approccio parametrico evita la ripetizione manuale del codice.

5.13 Implementazione del Line Trace (Sphere Trace)

5.13.1 Concetto

Un **trace** è una linea immaginaria (o una forma, es. sfera) usata per rilevare collisioni con altri oggetti. Oggi si userà uno **Sphere Trace by Channel** per determinare se il giocatore è nel raggio di puntamento della torretta. Se il trace interseca il player, la torretta potrà sparare.

5.13.2 Creazione dell'evento Shoot

- Creare un `Custom Event` chiamato `Shoot` nel Blueprint della torretta.
- Prima di eseguire il trace, verificare che il `TargetPlayer` sia valido (`IsValid`) per evitare errori.

5.13.3 Punto di partenza: Scene Component

Per definire con precisione il punto da cui parte il trace (es. tra le due canne del cannone):

- Aggiungere un componente `Scene` (`Add Component` → `Scene`) e attaccarlo come figlio del `Gun`.
- Rinominarlo (es. `TraceSpawn`) premendo `F2`.

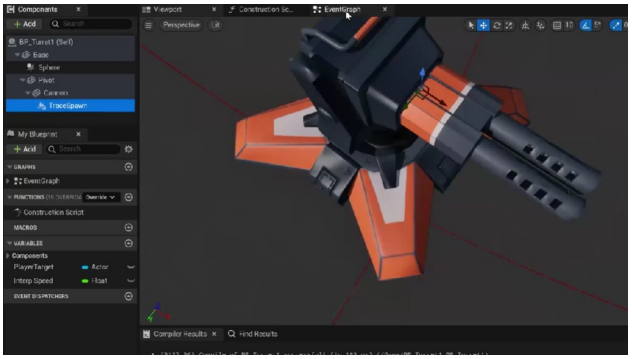


Figure 5.4: TraceSpawn creation.

- Il componente `Scene` non ha mesh, è un semplice punto di riferimento che seguirà la rotazione del cannone grazie alla gerarchia: `Base` → `Pivot` → `Gun` → `TraceSpawn`.

5.13.4 Direzione e lunghezza: Forward Vector

- Ottenere il **Forward Vector** (vettore locale che punta in avanti) del componente `TraceSpawn`: trascinare il componente nel grafo → `Get Forward Vector`. **Non** usare il `Get Actor Forward Vector` della torretta (rimarrebbe fisso).
- Il forward vector ha lunghezza unitaria (1). Per aumentare la portata del trace, moltiplicarlo per un valore (es. 800) tramite un nodo `Multiply` (float × vector). Se il nodo presenta pin vettoriali, fare tasto destro sul secondo pin e convertirlo in `Float`, inserendo poi 800.
- Il punto di **fine** del trace si ottiene **sommando** il punto di **inizio** al vettore direzione moltiplicato. Somma vettoriale: End = Start + (Forward × Lunghezza).

5.13.5 Nodo Sphere Trace by Channel

- **Start**: `Get World Location` del `TraceSpawn`.
- **End**: risultato della somma vettoriale (Start + Forward × Distanza).
- **Radius**: raggio della sfera di tracciamento (es. 80).
- **Trace Channel**: il canale di collisione da usare (di default `Visibility` o `Camera`). Per un rilevamento personalizzato, è meglio creare un canale dedicato.
- **Draw Debug Type**: per visualizzare il trace durante lo sviluppo, impostare su `For One Frame` e collegare all'esecuzione (ad es. `Event Tick` per il test).

5.13.6 Debug del tracciamento

Collegando il nodo `Shoot` (o il trace) all'`Event Tick` e impostando `Draw Debug Type` → `For One Frame`, è possibile vedere la sfera (o la linea) rossa nella viewport durante il gioco. La sfera diventa verde quando intercetta un oggetto nel canale di collisione impostato. Se non cambia colore, il canale non è configurato per rilevare il player.

5.13.7 Creazione di un Trace Channel personalizzato

Per far sì che il trace rilevi solo il giocatore e ignori il resto:

1. Menu `Edit` → `Project Settings` → sezione `Collision`.

Figure 5.5: Codice parziale del tracciamento della torretta.

2. Nella lista `Trace Channels`, cliccare `New Trace Channel`, dargli un nome (es. `DetectionTurret`).
3. Impostare `Default Response` su **Ignore**: di base il canale ignorerà tutte le collisioni.
4. Aprire il Blueprint del `ThirdPersonCharacter`, cercare la sezione `Collision` nei `Details` e, per il nuovo canale, cambiare la risposta da `Ignore` a **Block** (o **Overlap**). Così solo il personaggio risponderà al trace della torretta.

In questo modo il nodo `Sphere Trace by Channel` potrà usare il nuovo canale e attivarsi esclusivamente in presenza del giocatore.

5.14 Note finali e debug

- Se il forward vector non ruota insieme al cannone, assicurarsi di prenderlo dal componente `TraceSpawn` e non dall'intero attore.
- Se la linea del trace non compare o non segue il movimento, verificare i collegamenti: lo start deve essere la world location di `TraceSpawn`, l'end la somma vettoriale; il nodo `Draw Debug Type` deve essere eseguito (attaccare il pin di esecuzione a quello del trace, non solo i parametri).
- In caso di problemi di compilazione con tipi di pin (es. vettore invece di float), convertire il pin con tasto destro → `Convert Pin` (o cercare l'opzione `Float`).
- Per gestire l'offset della torretta (puntamento imperfetto) si può ricorrere al reimport del modello con rotazioni corrette (es. `Pivot` ruotato di $-113°$) oppure spostare manualmente il `TraceSpawn` dopo averlo attaccato al cannone.

Vedi Fig. 5.5.

5.15 Configurazione del canale di traccia personalizzato per la torretta

Dopo aver creato il nuovo `Trace Channel` (`DetectionTurret`) con risposta predefinita `Ignore`, occorre impostare il personaggio affinché risponda con `Block` su quel canale.

- Aprire il Blueprint del `ThirdPersonCharacter`.
- Nel pannello `Details` della `Mesh` (o del componente principale), cercare la sezione `Collision`.
- Se il canale personalizzato non compare, **compilare** il progetto (`Compile`).

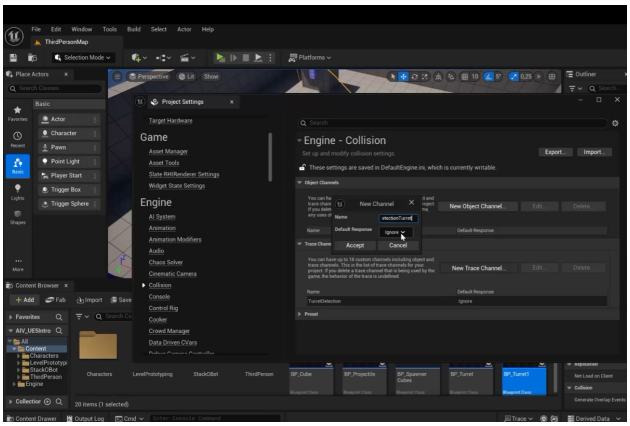


Figure 5.6: Schermata creazione DetectionTurret.

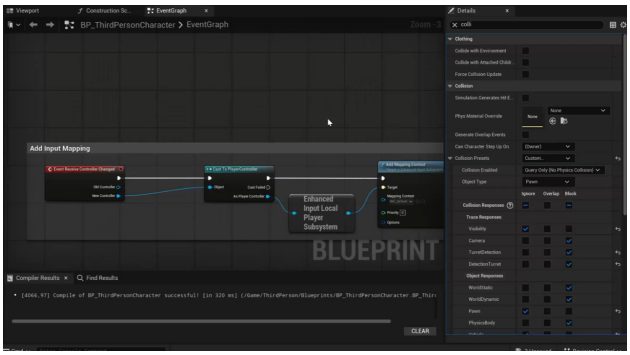


Figure 5.7: Blueprint del Player per vedere DetectionTurret.

- Trovare il canale `DetectionTurret` e impostare la risposta su `Block` (da `Ignore` a `Block`).
- Questo fa sì che solo il giocatore blocchi il trace; tutti gli altri oggetti lo ignoreranno perché la risposta predefinita del canale è `Ignore`.

5.15.1 Verifica del rilevamento

Tornati nel Blueprint della torretta, nel nodo `Sphere Trace by Channel` dell'evento `Shoot`:

- Se il canale `DetectionTurret` non appare nell'elenco `Trace Channel`, compilare il Blueprint.
- Selezionare `DetectionTurret`.
- Collegare un `Print String` al valore booleano di ritorno (`Return Value`) per visualizzare `True` (giocatore rilevato) o `False` (nessun rilevamento).

Durante l'esecuzione, la sfera di debug diventerà verde quando intercetta il giocatore, e il `Print String` mostrerà una sequenza di `True/False`.

5.16 Creazione del proiettile (BluePrint Actor)

Per permettere alla torretta di sparare, occorre creare un attore separato che rappresenti il proiettile.

1. Nel `Content Browser`, tasto destro → `Blueprint Class` → `Actor`, denominarlo `BP_Projectile`.

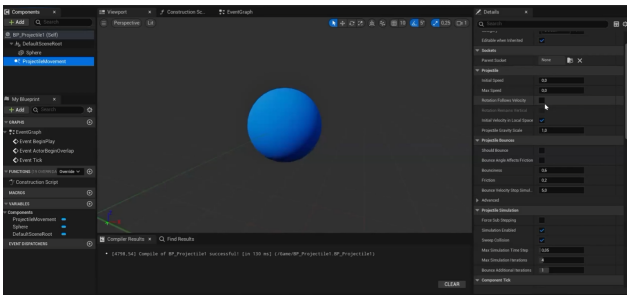


Figure 5.8: Creazione del Blueprint per il proiettile.

2. Aprire il Blueprint e aggiungere un componente `Static Mesh` (es. `Sphere`) come radice o figlio. Ridimensionare la sfera (es. `Scale = 0.3`) per ottenere un proiettile piccolo.
3. Aggiungere il componente **Projectile Movement**. Questo componente, fornito da Unreal, gestisce automaticamente il moto balistico.
4. Configurare i parametri principali:
   - `Initial Speed` e `Max Speed`: entrambi a `1000` (velocità elevata).
   - `Gravity Scale`: impostare a `0` per disabilitare la gravità e far sì che il proiettile viaggi in linea retta.

Se la gravità non viene azzerata, il proiettile seguirà una parabola e cadrà. Inoltre, senza un meccanismo di distruzione, il proiettile continuerebbe all'infinito, appesantendo la memoria.

5.17 Rilevamento delle collisioni del proiettile e autodistruzione

Per rilevare quando il proiettile colpisce qualcosa (muri, giocatore, ecc.) e distruggerlo, si utilizza un `Sphere Trace by Channel` sull'`Event Tick` del proiettile stesso.

5.17.1 Configurazione del Trace sul proiettile

- Nell'`Event Graph` di `BP_Projectile`, aggiungere un `Event Tick`.
- Inserire un nodo `Sphere Trace by Channel`.
- **Start** e **End**: entrambi collegati a `Get World Location` del proiettile (il trace diventa una sfera centrata sull'attore).
- **Radius**: impostare un valore come `30` (determina la grandezza della sfera di collisione).
- **Trace Channel**: scegliere un canale appropriato (vedi sotto).
- **Draw Debug Type**: impostare su `For One Frame` per visualizzare la sfera in fase di sviluppo.

5.17.2 Distruzione all'impatto

- Il pin `Return Value` del trace è un booleano: `True` se qualcosa viene colpito.
- Collegare un nodo `Branch` (premere `B` e cliccare) e connettere il `Return Value` al pin `Condition`.
- Sul ramo `True`, collegare un `Destroy Actor` con `Target` riferito al `Self` (l'attore stesso).

Il proiettile verrà eliminato non appena la sfera di traccia interseca un oggetto che risponde al canale impostato.

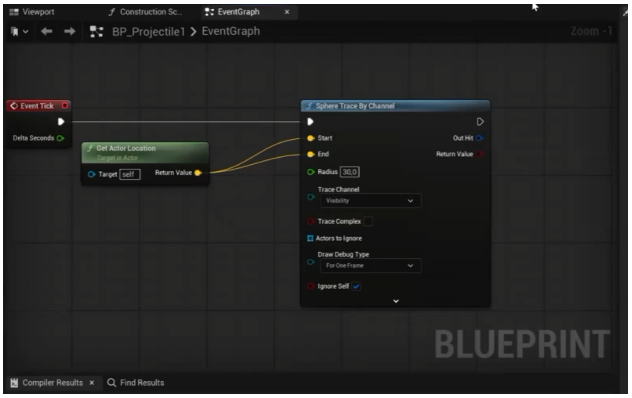


Figure 5.9: Blueprint per la distruzione del proiettile.

5.18 Canale di traccia per il proiettile

Il proiettile deve poter colpire sia i muri sia il giocatore, ma non la sfera di attivazione della torretta. Per ottenere questo comportamento si crea un nuovo `Trace Channel` personalizzato.

1. `Edit` → `Project Settings` → `Collision` → `New Trace Channel` (es. `ProjectileTrace`).
2. Impostare `Default Response` su `Block`: di base bloccherà tutto (muri, pavimenti, personaggio, ecc.).
3. Compilare il progetto.
4. Sul Blueprint della **torretta**, selezionare il componente `Sphere Collision` (quello per il range di attivazione). Nella sezione `Collision` → `Collision Responses`, trovare il canale `ProjectileTrace` e impostarlo su `Ignore`. Così la sfera di attivazione non intercetterà il proiettile.
5. Sul `BP_Projectile`, nel nodo `Sphere Trace by Channel`, selezionare `ProjectileTrace` come `Trace Channel`.

In questo modo il proiettile esplode su qualsiasi geometria (muri, pavimento) e sul giocatore, ma attraversa la sfera di attivazione della torretta senza distruggersi.

5.18.1 Nota sulla risposta del giocatore

Se il personaggio non viene rilevato, verificare che nella `Mesh` del `ThirdPersonCharacter` il canale `ProjectileTrace` sia impostato su `Block` (come tutte le altre geometrie). Poiché il canale ha default `Block`, il giocatore lo bloccherà automaticamente a meno che non sia stato modificato.

5.19 Affinità e regolazioni finali

- **Dimensione proiettile**: modificare la scala della mesh a `0.3` (o meno) per un aspetto più credibile.
- **Velocità**: riportare `Initial Speed` e `Max Speed` a `1000` per un comportamento da proiettile veloce.
- **Visibilità del trace di debug**: se la sfera di debug non compare, controllare che il `Radius` non sia `0` e che `Draw Debug Type` sia `For One Frame` con il flusso di esecuzione collegato al pin del trace (non solo i dati).
- **Proiettile che sposta il giocatore**: se il proiettile, anziché distruggersi, spinge il giocatore, verificare che il `Return Value` del trace sia collegato al `Branch` e che il `Destroy Actor` sia eseguito correttamente.

5.20 Ignorare il proiettile sulla torretta

Per evitare che il proiettile collida con la torretta che lo ha generato, occorre impostare la risposta a `Ignore` sul canale `ProjectileTrace` per tutti i componenti della torretta.

- Selezionare la `Sphere Collision` (sfera di attivazione) nel Blueprint della torretta.
- Nel pannello `Details`, alla voce `Collision Preset`, selezionare `Custom`.
- Nella sezione `Collision Responses`, individuare il canale `ProjectileTrace` e impostarlo su `Ignore`.
- Ripetere l'operazione per i componenti `Pivot`, `Base` e per la stessa `Sphere Collision` (se non già fatto), in modo che l'intera torretta venga ignorata dal proiettile.

5.21 Sessione di troubleshooting

5.21.1 Il tick dell'evento Shoot non funziona

Sintomo: il `Print String` legato al trace non compare e la sfera di debug non si vede.

- Verificare che il Blueprint della torretta non abbia un problema globale. Creare un nuovo `Actor` (es. `BP_Test`) con un `Event Tick` e un `Print String`: se il tick funziona nel nuovo attore, il problema è specifico della torretta.
- Controllare che non sia stato accidentalmente disabilitato il tick (`Enable Tick` nel componente principale).
- Rimuovere e reinserire l'attore della torretta nel livello: in alcuni casi la referenza interna può corrompersi.
- Se il problema persiste, sostituire il progetto con una copia funzionante fornita dal docente.

5.21.2 Orientamento del pivot e del trace

Sintomo: il cannone punta in direzione opposta al giocatore e il trace non è allineato.

- Verificare che la mesh del `Pivot` sia stata reimportata correttamente con rotazione $Z = -113°$.
- Se il reimport non è possibile, usare gli strumenti di modifica mesh: selezionare il `Pivot` nella scena, premere `Maiusc+5` per entrare in modalità `Transform`, e ruotare il componente in modo che l'asse rosso (X) punti in avanti. Confermare con `Accept`.
- Assicurarsi che il componente `TraceSpawn` (Scene) sia figlio del `Cannon` e abbia rotazione locale `0,0,0`.
- Dopo le modifiche, verificare che il trace segua la direzione del cannone e che la torretta ruoti verso il giocatore.

5.21.3 La torretta non segue il giocatore

Sintomo: la torretta ruota ma non punta correttamente, oppure la sfera di attivazione non si muove con la torretta.

- Controllare la gerarchia dei componenti: `Sphere Collision` deve essere figlia della `Base` (non viceversa).
- Se il `TraceSpawn` non è agganciato al `Cannon`, trascinarlo dentro il `Cannon` nella finestra `Components`.
- Verificare che la variabile `Interpolation Speed` sia impostata a un valore adeguato (es. 2.0) per una risposta reattiva.
- Assicurarsi che il `Find Relative Look at Rotation` riceva il `Transform` del `Pivot` (non del `Cannon` o della `Base`) e che il pin `Z` sia l'unico collegato.

5.22 Preparazione allo spawn del proiettile

Con la torretta ora correttamente orientata e il sistema di tracciamento funzionante, si può procedere con lo spawn effettivo del proiettile.

- Il nodo `Spawn Actor from Class` verrà inserito nell'evento `Shoot` (o in un nuovo `Custom Event`), utilizzando la classe `BP_Projectile` e il `Transform` calcolato a partire dalla posizione e direzione del `TraceSpawn`.
- Prima di spawnare, mantenere il controllo `IsValid` sul `TargetPlayer` e sul risultato dello `Sphere Trace` (per sparare solo se il giocatore è effettivamente rilevato).

La pausa è stata dedicata alla risoluzione dei problemi individuali; la lezione riprenderà con l'implementazione dello spawn e della logica di fuoco.

5.23 Spawning da due bocche con selezione casuale

Per simulare uno sparo da due canne, si creano due punti di spawn indipendenti e si sceglie casualmente quale utilizzare ad ogni colpo.

- Nel Blueprint della torretta, aggiungere due componenti `Scene` figli del `Cannon`, chiamandoli ad esempio `ProjectileSpawn1` e `ProjectileSpawn2`.
- Posizionarli ciascuno all'uscita di una bocca del cannone.
- Utilizzare un nodo `Select` (con tipo di ritorno `Transform` o `Vector`) pilotato da un `Random Bool`.
- `Random Bool` restituisce `true` o `false` in modo casuale; il nodo `Select` restituisce il `Get World Transform` del primo o del secondo punto di spawn in base al valore booleano.
- Collegare l'uscita del `Select` al nodo `Spawn Actor from Class` (campo `Spawn Transform`), dopo averne separato i pin con `Split Struct Pin`.

5.24 Controllo della cadenza di fuoco (Fire Rate)

Per evitare che la torretta spari ad ogni frame, si introduce un sistema di controllo basato su `Do Once` e `Delay`.

- A valle del `Branch` che verifica il `Return Value` dello `Sphere Trace` (quando `True`), inserire una `Sequence`.
- Dal primo pin `Then 0` della `Sequence`, collegare un nodo `Do Once`.
- Dal `Do Once` eseguire lo spawn del proiettile (collegare al flusso di esecuzione dello `Spawn Actor from Class`).
- Dal secondo pin `Then 1` della `Sequence`, collegare un `Delay` che ritornerà al `Reset` del `Do Once`.
  - La durata del `Delay` determina il *fire rate* (intervallo tra un colpo e il successivo). Promuovere il valore a variabile (`Promote to Variable`) e chiamarla `FireRate` per poterla regolare facilmente (es. 0.5 secondi).
- Il `Do Once` impedisce che il proiettile venga spawnato più volte durante lo stesso intervallo; il `Delay` riattiva la possibilità di sparare dopo il tempo stabilito.

Per far sì che il primo colpo parta immediatamente quando il giocatore viene rilevato, il `Do Once` deve essere resettato quando il giocatore non è più valido (vedi sezione successiva).

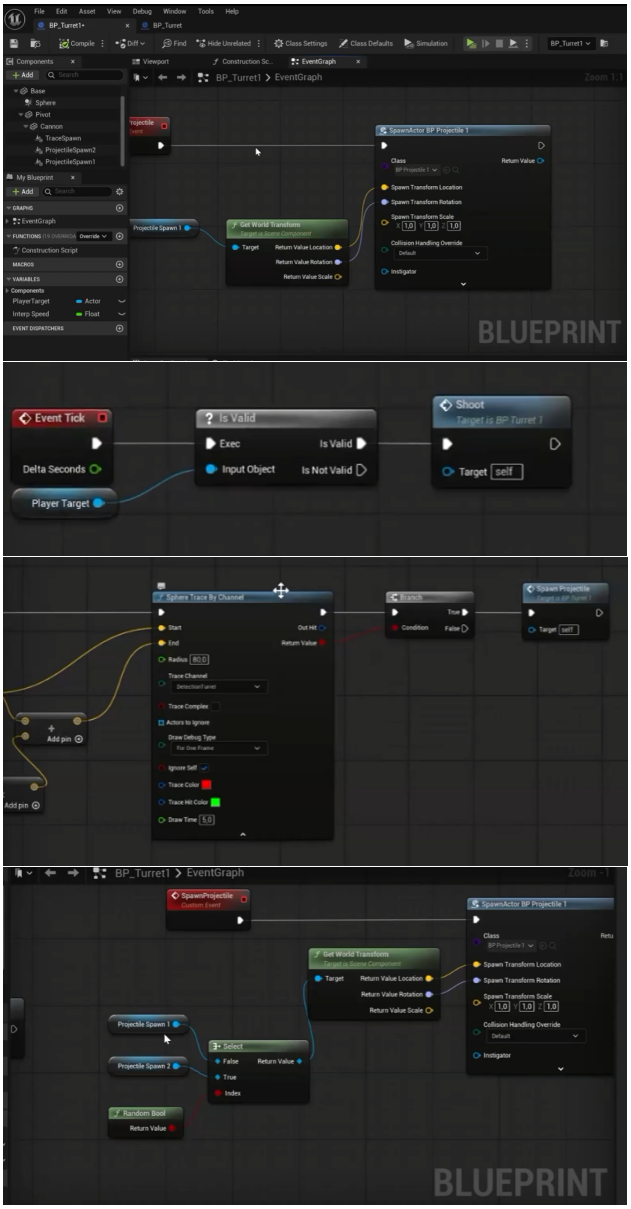


Figure 5.10: BP del proiettile.

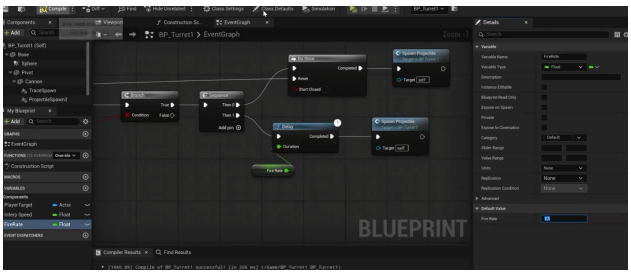


Figure 5.11: BP per far sparare con delay tra un proiettile e l'altro.

5.25 Arresto del fuoco all'uscita del giocatore

Quando il giocatore esce dalla sfera di attivazione, la torretta deve smettere di sparare.

- Selezionare il componente `Sphere Collision` e aggiungere l'evento `On Component End Overlap`.
- Nell'evento, recuperare `Other Actor` e verificare che abbia il tag `Player` (`Actor Has Tag`).
- Se vero, impostare la variabile `TargetPlayer` a **vuoto** (collegare un `Set` senza fornire alcun input, oppure trascinare il `Set` e non collegare il pin di input). Una variabile di tipo `Actor` non inizializzata risulta `IsValid = false`.
- A questo punto:
  - Il controllo `IsValid` su `TargetPlayer` nell'evento `Shoot` fallisce, interrompendo la logica di sparo.
  - Il `Do Once` deve essere resettato automaticamente. Per farlo, si può sfruttare il ramo `Is Not Valid` del controllo su `TargetPlayer` (a monte dello `Shoot`) e collegare il suo flusso di esecuzione al `Reset` del `Do Once`. In alternativa, si può resettare il `Do Once` nello stesso evento `End Overlap` dopo aver svuotato il target.

In questo modo la torretta smette di sparare immediatamente dopo che il giocatore esce dal raggio, e il `Do Once` viene ripristinato per un nuovo eventuale ingresso.

5.26 Risoluzione dei problemi più comuni

5.26.1 Proiettile che non si muove o si distrugge appena spawnato

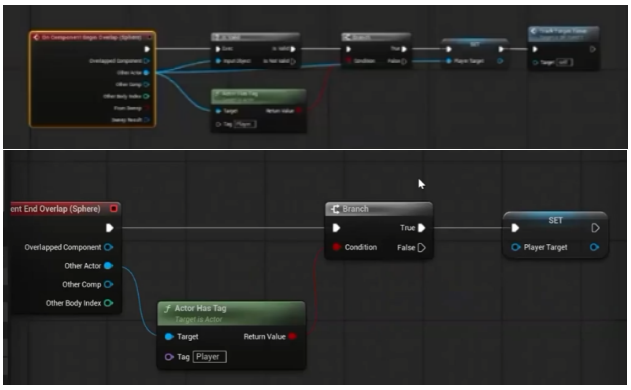
- Verificare che il `BP_Projectile` abbia il componente `Projectile Movement` con `Initial Speed` e `Max Speed` impostati (es. 1000).
- Assicurarsi che il proiettile non collida con se stesso: nel `BP_Projectile`, selezionare la `Sphere` (o la mesh) e nelle `Collision Responses` impostare il canale `ProjectileTrace` su `Ignore`.
- Assicurarsi che tutti i componenti della torretta (`Base`, `Pivot`, `Cannon`, `Sphere Collision`) ignorino il canale `ProjectileTrace`.
- Controllare che il raggio (`Radius`) del `Sphere Trace` sul proiettile non sia troppo grande (ad es. 30 potrebbe ancora colpire la torretta; ridurlo a 10 se necessario).

5.26.2 La torretta spara troppi proiettili o a raffica

- Il `Do Once` non è stato inserito o il `Delay` non lo resetta. Verificare il flusso: `Sequence` → `Do Once` → `Spawn`; `Sequence` → `Delay` → `Reset` del `Do Once`.
- Se il `Do Once` non viene resettato, il fuoco avviene una volta sola. Se manca del tutto, ogni `Tick` positivo spara un proiettile.

5.26.3 Torretta non spara pur rilevando il giocatore

- Controllare che il `Sphere Trace by Channel` dell'evento `Shoot` utilizzi il canale `DetectionTurret` e che il giocatore risponda `Block`.
- Verificare che il `Return Value` del trace sia collegato al `Branch` e che l'esecuzione successiva sia corretta.
- Controllare che la variabile `TargetPlayer` sia effettivamente popolata e valida all'ingresso del giocatore.
- Se il proiettile si distrugge subito, rivedere le collisioni (punto precedente).



5.27 Introduzione all'esercizio: matrice di cubi con For Loop

L'ultima parte della lezione introduce un nuovo esercizio: generare una griglia di cubi disposti ordinatamente (una matrice) utilizzando i cicli `For Loop`.

- Si partirà creando un nuovo `Actor` vuoto (es. `BP_CubeGrid`).
- All'interno del suo `Event BeginPlay` (o su chiamata manuale) si utilizzeranno due `For Loop` annidati.
- Il ciclo esterno gestirà le righe (asse Y), quello interno le colonne (asse X).
- Ad ogni iterazione interna si calcolerà la posizione del cubo come:

$$X = \text{indice_colonna} \times \text{spaziatura}, \quad Y = \text{indice_riga} \times \text{spaziatura}, \quad Z = 0$$

- La spaziatura sarà un valore costante (es. 200 unità).
- Si userà il nodo `Spawn Actor from Class` con classe `StaticMeshActor` (o un cubo predefinito) e un `Transform` costruito tramite `Make Transform` con la posizione calcolata e rotazione/scala di default.

L'implementazione pratica sarà completata nella prossima lezione.

5.28 Ciclo For per lo spawn di una fila di cubi

- Si crea un nuovo `Custom Event` chiamato `Spawner` (o `SpawnRow`) nel Blueprint dell'attore che gestirà la griglia.
- All'interno dell'evento si inserisce un nodo `For Loop` (da `Flow Control`).
- Impostare `First Index` a `1` e `Last Index` a `10` (poi eventualmente modificabile; con `Last Index = 20` si ottengono 20 iterazioni).
- Ad ogni iterazione, il pin `Index` restituisce il valore corrente (1, 2, 3, ...). Moltiplicando l'Index per una spaziatura costante (es. `100`) si ottiene uno spostamento progressivo:

$$\text{Offset} = \text{Index} \times 100$$

- Per calcolare la posizione di spawn:
  - Ottenere la posizione attuale dell'attore (`Get Actor Location`) e separarla nei tre assi (`Split Struct Pin`).
  - Sommare l'Offset all'asse X (o all'asse desiderato) tramite un nodo `Add` (float + float).
  - Ricostruire il vettore posizione con `Make Vector` (o un `Make Transform`) lasciando invariati Y e Z.
- Il valore `Last Index` determina il numero totale di elementi: con `Last Index = 10` si avranno 10 posizioni (da 1 a 10). Per ottenere più posizioni basta aumentare il `Last Index`.

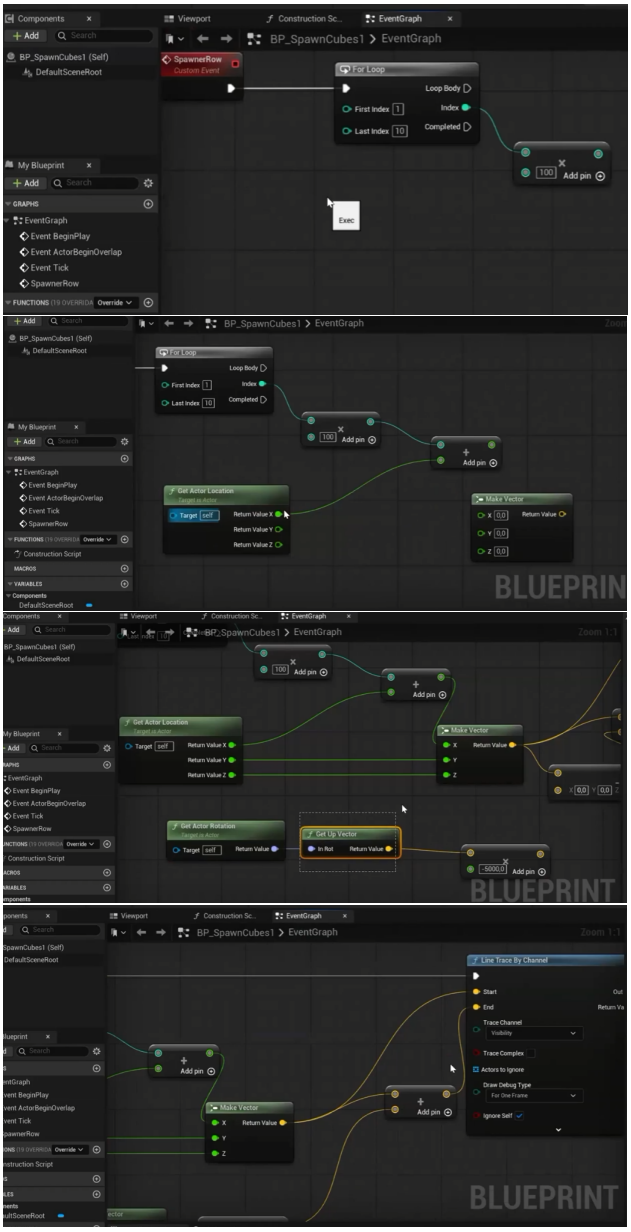


Figure 5.12: Inizio BP della griglia dei cubi.

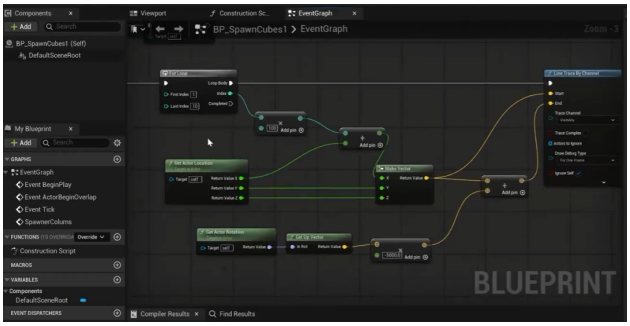


Figure 5.13: Codice finale della torretta che spara.

5.29 Visualizzazione delle posizioni con Line Trace

Per verificare la distribuzione delle posizioni calcolate si può usare un `Line Trace by Channel` disegnato per ogni punto, con una linea verticale verso il basso.

- All'interno del corpo del `For Loop`, dopo aver costruito il vettore di posizione, aggiungere un nodo `Line Trace by Channel`.
- **Start**: il vettore appena calcolato (posizione X incrementata, Y e Z originali).
- **End**: lo stesso vettore a cui viene sommato un vettore diretto verso il basso.
- Per ottenere un vettore verso il basso relativo all'attore:
  - Prendere la rotazione dell'attore (`Get Actor Rotation`) e da essa ricavare il vettore `Up` (`Get Up Vector`).
  - Moltiplicare `Up Vector` per un valore negativo molto grande (es. `-5000`) per ottenere un vettore che punta verso il basso e lungo a sufficienza per essere visibile.
  - Sommare questo vettore al punto di Start per ottenere l'`End`.
- Impostare `Draw Debug Type` su `For One Frame` e collegare il flusso di esecuzione.
- Eseguendo l'evento (es. collegandolo momentaneamente a `Event Tick` per test), si vedranno tante linee verticali quante sono le iterazioni, equispaziate sull'asse X.
- Se le linee risultano oblique o non allineate, verificare che il vettore `Up` sia ricavato dalla rotazione attuale e moltiplicato per un valore negativo, e che la somma sia corretta.

5.30 Regolazione del puntamento della torretta (approfondimento)

Uno studente ha notato che la torretta, quando il giocatore è lontano, tende a mirare in basso (ai piedi). Si discutono due possibili soluzioni per alzare il punto di mira.

5.30.1 Offset manuale sulla posizione del target

- Nel nodo `Find Relative Look at Rotation` (o `Find Look at Rotation`), invece di usare direttamente la posizione del giocatore (`Get Actor Location`), si può sommare un offset alla componente Z.
- Ad esempio, aggiungere un valore positivo (es. `+100`) alla Z del target, in modo che la torretta punti più in alto.
- Lo studente ha risolto calcolando la differenza di altezza tra il pivot della base e il pivot della canna, e sottraendo quel valore alla Z del target: `Target.Z -= AltezzaBase - AltezzaCanna`. In questo modo la mira risulta allineata indipendentemente dalla distanza.

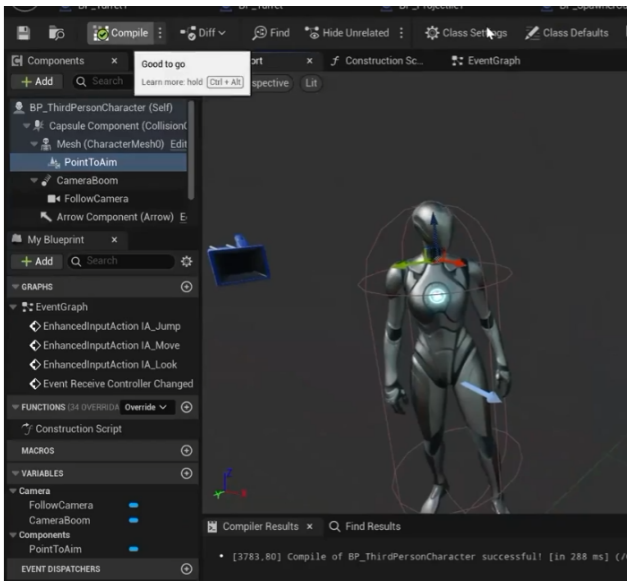


Figure 5.14: Creazione actor da colpire.

5.30.2 Uso di un punto di riferimento sul giocatore

Alternativamente, si può creare un componente `Scene` posizionato sul corpo del giocatore (es. all'altezza del collo o della testa) e usare la sua world location come target per il `Find Look at Rotation`. Questo richiede una referenza al componente specifico del `ThirdPersonCharacter`, da memorizzare in una variabile di tipo `Scene Component` o `Actor`.

5.31 Conclusioni e prossima lezione

- Il `For Loop` è stato utilizzato per generare una fila di posizioni (asse X) con spaziatura costante. La logica sarà estesa nella prossima lezione per creare una griglia bidimensionale (righe e colonne) tramite un secondo `For Loop` annidato, che gestirà l'asse Y.
- Verrà completato l'esercizio della matrice di cubi, con lo spawn effettivo di attori (`Spawn Actor from Class`) alle coordinate calcolate.
- La prossima lezione riprenderà dall'inizio dell'argomento, con un breve ripasso della logica dei cicli e della costruzione delle coordinate, per consolidare i concetti.

Chapter 6

Fondamenti di Programmazione e Interattività in Unreal Engine 5: Timeline, Collisioni e Audio

6.1 Introduzione e comunicazioni

- **Materiale didattico:** è stato condiviso su Discord un file audio (formato `WAV`, l'unico accettato da Unreal Engine per l'importazione diretta; il formato `MP3` non è supportato) da utilizzare durante l'esercitazione odierna su audio e piattaforme mobili.
- **Problema tecnico ricorrente:** all'apertura dell'Epic Games Launcher compare un messaggio che avvisa che i file di progetto non sono associati. È sufficiente cliccare `Fix Now` per risolvere temporaneamente; il problema sembra essere comune dopo gli ultimi aggiornamenti e non comporta rischi per i progetti.
- **Registrazioni:** le lezioni registrate rimarranno disponibili sulla piattaforma fino a settembre.
- **Organizzazione della lezione:** si completerà l'esercizio sulla matrice di cubi con i `For Loop`, per poi passare a un nuovo argomento: la creazione di una piattaforma mobile, l'uso delle **Timeline** e l'integrazione di effetti audio.

6.2 Riepilogo teorico

6.2.1 Ciclo `For Loop`

- È una struttura iterativa che esegue un'azione per un numero determinato di volte.
- I parametri principali sono `First Index` (indice di partenza) e `Last Index` (indice finale). A ogni iterazione l'indice viene incrementato automaticamente.
- I cicli possono essere **annidati** (un `For Loop` dentro un altro) per generare griglie bidimensionali (righe $\times$ colonne).
- Utilizzi tipici: spawnare un numero prefissato di nemici, creare file di oggetti, controllare condizioni iterative.

6.2.2 Collisioni: Ignore, Overlap, Block

Le collisioni in Unreal Engine possono assumere tre modalità principali, configurabili tramite i **Collision Preset** e le **Object Responses**.

- **Ignore**: nessuna interazione né fisica né di notifica. L'oggetto viene completamente ignorato.
- **Overlap**: non c'è blocco fisico (gli oggetti si attraversano), ma l'engine genera una notifica (*evento*) di inizio o fine sovrapposizione. È ampiamente usato per:
  - Attivare cutscene (volumi invisibili).
  - Rilevare la presenza del giocatore in un'area (come nella sfera della torretta).
  - Trigger per porte, trappole, cambi di musica.
- **Block**: collisione fisica completa (impedisce il movimento) e contemporaneamente genera l'evento di collisione. È il comportamento predefinito per muri, pavimenti e oggetti solidi.

6.2.3 Object Type e Object Responses

- Ogni oggetto possiede un **Object Type** (es. `WorldStatic`, `WorldDynamic`, `Pawn`, `PhysicsBody`) che ne definisce la categoria.
- Le **Object Responses** specificano come l'oggetto risponde a ciascuna categoria: `Ignore`, `Overlap` o `Block`.
- Queste impostazioni si trovano nel pannello `Details` dell'attore, nella sezione `Collision`. È possibile scegliere un `Collision Preset` predefinito oppure impostare `Custom` per regolare manualmente le risposte.

6.2.4 Eventi di collisione: Begin Overlap e End Overlap

- `On Component Begin Overlap` e `On Actor Begin Overlap`: si attivano quando un altro attore o componente inizia a sovrapporsi.
- `On Component End Overlap` e `On Actor End Overlap`: si attivano quando la sovrapposizione termina.
- I nodi forniscono riferimenti all'attore e al componente che hanno causato l'overlap, utili per identificare chi ha attivato l'evento (es. controllare il tag `Player`).

6.2.5 Branch (condizione)

Il nodo `Branch` rappresenta una diramazione condizionale (if/else): se la condizione booleana è `True` esegue il ramo `True`, altrimenti quello `False`. Già utilizzato nella torretta per decidere se sparare o meno.

6.3 Timeline

6.3.1 Definizione e caratteristiche

Una **Timeline** è un oggetto che consente di eseguire animazioni e interpolazioni di valori lungo una linea temporale definita.

- Funziona in modo **asincrono** rispetto all'`Event Tick`: non dipende dal frame rate, ma da un tempo proprio.
- Permette di animare parametri (posizione, rotazione, colore, ecc.) su una durata specifica (es. 2 secondi).
- È composta da una o più **tracce** (`Tracks`), ciascuna delle quali può gestire un tipo di dato: `Float`, `Vector`, `Color`, `Event`.
- Ogni traccia contiene una **curva** che definisce l'andamento del valore nel tempo (es. lineare, ease-in, ease-out).

- Eventi esposti dalla Timeline:
  - `Update`: chiamato a ogni variazione del valore, fornisce il valore corrente della traccia.
  - `Finished`: eseguito al termine della Timeline.
  - `Play`, `Play from Start`, `Stop`, `Reverse`: controlli per avviare/fermare la Timeline.

6.3.2 Float Track

La traccia più semplice e quella che verrà utilizzata oggi è la **Float Track**: gestisce un singolo valore numerico (es. da 0 a 1, o da -90 a 90) lungo il tempo. Può essere usata per ruotare una piattaforma, muovere un oggetto lungo un asse, modulare l'intensità di una luce, ecc.

6.4 Approfondimento sui Trace Channels (domanda studente)

Domanda: In un gioco reale, quanti `Trace Channel` vengono creati? Si cerca di ottimizzare o se ne crea uno per ogni oggetto?

Risposta:

- I `Trace Channels` vanno usati con parsimonia: sono uno strumento preciso ma potenzialmente pesante se abusato (specialmente se eseguiti ogni frame).
- Vengono impiegati per azioni che richiedono elevata precisione, come il **coyote time** nei platform (la possibilità di saltare anche se si è già oltre il bordo per pochi frame) o il rilevamento del terreno sotto il personaggio.
- In un gioco tipico, non si crea un canale per ogni oggetto, ma si definiscono pochi canali specifici (es. `Visibility`, `Camera`, più alcuni personalizzati per meccaniche particolari come il rilevamento del giocatore per la torretta).
- Un utilizzo eccessivo di `Trace` in **Blueprint** può portare a problemi di performance; in progetti complessi, le funzioni critiche vengono spesso implementate in **C++** per un controllo più fine.
- Buona pratica: **isolare** la logica degli attori, limitare l'uso di tracce solo quando necessario, e **disabilitare il** `Tick` quando l'attore non deve eseguire alcuna operazione (per ridurre il carico sulla CPU).

6.5 Dimostrazione pratica: configurazione delle collisioni

- Il docente mostra un cubo (`WorldDynamic`) e una sfera (`PhysicsBody`) nella scena.
- Selezionando la sfera e modificando le `Object Responses` (ad esempio impostando `Block` verso `WorldDynamic`), la sfera rimbalzerà contro il cubo; impostando `Overlap`, lo attraverserà ma genererà un evento.
- Le collisioni possono essere configurate anche tramite i `Collision Presets` (es. `BlockAll`, `OverlapAll`, `PhysicsActor`) o tramite la modalità `Custom` per regolare manualmente ogni risposta.
- La stessa logica si applica ai componenti all'interno di un Blueprint: ogni `Static Mesh`, `Sphere Collision`, `Box Collision` può avere le proprie impostazioni di collisione.

6.6 Esercitazioni previste

1. Completamento dell'esercizio sulla matrice di cubi con `For Loop` annidati (spawn di una griglia di attori).
2. Creazione di una piattaforma mobile con l'ausilio di una `Timeline` (Float Track) per animare la posizione o la rotazione.

3. Integrazione di un effetto audio (`WAV`) al movimento della piattaforma.

Il materiale audio necessario è già disponibile nel file condiviso su Discord. La lezione riprende in engine con la pratica.

6.7 Collisioni: Overlap, Ignore e Block

6.7.1 Dimostrazione pratica

Per illustrare i concetti di collisione, vengono utilizzati una sfera (`PhysicsBody`) e un cubo (`WorldDynamic`).

- Impostando la risposta `WorldDynamic` della sfera verso il cubo su **Overlap**, i due oggetti si attraversano fisicamente ma l'engine genera una notifica.
- Aggiungendo un evento `On Component Begin Overlap` nel cubo e collegandovi un `Print String`, ogni volta che la sfera entra in contatto con il cubo viene stampato un messaggio, pur senza blocco fisico.
- Impostando la risposta su **Ignore**, non si verifica alcuna interazione: né fisica, né di notifica.
- Impostando su **Block**, si ha sia la collisione fisica (la sfera rimbalza) sia la notifica dell'evento.

6.7.2 Gestione dei conflitti tra risposte

Quando due oggetti hanno impostazioni di collisione in conflitto (es. uno `Block` e l'altro `Ignore`), prevale la risposta più permissiva? In realtà, la logica di Unreal richiede che entrambe le parti concordino: se una delle due ignora, la collisione viene ignorata. Per evitare ambiguità, è buona pratica:

- Creare **Object Channel** personalizzati (`Project Settings` → `Collision`).
- Assegnare risposte coerenti a entrambi gli oggetti coinvolti.
- Se l'oggetto in movimento (es. proiettile) deve overlappare, impostare l'overlap su entrambi.

6.8 Completamento esercizio: spawn di cubi con `For Loop`

6.8.1 Riepilogo della logica

L'obiettivo è disporre una fila di cubi con spaziatura costante utilizzando un `For Loop`.

- Il `Custom Event` (es. `Spawner`) contiene un `For Loop` con `First Index = 1` e `Last Index = 10`.
- A ogni iterazione, l'`Index` (1, 2, 3, ...) viene moltiplicato per `100` (spaziatura in centimetri, ovvero 1 metro).
- L'offset risultante viene sommato alla componente `X` della `Get Actor Location`.
- Si costruisce un `Make Vector` con la nuova `X`, mantenendo invariate `Y` e `Z`.
- Si esegue un `Line Trace by Channel` dal punto calcolato verso il basso (usando `Get Up Vector` moltiplicato per un valore negativo, es. `-5000`, e sommandolo al punto di start). Il debug `For One Frame` mostra le linee verticali equidistanti.

6.8.2 Spawn del cubo al punto di impatto

Quando il `Line Trace` colpisce una superficie (es. il terreno), si vuole spawnare un cubo in quel punto esatto.

- Dal nodo `Line Trace by Channel` si ottiene il `Out Hit` (struttura dati). Utilizzando `Break Hit Result` si estrae il campo **Location**, che rappresenta il punto di impatto.
- Un nodo `Branch` verifica il `Return Value` (booleano) del trace: se `True` (impatto avvenuto), procede con lo spawn.
- Si usa `Spawn Actor from Class`, scegliendo come classe un Blueprint `BP_Cube`.

6.8.3 Creazione del Blueprint BP_Cube

- Nuovo Actor → BP_Cube.
- Aggiungere un componente Static Mesh (cubo).
- Ridimensionare: Scale = 0.5 su tutti gli assi.
- Posizionare: Z Location = 50 per sollevarlo ed evitare che sia interrato.
- Chiudere il Blueprint e usarlo come classe nello Spawn Actor from Class dello spawner.

6.8.4 Problema: i cubi spawnati interferiscono con i trace successivi

Quando il ciclo tenta di spawnare più cubi, i primi cubi generati bloccano il Line Trace dei successivi, causando uno spawn a cascata sullo stesso punto (effetto "pa-pa-pa").

Soluzione: Modificare le impostazioni di collisione del BP_Cube.

- Aprire BP_Cube, selezionare la Static Mesh.
- Nel pannello Details, sotto Collision, impostare Collision Preset su Custom.
- Nella sezione Trace Responses, trovare il canale Visibility e impostarlo su **Ignore**.
- In questo modo i cubi spawnati vengono ignorati dal Line Trace (che usa il canale Visibility) e non interferiscono con le rilevazioni successive.
- In alternativa, si può impostare il preset No Collision se il cubo non deve interagire con nulla.

Salvando e rieseguendo, i cubi vengono disposti correttamente in fila, ognuno al punto di impatto del proprio trace.

6.8.5 Introduzione al secondo ciclo: le righe

Per trasformare la fila in una matrice (colonne × righe), si introduce una **Sequence**.

- Il nodo Sequence permette di eseguire più flussi di esecuzione in parallelo (pin Then 0, Then 1, ecc.).
- Al momento dello spawn di ogni colonna, si vuole anche avviare un secondo For Loop che iteri sull'asse Y (righe), in modo da generare l'intera griglia.
- Questo secondo loop, annidato nel primo, gestirà lo spostamento lungo Y in modo analogo a quanto fatto per X: Y offset = Index_riga * Spaziatura.
- L'implementazione completa sarà oggetto della prossima lezione.

6.9 Completamento della matrice di cubi con For Loop annidati

6.9.1 Logica della griglia

Per generare una matrice 10×10 di cubi (o dimensione variabile), si utilizza un For Loop per le colonne (asse X) e un secondo For Loop annidato per le righe (asse Y).

- **Primo loop (colonne)**: itera su X con First Index = 1 e Last Index desiderato (es. 10). Ad ogni iterazione calcola $X_{\text{offset}} = \text{Index} \times \text{Spaziatura}$ e lo somma alla X della posizione base.
- **Secondo loop (righe)**: inserito all'interno del corpo del primo loop, itera su Y con gli stessi indici. Somma l'offset alla Y della posizione base, **mantenendo la stessa** X calcolata dal primo loop.
- In questo modo, per ogni colonna X, vengono generate tutte le righe Y, coprendo l'intera area.

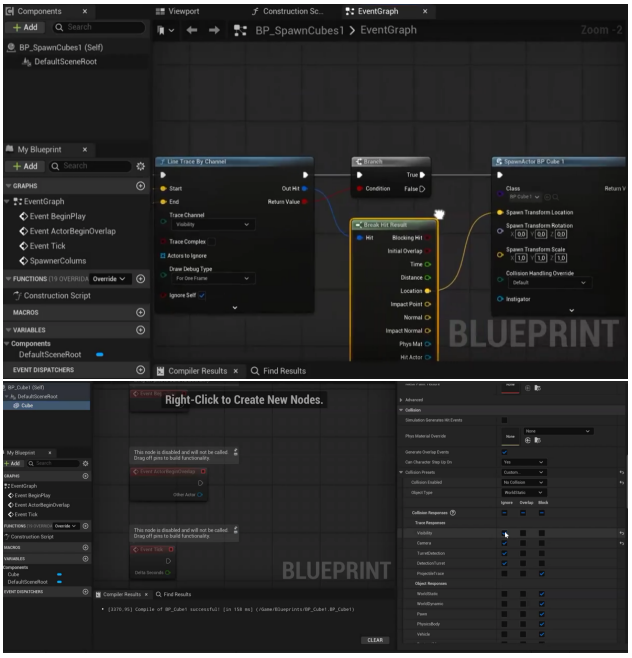


Figure 6.1: BP del BreakHit Result. Nota: bisogna modificare il collision handling.

6.9.2 Realizzazione pratica

1. Nel `Custom Event` (es. `Spawner`), dopo il primo `For Loop` che calcola X, aggiungere una `Sequence` per gestire i due flussi in parallelo.

2. Dal pin `Then 1` della `Sequence`, inserire un secondo `For Loop` con gli stessi `First Index` e `Last Index`.

3. All'interno del secondo loop:

   - Moltiplicare l'`Index` per la spaziatura (es. 100) per ottenere l'offset Y.
   - Sommare questo offset alla componente Y della posizione base (`Get Actor Location` → `Split Struct Pin`, usare Y originale).
   - Per la X, **non** usare la X originale, ma quella già calcolata nel primo loop (trascinare il nodo della X calcolata fino al `Make Vector` del secondo loop).
   - La Z rimane invariata.

4. Costruire il vettore di partenza con `Make Vector` (usando la X del primo loop, la Y offsettata, la Z originale).

5. Eseguire il `Line Trace by Channel` verso il basso (come nel caso di una sola fila) per trovare il punto di impatto.

6. All'uscita del `Line Trace` (se `Return Value = True`), spawnare il `BP_Cube` con `Spawn Actor from Class` usando la `Location` dell'`Break Hit Result`.

7. Collegare correttamente i nodi di esecuzione: `Loop Body` del secondo loop → `Line Trace` → `Spawn Actor`.

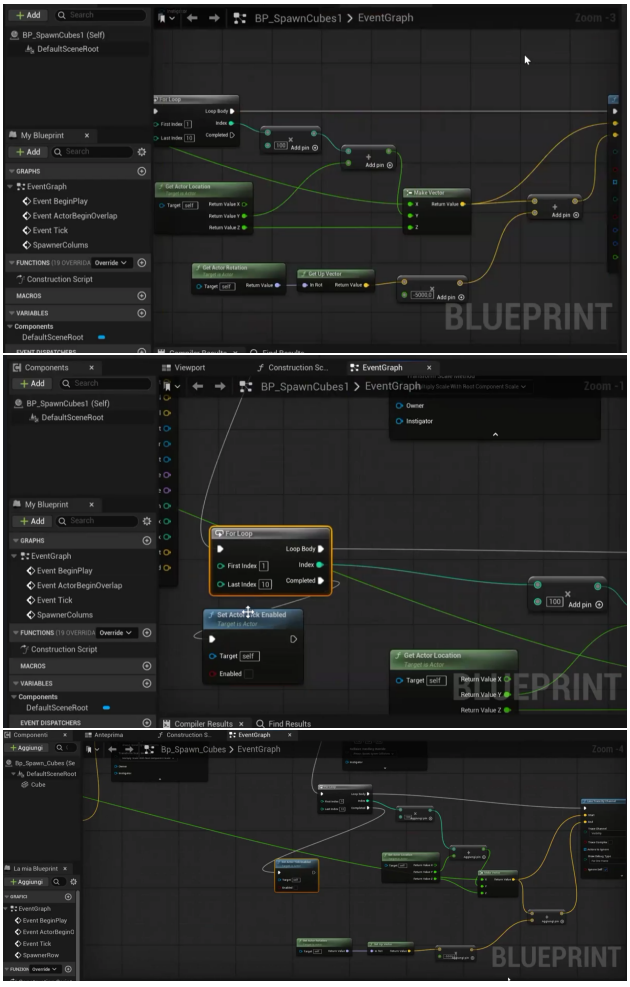


Figure 6.2: Parte del BP della creazione cubi.

6.9.3 Ottimizzazione: disabilitare il Tick

Al termine dell'intera operazione (pin `Completed` del **secondo** `For Loop`), disabilitare il `Tick` dell'attore per evitare esecuzioni successive.

`Completed` → `Set Actor Tick Enabled` (impostare `Enabled = False`)

Questo riduce il carico computazionale una volta che la griglia è stata generata.

6.9.4 Risoluzione problemi comuni

- **I cubi formano un muro compatto invece di una griglia**: probabilmente i cubi sono troppo grandi o la spaziatura è insufficiente. Ridurre la `Scale` del `BP_Cube` (es. 0.2 su tutti gli assi) o aumentare la distanza di spaziatura.
- **Il secondo loop non esegue**: verificare che il `Loop Body` sia collegato al nodo successivo (`Line Trace`) e che tutti i pin di esecuzione siano connessi.
- **Manca il movimento verso il basso nel trace**: assicurarsi di aver aggiunto il nodo `Add` tra il `Make Vector` e il vettore diretto verso il basso (`Get Up Vector` × `-5000`) per calcolare il punto di `End` del `Line Trace`.
- **I cubi spawnati interferiscono con i trace successivi**: nel `BP_Cube`, impostare la `Collision Preset` su `Custom` e nella `Trace Responses` impostare `Visibility` su `Ignore` (come già visto).

6.10 Nuovo esercizio: piattaforma mobile con Timeline

6.10.1 Obiettivo

Realizzare una piattaforma che si attivi (ad es. inizi a muoversi) quando il giocatore vi sale sopra e ritorni alla posizione iniziale quando il giocatore scende, utilizzando una `Timeline` per animare il movimento e integrando un effetto audio.

6.10.2 Creazione del Blueprint `BP_Platform`

- Creare un nuovo `Actor` e chiamarlo, ad esempio, `BP_Platform`.
- Aggiungere un componente `Static Mesh` e assegnargli la mesh `SM_ModularPlatform` (o simile).
- Aggiungere un componente **Box Collision** e posizionarlo sopra la piattaforma, ridimensionandolo per coprire l'area di attivazione (es. Scale $X = 7$, $Y = 7$, Z opportuna). La `Box Collision` deve essere **figlia** della `Static Mesh` in modo che la segua nei movimenti.
- Rinominare i componenti per chiarezza: `Platform` (Static Mesh) e `TriggerBox` (Box Collision).

6.10.3 Rilevamento del giocatore con Overlap

- Selezionare la `Box Collision` e aggiungere gli eventi `On Component Begin Overlap` e `On Component End Overlap` nel `Event Graph`.
- In entrambi gli eventi, trascinare il pin `Other Actor` e creare un nodo `Equal (Actor)` (operatore `==`) per confrontarlo con il riferimento al giocatore.
- Per ottenere il riferimento al giocatore, usare il nodo `Get Player Character` (index 0) o, in alternativa, controllare un tag (`Actor Has Tag` con `"Player"`). Il metodo `Get Player Character` è immediato e diretto.
- Collegare il risultato del confronto a un `Branch`: se `True`, procedere con l'attivazione o disattivazione della `Timeline`.

6.10.4 La Timeline

- Aggiungere un nodo `Timeline` nell'`Event Graph` e dargli un nome (es. `TL_Platform`).
- Fare doppio clic sul nodo per aprire l'editor della Timeline.
- Aggiungere una **Float Track** (premere il pulsante `f` o cliccare su `Add Float Track`). Assegnarle un nome (es. `Alpha`).
- Definire la curva: due keyframe, uno al tempo 0 con valore 0, l'altro al tempo finale (es. 2 secondi) con valore 1. L'andamento può essere lineare o con easing (es. ease in/out).
- La Timeline espone automaticamente i pin di controllo:
  - `Play`: avvia la Timeline dalla posizione corrente.
  - `Play from Start`: reimposta a 0 e avvia.
  - `Stop`: ferma l'esecuzione.
  - `Reverse`: inverte la direzione.
  - `Reverse from End`: reimposta alla fine e inverte.
  - `Set New Time`: imposta un tempo specifico.
- Il pin `Update` fornisce il valore corrente della traccia (es. il float da 0 a 1).
- Il pin `Finished` viene attivato al termine della Timeline.

6.10.5 Integrazione nella logica della piattaforma

- Nell'evento `On Component Begin Overlap` (dopo aver verificato che l'attore sia il giocatore), collegare il flusso a `Play from Start` della Timeline. In questo modo la piattaforma inizierà a muoversi dalla posizione iniziale.
- Nell'evento `On Component End Overlap` (sempre dopo la verifica), collegare il flusso a `Reverse from End` per far tornare la piattaforma alla posizione di partenza.
- Nel corpo dell'`Update` della Timeline, utilizzare il valore `Alpha` (compreso tra 0 e 1) per interpolare la posizione o la rotazione della piattaforma. Ad esempio, con un `Lerp (Vector)` tra la posizione iniziale e quella finale, oppure con un semplice offset sulla Z:

$$\text{Nuova Posizione Z} = \text{Z iniziale} + \alpha \times \text{AltezzaMassima}$$

- Applicare la nuova posizione al componente `Platform` (o all'intero attore) tramite `Set Relative Location` o `Set Actor Location`, assicurandosi che la mobilità sia `Movable`.

6.10.6 Audio (cenni)

Successivamente si integrerà un suono (file `WAV`) da riprodurre durante il movimento della piattaforma. L'audio è già stato distribuito su Discord. Verrà utilizzato un componente `Audio` o un nodo `Play Sound at Location` per riprodurre il suono quando la piattaforma inizia a muoversi.

6.11 Riepilogo

- La matrice di cubi è stata completata usando `For Loop` annidati, con attenzione a condividere la coordinata X tra i due cicli e a disabilitare il `Tick` al termine.
- La piattaforma mobile utilizza `Box Collision` per rilevare il giocatore e una `Timeline` con `Float Track` per animare il movimento in modo fluido e indipendente dal frame rate.
- I prossimi passi prevedono il completamento dell'animazione della piattaforma e l'aggiunta dell'audio.

6.12 Completamento della piattaforma mobile: Timeline e Lerp

6.12.1 Riepilogo: movimentazione lungo l'asse Z

La piattaforma deve muoversi verticalmente (asse Z) in un intervallo di tempo definito, attivandosi quando il giocatore entra nella `Box Collision` e tornando alla posizione iniziale quando esce. Il movimento è gestito da una `Timeline` con una `Float Track`.

6.12.2 Creazione e configurazione della Timeline

- Doppio clic sul nodo `Timeline` per aprire l'editor.
- Aggiungere una `Float Track` (pulsante `f` o `Add Float Track`) e chiamarla, ad esempio, `MovementPlatformZ`.
- Impostare la lunghezza della Timeline (`Length`) a `1.0` secondi (modificabile a piacere).
- Inserire due keyframe sulla curva:
  - Tempo 0.0, Valore 0.0 (posizione di partenza).
  - Tempo 1.0, Valore 1.0 (posizione di arrivo, corrispondente all'alpha massimo).
- Tenere premuto `Shift` e cliccare sul grafico per aggiungere un punto; trascinarlo per regolare tempo e valore.
- La curva può essere lineare o smussata (ease in/out) a seconda delle necessità.

6.12.3 Utilizzo del valore della Timeline: il nodo Lerp (Interpolazione Lineare)

Il pin `Update` della Timeline fornisce ad ogni frame il valore corrente della traccia (compreso tra 0 e 1). Questo valore, detto **alpha**, indica la posizione percentuale lungo la linea del tempo:

- alpha = 0: inizio del movimento.
- alpha = 0.5: metà del movimento.
- alpha = 1: fine del movimento.

Per spostare la piattaforma in modo proporzionale, si utilizza un nodo **Lerp (Float)** o una combinazione di **Add** e **Multiply**. La soluzione più semplice e robusta adottata è:

1. Ottenere la posizione relativa iniziale della piattaforma: trascinare il componente `Platform` nel grafo, chiamare `Get Relative Location`, separare i pin (`Split Struct Pin`) e prelevare solo il valore Z.
2. Collegare questo Z al pin `A` del `Lerp (Float)`.
3. Creare una variabile (o una costante) `MaxZOffset` (es. `70.0`) che rappresenta l'altezza massima aggiuntiva.
4. Sommare `MaxZOffset` al Z iniziale: nodo `Add` (Z + MaxZOffset) e collegare il risultato al pin `B` del `Lerp`.
5. Collegare il valore `Alpha` (dalla Timeline) al pin `Alpha` del `Lerp`.
6. Il `Lerp` restituirà un valore Z compreso tra A e B in base all'alpha corrente.

6.12.4 Ricostruzione del vettore e applicazione del movimento

- Creare un nodo `Make Vector`.
- Collegare:
  - X: il valore X originale della `Relative Location`.
  - Y: il valore Y originale della `Relative Location`.
  - Z: l'uscita del `Lerp`.
- Trascinare nuovamente il componente `Platform` e chiamare `Set Relative Location`, collegando l'uscita del `Make Vector` al pin `New Location`.
- Assicurarsi che la mobilità del componente `Platform` sia impostata su `Movable`.

6.12.5 Attivazione e disattivazione con Overlap

- Nell'evento `On Component Begin Overlap` (dopo aver verificato che l'attore sia il giocatore tramite `Get Player Character` e `Equal`), collegare l'esecuzione a `Play from Start` della Timeline. La piattaforma inizierà a salire.
- Nell'evento `On Component End Overlap`, collegare l'esecuzione a `Reverse from End`. La piattaforma tornerà alla posizione iniziale.
- Grazie alla Timeline, il movimento è fluido e indipendente dal frame rate.

6.12.6 Risoluzione dei problemi comuni

- **La piattaforma non si muove o si muove a scatti**: verificare che il `Lerp` sia correttamente collegato (A = Z iniziale, B = Z iniziale + offset, Alpha = uscita Timeline). Controllare che il `Make Vector` e il `Set Relative Location` siano eseguiti ogni frame tramite il pin `Update` della Timeline.
- **La piattaforma sale troppo in alto**: regolare il valore di `MaxZOffset` (es. 70 → 30) in base alle dimensioni della scena.
- **La piattaforma non torna alla posizione esatta**: verificare che il pin `A` del `Lerp` sia il Z iniziale **non** alterato (no offset). Se si usa un `Add` prima del `Lerp`, assicurarsi che A sia proprio il valore base.
- **Errori di compilazione sul tipo dei pin**: `Lerp (Float)` richiede pin `Float`. Se qualche collegamento è errato (es. `Vector` invece di `Float`), separare i pin con `Split Struct Pin`.

6.13 Pausa e comunicazioni

- La lezione prosegue dopo una pausa di circa 20 minuti. Durante la pausa si sono discussi annunci videoludici recenti (es. *Kingdom Hearts 4*, *Ocarina of Time Remake*) come momento di svago.
- Si ricorda agli studenti che le registrazioni delle lezioni resteranno disponibili sulla piattaforma fino a settembre.
- L'argomento della prossima lezione sarà il perfezionamento della piattaforma e l'integrazione dell'audio.

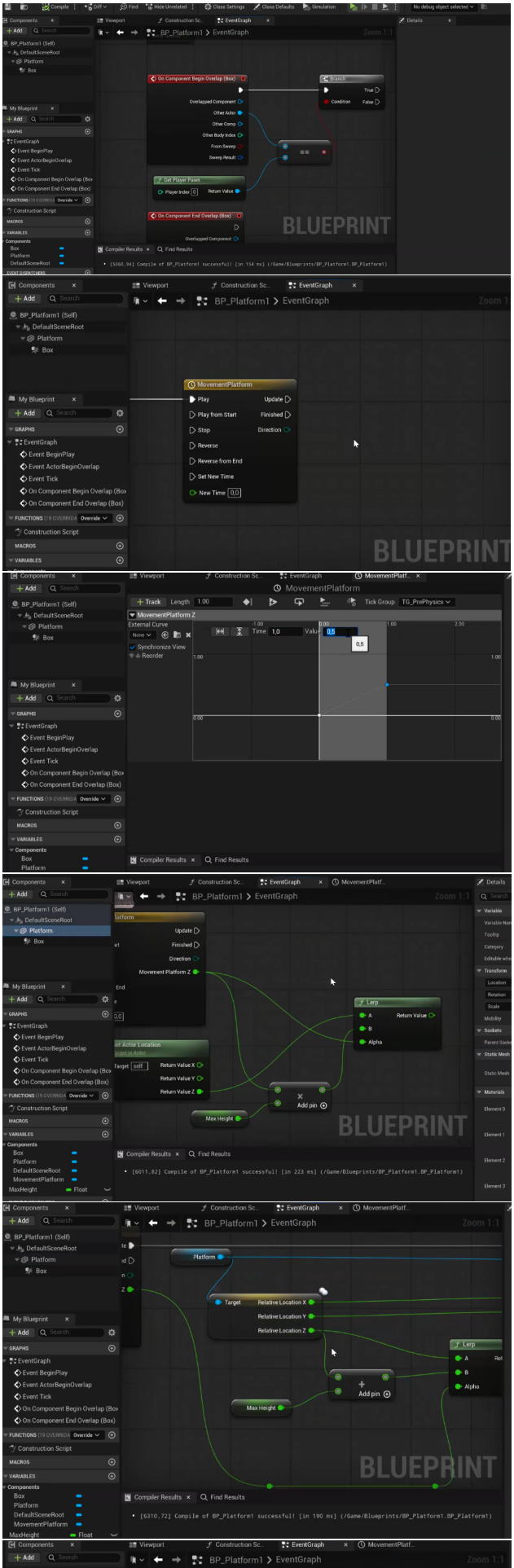

BP_Platform1 > EventGraph
On Component Begin Overlap (Box)
Branch
Get Player Pawn
On Component End Overlap (Box)
BLUEPRINT
MovementPlatform
Play
Play from Start
Stop
Reverse
Reverse from End
Set New Time
Update
Finished
Direction
New Time
MovementPlatform
Track
Length
Tick Group
TG_PrePhysics
MovementPlatform Z
External Curve
Synchronize View
Reorder
Lerp
A
B
Alpha
Return Value
Add pin
Max Height
Return Value X
Return Value Y
Return Value Z
Platform
Target
Relative Location X
Relative Location Y
Relative Location Z
Compiler Results
Find Results

Figure 6.4: BP per la velocità del giocatore.

6.14 Perfezionamento della piattaforma: controllo della velocità e posizionamento preciso

6.14.1 Regolazione del Play Rate della Timeline

Per modificare la velocità di esecuzione della Timeline senza alterare la lunghezza della traccia, si utilizza il nodo `Set Play Rate` sul componente Timeline.

- Ottenere un riferimento alla Timeline: nel `Event Graph`, trascinare il nodo della Timeline (es. `MovementPlatform`) e chiamare `Set Play Rate`.
- `Play Rate = 1.0`: velocità normale (la Timeline impiega il tempo definito dalla sua lunghezza).
- `Play Rate = 0.5`: velocità dimezzata (durata doppia).
- `Play Rate = 2.0`: velocità doppia (durata dimezzata).
- Il `Play Rate` non modifica la lunghezza della Timeline ma la velocità di scorrimento del tempo interno.

6.14.2 Variabile `Duration` e calcolo automatico del Play Rate

Per esporre la durata desiderata come parametro di design, si può calcolare il `Play Rate` in modo che la Timeline si adatti automaticamente.

- Creare una variabile `Float` chiamata `Duration` (es. valore di default 1.0 secondi).
- Ottenere la lunghezza della Timeline con il nodo `Get Timeline Length` (dal riferimento alla Timeline).
- Calcolare il `Play Rate` come:

$$\text{PlayRate} = \frac{1.0}{\text{Duration}} \times \text{TimelineLength}$$

- Se la lunghezza della Timeline è 1.0 secondo, il `Play Rate` diventa semplicemente 1/Duration.
- Applicare il valore calcolato a `Set Play Rate` durante l'`Event Begin Play`.
- Con `Duration = 2.0` la piattaforma impiegherà 2 secondi per completare la corsa (velocità dimezzata); con `Duration = 0.5` impiegherà mezzo secondo (velocità doppia).

6.14.3 Problema del metodo additivo e introduzione del Billboard

Il metodo precedente basato su `Add` (posizione iniziale $+\alpha \times$ offset) può risultare meno flessibile e portare a imprecisioni se la Timeline non viene eseguita completamente o se la velocità non è quella attesa. Per un controllo più robusto e visivo della posizione di arrivo si introduce un **Billboard**.

- Un `Billboard` è un componente simile a una `Scene`, ma dotato di uno sprite visibile nell'editor (utile come riferimento).
- Aggiungere un componente `Billboard` al Blueprint della piattaforma, posizionarlo nel punto esatto che si vuole raggiungere (ad esempio spostandolo sull'asse Z di 500 unità).
- Il `Billboard` deve essere **figlio** del componente radice (es. `DefaultSceneRoot`) affinché non si muova insieme alla piattaforma durante l'animazione (se invece fosse figlio della mesh, si sposterebbe con essa, causando un movimento infinito).
- Per sicurezza, salvare le posizioni di partenza e di arrivo in variabili all'inizio del gioco.

6.14.4 Salvataggio delle posizioni in variabili

Per evitare che i riferimenti diretti alle coordinate vengano alterati durante il movimento:

- In `Event Begin Play`:
  - Ottenere la `Get Relative Location` del componente `Platform` e promuoverla a variabile (`Promote to Variable`) chiamata `StartLocation`.
  - Ottenere la `Get Relative Location` del `Billboard` e promuoverla a variabile chiamata `FinishLocation`.
- Queste variabili conterranno i valori fissi iniziali e non cambieranno durante il gioco.

6.14.5 Interpolazione vettoriale con `VLerp`

Invece di interpolare solo la componente Z, si utilizza un `Lerp` vettoriale per spostare l'intera piattaforma dal punto di partenza a quello di arrivo.

- Nodo `VLerp` (Vector Lerp):
  - `A`: collegare la variabile `StartLocation`.
  - `B`: collegare la variabile `FinishLocation`.
  - `Alpha`: collegare l'uscita `Update` della Timeline (valore tra 0 e 1).
- Il risultato è un vettore che si sposta linearmente da `A` a `B` al variare di α.
- Applicare la nuova posizione al componente `Platform` con `Set Relative Location`.
- Questo metodo permette di muovere la piattaforma in **qualsiasi direzione** (non solo verticale) semplicemente posizionando il `Billboard` dove si desidera, senza ulteriori calcoli matematici.
- Inoltre, il movimento rimane corretto indipendentemente dalla velocità di esecuzione della Timeline, perché il valore α mappa sempre l'intera escursione tra i due punti.

6.14.6 Gestione del ritorno con `Reverse`

Quando il giocatore esce dalla `Box Collision`, la piattaforma deve tornare alla posizione iniziale.

- Nell'evento `On Component End Overlap` (dopo aver verificato che si tratti del giocatore), collegare il flusso a `Reverse` della Timeline.
- `Reverse` ripercorre la Timeline dall'attuale posizione verso l'inizio, riportando la piattaforma da `B` ad `A`.
- Non è necessario creare una seconda Timeline o un nuovo `Lerp`: la stessa traccia, invertita, gestisce il ritorno.

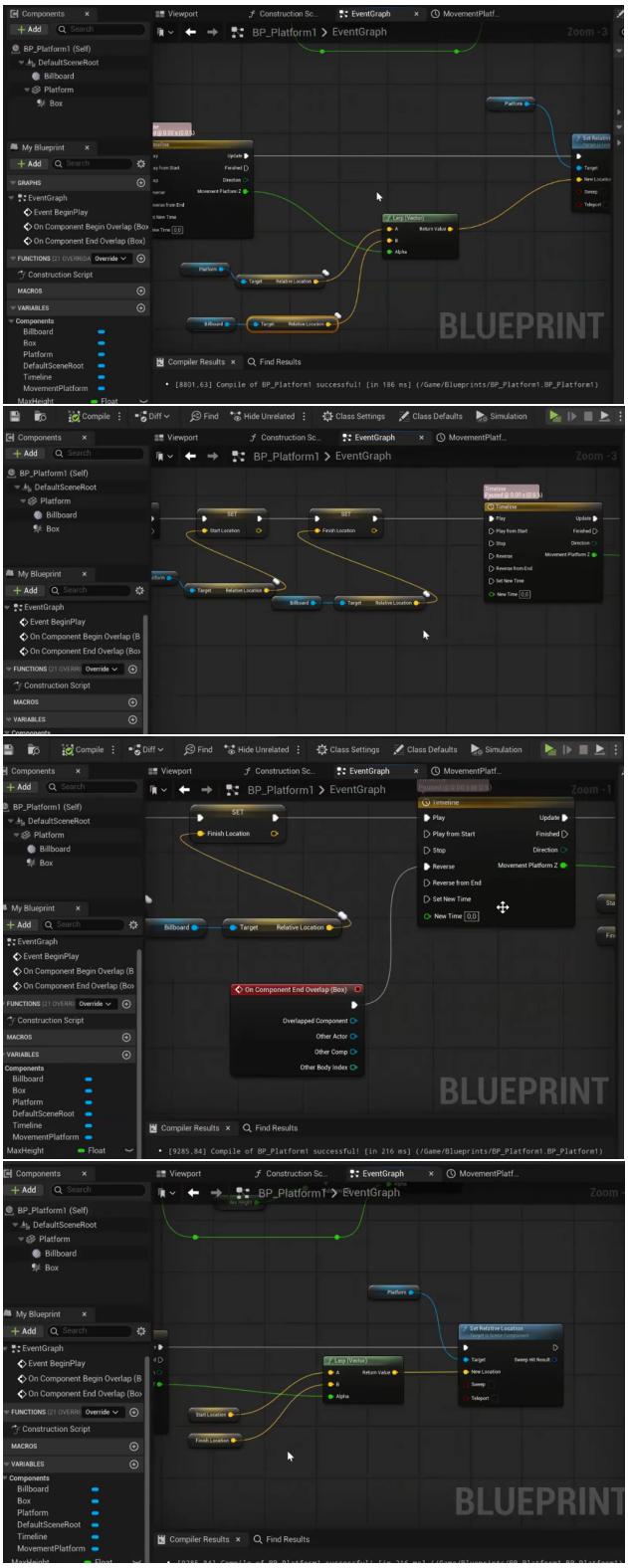


Figure 6.5: BP della logica della piattaforma che raggiunge il Billboard.

6.15 Integrazione dell'audio (introduzione)

- Sul server Discord del corso è stato fornito il file audio `sfx_moving_platform.wav` (formato WAV, l'unico importabile direttamente in Unreal).
- L'audio verrà riprodotto durante il movimento della piattaforma, utilizzando un nodo `Play Sound at Location` o un componente `Audio`.
- I dettagli implementativi verranno affrontati nella prossima lezione.

6.16 Integrazione dell'audio nella piattaforma mobile

6.16.1 Regolazione del Play Rate e risoluzione problemi

Si è osservato che il nodo `Set Play Rate` non influenzava il movimento. Il motivo era un errore di riferimento: la Timeline utilizzata nell'esempio precedente si chiamava `MovementPlatform`, mentre la nuova Timeline creata per la piattaforma aveva un nome diverso (es. `Timeline`). Ogni Timeline ha un **nome univoco** e per modificare il `Play Rate` occorre referenziare la Timeline corretta.

- Ottenere il riferimento alla Timeline dal pannello `Components` (si trova sotto la voce `Timeline` o con il nome assegnato, ad es. `TL_Platform`).
- Collegare il nodo `Set Play Rate` a questo riferimento.
- Con `Duration = 3.0`, il movimento diventa più lento (la Timeline impiega 3 secondi).
- Se non si ottiene l'effetto desiderato, verificare che il nome della Timeline nel nodo coincida con quello nella lista dei componenti.

6.16.2 Importazione del file audio

- Il file fornito su Discord è `sfx_moving_platform.wav` (formato WAV, l'unico importabile direttamente in Unreal).
- Creare una cartella `Audio` nel `Content Browser` (tasto destro → `New Folder`) per organizzare le risorse.
- Trascinare il file WAV all'interno della cartella: comparirà come asset con l'icona a forma di linea ondulata.
- L'importazione non richiede ulteriori passaggi.

6.16.3 Creazione della Sound Cue

La **Sound Cue** è un contenitore programmabile per il suono, che permette di modulare parametri come volume e pitch, applicare loop, randomizzazioni, ecc.

- Tasto destro sull'asset audio importato → `Create Cue`. Si ottiene un nuovo asset con lo stesso nome ma di tipo `Sound Cue` (icona con un altoparlante e un ingranaggio).
- Aprire la `Sound Cue` con doppio clic: si presenta un `Graph` simile a quello dei materiali, con un nodo `Wave Player` collegato all'output.
- Per far sì che il suono venga riprodotto in **loop** continuo: selezionare il nodo `Wave Player`, nel pannello `Details` spuntare la voce `Looping`.
- Per fermare la riproduzione di test, premere la **barra spaziatrice**; per riavviarla, premerla di nuovo.

6.16.4 Modulazione del pitch e del volume con il Modulator

Per rendere il suono meno ripetitivo, si possono randomizzare entro certi intervalli il **pitch** (tonalità e velocità) e il **volume**.

- Fare tasto destro nel grafo della `Sound Cue` e aggiungere un nodo `Modulator` (cercare `modulator`).
- Collegare l'uscita del `Wave Player` all'ingresso del `Modulator` e l'uscita del `Modulator` all'output.
- Nel `Modulator` si possono impostare:
  - **Pitch Min / Pitch Max**: ad esempio 1.7 e 2.0 per una variazione casuale del tono.
  - **Volume Min / Volume Max**: ad esempio 1.0 e 1.2.
- Ad ogni attivazione, il suono avrà un pitch e un volume estratti casualmente all'interno dei range specificati.
- Il pitch influenza sia la tonalità che la velocità di riproduzione (valori inferiori a 1: suono più grave e lento; superiori a 1: più acuto e veloce).

6.16.5 Spawn del suono e controllo della riproduzione

Per poter fermare il suono quando la piattaforma smette di muoversi, è necessario ottenere un **riferimento** all'istanza audio in esecuzione. Il nodo `Play Sound 2D` riproduce il suono ma non restituisce alcun riferimento, quindi si utilizza `Spawn Sound 2D` (o `Spawn Sound at Location`).

- `Spawn Sound 2D` (o `Spawn Sound at Location`):
  - **Sound**: selezionare la `Sound Cue` creata.
  - **Location**: per un suono 2D non dipende dalla posizione; per `at Location` si può usare la posizione dell'attore (`Get Actor Location`).
- Il nodo restituisce un `Return Value` di tipo `Audio Component`. Promuoverlo a variabile chiamata `SoundPlatform` per conservare il riferimento.
- Poiché il suono deve partire all'inizio del movimento e fermarsi al termine, si utilizza il nodo `Do Once` per avviarlo una sola volta.
- Alla fine della Timeline (pin `Finished`), fermare il suono: trascinare la variabile `SoundPlatform` → `Stop`.
- Per consentire che il suono venga riprodotto di nuovo al successivo movimento, resettare il `Do Once` subito dopo lo `Stop`: collegare `Stop` → `Reset` del `Do Once`.
- Il suono ora parte all'ingresso del giocatore (`On Component Begin Overlap`), si ferma quando la Timeline raggiunge il termine, e il `Do Once` viene ripristinato per il prossimo utilizzo.

6.16.6 Cambiare il pitch in base alla direzione della Timeline

Per differenziare ulteriormente il suono in salita e in discesa, si può modificare il pitch dinamicamente usando la direzione della Timeline.

- Dal nodo della Timeline, trascinare il pin `Direction` e creare un nodo `Switch` (si cerchi `switch` e si selezioni `Switch on Timeline Direction`).
- Lo `Switch` espone due rami di esecuzione:
  - **Forward**: la Timeline sta andando in avanti (da 0 a 1).
  - **Backward**: la Timeline sta andando all'indietro (da 1 a 0).

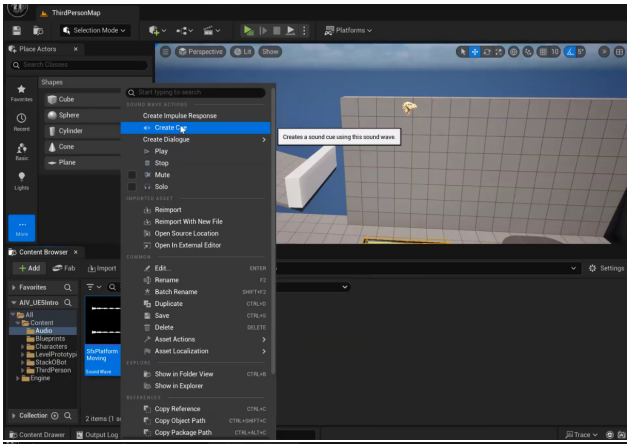
- Inserire questa logica all'interno del flusso del `Do Once` (dopo lo spawn del suono):
  - Sul ramo **Forward**: trascinare la variabile `SoundPlatform` → `Set Pitch Multiplier` e impostare un valore, ad esempio `1.0` (normale).
  - Sul ramo **Backward**: collegare un altro `Set Pitch Multiplier` con un valore più basso, ad esempio `0.4` o `0.8`, per ottenere un suono più grave durante la discesa.
- I nodi `Set Pitch Multiplier` devono essere posizionati dopo lo spawn ma prima che il suono venga fermato; vanno quindi collocati all'interno del `Do Once`, in parallelo all'esecuzione principale (ad esempio, subito dopo lo spawn, in modo che il pitch venga impostato prima che la Timeline termini).
- Se compare un errore di compilazione legato a `SoundPlatform` non creato, verificare che la variabile sia stata effettivamente promossa dall'uscita di `Spawn Sound 2D` e che il nome corrisponda.

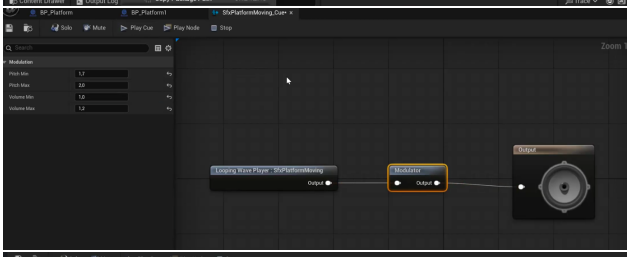
6.16.7 Verifica finale

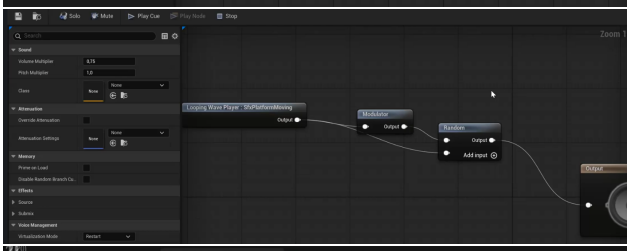
- Entrare con il giocatore nella `Box Collision`: la piattaforma sale, il suono si avvia con pitch casuale nell'intervallo definito e con la tonalità normale (Forward).
- Raggiunta la posizione finale, la Timeline termina, il suono si ferma e il `Do Once` viene resettato.
- Uscendo dalla piattaforma (`End Overlap`), la Timeline inverte la direzione (Reverse), il suono riparte con pitch più basso (Backward) e si ferma al termine.
- La piattaforma torna alla posizione iniziale e il sistema è pronto per un nuovo ciclo.

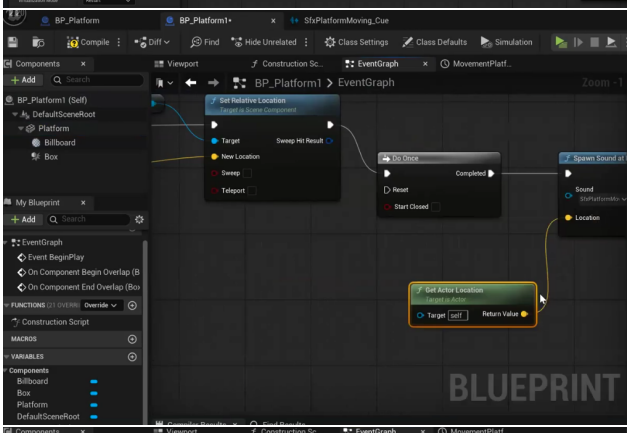
6.17 Conclusione e saluti

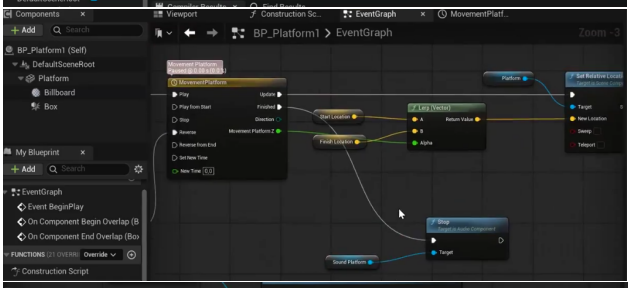
- La lezione si conclude con l'integrazione completa dell'audio e la gestione delle direzioni della Timeline.
- Eventuali problemi residui verranno affrontati nella prossima lezione.
- La prossima sessione è prevista per giovedì; gli studenti vengono salutati e ringraziati per l'attenzione.

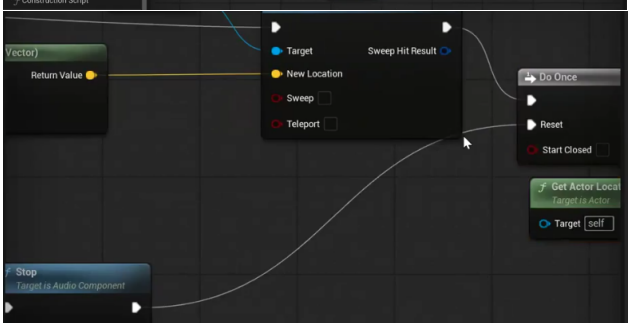







BLUEPRINT




Target
Sweep Hit Result
New Location
Sweep
Teleport
Return Value
Do Once
Reset
Start Closed
Stop

Chapter 7

Blueprint Avanzati: Trigger, Ereditarietà, Timeline ed Event Dispatcher in UE5

7.1 Risoluzione di problemi tecnici pregressi

7.1.1 Errore "Sound Wave not found" e accesso nullo

- Sintomo: messaggio di errore runtime/blueprint "accesso nullo" (in inglese `null access`) dopo la riproduzione di un suono, con riferimento a `Sound Wave` non trovato.
- Causa: il suono non esiste o non è valido al momento dell'esecuzione. Nel blueprint, la logica che creava e modificava il `Sound Platform` era posizionata subito dopo il nodo `Do Once`, prima di aver realmente istanziato la wave.
- Soluzione: spostare tutta la sezione (switch, timeline, direction) *dopo* il `Do Once` e dopo che il suono è stato effettivamente creato. In questo modo si modifica solo quando il suono esiste già.

7.1.2 Piattaforma che si sotterra o si teletrasporta allo zero del mondo

- Problema segnalato: la piattaforma, dopo ripetuti passaggi, affonda nel terreno; oppure, quando vengono impostate le locazioni di start e finish, la piattaforma si sposta in modo erratico, arrivando addirittura alla coordinata $(0, 0, 0)$ del mondo.
- Verifiche effettuate:
  - Salvataggio dei valori di `Start Location` e `Finish Location`.
  - Controllo del `Billboard` esterno: posizione $Z = 500$ (confermata da un `Print String` che stampava `Finish Location` come stringa, mostrando `00500`).
  - Ispezione dell'asset nel mondo: la box di collisione risultava minuscola e posizionata troppo in basso; la piattaforma sembrava uscire dal volume.
- Azioni correttive:
  - Ridimensionare la box di trigger in modo che avvolga solo la parte superiore della piattaforma, permettendo al giocatore di entrarci sopra. La box deve rimanere *sopra* la piattaforma, non sotto, altrimenti il giocatore non la attiva mai.
  - Verificare che il blueprint non avesse perso il componente `DefaultSceneRoot`: in alcuni casi era stato eliminato accidentalmente. Aggiungere una `Scene Component` come nuova radice e agganciare la piattaforma ad essa risolve il problema di riferimento alla world location.

7.1.3 Problema di collisione con la piattaforma e attivazione

- Osservazione: camminando verso la piattaforma, il personaggio urta la box di collisione e non riesce a salire; solo saltando si attiva.
- Cause e soluzioni proposte:
  - Box troppo piccola o troppo grande: regolarne dimensioni e posizione affinché il personaggio possa entrarvi naturalmente quando sale sulla piattaforma.
  - In alternativa, interrare leggermente la piattaforma (abbassarla un po') in modo che il piano di calpestio sia quasi a livello del suolo, eliminando lo scalino.
  - Approfondimento futuro: modifica delle collisioni degli asset, applicabile anche a questa piattaforma.

7.1.4 Bug audio: suono continua dopo interruzione del movimento

- Comportamento osservato: se la piattaforma si sta alzando e il giocatore scende prima che raggiunga il punto massimo, il suono di salita persiste e non parte l'altro suono (discesa).
- Spiegazione: è un comportamento normale derivante dalla logica attuale; il `Play Rate` del suono rimane a 1 anche se l'animazione/ timeline si interrompe.
- Correzione pianificata: impostare il `Play Rate` a 0 quando il movimento viene interrotto, oppure gestire in modo più fine lo stato del suono. Verrà risolto in seguito.

7.2 Lezione del giorno: Array, Ereditarietà e Pulsante Interattivo

7.2.1 Introduzione agli array

- Analogia grafica: se una variabile è una scatola, un array è uno scaffale che contiene più scatole dello stesso tipo.
- Esempio IKEA: ogni mobile ha un numero identificativo (tipo). Lo scaffale corrisponde alla tipologia di variabile (es. `float`), ma contiene più valori dello stesso tipo (i vari pezzi del mobile).
- In programmazione, un array raggruppa variabili di un medesimo tipo. È utile quando si hanno molti oggetti simili (es. tanti nemici) per evitare di creare una variabile separata per ciascuno. Si dichiara un array di tipo `Actor` (o `Enemy`) e vi si inseriscono tutti i nemici.
- Utilizzo con i cicli: l'array sarà spesso accompagnato dal ciclo `For Each` (visto in precedenza) per eseguire operazioni su ogni elemento.

7.2.2 Ereditarietà (Parent e Child)

- Concetto di padre e figlio: un oggetto `Child` deriva da un `Parent` e ne eredita caratteristiche e comportamenti, ma può aggiungerne di nuovi o sovrascrivere quelli esistenti.
- Esempio concreto: un `BP_Cube` (padre) ha una `Static Mesh` cubica. Creando un `Child` di quel blueprint, si può cambiare la mesh in un cilindro e mantenere il resto del comportamento. Il padre rimane un cubo, il figlio diventa un cilindro ma con la stessa logica di base.
- In Unreal Engine è possibile creare classi figlie direttamente dall'Editor, ereditando da qualsiasi blueprint.

7.2.3 Esercizio: Pulsante che attiva luce e piattaforma

- Obiettivo: realizzare un pulsante (`BP_Button`) che, quando premuto (cioè quando il giocatore vi sale sopra), accende una luce e aziona una piattaforma.
- Meccanica:
  - Il pulsante possiede un volume (box) sovrastante che funge da trigger.
  - Quando il giocatore entra nella box, una timeline fa abbassare il pulsante (movimento verso il basso).
  - Il pulsante comunica con altri oggetti definiti come `Attivabili`, ovvero appartenenti a una stessa classe base che condividono l'interfaccia di attivazione ma possono avere comportamenti diversi.
  - Esempio: la luce si accende, la piattaforma si muove (su e giù) in modo indipendente, ma entrambi vengono attivati dallo stesso evento.
- Perché usare una box separata e non la collisione della mesh del pulsante:
  - Le collisioni delle mesh sono spesso troppo precise e aderenti alla superficie; quando il giocatore è a contatto, non si verifica un overlap di volume ma un semplice contatto superficie-superficie, soggetto a micro-variazioni e facilmente inaffidabile.
  - Un volume dedicato (box) fornisce un controllo più ampio e tollerante, garantendo che l'overlap venga rilevato in modo affidabile e con meno errori.

7.2.4 Creazione del Blueprint Pulsante (inizio)

- Dalla `Content Browser`, nella cartella `Blueprints`, creare un nuovo `Blueprint Class` di tipo `Actor` e chiamarlo ad esempio `BP_ButtonOne`.
- Il comportamento di base prevede:
  - Un componente `Box Collision` posizionato sopra la mesh del pulsante.
  - Una `Timeline` che controlla l'animazione di pressione (traslazione verso il basso della mesh).
  - Nodi di evento `OnComponentBeginOverlap` e `OnComponentEndOverlap` per avviare e invertire la timeline.
- L'attivazione di altri oggetti (luce, piattaforma) sarà gestita tramite riferimenti e array di oggetti `Activatable`, sfruttando il ciclo `For Each` e il concetto di ereditarietà per applicare l'azione a tutti gli elementi collegati.

7.3 Realizzazione del pulsante con due mesh statiche e Timeline

7.3.1 Perché due mesh distinte: frame e bottone

- Il pulsante è composto da due `Static Mesh`:
  - `Frame` (la cornice fissa);
  - `Platform` (la parte che si abbassa, chiamata anche *pressure plate*).
- Motivazione: se si usasse un unico asset, quando il pulsante viene premuto l'intero oggetto (cornice inclusa) si muoverebbe verso il basso, penetrando nel pavimento. Separando le due parti, solo il bottone interno si abbassa, mentre il frame rimane fermo, dando un effetto visivo corretto.
- Regola generale per asset interagibili con movimentazione: dividere sempre le parti mobili da quelle fisse, per evitare compenetrazioni e semplificare la gestione.

7.3.2 Creazione e posizionamento delle mesh

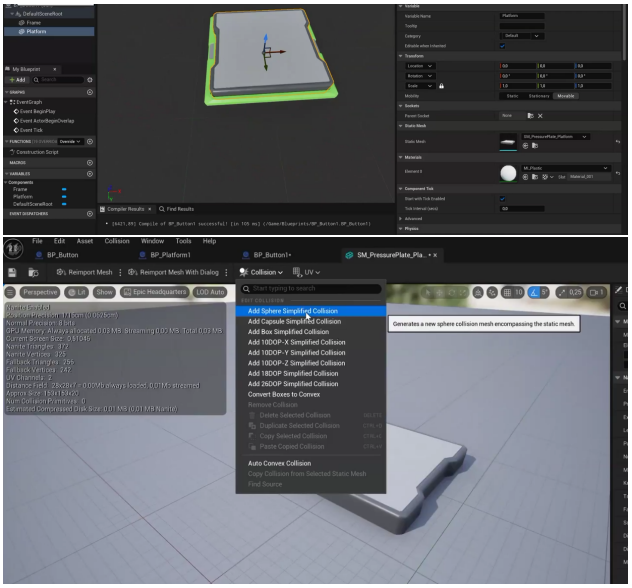
- Nel `Content Browser`, creare due `Static Mesh`:
  - `SM_ButtonFrame` (cornice)
  - `SM_ButtonPlatform` (bottone premibile)
- Le due mesh sono già allineate in origine; trascinandole nella scena si sovrappongono correttamente senza ulteriori aggiustamenti.
- Aggiungere un `Box Collision` sopra la mesh del bottone, scalarlo in modo che avvolga l'area di attivazione (leggermente più grande del pulsante stesso).
- Attenzione all'asse Z: il volume deve trovarsi sopra la pedana, in modo che il giocatore possa entrarvi camminandoci sopra.

7.3.3 Gestione della collisione per evitare blocchi del giocatore

- Problema comune: la collisione di default del bottone può essere troppo alta, impedendo al personaggio di salirvi a causa del parametro `Max Step Height` del `Character`. Il giocatore si blocca con i piedi contro il bordo.
- Soluzione: semplificare la collisione della `Static Mesh` del bottone (`SM_ButtonPlatform`).
- Passaggi nell'editor della mesh:
  1. Aprire la `Static Mesh` e andare alla finestra delle collisioni (`Collision`).
  2. Rimuovere la `Simple Collision` esistente (quella verde, più complessa). *Non* cancellare la `Complex Collision` (azzurra).
  3. Aggiungere una nuova `Box Collision` dal menu `Collision > Add Box Simplified Collision`.
  4. Scalare e spostare la box in modo che sia molto sottile e aderente alla superficie superiore del bottone, riducendo l'altezza totale. Così il giocatore può camminarci sopra senza ostacoli.
- Differenza tra `Simple` e `Complex` collision:
  - `Simple`: usate per interazioni di base (es. overlap, fisica semplificata). Sono più performanti e vengono utilizzate di default quando si inserisce un asset nella viewport.
  - `Complex`: contengono il dettaglio esatto della geometria. Servono per operazioni di precisione (es. sistema di scalata, rilevamento punto esatto di contatto). Sono più pesanti e vanno usate con cautela.
- In questo caso è sufficiente una `Box` semplice e ribassata.

7.3.4 Logica di pressione con Timeline

- Nel `Blueprint` del bottone, si usano gli eventi della `Box Collision`:
  - `OnComponentBeginOverlap`
  - `OnComponentEndOverlap`
- Per entrambi, verificare che l'attore che entra/esce sia il `Player`: ottenere il `Player Pawn` (ad esempio con `Get Player Pawn`) e confrontarlo con l'`Other Actor` tramite un `Branch`. Se vero, procedere.
- Creare una `Timeline` (chiamata ad es. `PushButton`) di durata `0.25` secondi.
- Aggiungere una `Float Track` e rinominarla (es. `PushButtonZ`).



- Impostare due chiavi:
  - Tempo `0.0` → valore `0.0`
  - Tempo `0.25` → valore `-10.0`
- Ciò farà sì che il valore passi da `0` a `-10` in un quarto di secondo.
- L'output della traccia va collegato a un nodo `Make Vector` (X e Y a 0, Z collegata al float della timeline). Il vettore risultante viene collegato all'input `New Location` di un `Set Relative Location` della `Platform` (il componente mesh del bottone).
- In questo modo la piattaforma si abbassa lungo l'asse Z *relativamente* alla sua posizione iniziale (non coordinate mondo).
- Per far risalire il bottone quando il giocatore esce, collegare `OnComponentEndOverlap` al nodo `Reverse` della timeline.

7.3.5 Modifica delle curve di interpolazione

- Le chiavi della timeline hanno di default un'interpolazione `Linear`: il movimento è a velocità costante.
- Facendo clic destro su un punto e scegliendo `Key Interpolation > Auto` (o `User`) si può trasformare la retta in una curva `Smooth`, con maniglie di tangente regolabili.
- Modificando la forma della curva si ottengono effetti di easing (accelerazione/decelerazione) o addirittura piccoli rimbalzi. Esempio: una curva che scende e poi risale leggermente prima di fermarsi dà un effetto *overshoot*.
- Questa tecnica è utile anche per la piattaforma mobile realizzata in precedenza.
- Per il pulsante, l'interpolazione lineare è adeguata; le curve verranno sfruttate in seguito per movimenti più complessi.

7.3.6 Verifica e debug

- Se il pulsante non si muove, controllare:
  - che l'output della timeline (float track) sia effettivamente collegato all'input Z del `Make Vector`;

  - che il `Set Relative Location` agisca sul componente corretto (la mesh della `Platform`);
  - che il nome della traccia nella timeline corrisponda (a volte rimane `New Track 0` e va rinominato).
- Test in gioco: salendo sulla box il bottone si abbassa, scendendo ritorna su. Se non si alza, verificare che l'evento `OnComponentEndOverlap` sia collegato al `Reverse` della timeline.

7.4 Oggetti attivabili e comunicazione tramite array

7.4.1 Creazione della classe base `BP_Activable`

- Nuovo `Blueprint Class` di tipo `Actor`, chiamato `BP_Activable`.
- All'interno si inseriscono due `Custom Event`:
  - `Activation`
  - `Deactivation`
- Per il momento, a ciascun evento viene collegato un `Print String` che stampa "Attivato" e "Disattivato" rispettivamente, così da verificare il funzionamento.
- Questo attore rappresenta un generico *oggetto attivabile* che risponderà agli impulsi del pulsante.

7.4.2 Variabile array pubblica nel pulsante

- Nel `BP_Button` si crea una variabile denominata `ActivableItems`.
- **Tipo variabile**: si imposta su `BP_Activable` (il riferimento alla classe appena creata).
- **Da singola ad array**: accanto al tipo variabile si apre il menù a tendina con l'icona del cilindro (o ellisse) e si seleziona `Array`. La variabile diventa una matrice, in grado di contenere più riferimenti a oggetti attivabili.
- **Visibilità pubblica**: si clicca sull'icona dell'occhietto accanto al nome della variabile per renderla `Public` (occhio aperto). In questo modo la variabile sarà esposta nei dettagli del pulsante nell'editor, permettendo di aggiungere comodamente gli attori dalla viewport.

7.4.3 Logica di attivazione/disattivazione con Timeline e `For Each Loop`

- Al termine della Timeline (`Finished` pin), si vuole attivare o disattivare tutti gli oggetti in base alla direzione del movimento.
- Si utilizza un nodo `Switch` collegato all'output `Direction` della Timeline, che distingue tra `Forward` (premuto) e `Backward` (rilasciato).
- Ramo `Forward` (pulsante premuto):
  - Ottenere l'array `ActivableItems` con un `Get`.
  - Collegarlo a un `For Each Loop`.
  - Dal loop si estrae `Array Element` (il singolo `BP_Activable`) e si chiama su di esso il `Custom Event Activation` (nodo `Activation`).
- Ramo `Backward` (pulsante rilasciato):
  - Stessa struttura, ma si chiama `Deactivation` su ogni elemento.
- Il ciclo `For Each` esegue l'azione per ogni elemento dell'array, garantendo che tutti gli attivabili vengano raggiunti.

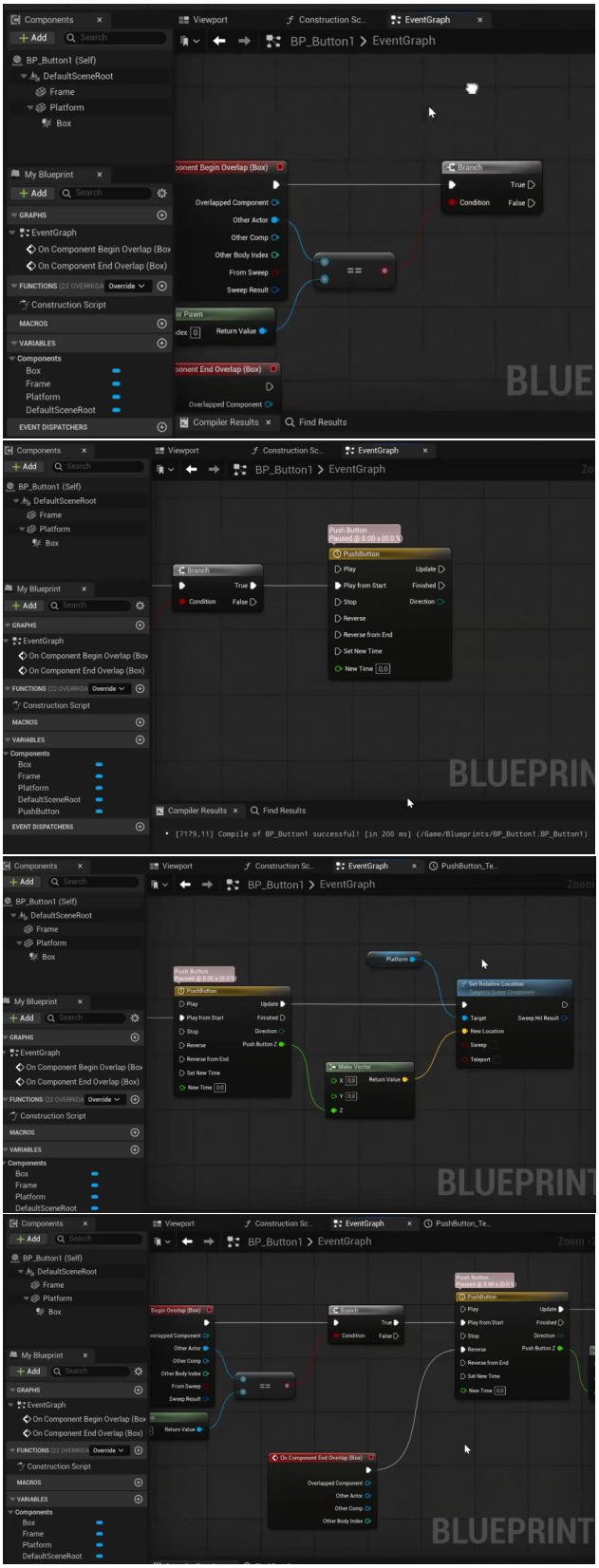


Figure 7.1: BP della piattaforma attivabile.

7.4.4 Assegnazione degli oggetti nella scena

- Trascinare un'istanza di `BP_Activable` nel livello.
- Selezionare il pulsante e, nel pannello `Details`, individuare la variabile `Activable Items`.
- Aggiungere elementi all'array cliccando sul `+` e assegnare il riferimento tramite:
  - Il contagocce (dropper): si clicca sull'icona del contagocce e poi sull'attore nella viewport; Unreal mostra quali classi sono compatibili, impedendo di assegnare tipi errati.
  - Oppure scegliendo direttamente dalla lista degli attori presenti in scena.
- Salvare, compilare e testare: salendo sul pulsante si stampa "Attivato", scendendo "Disattivato".

7.5 Ereditarietà: luce attivabile come classe figlia

7.5.1 Creazione del Blueprint `BP_ActivatableLight`

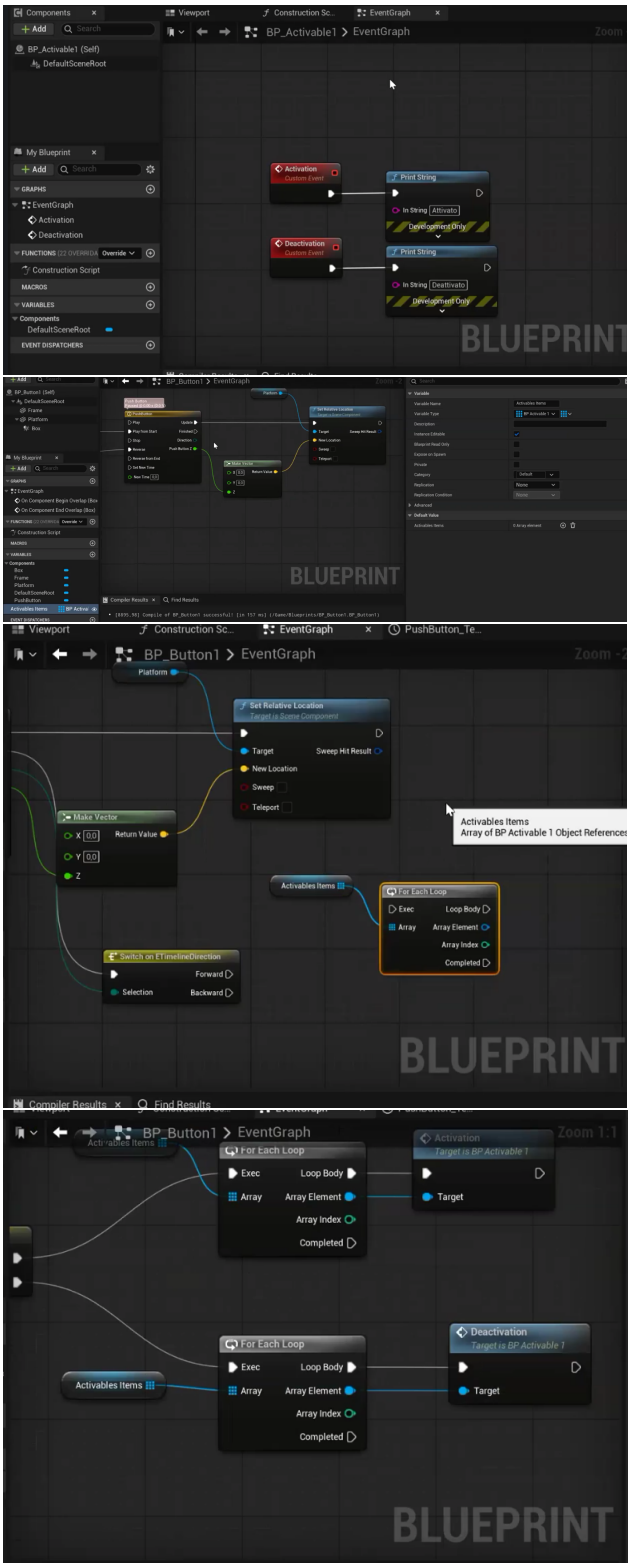
- Non si crea un nuovo `Actor` da zero, ma un **figlio** di `BP_Activable`.
- Procedura: tasto destro su `BP_Activable` → `Create Child Blueprint Class`. Rinominare il nuovo blueprint, ad esempio `BP_ActivatableLight`.
- Il nuovo attore eredita tutte le variabili, gli eventi e la logica del genitore (in particolare i `Custom Event Activation` e `Deactivation`), ma può essere esteso con componenti e comportamenti specifici (in questo caso una luce).

7.5.2 Ereditarietà degli eventi e `BeginPlay`

- Se nel genitore (`BP_Activable`) si trova un nodo `Event BeginPlay`, il figlio può decidere se eseguire anche quello del padre o solo il proprio.
- Nel `Child`, se si aggiunge un `Event BeginPlay` e lo si collega al nodo `Parent: Event BeginPlay` (visibile come `Call Parent`), allora entrambi i `BeginPlay` verranno eseguiti: prima quello del figlio, poi quello del padre.
- Nell'esercizio, il `BP_Activable` non contiene azioni nel `BeginPlay`, quindi non è necessario chiamare il padre.

7.5.3 Configurazione della `Point Light`

- Nel `BP_ActivatableLight` si aggiunge un componente `Point Light`.
- Parametri principali regolabili:
  - `Intensity`: luminosità della sorgente.
  - `Attenuation Radius`: raggio entro cui la luce si attenua; aumenta l'area illuminata.
  - `Source Radius`: raggio della parte centrale della luce, visibile come alone.
  - `Light Color`: colore della luce, modificabile con il selettore cromatico.
- Per l'esercizio si sceglie un colore blu e un'intensità elevata (es. `200000`) per rendere l'effetto ben visibile.


BP_Activable1 > EventGraph
Activation
Print String
In String Attivato
Development Only
Deactivation
In String Deattivato
BLUEPRINT
BP_Button1 > EventGraph
Platform
Set Relative Location
Target is Scene Component
Target
Sweep Hit Result
New Location
Sweep
Teleport
Make Vector
Return Value
Activables Items
Array of BP Activable 1 Object References
For Each Loop
Exec
Loop Body
Array
Array Element
Array Index
Completed
Switch on ETimelineDirection
Forward
Selection
Backward
Target is BP Activable 1
Zoom 1:1

7.5.4 Salvataggio dell'intensità iniziale

- All'avvio (`Event BeginPlay`) si vuole memorizzare l'intensità impostata nel componente luce per poterla ripristinare alla disattivazione.
- Operazioni:
  - Trascinare il componente `Point Light` nel grafo, tirare fuori il pin `Get Intensity`.
  - Promuovere a variabile con `Promote to Variable`, nominandola ad esempio `IntensityStart`.
  - Collegare il nodo `Set` (che viene creato automaticamente) al flusso del `BeginPlay`.
- Questa variabile conterrà il valore di intensità di default e potrà essere usata nei `Custom Event` per accendere (ripristinare l'intensità) o spegnere (portare a zero) la luce.

7.5.5 Note sulla timeline: `Play` vs `Play from Start`

- `Play from Start`: ogni volta che la timeline viene fatta partire (ad esempio con `Play`), la riproduzione ricomincia dall'inizio (tempo `0`). Se il player esce e rientra durante il movimento, la timeline ripartirà da zero, evitando comportamenti indesiderati.
- `Play`: la timeline riprende dal punto in cui era stata eventualmente interrotta. Può essere utile in altri contesti, ma per il pulsante si preferisce il reset completo.

7.6 Override degli eventi e implementazione della luce attivabile

7.6.1 Sovrascrivere gli eventi Activation e Deactivation

- Nel `BP_ActivatableLight`, per personalizzare il comportamento ereditato, si utilizza la funzione `Override`.
- Nel pannello `Functions` (sotto `Override`), compaiono gli eventi del genitore: `Activation` e `Deactivation`. Selezionandoli, Unreal li aggiunge al grafico, permettendo di *sovrascriverli*.
- Quando un evento è sovrascritto, il comportamento originale del padre (ad esempio la stampa "Attivato"/"Disattivato") viene ignorato. Se l'evento sovrascritto è vuoto, non succede nulla.

7.6.2 Logica della luce

- **Activation** (luce accesa):
  - Si richiama il componente `Point Light` e si usa un nodo `Set Intensity`.
  - Il valore da impostare è la variabile `IntensityStart` (salvata in `BeginPlay`), che ripristina l'intensità originale.
- **Deactivation** (luce spenta):
  - Sempre tramite `Set Intensity` su `Point Light`, ma con valore `0.0`.
- **Inizializzazione** in `BeginPlay`:
  1. Salvataggio dell'intensità iniziale in `IntensityStart` (`Get Intensity` → `Set IntensityStart`).
  2. Subito dopo, chiamare `Deactivation` per spegnere la luce all'avvio del gioco.
- Motivo: si vuole che la luce parta spenta e si accenda solo quando il pulsante viene premuto. Se si chiamasse `Deactivation` prima del salvataggio, `IntensityStart` diventerebbe `0`, perdendo il valore di default.
- Test: avviando il gioco la luce è spenta; salendo sul pulsante la luce si accende; scendendo si spegne. I messaggi di stampa del padre non compaiono più.

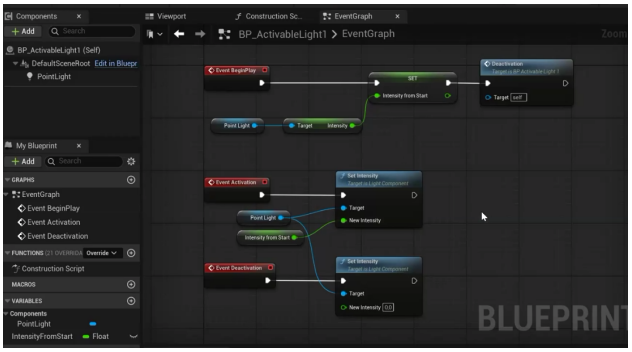


Figure 7.2: BP completo della luce.

7.6.3 Debug di problemi comuni

- Luce sempre accesa nonostante la pressione: verificare di aver assegnato correttamente l'attore `BP_ActivatableLight` nell'array `ActivableItems` del pulsante, usando il contagocce.
- Luce invisibile a inizio partita: è normale, perché `BeginPlay` chiama `Deactivation`. Se la luce deve partire accesa, si può omettere quella chiamata o invertire la logica.
- Disallineamento accensione/spegnimento (es. luce si accende quando si scende): controllare che nel loop `For Each` del pulsante il ramo `Forward` chiami `Activation` e il ramo `Backward` chiami `Deactivation`, e che il collegamento non sia invertito.

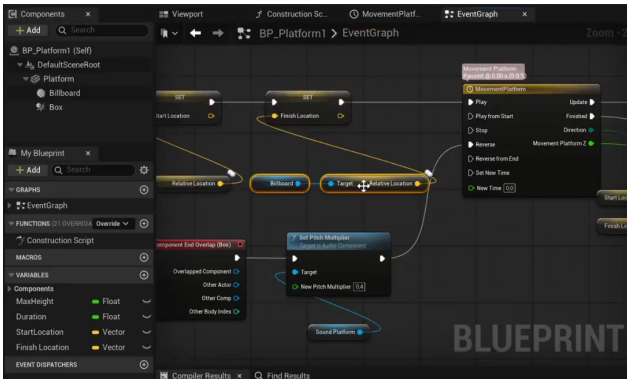
7.7 Fix audio per la piattaforma mobile

7.7.1 Creazione del suono con `Spawn Sound at Location`

- Nel `Blueprint` della piattaforma, all'inizio dell'overlap (`Do Once`), si genera un suono con `Spawn Sound at Location`.
- Il pin `Location` si collega al risultato di `Get Actor Location` (posizione della piattaforma).
- Selezionare la `Sound Wave` desiderata nell'apposito slot.
- Dal nodo `Spawn Sound`, trascinare il pin di ritorno (`Return Value`) e promuoverlo a variabile, chiamandola `SoundPlatform`. Questa variabile contiene il riferimento all'istanza sonora appena creata e permette di controllarla successivamente.

7.7.2 Impostazione del Pitch in base alla direzione

- Dalla `Timeline` si estrae il pin `Direction` e lo si collega a un nodo `Switch` con rami `Forward` e `Backward`.
- `Forward` (piattaforma in salita): ottenere la variabile `SoundPlatform` e chiamare `Set Pitch Multiplier` con valore `1.0`.
- `Backward` (piattaforma in discesa): stesso nodo ma con valore `0.4` (suono più grave o distorto, a seconda della wave).
- Attenzione: il pin di esecuzione del `Switch` deve essere collegato in modo che il flusso arrivi ai `Set Pitch Multiplier` (i nodi vanno inseriti nel ramo corrispondente, non prima).



7.7.3 Interruzione del suono al termine del movimento

- Al `Finished` della `Timeline`, prendere la variabile `SoundPlatform` e chiamare `Stop` per fermare la riproduzione.
- Per consentire una nuova attivazione, collegare `Finished` anche al `Reset` del nodo `Do Once` (così al successivo overlap il suono viene rigenerato).

7.7.4 Bug del Pitch quando il giocatore abbandona la pedana

- Problema: se il giocatore scende (o salta via) mentre la piattaforma è ancora in movimento, il Pitch rimane impostato sul valore della direzione corrente e il suono non si corregge al prossimo utilizzo.
- Soluzione: nell'evento `OnComponentEndOverlap`, aggiungere un `Set Pitch Multiplier` sulla variabile `SoundPlatform` con valore `0.4`. Così, ogni volta che il giocatore lascia il volume, il pitch viene riportato al valore di discesa, pronto per il prossimo movimento.
- In questo modo anche un'uscita anticipata sistema lo stato del suono.

7.8 Event Dispatcher: comunicazione inversa tra pulsante e luce

7.8.1 Creazione dell'Event Dispatcher nel pulsante

- Nel `BP_Button`, nel pannello `Variables` si trova la sezione `Event Dispatchers`.
- Cliccare su `+` per crearne uno nuovo, rinominandolo ad esempio `CheckStatus`.
- Aggiungere un parametro di input: nella finestra dell'event dispatcher, sotto `Inputs`, premere `+` e denominare il parametro `IsActive`. Il tipo di default è `Boolean`, adatto allo scopo.
- Questo dispatcher servirà a notificare a chiunque sia in ascolto lo stato del pulsante (premuto = `true`, rilasciato = `false`).

7.8.2 Bind dell'evento nella luce attivabile

- In `BP_ActivatableLight`, durante l'`Event BeginPlay`, si cerca il pulsante nel mondo con `Get Actor of Class`, specificando la classe `BP_Button`.
- Dal risultato, si chiama il nodo `Bind Event to CheckStatus` (il nome varia in base al dispatcher creato).
- Questo nodo restituisce un pin evento: trascinandolo fuori e scegliendo `Add Custom Event`, si crea un evento personalizzato che Unreal configura automaticamente con il parametro booleano `IsActive`.

- Il custom event viene rinominato, ad esempio `StatusEvent`.
- All'interno, si usa un `Branch` sul valore di `IsActive`:
  - Se `true` → chiamare `Activation` (accendi luce).
  - Se `false` → chiamare `Deactivation` (spegni luce).
- In questo modo è la luce a "osservare" lo stato del pulsante, reagendo di conseguenza.

7.8.3 Chiamata del dispatcher nel pulsante

- Nel `BP_Button`, dopo il `Finished` della `Timeline` (e dopo lo `Switch` su `Direction`), si inserisce il nodo `Call CheckStatus`.
  - Ramo `Forward`: `IsActive` = `true` (checkbox spuntata).
  - Ramo `Backward`: `IsActive` = `false` (checkbox deselezionata).
- Il nodo `Call` invoca il dispatcher, che a sua volta esegue tutti gli eventi bindati ad esso (nel nostro caso, il `StatusEvent` della luce).
- Il `Bind` avviene una sola volta in `BeginPlay`, ma l'ascolto è continuo: ogni `Call` attiva l'evento collegato, consentendo di attivare/disattivare la luce senza che il pulsante debba conoscere direttamente l'attore luce.

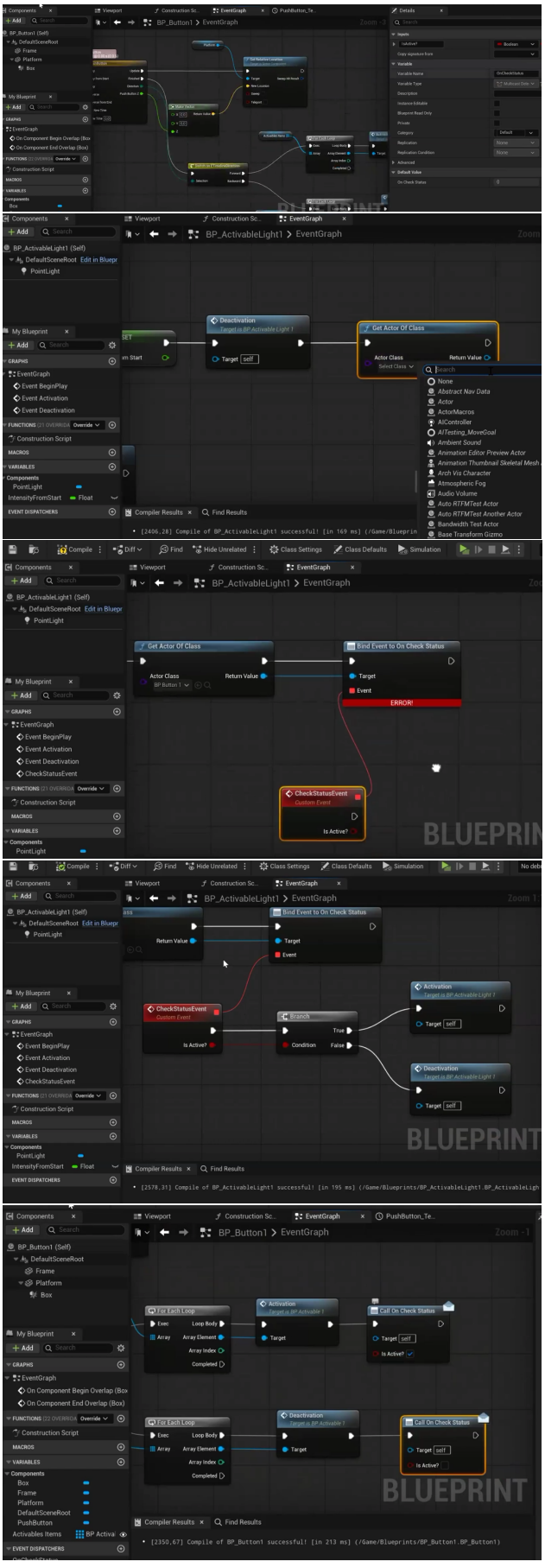
7.8.4 Vantaggi degli Event Dispatcher

- Disaccoppiamento: il pulsante non ha bisogno di un riferimento diretto alla luce (o a ogni singolo oggetto attivabile); semplicemente lancia un segnale, e chi è interessato si "iscrive" (`Bind`) per riceverlo.
- Permettono di collegare più comportamenti diversi a un singolo evento, semplicemente bindando più attori allo stesso dispatcher.
- Rispetto ai `Custom Event` chiamati direttamente, offrono maggiore ordine e flessibilità nella gestione di eventi globali (salto, sparo, cambi di stato).

7.9 Override degli eventi e implementazione della luce attivabile

7.9.1 Sovrascrivere gli eventi `Activation` e `Deactivation`

- Nel `BP_ActivatableLight` si utilizzano gli **Override** per personalizzare il comportamento ereditato dal genitore `BP_Activable`.
- Nel pannello `Functions` (sezione `Override`) compaiono gli eventi del padre: selezionandoli, Unreal Engine li aggiunge al grafico e ne permette la *sovrascrittura*.
- Quando un evento è sovrascritto, il comportamento originale (es. le stampe `Print String` con "Attivato"/"Disattivato") viene completamente ignorato. Se l'Override è vuoto, l'evento non produce alcun effetto.


BP_Button1 > EventGraph
BP_ActivableLight1 > EventGraph
Deactivation
Get Actor Of Class
Bind Event to On Check Status
ERROR!
CheckStatusEvent
Branch
Activation
For Each Loop
Call On Check Status
BLUEPRINT

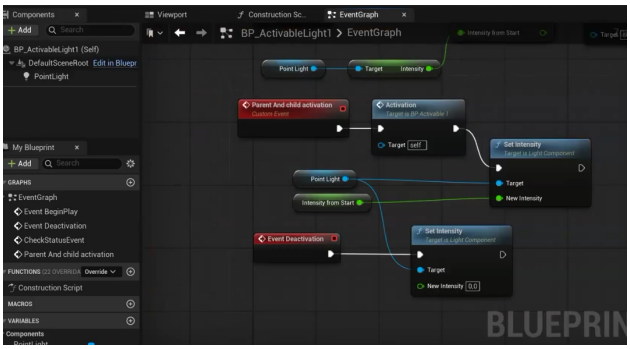


Figure 7.3: BP per chiamare padre e figlio.

7.9.2 Logica della luce

- **Accensione** (`Activation`):
  - Si richiama il componente `Point Light` e si imposta l'intensità con il nodo `Set Intensity`.
  - Il valore da applicare è contenuto nella variabile `IntensityStart`, salvata in precedenza.
- **Spegnimento** (`Deactivation`):
  - Sempre tramite `Set Intensity` sul componente luce, ma con valore `0.0`.
- **Inizializzazione in** `BeginPlay`:
  1. Salvare l'intensità attuale: dal componente `Point Light` si usa `Get Intensity` e si collega al nodo `Set` della variabile `IntensityStart` (creata con `Promote to Variable`).
  2. Subito dopo, chiamare l'evento `Deactivation` per spegnere la luce all'avvio del gioco.
- **Motivazione dell'ordine**: se si chiamasse `Deactivation` prima di salvare l'intensità, `IntensityStart` diventerebbe `0`, perdendo il valore di default impostato nell'editor.
- Test: a partita avviata la luce è spenta; salendo sul pulsante si accende, scendendo si spegne. I messaggi di stampa del padre non vengono più eseguiti.

7.9.3 Debug di problemi comuni

- **Luce sempre accesa**: verificare di aver assegnato correttamente l'attore `BP_ActivatableLight` all'array `ActivableItems` del pulsante usando il contagocce.
- **Luce invisibile a inizio partita**: comportamento normale dovuto alla chiamata di `Deactivation` in `BeginPlay`. Se si desidera che parta accesa, omettere quella chiamata o invertire la logica.
- **Accensione/spegnimento invertiti**: controllare che nel `For Each Loop` del pulsante il ramo `Forward` chiami `Activation` e il ramo `Backward` chiami `Deactivation`, e che i collegamenti non siano scambiati.

7.10 Fix audio per la piattaforma mobile

7.10.1 Creazione del suono con `Spawn Sound at Location`

- Nel `Blueprint` della piattaforma, in corrispondenza del primo overlap (`Do Once`), si genera un suono con il nodo `Spawn Sound at Location`.
- Il pin `Location` va collegato a `Get Actor Location` (posizione della piattaforma).

- Selezionare la `Sound Wave` desiderata nell'apposito slot.
- Dal nodo `Spawn Sound` si trascina il `Return Value` e lo si promuove a variabile, nominandola `SoundPlatform`. Questa variabile mantiene il riferimento all'istanza sonora per i controlli successivi.

7.10.2 Impostazione del `Pitch` in base alla direzione

- Dalla `Timeline` si estrae il pin `Direction` e lo si collega a un nodo `Switch` con i rami `Forward` e `Backward`.
- **Ramo `Forward` (salita)**: ottenere la variabile `SoundPlatform` e chiamare `Set Pitch Multiplier` con valore `1.0`.
- **Ramo `Backward` (discesa)**: stesso nodo ma con valore `0.4` (tonalità più grave o effetto voluto).
- Il flusso di esecuzione deve entrare nei `Set Pitch Multiplier` dopo lo `Switch`, non prima.

7.10.3 Interruzione del suono al termine del movimento

- Collegare il pin `Finished` della `Timeline` a un nodo `Stop` applicato alla variabile `SoundPlatform`, in modo da fermare il suono quando l'animazione termina.
- Per permettere una nuova attivazione, connettere `Finished` anche al `Reset` del nodo `Do Once` (così al successivo overlap il suono verrà rigenerato).

7.10.4 Correzione del bug del `Pitch` all'uscita anticipata

- **Problema**: se il giocatore scende o salta via mentre la piattaforma è ancora in movimento, il `Pitch` rimane sul valore dell'ultima direzione e non viene ripristinato correttamente al prossimo utilizzo.
- **Soluzione**: nell'evento `OnComponentEndOverlap` aggiungere un `Set Pitch Multiplier` sulla variabile `SoundPlatform` con valore `0.4`. In questo modo, ogni volta che il giocatore lascia il volume, il pitch viene forzato al valore di discesa, preparandolo per la prossima attivazione.

7.11 Event Dispatcher: comunicazione inversa tra pulsante e luce

7.11.1 Creazione dell'Event Dispatcher nel pulsante

- Nel `BP_Button`, nella sezione `Variables` si trova il pannello `Event Dispatchers`.
- Cliccare `+` per crearne uno nuovo, rinominarlo ad esempio `CheckStatus`.
- Aggiungere un parametro di input: nell'editor del dispatcher, sotto `Inputs`, premere `+` e denominare il parametro `IsActive`. Il tipo predefinito `Boolean` è adatto allo scopo.
- Questo dispatcher segnalerà lo stato del pulsante: `true` quando premuto, `false` quando rilasciato.

7.11.2 Bind dell'evento nella luce attivabile

- Nel `BP_ActivatableLight`, durante l'`Event BeginPlay`, si cerca il pulsante nel mondo con il nodo `Get Actor of Class`, specificando la classe `BP_Button`.
- Dal risultato si utilizza il nodo `Bind Event to CheckStatus` (il nome varia in base al dispatcher creato).
- Il nodo fornisce un pin evento: trascinandolo fuori e scegliendo `Add Custom Event` si crea un nuovo evento personalizzato. Unreal lo configura automaticamente con il parametro booleano `IsActive`.
- Rinominare il custom event, ad esempio `StatusEvent`.

- All'interno di `StatusEvent` si inserisce un `Branch` basato su `IsActive`:
  - Se `true` → chiamare `Activation` (accendi luce).
  - Se `false` → chiamare `Deactivation` (spegni luce).
- In questo modo è la luce a "osservare" lo stato del pulsante, reagendo di conseguenza.

7.11.3 Chiamata del dispatcher nel pulsante

- Nel `BP_Button`, al termine della `Timeline` (dopo lo `Switch` su `Direction`), si inserisce il nodo `Call CheckStatus`.
  - Ramo `Forward`: `IsActive` = `true` (checkbox spuntata).
  - Ramo `Backward`: `IsActive` = `false` (checkbox deselezionata).
- Il nodo `Call` invoca il dispatcher, che esegue tutti gli eventi a esso bindati (nel nostro caso, `StatusEvent` della luce).
- Il `Bind` avviene una sola volta in `BeginPlay`, ma l'ascolto rimane attivo: ogni `Call` successiva attiva l'evento collegato, permettendo di controllare la luce senza che il pulsante abbia un riferimento diretto ad essa.

7.11.4 Vantaggi degli Event Dispatcher

- **Disaccoppiamento**: il pulsante non ha bisogno di conoscere ogni singolo oggetto attivabile; emette un segnale e chi è interessato si "iscrive" con il `Bind`.
- **Multi-sottoscrizione**: è possibile collegare più comportamenti diversi allo stesso dispatcher, semplicemente bindando più attori.
- **Ordine e flessibilità**: rispetto ai `Custom Event` chiamati direttamente, i dispatcher offrono una gestione più strutturata di eventi globali (es. cambi di stato, input di gioco).

7.12 Esercizio pratico: oggetto attivato da pulsante con Timeline

7.12.1 Opzioni per l'esercizio

- **Variante avanzata**: riadattare la piattaforma mobile esistente in un child di `BP_Activable`, mantenendo mesh e box, ma facendo in modo che il movimento sia innescato dal pulsante invece che dall'overlap diretto del giocatore.
- **Variante semplificata**: creare un nuovo `Blueprint` (es. `BP_BallMoving`) che rappresenti un qualsiasi oggetto (cubo, sfera) e che si muova lungo una `Timeline` quando il pulsante viene premuto, e torni indietro al rilascio.

7.12.2 Implementazione base con Timeline

- Nel nuovo `BP_BallMoving` (o nella piattaforma modificata) si devono sovrascrivere gli eventi `Activation` e `Deactivation` ereditati da `BP_Activable`.
- **Attivazione (`Activation`)**:
  - Eseguire il nodo `Play` della `Timeline` (per avviare il movimento in avanti).
  - L'oggetto si sposterà dalla posizione iniziale a quella finale lungo la traccia.
- **Disattivazione (`Deactivation`)**:

  - Eseguire il nodo `Reverse` della `Timeline` (per tornare alla posizione di partenza).

- La `Timeline` va configurata con una `Float Track` (o `Vector Track`) e con i valori di spostamento desiderati (es. modifica della posizione, rotazione).
- **Nota**: per oggetti semplici che devono andare da A a B e tornare indietro, `Play` e `Reverse` sono sufficienti; non serve il `Finished` o loop complessi.

7.12.3 Note specifiche per la piattaforma (variante avanzata)

- La piattaforma deve essere figlia di `BP_Activable`, quindi va creato un `Child Blueprint Class` a partire da `BP_Activable` e vanno ripristinate le mesh (`Static Mesh` della piattaforma) e la `Box Collision` (se necessaria per l'interazione con il giocatore sulla pedana).
- La logica di movimento preesistente (basata su `BeginOverlap` e `EndOverlap`) viene sostituita dalla risposta agli eventi `Activation`/`Deactivation` ereditati.
- Se il comportamento desiderato è che la piattaforma vada giù alla pressione del pulsante e risalga solo quando il giocatore vi sale sopra, occorre combinare l'attivazione da pulsante con l'overlap della box (es. il pulsante attiva una "modalità abbassata", e l'overlap del giocatore sulla piattaforma la fa risalire). In questo caso l'esercizio richiede di gestire sia gli eventi dell'`Activable` sia l'`Overlap` locale.
- In fase di test, se il movimento appare invertito (es. la piattaforma parte dalla posizione finale), controllare i valori di `Start Location` e `Finish Location` nel `Billboard` o nel `Lerp` e, se necessario, invertirli.

7.12.4 Consegna e suggerimenti

- L'esercizio può essere realizzato usando indifferentemente il metodo delle chiamate dirette agli eventi (`For Each Loop` con `Activation`/`Deactivation`) oppure il metodo con `Event Dispatcher` e `Bind`, a scelta dello studente.
- Si raccomanda di procedere per passi: prima far funzionare il movimento base con la timeline, poi collegare l'attivazione al pulsante, infine aggiungere eventuali suoni o rifiniture.
- In caso di errori di compilazione dovuti alla copia di codice da un blueprint all'altro (es. variabili non esistenti), ricreare le variabili mancanti con `Create Variable` e riassegnare i riferimenti (mesh, box, durate, ecc.).

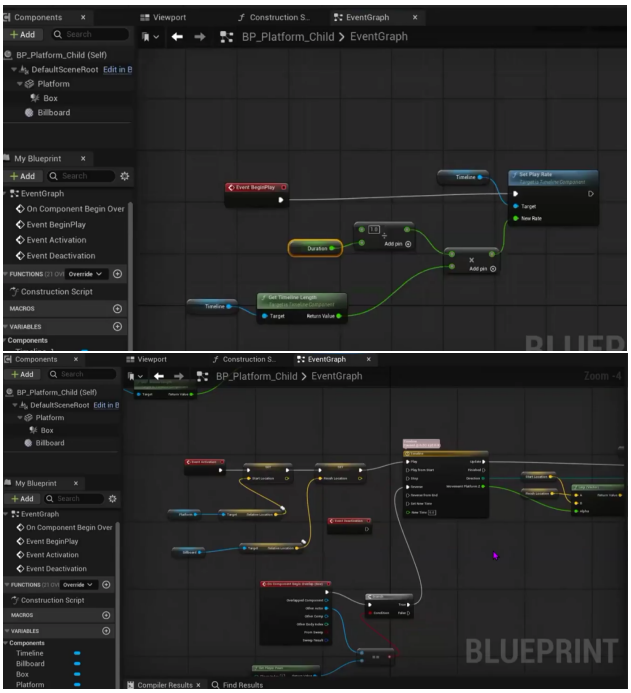


Figure 7.4: Una possibile soluzione al problem. Nota: il Billboard era al contrario e il BP può essere rimaneggiato.

Chapter 8

Gameplay Framework, Dati Globali e Animazioni dei Personaggi in Unreal Engine 5

8.1 Problemi tecnici iniziali e raccomandazioni

- L'importanza di usare variabili chiare e concise: dimenticare di assegnare un parametro (ad esempio il tempo) blocca completamente il movimento e costringe a rivedere più volte il codice.
- Difficoltà con la condivisione dello schermo e la webcam (problema con l'altro PC, possibile conflitto con UBS). Alla fine la condivisione funziona ma la webcam resta nera.

8.2 Mixamo come risorsa gratuita per animazioni

- Mixamo è un servizio Adobe gratuito, molto utile per animare personaggi.
- Le animazioni di Mixamo sono compatibili con i modelli di Unreal Engine e ben replicate.
- Esistono pack a pagamento più curati, ma Mixamo rappresenta un'ottima base, soprattutto per le *game jam* (eventi in cui si sviluppa un videogioco in pochi giorni, da tre giorni a una settimana).

8.3 Debug dell'errore `Sound Platform`

Problema segnalato da Filippo: quando sale su una piattaforma, appare l'errore "tentativo di accesso ad `AutoComponent` tramite la proprietà del `Sound Platform`, ma non è valida". Il suono funziona, ma l'errore compare solo la prima volta che si sale sulla piattaforma.

Tentativo di soluzione:

1. Si ipotizza che l'errore sia dovuto a `Sound Platform` non ancora inizializzato. Si prova a rimuovere i nodi `Get` e a collegare direttamente il target.
2. L'errore persiste e il messaggio diventa più preciso: "`Audio Component 0` non è valida".
3. Si conclude che il vero problema è l'inesistenza dell'`Audio Component`, non della `Sound Platform`.
4. Si decide di mettere in pausa la correzione perché richiede un'analisi più approfondita, ma non impatta il resto del progetto.

8.4 Il Gameplay Framework di Unreal Engine

Un *framework* è uno schema di sviluppo che definisce processi e gerarchie. In Unreal Engine esiste una gerarchia di entità sempre presenti (o persistenti in determinati contesti) che gestiscono il flusso del gioco.

8.4.1 Game Instance

- È la prima entità creata all'avvio del progetto e **rimane attiva per tutta la durata dell'applicazione**, indipendentemente dal livello caricato.
- Permette di salvare dati in modo *globale*, cioè accessibili da qualsiasi parte del codice e da qualsiasi mappa.
- **Esempio pratico:** numero di oggetti collezionabili (gemme, frutti, ecc.). Se il valore viene conservato nella Game Instance, passando da un livello all'altro il conteggio rimane; altrimenti, con variabili locali alla mappa o al personaggio, il dato andrebbe perso ad ogni cambio livello.
- Nell'esercizio mostrato: il giocatore raccoglie `collectable` (palline), preme `E` per visualizzare il conteggio, attraversa una porta che carica un nuovo livello vuoto e, tornando indietro, il numero di oggetti raccolti rimane memorizzato. Inoltre, si può salvare anche lo stato di ogni `collectable` per evitare che riappaia se già raccolto (simile al salvataggio dei cristalli e delle gemme in Crash Bandicoot).

8.4.2 Game Mode

- Definisce le **regole del gioco**, gli stati del giocatore e le condizioni di vittoria/sconfitta.
- È associata alla mappa e può cambiare durante l'esecuzione (es. passaggio dal gameplay normale a un minigioco di carte come Gwent in The Witcher 3). In quel caso cambiano punteggi, regole e condizioni.

8.4.3 Game State e Player State

- **Game State**: contiene i dati globali della partita in corso (es. tempo rimanente, punteggio di squadra).
- **Player State**: memorizza lo stato del singolo giocatore (es. numero di uccisioni/morti in un gioco online). È fondamentale per lo scambio di informazioni tra client e server in multiplayer: ogni client comunica con il server e condivide il proprio Player State per aggiornare, ad esempio, il numero di *death* o *kill* in un MOBA.

8.4.4 Pawn e Character

- **Pawn**: non è il personaggio in sé, ma l'entità che **possiede il personaggio** e ne riceve il controllo. Può essere pilotato da input del giocatore o da un *behavior tree* (per l'IA). Contiene componenti come mesh, impostazioni di navigazione, materiali fisici, e definisce la telecamera (*view target*). Il Pawn viene individuato tramite nodi come `Get Player Pawn`.
- **Character**: è una specializzazione del Pawn che include il **Character Movement Component**, il quale gestisce le capacità di movimento:
  - Velocità massima.
  - Movimento in volo (*flying movement*).
  - Nuoto (*swimming movement*).
  - Movimenti personalizzati (scivolata, ecc.).
- Oggi si creerà un nuovo Character da zero con Mixamo e si applicheranno animazioni personalizzate; successivamente si sistemeranno i collegamenti con il resto del progetto.

8.5 Skeletal Mesh, Scheletro e Animazioni

- **Skeletal Mesh**: una mesh (forma visibile) che contiene al suo interno uno **scheletro** (insieme di *bones*, ossa).
- Lo **scheletro** è l'insieme delle ossa e viene definito durante il *rigging* (lavoro dell'animatore che posiziona le ossa). L'animatore ruota queste ossa per creare le animazioni.
- La **Skeletal Mesh** è quindi l'unione di forma + scheletro.
- In Unreal è possibile visualizzare tutte le ossa del personaggio e controllarle tramite l'`Animation Graph` e il **Control Rig**.

8.5.1 Control Rig e Animation Blueprint

- Il **Control Rig** è uno strumento di Unreal che permette di manipolare lo scheletro con regole logiche (simile a quanto fa l'Animation Blueprint ma più orientato al controllo procedurale).
- Esempio: adattamento del piede su superfici inclinate (scalini). L'inclinazione del piede non è animata a mano, ma è gestita da un *animation programmer* che definisce regole del tipo: *se c'è un dislivello di 20°, ruota l'osso in questo modo e allinea la pianta del piede allo scalino.*
- Nel `Character` base di Unreal questa funzionalità è già presente.

8.6 Character Movement Component

- Il **Character Movement Component** definisce come il personaggio si muove.
- Contiene:
  - **Movement Mode** (modalità di movimento): camminare, volare, nuotare, movimento personalizzato (scivolata, ecc.).
  - **Movement Capabilities** (dati): velocità massima, *braking friction* (quanto impiega a frenare), ecc.
  - Interazioni fisiche.

8.7 Player Controller

- Il **Player Controller** è un'entità non fisica (Actor non visibile) che gestisce gli input e può essere associato a un personaggio.
- Permette di cambiare il personaggio controllato (switchare tra più Character) o di modificare i parametri di controllo (es. per effetti di allucinazione).

8.8 Sistema di Input

- Esiste l'**Enhanced Input System** (non sarà usato nel corso), più flessibile, ma si utilizzeranno metodi più semplici.
- L'input rappresenta lo strato più esterno del Gameplay Framework: l'interazione tra macchina e giocatore.

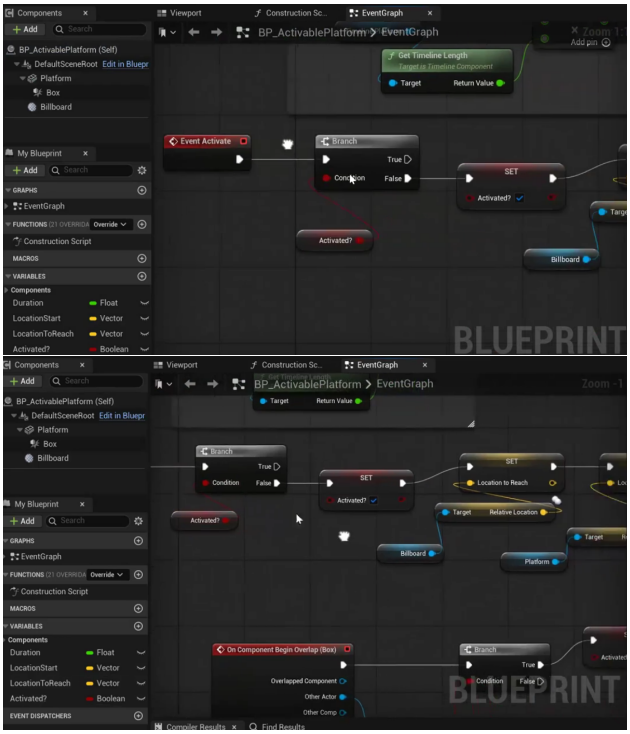


Figure 8.1: Fix ascensore della scorsa lezione.

8.9 Fix rapido per l'ascensore (booleano `Activated`)

Per risolvere il problema del blocco dell'ascensore che scendeva continuamente:

1. Nel Blueprint dell'ascensore, creare una variabile booleana chiamata `Activated`.
2. Nella funzione o evento che attiva il movimento (es. `Activate`), all'inizio mettere un `Branch` con condizione `Activated == false`. Se falso, procedere con il movimento e settare `Activated = true`.
3. Nell'evento `OnBeginOverlap` (o dove si deve resettare), impostare `Activated = false`.

8.10 Game Instance personalizzata per dati globali

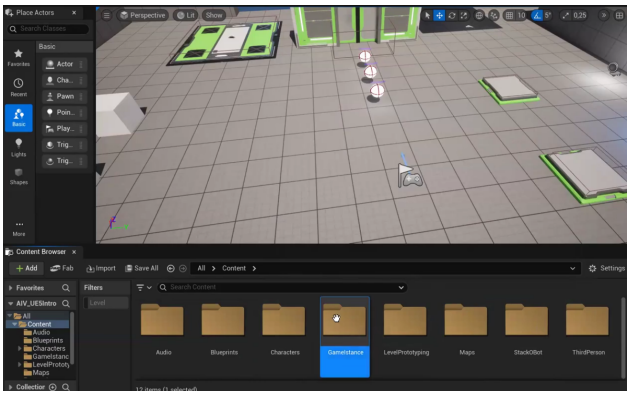
La Game Instance esiste per tutta la durata dell'applicazione e permette di salvare variabili accessibili da qualsiasi livello.

8.10.1 Creazione della classe Game Instance

1. Nella cartella dei Blueprint, creare una nuova `Blueprint Class` di tipo `Game Instance` (cercare "Game Instance" nel menu delle classi).
2. Chiamarla, ad esempio, `GI_Platform`.

8.10.2 Aggiunta della variabile globale

1. Aprire `GI_Platform`. Non ha viewport, è solo un'entità logica.
2. Creare una variabile di tipo `Integer` e chiamarla `Collectables` (rappresenta il numero di oggetti raccolti).



8.10.3 Impostare la Game Instance di progetto

1. Andare in `Edit` >`Project Settings`.
2. Cercare `Game Instance` nella barra di ricerca.
3. In `Project` >`Maps & Modes`, alla voce `Default Game Instance Class`, selezionare `GI_Platform`.
4. Ora il progetto userà questa Game Instance personalizzata invece di quella di default.

8.11 Creazione del `Collectable` (pallina raccoglibile)

- Creare un nuovo `Actor Blueprint` (nome `BP_Collectable`).
- Aggiungere un componente `Sphere` (chiamarlo `Mesh`) per l'aspetto visivo.
  - Impostare la scala a `0.5` su tutti gli assi (assicurarsi che il lucchetto sia chiuso per una modifica uniforme).
  - Assegnare un materiale (es. `M_Hologrid`).
- Aggiungere un componente `Sphere Collision` (chiamarlo `Collision`) per la rilevazione del giocatore.
  - Impostare il `Sphere Radius` a circa `52` (trascinando sul numero o scrivendo).
  - Disabilitare la collisione sulla `Static Mesh` (`Collision Preset = NoCollision`) per evitare doppie collisioni e micro-blocchi.

8.11.1 Logica di raccolta e comunicazione con la Game Instance

1. Nel grafo eventi, dal componente `Collision` aggiungere l'evento `OnComponentBeginOverlap`.
2. Collegare l'`Other Actor` a un nodo `Cast To ThirdPersonCharacter` (o al Character usato).
3. Dal pin `Exec` del cast riuscito, collegare un `Branch` (condizione: sempre `true` per sicurezza) e poi un nodo `Destroy Actor` (distrugge il collectable).
4. Per aggiornare il conteggio globale:
   - Aggiungere il nodo `Get Game Instance` (tasto destro >`Get Game Instance`).
   - Dal suo output, eseguire un `Cast To GI_Platform` (per penetrare la "scocca" e accedere alle variabili personalizzate).
   - Dal pin `As GI_Platform` del cast, trascinare per ottenere la variabile `Collectables` (`Get Collectables`).
   - Incrementare il valore: usare un nodo `+` (Integer + Integer) con l'input 1, quindi un `Set Collectables` per salvarlo.

Nota sul Cast: Il semplice `Get Game Instance` restituisce una referenza generica; per accedere ai membri specifici di `GI_Platform` è necessario il `Cast To GI_Platform`. Se il cast fallisce (pin `Cast Failed`) significa che la Game Instance in uso non è quella aspettata.

8.11.2 Troubleshooting: evento collisione non visibile

Se il nodo `OnComponentBeginOverlap` non compare:

- Verificare di aver selezionato il componente `Sphere Collision` nel pannello Componenti e di aver aggiunto l'evento da lì (non dalla mesh).
- Se era stato collegato alla mesh, scollegarlo e ri-aggiungerlo dalla collisione sferica.

8.12 Completamento del sistema di raccolta con Game Instance

8.12.1 Incremento e stampa dei `Collectable`

Dopo aver eseguito il `Cast To GI_Platform` e aver ottenuto `Collectables`:

1. Dal pin `As GI_Platform` trascinare per ottenere la variabile `Collectables`.
2. Utilizzare il nodo `+ (Integer + Integer)` con secondo operando `1` (incremento) e collegarlo a un nodo `Set Collectables` per aggiornare il valore globale.
3. **Suggerimento**: è possibile usare il nodo `Increment Int` (che incrementa di 1) o `Decrement Int` (decrementa di 1).
4. Per visualizzare il conteggio attuale (stile Crash Bandicoot), aggiungere un nodo `Print String`: dallo stesso `As GI_Platform` ottenere `Collectables`, convertirlo in stringa e collegarlo alla stampa.
5. Infine, distruggere l'attore con `Destroy Actor`.

8.12.2 Visualizzazione del conteggio con un tasto (es. `E`)

Nel `ThirdPersonCharacter` (o nel Character in uso):

1. Aggiungere l'evento di input `E` (cercare "E" tra gli eventi tastiera).
2. All'interno dell'evento, eseguire il `Cast To GI_Platform` (tramite `Get Game Instance`).
3. Dal cast riuscito, ottenere `Collectables` e passarlo a un `Print String`.
4. Comportamento: raccogliendo `collectable` e premendo `E` si visualizza il numero totale accumulato.

8.13 Correzione di errori comuni con la Game Instance (troubleshooting)

- **Errore riscontrato**: uno studente aveva creato una classe chiamata `GI_Platform` ma con parent class errata (non `Game Instance`), impedendo il cast e mostrando errori come "blueprint bad" o "istanza di gioco non valida".
- **Soluzione**:
  1. Creare una nuova `Blueprint Class` selezionando come classe genitore **Game Instance** (in italiano *Istanza di gioco*).
  2. Rinominarla in modo coerente (es. `GI`).

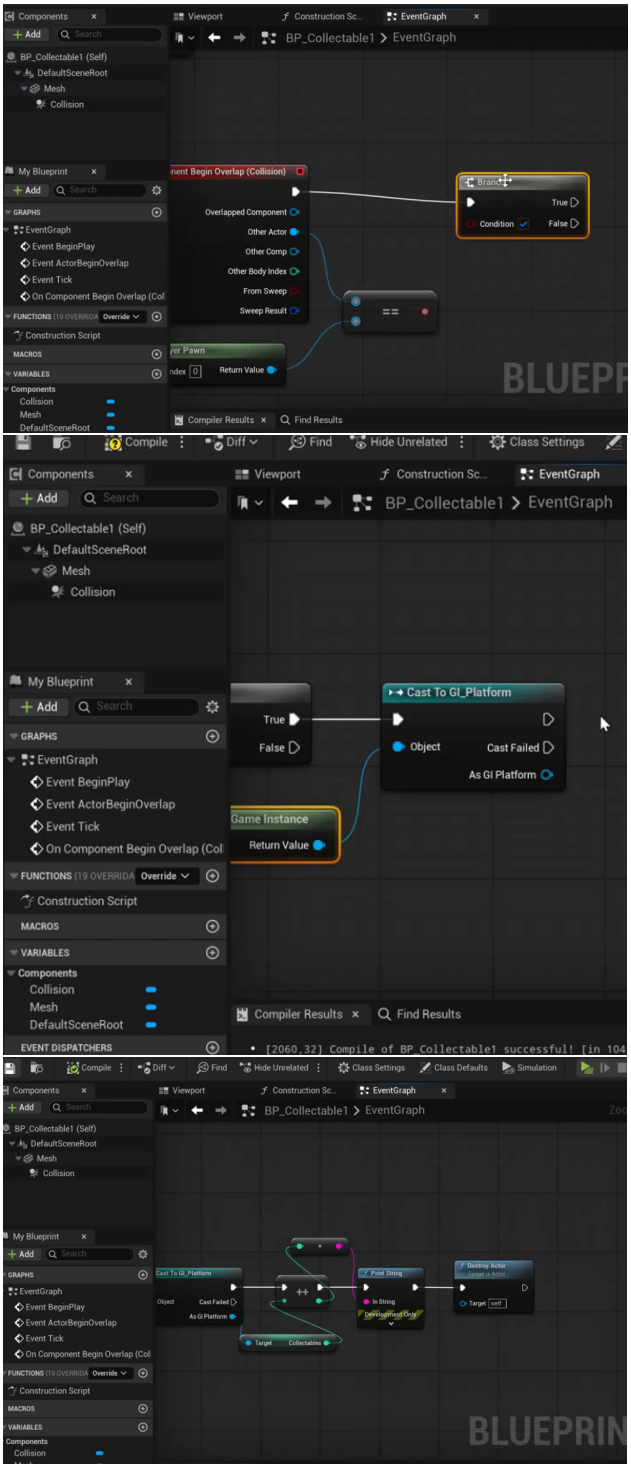
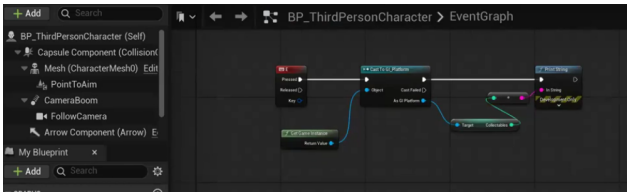


Figure 8.2: BP parziale del GameInstance.



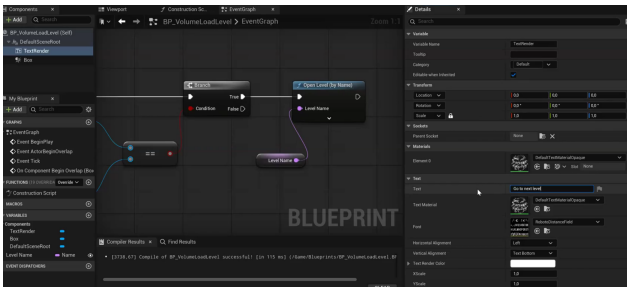
3. In `Edit > Project Settings > Maps & Modes` impostare la `Default Game Instance Class` con la nuova classe.
4. In tutti i blueprint dove si esegue il `Cast To GI_Platform`, sostituire il nodo di cast con `Cast To GI` (o il nuovo nome) e ricollegare i pin.
5. Ricollegare anche i nodi `Get Collectables` e `Set Collectables` al nuovo riferimento.

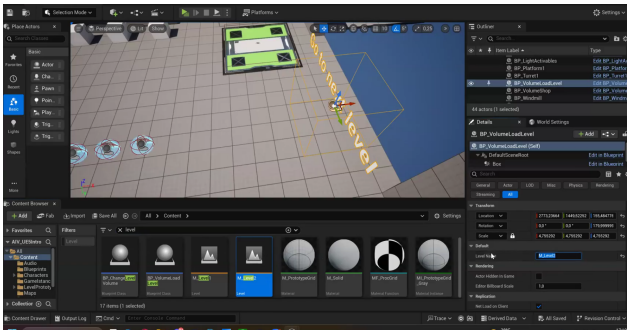
8.14 Volume per azzerare i `Collectable` (`Volume Shop`)

- Creare un nuovo `Actor Blueprint` (es. `BP_VolumeShop`).
- Aggiungere un `Box Collision` e un `Text Render` (impostare il testo a piacere, es. "Volume Shop").
- Nell'evento `OnComponentBeginOverlap` (dal Box Collision):
  1. Verificare che l'`Other Actor` sia il giocatore (`Cast To ThirdPersonCharacter`).
  2. Ottenere la `Game Instance` e fare `Cast To GI_Platform`.
  3. Dal cast riuscito, chiamare `Set Collectables` con valore `0`.
- (Opzionale) attivare la visibilità del volume in gioco disabilitando l'opzione `Hidden in Game` sul componente Box.

8.15 Volume per cambiare livello (`Volume Load Level`)

- Creare un `Actor Blueprint` (es. `BP_VolumeLoadLevel`).
- Aggiungere `Box Collision` e `Text Render` (testo "Next Level" o simile).
- Nell'evento `OnComponentBeginOverlap`:
  1. Verificare la collisione con il giocatore.
  2. Aggiungere il nodo `Open Level (by Name)`.
  3. Trascinare il pin `Level Name` e scegliere `Promote to Variable` (così da poterlo impostare dall'editor).
  4. Assegnare alla variabile il nome esatto del livello di destinazione (attenzione al **case sensitive**: "M_Level2" $\neq$ "m_level2").





8.15.1 Creazione di un nuovo livello

1. Creare una cartella `Maps` (consigliata).
2. `File > New Level` e scegliere **Basic** (livello vuoto).
3. Salvare immediatamente (`Ctrl+S`) con un nome preciso, es. `M_Level2`.
4. Per ottenere il nome esatto da inserire nel `Level Name` del volume:
   - Selezionare il file del livello nel Content Browser.
   - Premere `F2` per rinominarlo, copiare il nome (`Ctrl+C`) e incollarlo nel campo `Level Name` del volume.

8.16 Test del passaggio dati tra livelli

1. Disporre alcuni `BP_Collectable` nella mappa iniziale.
2. Raccoglierli e premere `E`: mostra il totale (es. 3).
3. Entrare nel volume `Volume Load Level` per caricare `M_Level2`.
4. Nella nuova mappa, posizionare altri `Collectable` e un volume per tornare indietro (con `Level Name` impostato al nome della mappa iniziale, es. `ThirdPersonMap`).
5. Raccogliere altri oggetti e premere `E`: il totale somma a quelli precedenti (es. 6).
6. **Osservazione**: i `Collectable` già presi nella mappa precedente **riappaiono** quando si torna indietro, perché non è stato salvato lo stato individuale di ciascun attore. Con la Game Instance si sta salvando solo il *conteggio*, non quali specifici oggetti siano stati raccolti.

8.17 Introduzione a Mixamo e pausa

- Il sistema basato su `Game Instance` permette di condividere dati globalmente (punteggi, personaggio scelto, ecc.).
- Esempio d'uso: selezione del personaggio (A, B, C) e mantenimento della scelta tra livelli.
- Dopo la pausa, si passerà alle animazioni con **Mixamo** (necessario un account Adobe).
- Si chiede di verificare di aver effettuato l'accesso a Mixamo.

8.18 Scelta e download del personaggio su Mixamo

- Mixamo presenta due sezioni: **Characters** e **Animations**. Per questa fase interessa la sezione Characters.
- Scegliere un personaggio preferibilmente umanoide (anche mostri umanoidi vanno bene). Esempio: *Paladin J Nordstrom*.
- Prima di scaricare, assicurarsi che il personaggio sia in **T-pose**: eliminare l'eventuale animazione predefinita cliccando sulla **X** accanto all'animazione.
- Formato di download: **FBX Binary**, posa **T-pose**.
- Dopo il download, pausa; si riprenderà con l'importazione in Unreal Engine.

8.19 Problemi tecnici e debugging durante la pausa

8.19.1 Posizionamento del volume a terra

- Nella viewport dei Blueprint la combinazione di tasti `FN + frecce` non funziona per spostare un oggetto al suolo.
- Occorre impostare manualmente la coordinata `Z` del Box Collision a circa `30--35` per collocarlo al livello del terreno.

8.19.2 Fix della piattaforma con booleano `Activated`

Problema: la piattaforma (ascensore) non risale dopo essere stata attivata.

- Creare una variabile booleana `Activated`.
- Nell'evento che attiva il movimento (es. `Event Activate`), aggiungere un `Branch` con condizione `Activated == false`. Se falso, procedere con il movimento e settare `Activated = true`.
- Per evitare che la piattaforma si riattivi durante il movimento, spostare il reset (`Activated = false`) sull'evento `Finished` della Timeline (invece che su `OnBeginOverlap`), collegando il pin `True` del `Branch` anche al `Reverse` della Timeline.
- Se la piattaforma scompare o non si muove, controllare la `Duration` e il `Play Rate` della Timeline.

8.19.3 Errore `Sound Platform / Audio Component` non valido

Sintomo: l'errore compare solo se la piattaforma arriva fino alla fine del movimento.

- L'errore indica che `Audio Component` (o la variabile `Sound Platform`) non è valido.
- Soluzione: eliminare la variabile `Sound Platform` esistente e tutte le connessioni `Set/Get`.
- Dal nodo che restituisce l'audio component (es. `Return Value` di un `Spawn Sound 2D`), creare una nuova variabile tramite `Promote to Variable`, chiamandola ad esempio `Sound`.
- Ricollegare tutti i `Set` e `Get` alla nuova variabile. In particolare, il `Target` del `Set Pitch Multiplier` deve ricevere la variabile `Sound` (non un'altra).

8.19.4 Errore `Movement Platform` e crash all'uscita

Problema di Marco: errori multipli all'uscita dal Play, legati a una Timeline chiamata `Movement Platform`.

- Controllare che non esistano più Timeline con lo stesso nome che causano conflitti.
- Se è stata duplicata una piattaforma, potrebbe esserci un riferimento a una Timeline rinominata in modo errato.
- Soluzione: eliminare la vecchia variabile Timeline (dal pannello delle variabili del Blueprint) e ricollegare il nodo `Get Timeline` alla Timeline corretta, aggiornando tutti i collegamenti.

8.20 Importazione del personaggio Mixamo in Unreal Engine

1. Prendere il file `.fbx` scaricato (es. `Paladin_J_Nordstrom.fbx`) e trascinarlo direttamente nel **Content Browser** di Unreal.
2. Si apre la finestra di importazione FBX:
   - Controllare le impostazioni di **Offset** (Translation e Rotation): devono essere tutte a `0.0`. Eventuali offset errati possono essere corretti prima dell'import.
   - Lasciare tutte le altre opzioni predefinite.
3. Cliccare `Import`. Unreal creerà automaticamente i seguenti asset:
   - **Skeletal Mesh** (la mesh con lo scheletro).
   - **Physics Asset** (per le collisioni fisiche).
   - **Skeleton** (lo scheletro).
   - **Animation** (di default la T-pose, colorata in verde scuro). **Questa animazione è inutile** (un solo frame fermo) e può essere eliminata.

8.20.1 Analisi dello scheletro

- Aprendo l'asset **Skeleton** si visualizzano tutte le ossa del personaggio.
- Ogni osso (bone) è rappresentato da una **spina** (linea) e da un **cerchio** (punto di giuntura, es. sulla spalla).
- **Regola fondamentale**: le ossa **si ruotano**, **non si traslano**. Spostare un osso produce deformazioni innaturali (effetto *Slenderman*).
- Per provare: selezionare una spina, premere `E` (Rotate) e ruotare. Il personaggio si piega correttamente attorno all'articolazione.
- Le animazioni sono quindi **sequenze di rotazioni** delle ossa, non di spostamenti.

8.21 Concetti di animazione e strumenti esterni

- Programmi come **Blender** e **Maya** gestiscono le animazioni in modo più avanzato, ma il principio fondamentale rimane la **rotazione delle ossa** (le ossa non si traslano, pena deformazioni innaturali).
- Il **Physics Skeleton** (Asset Fisico) gestisce le proprietà fisiche delle ossa (collisioni, materiali fisici, impatti) e non verrà trattato in dettaglio.

8.22 Scelta e download delle animazioni su Mixamo

8.22.1 Tipologie di animazioni richieste

- Passare alla sezione **Animations** di Mixamo e selezionare il personaggio precedentemente scaricato.
- Sono necessarie tre animazioni principali:
  - **Idle**: animazione di stasi (respirazione, movimenti minimi).
  - **Walk**: camminata.
  - **Run** (o **Sprint**): corsa.

8.22.2 Impostazioni dell'animazione su Mixamo

- **Overdrive**: velocizza o rallenta l'animazione (es. aumentandola l'animazione accelera, diminuendola rallenta).
- **Character Arm Space**: modifica la distanza delle braccia dal corpo (può causare clipping se eccessivo).
- **Trim**: ritaglia l'animazione, ma spesso produce risultati innaturali se manca il ritorno all'Idle.
- **Mirror**: specchia l'animazione (utile per invertire direzioni).
- **In Place** (fondamentale): attivare l'opzione **"In Place"** per bloccare l'animazione sul posto, rimuovendo il *Root Motion* (spostamento ereditato dall'animazione).
  - **Root Motion**: presente in giochi come *Dark Souls 3* dove l'animazione stessa sposta il personaggio. In Unreal, per il movimento standard, si usa la capsula del Character; l'animazione deve seguire la capsula, non muoverla.
  - Se un'animazione non supporta *In Place*, si consiglia di cambiare animazione.

8.22.3 Parametri di download

- **Formato**: `FBX Binary`.
- **Frames per Second**: `30 FPS`. **Importante** scaricare *tutte* le animazioni con lo stesso frame rate (per evitare problemi nel blending).
- **Skin**: **Without Skin** (lo scheletro e la mesh del personaggio sono già stati importati separatamente).
- **Keyframe Reduction**: non necessaria, lasciare predefinita.
- Dopo il download si ottengono file `.fbx` (es. `Walking.fbx`, `Idle.fbx`).

8.23 Importazione delle animazioni in Unreal Engine

1. Creare una cartella `Animations` (consigliata) e trascinarvi i file `.fbx` delle animazioni.

2. All'apertura della finestra di import, **non sarà possibile cliccare Import** subito perché l'animazione ha bisogno di uno scheletro di destinazione.

3. Scorrere fino alla sezione **Skeleton** e selezionare dal menu a tendina lo **scheletro del personaggio** importato in precedenza (es. `Paladin_J_Nordstrom_Skeleton`).

4. Una volta selezionato lo scheletro, il pulsante `Import` diventa attivo. Cliccarlo.

5. L'animazione viene "targettata" sullo scheletro scelto: ora è associata a quel personaggio e può essere aperta con doppio clic per la preview.

8.23.1 Impostazioni dell'asset di animazione

Aprendo l'animazione si accede all'editor con timeline e controlli:

- **Controlli di playback**: Play/Pausa, andare al primo/ultimo fotogramma, avanzare di un frame.

- **Proprietà principali**:

  - **Loop**: spuntare se l'animazione deve ripetersi ciclicamente (es. Idle, Walk, Run).
  - **Rate Scale**: moltiplicatore di velocità dell'animazione (1 = normale, 1.5 = più veloce, 2 = doppia velocità). Simile all'Overdrive di Mixamo.

- Verificare che l'opzione **Root Motion** sia disattivata (nessuna spunta su `Enable Root Motion` o `Root Motion`).

- Ripetere l'importazione e la configurazione per le altre due animazioni (Idle, Walk, Run).

8.24 Animation Blueprint (logica delle animazioni)

8.24.1 Creazione e setup iniziale

- Nella cartella del personaggio (o in una nuova cartella `Blueprints`) creare un **Animation Blueprint** (tasto destro `Animation > Animation Blueprint`).

- Assegnargli un nome (es. `ABP_Night`).

- Viene richiesto di selezionare lo **scheletro di destinazione**: scegliere lo stesso scheletro del personaggio importato.

- L'Animation Blueprint si compone di due ambienti principali:

  - **Event Graph**: logica di programmazione ad eventi (simile al normale Blueprint) dove si aggiornano variabili e si comunica con il Character.
  - **Anim Graph**: grafo delle animazioni, dove si collegano i nodi di animazione (es. *Blend Space*, *State Machine*) al nodo finale **Output Pose** (il risultato dell'animazione del personaggio).

8.24.2 Inizializzazione e comunicazione con il Character

- Nell'Event Graph, usare il nodo `Event Initialize Animation` (evento eseguito all'inizio, analogo a `BeginPlay`).
- Da esso, chiamare `Get Owning Actor` (o `Try Get Owner`) per ottenere il **Pawn** che possiede l'Animation Blueprint.
- Eseguire un `Cast To` il nome del **Character Blueprint** (non ancora creato! vedi sotto). Il cast serve per accedere alle variabili del Character (es. velocità, stato di salto).
- **Attenzione**: per eseguire questo passaggio è necessario **prima creare il Character Blueprint** corrispondente, altrimenti non esiste la classe su cui fare il cast.

8.25 Creazione del nuovo Character Blueprint

8.25.1 Procedura

1. Nella cartella del personaggio, creare un nuovo **Blueprint Class** di tipo **Character** (non semplice Actor). Chiamarlo ad esempio `BP_Night` (o `BP_Knight`).
2. Il Character Blueprint contiene di default:
   - **Capsule Component** (collisione principale).
   - **Mesh** (Skeletal Mesh Component, per il modello 3D scheletrico).
   - **Character Movement Component** (gestisce velocità, accelerazione, modalità di movimento, ecc.).

8.25.2 Assegnazione della Skeletal Mesh

- Selezionare il componente `Mesh` nel pannello Componenti.
- Nel pannello Dettagli, sotto la sezione `Mesh`, assegnare la **Skeletal Mesh** del personaggio importato (quella con lo scheletro).
- **Nota**: la Skeletal Mesh è l'unione di forma visibile e ossa; assegnandola, il Character acquisisce l'aspetto e lo scheletro corretti.

8.25.3 Collegamento con l'Animation Blueprint

- Sempre sul componente `Mesh`, impostare l'**Anim Class** (o `Animation Blueprint Generated Class`) selezionando l'Animation Blueprint creato in precedenza (`ABP_Night`).
- In questo modo il Character utilizzerà la logica di animazione definita nell'Animation Blueprint.
- Ora sarà possibile completare il `Cast To` nell'Animation Blueprint, perché la classe del Character è stata creata.

8.26 Configurazione del nuovo Character Blueprint

- Il Character, se importato con offset di posizione, può apparire ruotato. Basta riposizionarlo: usare **Snap** a 10 e allinearlo alla capsula (la freccia indica il forward vector).
- All'inizio il personaggio è in T-pose e privo di animazioni.

8.26.1 Assegnazione dell'Animation Blueprint al Character

1. Selezionare il componente **Mesh** del Character.
2. Nel pannello Dettagli, cercare la proprietà **Animation Class** (o *Anim Class*).
3. Scegliere l'Animation Blueprint creato in precedenza (es. `ABP_Night_1`).
4. Così facendo, il Character userà la logica definita in quell'Animation Blueprint.

8.27 Logica dell'Animation Blueprint

8.27.1 Inizializzazione e referenze (Event Graph)

- Nell'`Event Graph` dell'Animation Blueprint, dal nodo `Event Initialize Animation`:
  1. Eseguire `Try Get Owner` (o `Get Owning Actor`) per ottenere il Pawn proprietario.
  2. Fare un `Cast To BP_Night_1` (o il nome del proprio Character).
  3. Promuovere il risultato del cast a variabile (es. `BP_Night_1`) per avere una referenza diretta al Character ed evitare cast ripetuti a ogni frame.
  4. Dal Character, ottenere il `Character Movement Component` e promuoverlo a variabile (es. `CharacterMovement`).
- Per sicurezza, verificare che la referenza al Character sia valida con un nodo `Convert to Validated Get` (da `Get BP_Night_1`): si ottiene un'esecuzione condizionale *Is Valid / Is Not Valid*.

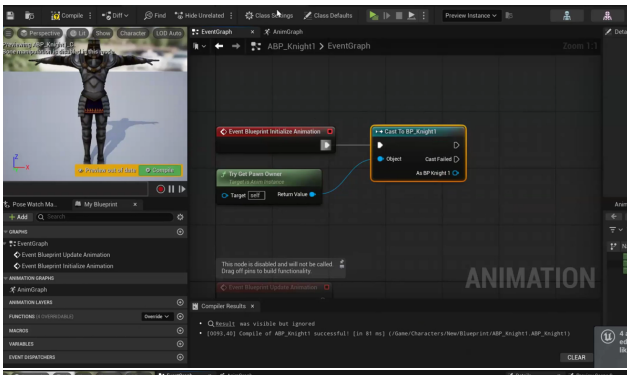
8.27.2 Aggiornamento a ogni frame (Event Blueprint Update Animation)

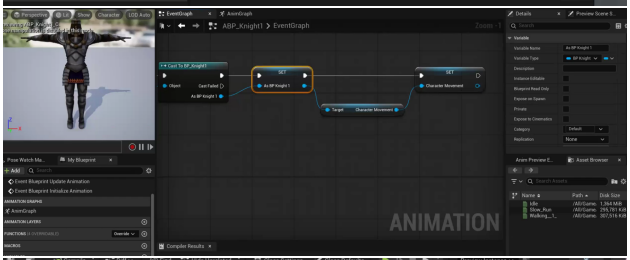
- `Event Blueprint Update Animation` viene chiamato **ogni frame** (simile a un *Tick*), ogni volta che l'animazione si aggiorna.
- **Non** si deve ripetere il cast a ogni frame: si usano le variabili salvate all'inizializzazione, controllando prima la validità con l'`Is Valid Sequence` ottenuto dal *Validated Get*.
- All'interno del ramo *Is Valid*:
  - **Ottenere la velocità orizzontale**:
    1. Dal `Character Movement` (variabile), chiamare `Get Velocity`.
    2. Per ignorare la componente verticale (salto/caduta), usare il nodo `Vector Length XY`: restituisce un `float` che rappresenta la velocità sul piano orizzontale.
    3. Promuovere questo valore a variabile e chiamarla `GroundSpeed`.
  - **Stato di caduta**:
    1. Sempre dal `Character Movement`, ottenere il booleano `Is Falling`.
    2. Promuoverlo a variabile (es. `IsFalling`) e settarlo a ogni frame per sapere se il personaggio è in aria.
  - **Velocità completa** (opzionale): promuovere anche il vettore `Velocity` a variabile per eventuali utilizzi futuri.

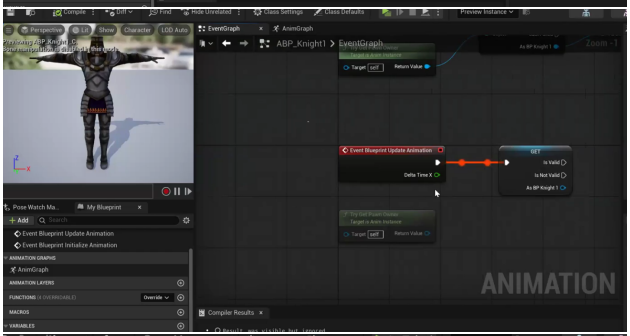
8.28 Animation Graph e Blend Space (introduzione)

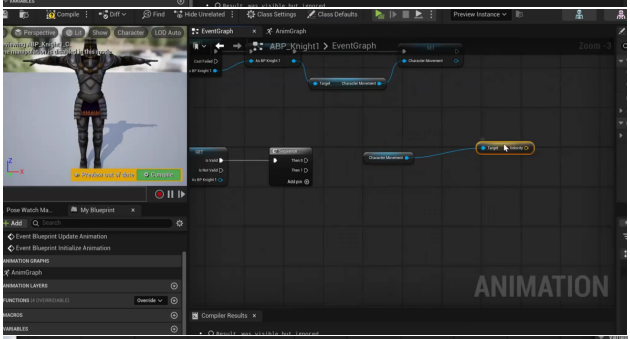
8.28.1 Collegamento diretto di un'animazione

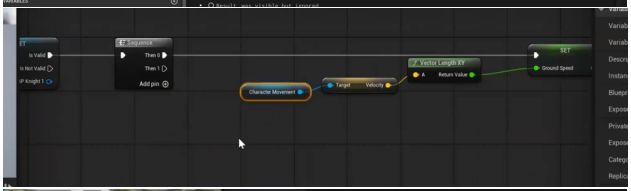
- Nell'`Anim Graph`, trascinare un'animazione (es. `Walking_1`) dall'`Asset Browser` e collegare il suo pin di output al nodo `Output Pose`.
- Impostare l'animazione in **Loop** (spuntare `Loop Animation` nei dettagli) per vederla ciclica.
- A questo punto il Character nella viewport (e in gioco) eseguirà solo quella singola animazione.


ANIMATION


ANIMATION


ANIMATION


ANIMATION



8.28.2 Blend Space 1D (anteprima)

- Per fondere più animazioni in base a un parametro (es. velocità) si usa un **Blend Space 1D**.
- Si crea con `Blend Space 1D` selezionando lo stesso scheletro.
- All'interno, si posizionano le animazioni (es. *Idle* a velocità 0, *Walk* a un valore medio, *Sprint* a velocità massima). Il risultato è un'animazione che si interpola in modo fluido al variare del parametro (la `GroundSpeed`).
- La prossima lezione sarà dedicata alla costruzione del Blend Space e al collegamento con le variabili dell'Event Graph.

8.29 Preparazione per la lezione successiva

- Obiettivo: completare il sistema di animazione con Blend Space per Idle/Walk/Sprint, integrare l'animazione di salto e caduta (retargeting di animazioni già disponibili in Unreal).
- Importanza di avere tutte le animazioni allo stesso framerate (30 FPS) per un blending corretto.

8.30 Risoluzione problema materiali personaggio

Problema: alcuni personaggi importati da Mixamo non hanno i materiali assegnati, oppure compaiono solo le texture.

- Aprendo la Skeletal Mesh, nel pannello Dettagli sono elencati gli **slot di materiale** (es. `Element 0`, `Element 1`...).
- Se mancano i materiali, occorre crearli manualmente:
  1. Tasto destro nel Content Browser: `Material` (nominarlo, es. `M_Archer_Clothing`).
  2. Aprire il materiale e trascinare al suo interno le texture necessarie (base color, normal, ecc.).
  3. Collegare la texture **Diffuse** (o *Albedo*) al pin `Base Color` e la **Normal** al pin `Normal`.
  4. Ripetere per ogni slot (es. uno per i vestiti, uno per la pelle) e assegnare i materiali corretti agli slot corrispondenti della Skeletal Mesh.
- **Nota**: per una corretta importazione di texture e materiali direttamente dal file FBX, nella finestra di import spuntare `Import Materials` e `Import Textures`.
- Se Mixamo non risponde, riprovare o scaricare nuovamente il personaggio.

8.31 Chiusura e compiti

- Si ricorda di inviare eventuali screenshot di problemi sul canale Discord *Materiale Didattico* per ricevere assistenza.
- Prossima lezione: completamento delle animazioni (Blend Space, retargeting).

Chapter 9

Introduzione ai Sistemi di Animazione del Personaggio in Unreal Engine 5

9.1 Introduzione e obiettivi della lezione

Proseguimento diretto dall'interruzione precedente: creazione di animazioni partendo dal Blendspace già impostato. Si decide di affrontare la realizzazione di menu e UI, con la possibilità, in base agli interessi dei presenti, di dedicare parte della lezione ad argomenti specifici (es. menu, bottoni, funzionalità). In assenza di richieste si procede con il menu classico.

9.2 Animation Blueprint: eventi fondamentali

9.2.1 Initialize Animation

- Equivalente del `BeginPlay` per le animazioni.
- Viene eseguito una sola volta all'inizializzazione dell'animazione, indipendentemente dal frame.
- Utile per configurare riferimenti iniziali.

9.2.2 Update Animation (Event Blueprint Update Animation)

- Equivalente del `Tick` per le animazioni.
- Chiamato a ogni frame per aggiornare qualsiasi parametro dell'animazione (es. rotazione di un arto).

9.3 Ottenere i dati dal personaggio

9.3.1 Cast al Pawn Owner

- Il `Pawn Owner` è l'entità che controlla il personaggio (il `Character` possiede mesh, scheletro, ecc.).
- Tramite un nodo `Cast To BP_Knight` si ottiene un riferimento diretto al Blueprint del cavaliere.
- Si memorizza anche il riferimento al `Character Movement Component`, contenente tutte le proprietà di movimento (gravità, accelerazione massima, altezza massima del passo, ecc.).

9.3.2 Velocità (Ground Speed)

- Nell'evento `Update Animation`, usando una `Sequence`, si preleva la `Velocity` del `Character` e la si memorizza in una variabile chiamata `GroundSpeed`.
- Tale variabile verrà utilizzata per pilotare le transizioni e il Blend Space.

9.3.3 Verifica dello stato di caduta

- Dal `Character Movement` si controlla se il personaggio sta cadendo (`Is Falling`) e si memorizza il risultato in un booleano (`True/False`).
- Questo booleano servirà per distinguere lo stato di salto/caduta nella State Machine.

9.4 State Machine per la locomozione

9.4.1 Creazione della State Machine

- Nell'`AnimGraph` si aggiunge un nodo `State Machine` (tasto destro → `Animation State Machine`) e lo si rinomina `Locomotion` (nome classico per la gestione del movimento).
- Facendo doppio clic si entra nel suo grafo.

9.4.2 Entry e stati

- Il nodo `Entry` rappresenta l'ingresso.
- Da esso si trascina un collegamento per creare un nuovo stato, ad esempio `WalkRun`.
- All'interno di ogni stato si definisce l'`Output Animation Pose`, cioè l'animazione (o Blend Space) da riprodurre.
- Per il momento si inserisce un'animazione di esempio (es. `Walking1`) con l'opzione `Loop` attivata.
- Si creano ulteriori stati: `Jump` e `Fall`.
- Le transizioni (frecce) permettono di passare da uno stato all'altro e, volendo, di tornare indietro (loop di stati) in base a condizioni.

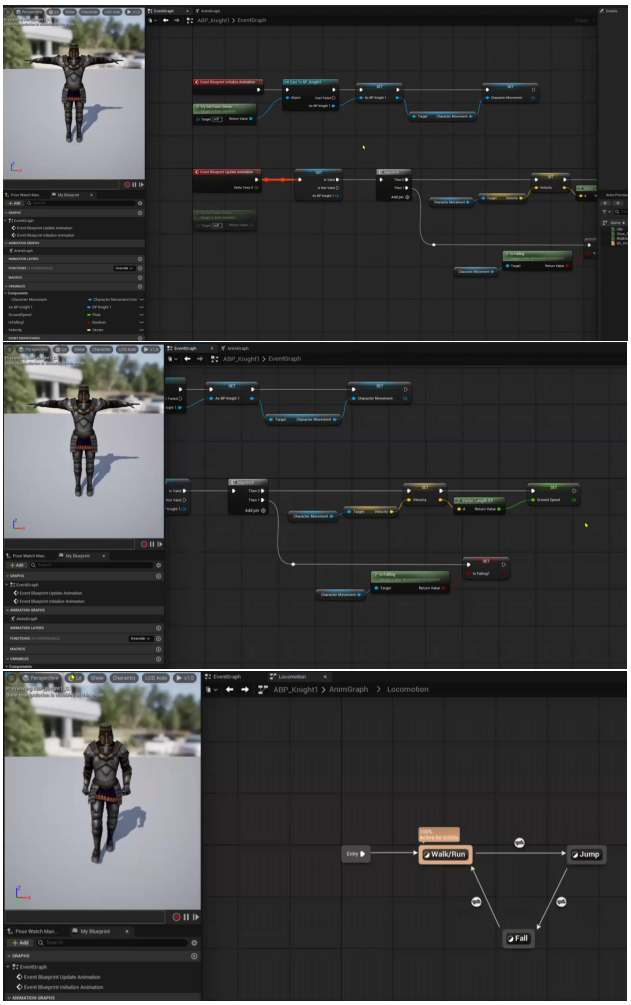
9.4.3 Condizioni di transizione

- I nodini di transizione tra due stati definiscono **quando** avviene il cambio.
- Esempi:
  - Da `WalkRun` a `Jump` se il personaggio sta saltando/cadendo.
  - Da `Fall` a `WalkRun` se non sta più cadendo.
- Le condizioni si basano sulle variabili calcolate nell'Animation Blueprint (`GroundSpeed` e `IsFalling`).

9.5 Blend Space 1D per camminata e corsa

9.5.1 Scopo

- `WalkRun` non deve contenere una singola animazione, ma miscelare dinamicamente tre animazioni: `Idle`, `Walk` e `Run`.
- La miscelazione deve dipendere dalla velocità (`GroundSpeed`).


ABP_Knight1 ❯ EventGraph
ABP_Knight1 ❯ EventGraph
ABP_Knight1 ❯ AnimGraph ❯ Locomotion
Entry
Walk/Run
Jump
Fall

9.5.2 Creazione

- Creare un `Blend Space 1D` (da **Animation → Blend Space 1D**).
- All'apertura si presenta il grafico dell'asset.
- Impostare nei dettagli l'**Horizontal Axis**:
  - Nome: `GroundSpeed`
  - Valore minimo: 0
  - Valore massimo: 600 (unità: cm/s, perché Unreal ragiona in centimetri)

9.5.3 Posizionamento delle animazioni

- Dal `Asset Browser` trascinare le animazioni sul grafico:
  - `Idle` sul valore 0.
  - `Walk` su 300 (metà del massimo, cioè velocità di camminata predefinita).
  - `Run` su 600 (massima velocità).
- Il Blend Space interpolerà automaticamente tra queste in base al valore di `GroundSpeed`.

9.5.4 Collegamento con l'Animation Blueprint

- Nello stato `WalkRun`, come `Output Animation Pose` si assegna il Blend Space creato.
- La variabile `GroundSpeed` calcolata nell'Update Animation viene passata al Blend Space tramite il nome dell'asse.

9.6 Regolazione della velocità di movimento

9.6.1 Modifica della Max Walk Speed

- Nel Blueprint del personaggio (es. `BP_Knight`), componente `Character Movement`, individuare la proprietà `Max Walk Speed` (sotto `Walking`).
- Il valore predefinito è 600 cm/s.
- Per far sì che la camminata sia lo stato di default, ridurla a **300** cm/s.
- In questo modo:
  - A velocità 0–300 l'animazione è `Idle` → `Walk`.
  - Per la corsa si prevede di aumentare temporaneamente la `Max Walk Speed` tramite input (es. tasto Shift), portandola a 600. Questo permette al Blend Space di raggiungere l'animazione di `Run`.

9.6.2 Pianificazione dello sprint

- In futuro si implementerà un input che modifica la `Max Walk Speed` per differenziare le animazioni.
- Il Blend Space reagirà automaticamente al nuovo valore di `GroundSpeed`.

9.7 Problemi tecnici e soluzioni

9.7.1 Errore "Questo asset non ha schede ancorate"

- Si verifica all'apertura di un Blueprint, impedendo la visualizzazione dei pannelli.
- Possibili soluzioni:
  1. Ripristinare il layout di default: nella barra superiore dell'editor dell'asset, cliccare su `Window` → `Load Layout` → scegliere uno dei layout predefiniti (es. `Default Editor Layout`).
  2. Aprire l'editor blueprint completo (se il blueprint risulta consolidato).

9.7.2 Cambio lingua dell'editor

- Per passare da italiano a inglese (o viceversa):
  - `Edit` → `Editor Preferences` → cercare `Language`.
  - In alternativa: `Project Settings` → `Language`.

9.8 Collegamento del Blend Space nell'Animation Blueprint

9.8.1 Utilizzo del Blend Space Player

- Nell'AnimGraph, dopo aver creato il Blend Space 1D (es. `BS_Knight`), lo si trascina nel grafo dell'Animation Blueprint.
- A differenza di un'animazione normale, il nodo `Blend Space Player` presenta un ingresso per il valore dell'asse (es. `GroundSpeed`).
- Si prende la variabile `GroundSpeed` (creata nell'Event Graph) e la si collega:
  - All'ingresso `GroundSpeed` del Blend Space Player.
  - Al nodo `Result` per l'output finale della posa.

9.8.2 Scorciatoie per variabili nei Blueprint

- Per ottenere un nodo `Get` di una variabile: tenere premuto **Ctrl** e trascinare la variabile nel grafo.
- Per ottenere un nodo `Set`: tenere premuto **Alt** e trascinare la variabile (non disponibile nell'Animation Blueprint per alcune variabili, ma valido nei Blueprint normali).

9.8.3 Ordinamento dei nodi

- Selezionando due o più nodi e premendo **Q**, i nodi vengono allineati automaticamente.

9.9 Test del Blend Space e anteprima

- Con `GroundSpeed` a 0 l'anteprima mostra l'animazione di Idle.
- Impostando manualmente `GroundSpeed` a 300, il personaggio cammina (animazione Walk).
- A 600, il personaggio corre (animazione Run).
- A valori intermedi (es. 200) si verifica un'interpolazione tra Idle e Walk; è importante usare valori coerenti con il movimento effettivo per evitare pose innaturali.

9.10 Aggiunta della camera al personaggio

9.10.1 Assenza di camera nei nuovi Character

- I Blueprint di tipo Character non includono una camera di default.
- Per visualizzare correttamente il personaggio in gioco, bisogna aggiungere manualmente i componenti.

9.10.2 Componenti necessari

- **Spring Arm**: funge da braccio della camera, gestisce la distanza e le collisioni.
- **Camera**: componente figlio dello Spring Arm.
- Non modificare mai la posizione della Camera direttamente, ma regolare tutto tramite i parametri dello Spring Arm.

9.10.3 Configurazione dello Spring Arm

- `Target Arm Length`: distanza della camera dal personaggio (es. 300).
- `Socket Offset`: spostamento della camera rispetto al punto di attacco.
  - Modificare principalmente il valore sull'asse Z (es. 40), di poco e mai più di 20-30.
  - Preferire il Socket Offset al Target Offset perché gestisce meglio le collisioni (la camera si avvicina automaticamente quando incontra un ostacolo, riducendo l'arm length).

9.10.4 Abilitazione della rotazione della camera

- Per permettere al giocatore di ruotare la visuale, attivare `Use Pawn Control Rotation` sullo Spring Arm (non sulla Camera).
- Questo fa sì che la camera segua la rotazione del controller, abilitando il movimento libero del mouse.

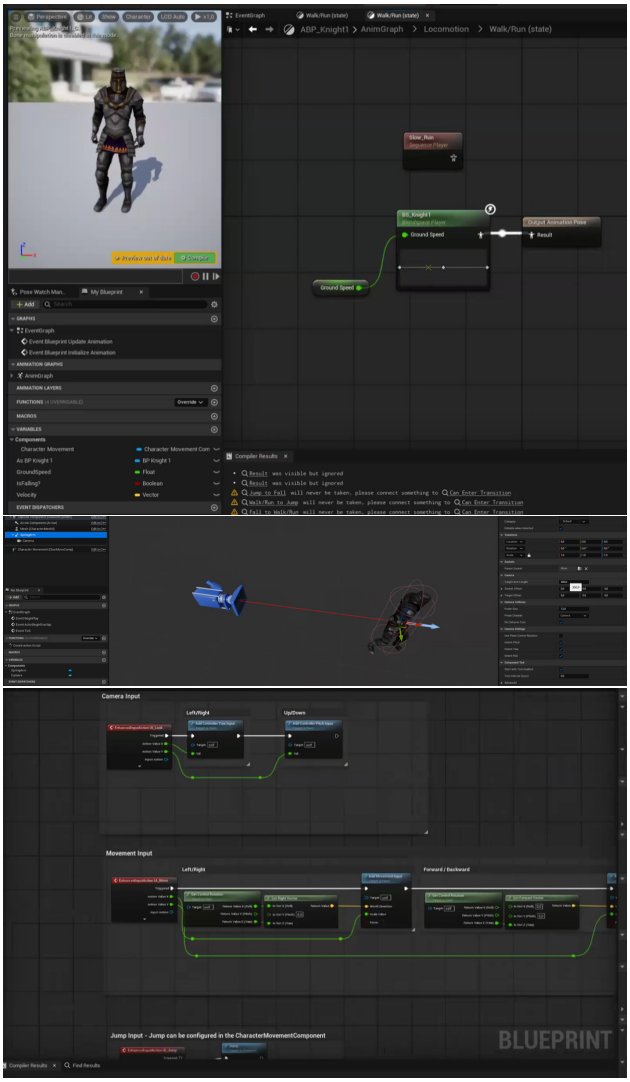
9.11 Movimento del personaggio

9.11.1 Copia dell'input dal Third Person Character di esempio

- Nel proprio Blueprint del personaggio (es. `BP_Knight`) mancano gli eventi di input.
- Soluzione rapida: aprire il Third Person Character predefinito di Unreal, copiare (Ctrl+C) i nodi principali dell'Event Graph:
  - **Input Mapping Context** (gestisce la mappatura dei tasti).
  - **Add Movement Input** (controlla il movimento in avanti/indietro e laterale).
  - **Jump** (azione di salto).
- Incollare (Ctrl+V) i nodi nell'Event Graph del proprio personaggio.

9.11.2 Spiegazione dell'Input Movement

- L'evento `IAMove` è legato a tasti (es. WASD) e restituisce due valori (X e Y).
- Tali valori diventano un vettore che viene passato a `Add Movement Input` per muovere il personaggio lungo gli assi forward/backward e left/right.


ABP_Knight1 > AnimGraph > Locomotion > Walk/Run (state)
Ground Speed
Result
Camera Input
Left/Right
Up/Down
Movement Input
Forward / Backward
Jump Input - Jump can be configured in the CharacterMovementComponent
BLUEPRINT

9.12 Orientamento del personaggio al movimento

- Se il personaggio si muove lateralmente senza ruotare (strafing), abilitare nel `Character Movement Component`:
  - `Orient Rotation to Movement`: fa ruotare il personaggio nella direzione del movimento.
  - Disabilitare `Use Controller Desired Rotation` per evitare conflitti.
- Questa combinazione garantisce che il personaggio giri nella direzione in cui cammina.

9.13 Suggerimento sul Field of View (FOV)

- Nelle impostazioni della Camera, il `Field of View` può essere ridotto (es. 80) per ottenere un effetto più claustrofobico, spesso usato nei giochi horror.

9.14 Retargeting delle animazioni di salto e caduta

9.14.1 Perché il retargeting

- Unreal fornisce animazioni di base (es. salto, caduta) per il manichino predefinito, non direttamente compatibili con il proprio scheletro (es. Paladino).
- Il retargeting adatta un'animazione da uno scheletro sorgente a uno destinazione.

9.14.2 Procedura

1. Nel Content Browser cercare l'animazione desiderata (es. `Jump` del manichino base).
2. Tasto destro sull'animazione → `Retarget Animation`.
3. Nella finestra di retargeting:
   - Viene mostrato lo scheletro sorgente (es. `Mannequin_Skeleton`).
   - Selezionare lo scheletro di destinazione (il proprio, es. quello del paladino).
4. L'anteprima mostra il risultato del retargeting.
5. Cliccare su `Export Animation` (non `Export Retarget Asset`, che creerebbe un asset diverso).
6. Scegliere la cartella di destinazione (es. `Character/Animation`) e salvare.

9.14.3 Note sul retargeting

- Il risultato può variare: con scheletri simili a quello di Unreal spesso funziona bene.
- L'animazione di salto potrebbe presentare un `Root Motion` imperfetto, ma per l'esercizio è accettabile.
- Ripetere il processo per l'animazione `Fall Loop` (caduta in loop).

9.15 Aggiornamento della State Machine

9.15.1 Assegnazione delle animazioni

- Nello stato `Jump` dell'Animation Blueprint, inserire l'animazione di salto retargettata.
- Nello stato `Fall`, inserire l'animazione di caduta retargettata (loop).

9.15.2 Configurazione delle transizioni

- **Da WalkRun a Jump**:
  - Condizione: `Get Is Falling` → `True` (il personaggio sta saltando/cadendo).
- **Da Jump a Fall**:
  - La transizione deve avvenire automaticamente al termine dell'animazione di salto.
  - Selezionare la freccia di transizione, nei dettagli abilitare `Automatic Rule Based on Sequence Player`.
  - In questo modo, appena l'animazione di Jump termina, si passa automaticamente a Fall.
- **Da Fall a WalkRun**:
  - Condizione: `Get Is Falling` → `Not Boolean` (quando non sta più cadendo, ossia atterra).
  - Il nodo `Not Boolean` inverte il valore booleano: se `Is Falling` è `False`, la transizione si attiva.

9.16 Test finale

- Avviare il gioco e muovere il personaggio.
- Verificare che:
  - Cammini e corra in base alla velocità.
  - Saltando passi all'animazione di Jump e poi automaticamente a Fall durante la caduta.
  - Atterrando torni allo stato WalkRun.
- Piccole imperfezioni nell'animazione di salto possono essere sistemate in seguito.

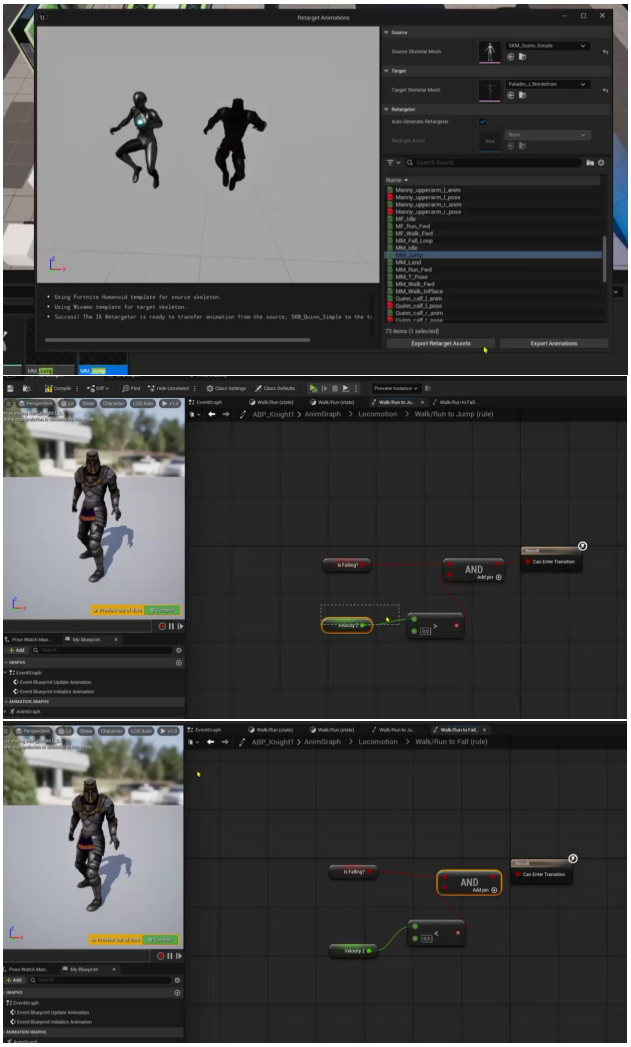
9.17 Perfezionamento delle transizioni di salto e caduta

9.17.1 Differenziazione tra salto e caduta tramite velocità verticale

- Finora la transizione `WalkRun` → `Jump` era attivata dal solo `IsFalling == True`, senza distinguere se il personaggio stesse saltando verso l'alto o cadendo.
- Per gestire anche le cadute accidentali (es. da un dirupo) è necessario separare i due casi usando la componente Z della velocità.
  - `Velocity Z` > 0 → il personaggio sta salendo (salto).
  - `Velocity Z` < 0 → il personaggio sta scendendo (caduta).

9.17.2 Calcolo della Jump Z Velocity nell'Animation Blueprint

- Nell'Event Graph dell'Animation Blueprint (`ABP_Knight`) si prende il vettore `Velocity` e lo si scompone con un nodo `Break Vector` (o `Split`) per ottenerne la componente Z.
- Si promuove il valore a variabile (es.`JumpZVelocity`) per poterlo utilizzare nelle condizioni della State Machine.
- Il nodo `Sequence` già utilizzato per `GroundSpeed` può essere esteso per includere anche questo calcolo.



9.17.3 Aggiornamento delle condizioni di transizione

- **Da WalkRun a Jump**: si utilizza un nodo `AND Boolean`.
  - Ingresso 1: `Get IsFalling == True`.
  - Ingresso 2: `JumpZVelocity > 0` (per sicurezza > 1 per evitare imprecisioni).
- **Da WalkRun a Fall**: per una caduta senza salto (es. si scivola giù da un bordo), si crea una transizione diretta con condizione `IsFalling == True` **ma** senza il vincolo sulla Z (o con `JumpZVelocity < 0`).
  - In questo modo, se il personaggio inizia a cadere senza aver saltato, entra subito nello stato `Fall`.
- La transizione `Jump` $\rightarrow$ `Fall` rimane automatica (`Automatic Rule Based on Sequence Player`).
- La transizione `Fall` $\rightarrow$ `WalkRun` utilizza `IsFalling == False`.

9.18 Risoluzione del problema di rotazione del personaggio

9.18.1 Sintomo

- Il personaggio si muoveva in strafing (es. verso destra) senza ruotare nella direzione di movimento.

9.18.2 Soluzione

- Eliminare il nodo `Tick` che forzava l'uso di `Use Controller Yaw` (o qualsiasi logica che bloccava la rotazione).
- Nel `Character Movement Component` del Blueprint del personaggio (`BP_Knight`):
  - Mantenere `Orient Rotation to Movement` **attivo**.
  - Assicurarsi che `Use Controller Desired Rotation` sia **disattivo**.
- In questo modo il personaggio ruoterà sempre nella direzione del movimento impartito dall'input.

9.19 Implementazione della corsa (Sprint)

9.19.1 Creazione dell'Input Action IA_Run

- Nel Content Browser, navigare nella cartella degli input (`Input/Actions`) e duplicare l'`IA_Jump` (Ctrl+C, Ctrl+V).
- Rinominare il duplicato in `IA_Run`.
- Aprirlo e verificarne le proprietà:
  - `Value Type`: `Digital Bool` (true/false in base alla pressione del tasto).
  - `Triggers`: mantenere `Pressed` e `Released` (attivato alla pressione e al rilascio).

9.19.2 Aggiunta del mapping al contesto di input (IMC_Default)

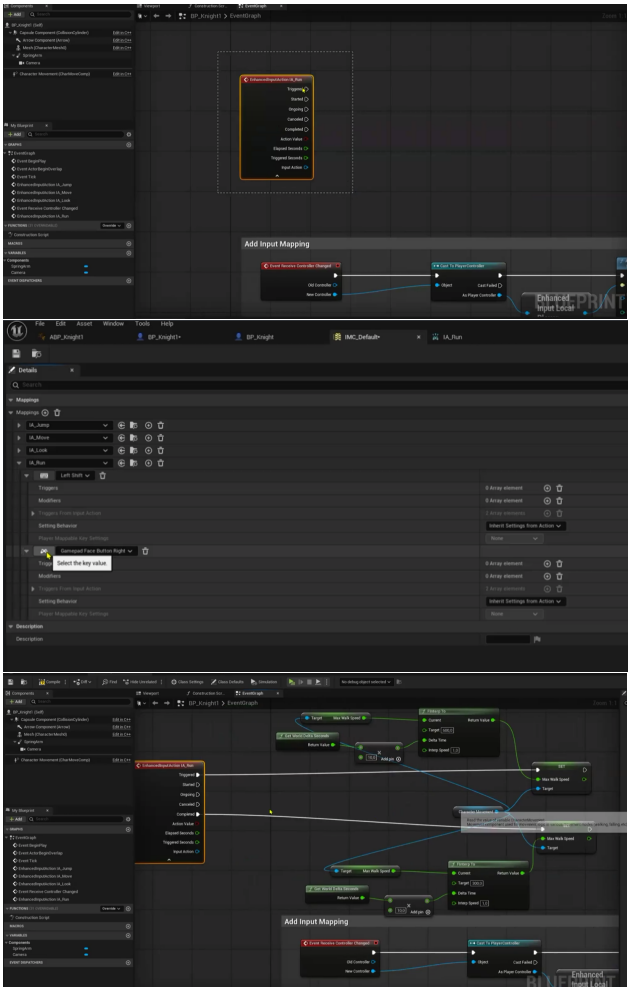
- Aprire l'`IMC_Default` (o il proprio Input Mapping Context).
- Nella lista dei mappings, aggiungere un nuovo elemento:
  - `Input Action`: `IA_Run`.
  - Assegnare un tasto, ad esempio `Left Shift` (Shift sinistro).
  - Opzionalmente aggiungere un secondo mapping per il gamepad (es. `Face Button Right` o `Circle` per controller PlayStation, per ottenere una configurazione simile a Dark Souls).

9.19.3 Logica nel Blueprint del personaggio (BP_Knight)

- L'`IA_Run` viene automaticamente rilevata perché il contesto è già caricato (nodo `Add Mapping Context` copiato dal Third Person Character di esempio).
- Aggiungere i nodi evento per `IA_Run` (cliccare sull'azione e selezionare `Triggered` e `Completed`):
  - **Triggered (Pressed)**: → chiamare `Set Max Walk Speed` sul `Character Movement Component`, impostando il valore a **600**.
  - **Completed (Released)**: → `Set Max Walk Speed` a **300** (ritorno alla camminata).

9.19.4 Transizione morbida con FInterpTo

- Per evitare il cambio istantaneo di velocità si può usare il nodo `FInterp To` (Float Interpolation).
- Parametri:
  - `Current`: attuale `Max Walk Speed`, ottenuta con un `Get Max Walk Speed`.
  - `Target`: velocità desiderata (600 in pressione, 300 al rilascio).
  - `Delta Time`: utilizzare `World Delta Seconds`.



  - `Interp Speed`: moltiplicatore per controllare la rapidità dell'interpolazione (es. 10 o valori più alti per un effetto più rapido).

- **Nota**: poiché `FInterpTo` necessita di essere chiamato ogni frame per interpolare correttamente, e gli eventi `Triggered/Completed` sono puntuali, la semplice connessione diretta non produrrà un'interpolazione continua.

  - Soluzione: usare un timer o spostare la logica su `Event Tick`, controllando un booleano di stato (es. `IsSprinting`) per determinare il valore target.

- Per il momento si mantiene il `Set` diretto, lasciando il perfezionamento con `FInterpTo` come argomento successivo.

9.20 Scaricamento delle animazioni per Blend Space 2D e Montaggi

9.20.1 Obiettivi

- Prossima lezione: creare un **Blend Space 2D** per la locomozione (movimento in tutte le direzioni) e i **Montages** per gli attacchi.

- Servono animazioni specifiche da Mixamo.

9.20.2 Animazione di attacco con Root Motion

- Su Mixamo, scegliere un'animazione di attacco (es. con spada) che **non sia in-place**, cioè che contenga **root motion**.
- L'animazione deve spostare il personaggio (movimento ereditato), caratteristica tipica degli attacchi in stile Dark Souls.
- Scaricare in formato FBX, opzione `Without Skin`, a 30 fps.
- Questa animazione verrà utilizzata per i montaggi.

9.20.3 Animazioni direzionali in-place per il Blend Space 2D

- Servono animazioni di camminata nelle otto direzioni (avanti, indietro, destra, sinistra e quattro diagonali).
- Su Mixamo cercare:
  - `Walk Forward`
  - `Walk Backward`
  - `Strafe Right` (o `Strafe Left` e specchiare)
  - `Strafe Left` (o utilizzare il mirror)
  - `Walk Forward Right` (diagonale)
  - `Walk Forward Left`
  - `Walk Backward Right`
  - `Walk Backward Left`
- Tutte queste animazioni **devono essere in-place**, senza spostamento root (opzione `In Place` attiva su Mixamo).
- Consiglio: per ottenere sia destra che sinistra da una singola animazione, scaricare `Strafe Left` e poi utilizzare l'opzione **Mirror** durante il download o dopo l'importazione in Unreal. In questo modo si raddoppiano le animazioni con coerenza stilistica.
- Totale animazioni: circa 6–8 (se si specchiano le laterali e le diagonali, si possono ridurre i download a 4-5).

9.20.4 Importazione e preparazione

- Le animazioni verranno importate e successivamente organizzate in Blend Space 2D.
- La lezione riprenderà dopo la pausa con la creazione del Blend Space 2D e dei montaggi.

9.21 Discussione e feedback sul corso

- Viene ribadita la possibilità di fornire feedback sul ritmo, i contenuti o la struttura delle lezioni.
- La lezione del 25 giugno sarà l'ultima tenuta da questo docente; successivamente Davide proseguirà con la parte di level design, utilizzando gli asset creati (piattaforme, pulsanti) per costruire livelli e sperimentare meccaniche di gioco.
- Su richiesta di uno studente riguardo a inventari e UI, si chiarisce che tali argomenti non saranno trattati in profondità in questo corso.
- Esempio di progetto personale: un platformer con una pallina che cambia materiale (metallo/gomma) e la cui fisica (es. accelerazione su pendenze) è simulata via Blueprint tramite rilevamento della pendenza e modifica dinamica dei valori.

9.22 Preparazione per il Blend Space 2D

9.22.1 Animazioni necessarie

- Per un Blend Space 2D completo occorrono animazioni di camminata (o jog) in **otto direzioni**: avanti, indietro, destra, sinistra e le quattro diagonali.
- **Avanti**: si può riutilizzare l'animazione di walk forward già in uso.
- **Tutte devono essere in-place** (nessun root motion), per evitare conflitti con il movimento controllato dal Character Movement Component.
- Le animazioni possono essere scaricate da Mixamo cercando ad esempio `Jog Forward`, `Jog Backward`, `Strafe Left/Right`, `Forward Diagonal` ecc.
- Per risparmiare download, le versioni destra/sinistra possono essere specchiate (opzione `Mirror` su Mixamo o successivamente in Unreal).

9.23 Calcolo della direzione nell'Animation Blueprint

9.23.1 Estensione della Sequence

- Nell'Event Graph dell'`ABP_Knight`, aggiungere un nuovo pin alla `Sequence` (tramite il pulsante `+` → `Then 2`) per elaborare la direzione.

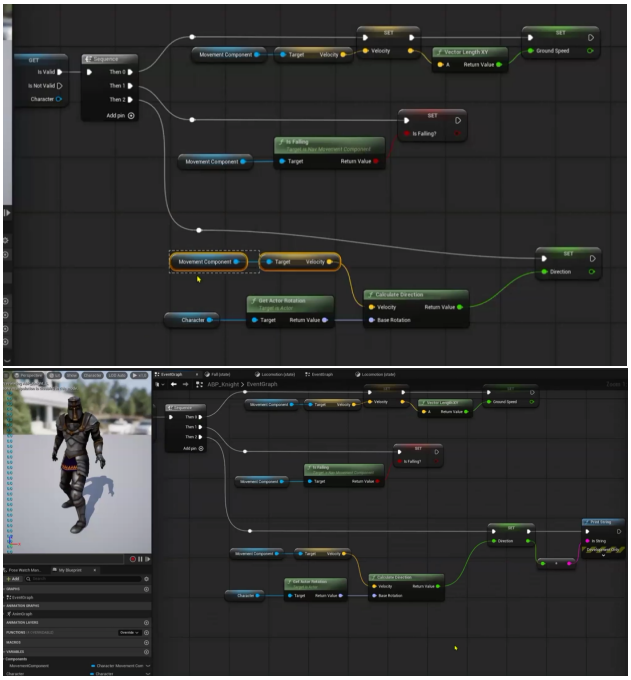
9.23.2 Nodo Calculate Direction

- Prelevare il `Movement Component` e da esso la `Velocity` attuale tramite `Get Velocity`.
- Ottenere la rotazione dell'attore con `Get Actor Rotation`.
- Collegare `Velocity` e `Actor Rotation` al nodo `Calculate Direction`.
- Il nodo restituisce un **float** compreso tra **-180** e **180**, che rappresenta l'angolo della velocità rispetto all'orientamento del personaggio:
  - 0 = avanti,
  - 90 = destra piena,
  - -90 = sinistra,
  - 180/-180 = indietro.
- Promuovere il risultato a variabile, ad esempio `Direction`.

9.24 Creazione del Blend Space 2D

9.24.1 Impostazioni iniziali

- Nel Content Browser, creare un `Blend Space` (senza specificare 1D, di default è 2D) assegnando il proprio scheletro (es. `BS_2D_Knight`).
- Configurare gli assi nei dettagli:
  - **Asse orizzontale (X)**: nome `Direction`, valori min -180, max 180.
  - **Asse verticale (Y)**: nome `GroundSpeed`, valori min 0, max 600.
- Impostare `Grid Division` a **8** per entrambi gli assi, per semplificare il posizionamento preciso delle animazioni sui nodi della griglia.
- Impostare uno `Smoothing Time` di **0.2** su entrambi gli assi per transizioni più morbide.



9.24.2 Posizionamento delle animazioni

- **Idle**: posizionare tre copie dell'animazione di idle nei punti a velocità 0 (ad es. (-180,0), (0,0), (180,0)) per coprire tutto il range di direzione da fermi.
- **Forward**: `Direction=0, GroundSpeed=600`.
- **Backward**: `Direction=-180` e `Direction=180` a `GroundSpeed=600` (entrambi rappresentano il movimento all'indietro).
- **Strafe Left**: `Direction=-90, GroundSpeed=600`.
- **Strafe Right**: `Direction=90, GroundSpeed=600`.
- **Diagonali avanti**:
  - Avanti-destra: `Direction=45, GroundSpeed=600`.
  - Avanti-sinistra: `Direction=-45, GroundSpeed=600`.
- **Diagonali indietro**:
  - Indietro-destra: `Direction=135, GroundSpeed=600`.
  - Indietro-sinistra: `Direction=-135, GroundSpeed=600`.
- Tenere premuto **Ctrl** e muovere il cursore sulla griglia per verificare in anteprima l'interpolazione tra le animazioni.

9.25 Integrazione nell'Animation Blueprint

9.25.1 Sostituzione nello stato WalkRun

- Nell'`AnimGraph`, aprire la State Machine `Locomotion` e lo stato `WalkRun`.
- Rimuovere il vecchio `Blend Space Player 1D` e inserire il nuovo `Blend Space Player` per il `BS_2D_Knight`.

- Collegare le variabili:
  - `Direction` → ingresso `Direction`.
  - `GroundSpeed` → ingresso `GroundSpeed`.
- Collegare l'uscita `Result` al nodo di output della posa.

9.25.2 Configurazione del Character

- Nel `BP_Knight`, assicurarsi che nel `Character Movement` la `Max Walk Speed` sia **600** per permettere al Blend Space di raggiungere tutte le animazioni alla velocità massima.
- Riattivare `Use Controller Rotation Yaw` nel `Character Movement` per consentire al personaggio di muoversi liberamente in tutte le direzioni senza ruotare automaticamente verso la direzione di input. Questo permette di vedere le animazioni di strafe e indietro.

9.26 Test e problemi di interpolazione

- Avviando il gioco, il personaggio esegue le animazioni corrispondenti alla direzione e velocità (destra, sinistra, diagonale, indietro).
- Si nota un leggero scatto o una scelta non ottimale dell'animazione in alcune transizioni.
- Possibile causa: la direzione è calcolata sulla base dell'orientamento del Character e non della mesh scheletrica, introducendo disallineamenti.
- Soluzione proposta (da approfondire): utilizzare la rotazione della mesh (`Get Mesh` → `Get Component Rotation`) oppure aumentare ulteriormente lo `Smoothing Time` nei parametri del Blend Space.

9.27 Riepilogo del Blend Space 2D e regolazione parametri

9.27.1 Impostazione dello Smoothing

- Si imposta lo `Smoothing Time` a `0.5` sull'asse `Direction` per transizioni più morbide.
- Il personaggio ora si muove in tutte le direzioni (avanti, indietro, laterale, diagonali) utilizzando il Blend Space 2D.

9.27.2 Spiegazione del sistema di direzione

- Il valore di `Direction` (output del nodo `Calculate Direction`) va interpretato come un cerchio:
  - `0` → avanti.
  - `90` → destra piena.
  - `-90` → sinistra piena.
  - `180` o `-180` → indietro.
  - `45` → diagonale avanti-destra.
  - `-45` → diagonale avanti-sinistra.
  - `135` → diagonale indietro-destra.
  - `-135` → diagonale indietro-sinistra.
- Tutti i punti di animazione nel Blend Space sono impostati a `GroundSpeed = 600` (velocità massima).

9.28 Animation Montages

9.28.1 Scopo e descrizione

- Un **Animation Montage** è un asset che consente di montare più animazioni in sequenza, creare sezioni, aggiungere notifiche (suoni, VFX) e gestire animazioni modulari (es. ricarica di un'arma suddivisa in drop, reload, lock).

- Permette di eseguire animazioni "una tantum" (es. un attacco) interrompendo momentaneamente il ciclo di locomozione, per poi riprendere lo stato precedente.

9.28.2 Creazione del Montage

- Nel Content Browser, tasto destro sull'animazione di attacco importata (es. `Sword and Shield Attack`) e selezionare `Create Animation Montage`.

- Si apre l'editor del montage, con una timeline e diverse tracce.

9.28.3 Sezioni del Montage

- La timeline mostra l'animazione originale (barra verde).

- È possibile creare più sezioni (`New Montage Section`) per dividere l'animazione in fasi (es. "Default", "Attack"). Ogni sezione può contenere animazioni diverse.

- Nell'esempio viene mostrato come aggiungere un'altra animazione (es. `Jump`) dopo la fine dell'attacco, creando una sequenza di due animazioni consecutive, sebbene il risultato sia esteticamente brutto per la mancanza di un'animazione di ritorno in idle.

9.28.4 Notify

- I **Notify** sono marcatori posizionabili sulla timeline che scatenano eventi in momenti precisi dell'animazione.

- Possono essere usati per suoni, effetti visivi (VFX), o qualsiasi altra logica.

- Per aggiungere un Notify:

  - Creare una traccia di notifica (`Add Notify Track`).
  - Sulla traccia, tasto destro → `Add Notify` → scegliere il tipo (es. `Play Sound`).
  - Assegnare la risorsa sonora.

- Durante l'esecuzione, quando l'animazione raggiunge quel punto, il suono viene riprodotto.

9.28.5 Slot di animazione

- Ogni montage appartiene a uno **Slot** (di default `DefaultSlot`).

- Lo slot predefinito interessa tutto il corpo. Volendo, si possono creare slot personalizzati che coinvolgono solo alcune ossa (es. parte superiore) per animazioni parziali (es. ricaricare mentre si cammina).

- Nel caso dell'attacco, si utilizza `DefaultSlot`.

9.29 Integrazione del Montage nell'Animation Blueprint

9.29.1 Cache della posa di locomozione

- Nell'`AnimGraph` dell'ABP del personaggio, dopo la State Machine `Locomotion`, si inserisce un nodo `Save Cache Pose`.
- Lo si rinomina (es. `Locomotion Cache`) e lo si collega all'output della State Machine.
- Questo nodo salva in memoria la posa calcolata dalla locomozione, permettendo di riutilizzarla in un altro ramo.

9.29.2 Ramo del montage

- Si aggiunge un nodo `Slot` (di default `Default Slot`) e lo si collega all'output finale.
- Il nodo `Use Cached Pose` `Locomotion Cache` va collegato all'ingresso dello `Slot`.
- In questo modo, quando viene riprodotto un montage, esso si sovrappone (o sostituisce temporaneamente) la posa della locomozione.

9.30 Attivazione del Montage da Blueprint

9.30.1 Input e chiamata a Play Montage

- Nell'Event Graph del personaggio (`BP_Knight`), si crea un evento per un tasto (es. `F`).
- Si utilizza il nodo `Play Montage`:
  - `Mesh Component`: la mesh scheletrica del personaggio.
  - `Montage to Play`: l'Asset Montage creato.
- Il nodo espone pin di esecuzione per:
  - `On Completed`: quando l'animazione è terminata.
  - `On Blend Out`: quando l'animazione sta sfumando.
  - `On Interrupted`: se l'animazione viene interrotta.
  - `On Notify Begin/End`: per ogni Notify presente nel montage.

9.30.2 Gestione del movimento e input durante l'attacco

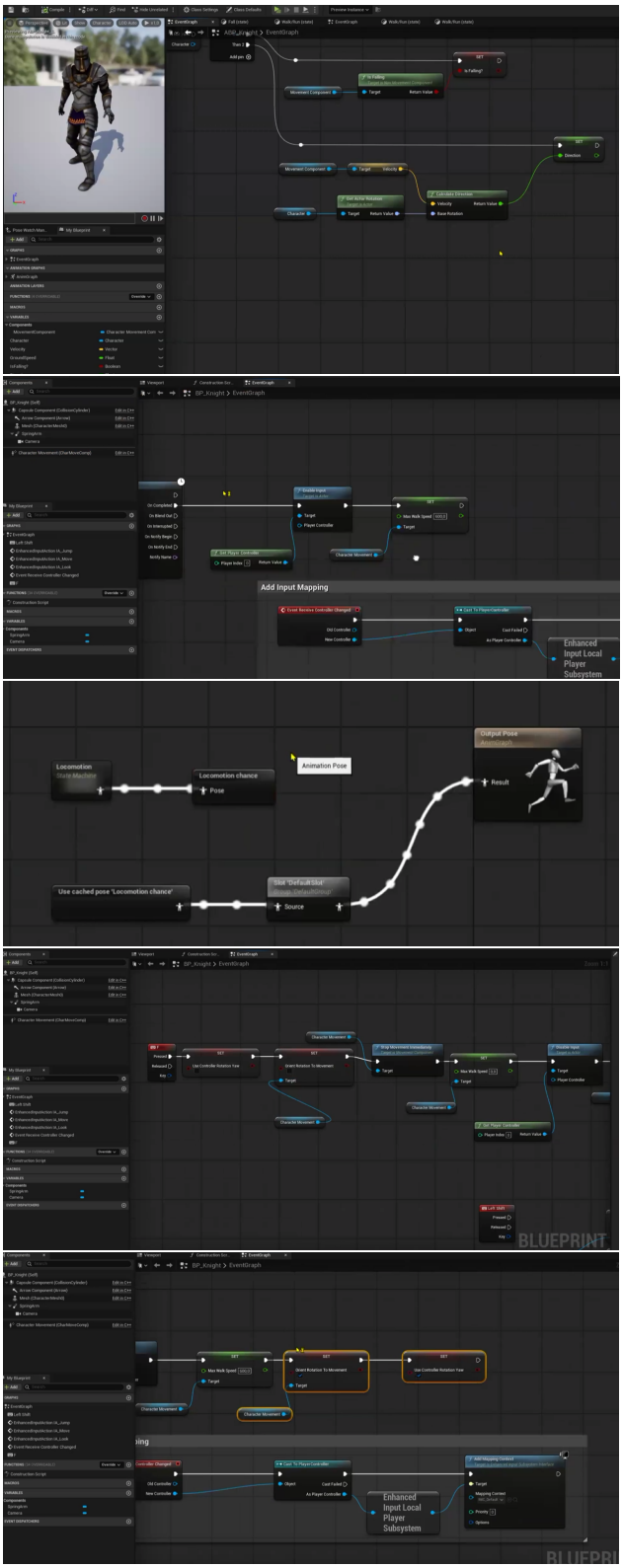
- **All'inizio del montage** (subito dopo `Play Montage`):
  - Impostare `Max Walk Speed` a `0` per fermare il movimento.
  - Chiamare `Get Player Controller` → `Disable Input` per disabilitare tutti gli input.
- **Al completamento** (pin `On Completed`):
  - Ripristinare `Max Walk Speed` a `600` (o il valore di default).
  - Chiamare `Enable Input` sul Player Controller.

9.30.3 Blocco della rotazione e della camera

- Durante l'attacco, per evitare che il personaggio ruoti o che la camera si muova in modo indesiderato:
  - Disattivare `Orient Rotation to Movement` sul `Character Movement Component` (`Set Orient Rotation to Movement = False`).
  - Disattivare `Use Controller Yaw` sulla Pawn (`Set Use Controller Yaw = False`).
- Al termine dell'animazione (`On Completed`), riattivare entrambi (`True`).
- Questo assicura che il personaggio resti immobile e non ruoti verso la direzione di input mentre esegue l'attacco.

9.31 Task per la prossima lezione

- Creare un nuovo personaggio partendo dal blueprint **Stack-o-Bot** già presente nel progetto (cartella `Blueprints/Character/BotCharacter`).
- Realizzare un **Animation Blueprint** per tale personaggio, collegando tutte le sue animazioni e configurando un Blend Space (1D o 2D) e la relativa State Machine.
- L'obiettivo è avere un ciclo di animazioni completo per Stack-o-Bot, che verrà utilizzato nella prossima lezione.


Add Input Mapping
Enhanced Input Local Player Subsystem
Locomotion
Locomotion chance
Pose
Animation Pose
Output Pose
Result
Use cached pose 'Locomotion chance'
Slot 'DefaultSlot'
Source
BLUEPRINT
Enhanced Input Local Player Subsystem

Chapter 10

Sviluppo di un Bot in Unreal Engine: Animazioni e Physical Materials

10.1 Risoluzione dei problemi riscontrati nel progetto

10.1.1 Problemi con l'Input Action e l'Input Mapping Context

- Manca l'**Enhanced Input Action** `IA_Move`: è stata accidentalmente cancellata o sostituita con `IA_Run`.
- Verificare nel *Content Browser* la presenza di `IA_Run` e controllare l'*Input Mapping Context* (cercare `IMC`).
- Nell'IMC sono state inserite molte azioni errate; è sufficiente tenere solo quelle necessarie.
- `IA_Move` deve essere ricreata: duplicare `IA_Look`, rinominarla in `IA_Move` e assegnarla nell'IMC.
- Nell'IMC, per `IA_Run` aggiungere il modificatore di input **Shift** (tramite il pulsante `+` nell'editor, poi selezionare il tasto `Shift`).
- Sostituire nel grafo degli eventi i riferimenti errati: al nodo `Action Value` collegare `IA_Move` e dividere il pin per ottenere `X` e `Y`.
- Ricreare il nodo `Add Movement Input` (se cancellato) e collegarlo:
  - `Scale Value` al risultato di `IA_Move` opportunamente scomposto.
  - `World Direction` al nodo `Get Actor Forward Vector` (arancione).
  - Flusso di esecuzione: dal `Triggered` di `IA_Move` collegare in sequenza tutti i nodi di movimento.
- Compilare per verificare la risoluzione degli errori.

10.1.2 Correzione della telecamera (Spring Arm)

- La telecamera appare spostata perché lo **Spring Arm** è stato mosso dalla posizione originale.
- Non spostare mai direttamente Spring Arm e Camera dal pivot; si rischia di rompere il funzionamento.
- Per correggere: selezionare lo Spring Arm, azzerare tutti i valori di `Location` (X, Y, Z = 0).
- Per alzare la telecamera usare il parametro `Target Offset` → impostare `Z = 60`.
- Abilitare `Control Rotation` sullo Spring Arm: `Use Pawn Control Rotation = True`.
- Compilare e verificare che la visuale sia corretta.

10.1.3 Problemi con le animazioni e l'interazione

- Il personaggio corre ma manca l'animazione di corsa: verrà affrontato durante la creazione del nuovo personaggio.
- L'interazione con alcuni oggetti (ascensore, torretta) non funziona più a causa del cambio del personaggio:
  - La torretta usa un **tag** per riconoscere il giocatore. Sul nuovo personaggio occorre aggiungere il tag `Player` (con `Add Tag`).
  - Vanno riconfigurate le collisioni e la risposta ai *trace* (es. sfera per l'interazione).
- Durante l'esecuzione di un *Animation Montage* (es. interagire con un bottone) il personaggio si ferma ma l'animazione non parte: verificare nell'*Animation Blueprint* (ABP) che sia impostato correttamente lo **Slot** di default per il mixaggio delle animazioni. Aggiungere o abilitare il nodo `Default Slot` nell'Anim Graph.
- Se il personaggio ruota su sé stesso durante l'interazione, usare `Use Control Rotation` per bloccare la rotazione indesiderata.

10.2 Lezione del giorno: Suoni dei passi con Physic Material

10.2.1 Introduzione ai Physic Material

- Unreal Engine permette di assegnare materiali fisici (**Physic Material**) alle superfici.
- Creando diversi Physic Material (es. `PM_Rock`, `PM_Water`) si possono differenziare le reazioni in base al tipo di superficie toccata, ad esempio:
  - Suono dei passi su roccia.
  - Suono dei passi in acqua.
  - Suono dei passi su superficie generica.
- Per crearne uno: *Content Browser* → tasto destro → *Physics* → *Physical Material*.
- Il sistema funziona tramite un *trace* sotto i piedi del personaggio che rileva il materiale ad ogni passo.

10.2.2 Creazione del nuovo personaggio (Bot2)

Obiettivo: ricreare da zero un personaggio (Bot2) che utilizzeremo come default, integrando poi i suoni dei passi e sistemando le funzionalità.

1. Creare una nuova cartella per il Bot2.
2. **Animation Blueprint**:
   - Creare un nuovo ABP (es. `ABP_Bot2`) associato alla *Skeletal Mesh* `SK_Bot`.
   - Per l'Event Graph è possibile copiare la logica di calcolo già fatta per il Knight (velocità, caduta, direzione).
   - Andare nell'ABP del Knight, selezionare tutti i nodi nell'Event Graph, copiare (`Ctrl+C`) e incollare (`Ctrl+V`) nell'Event Graph del nuovo ABP.
   - L'Anim Graph verrà ricostruito in seguito secondo le nuove esigenze.
3. Successivamente si procederà a:
   - Aggiungere i tag necessari (es. `Player`).
   - Configurare collisioni e trace per interazioni e ascensori.
   - Implementare i suoni dei passi variabili usando i Physic Material.

10.2.3 Materiale didattico

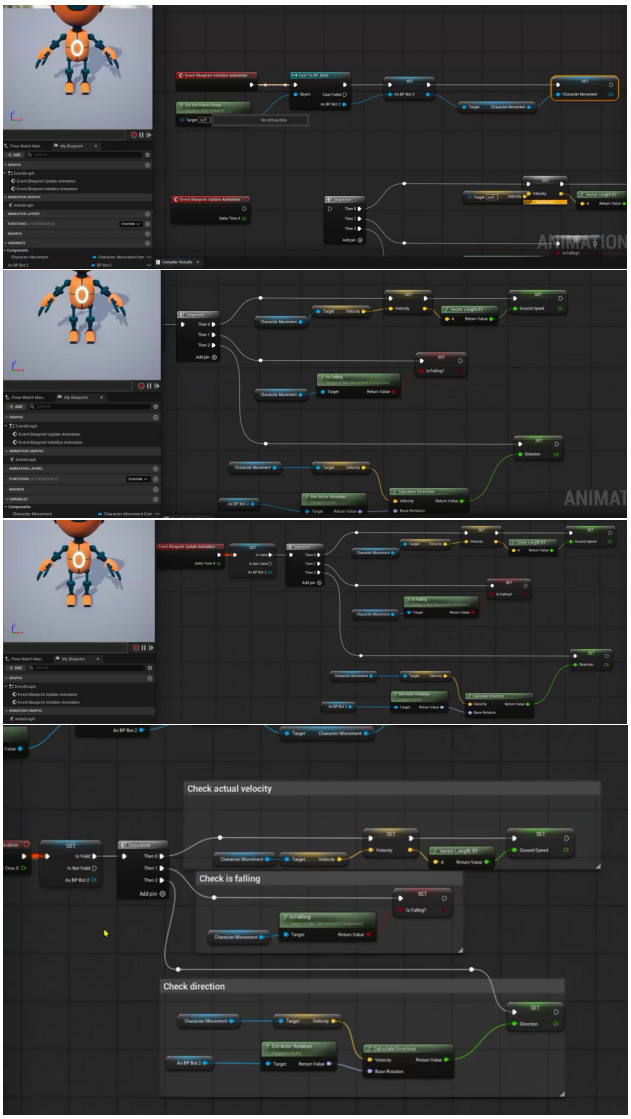
- Nella cartella del Drive è disponibile una sotto-cartella chiamata `Footstep Sound` contenente i file audio necessari per i passi (acqua, roccia, default).
- Scaricare e decomprimere il contenuto per la lezione.

10.2.4 Creazione del nuovo Character Blueprint (Bot2)

- Creare un nuovo **Character Blueprint**: `BP_Bot2` (o semplicemente `Bot`).
- Assegnare la **Skeletal Mesh** del bot (`SK_Bot`) e ruotarla di 90° verso il basso, se necessario.
- Collegare l'**Animation Blueprint** appena creato (`ABP_Bot2`) alla Mesh: selezionare la Mesh, impostare `Animation Class` → `ABP_Bot2`.
- Nell'Animation Blueprint, recuperare le referenze necessarie tramite **Event Initialize Animation**:
  1. Eseguire un **Cast** dal `Get Owning Actor` al `BP_Bot2`.
  2. Promuovere a variabile il risultato del cast (es. `As BP Bot 2`).
  3. Prendere il `Character Movement` del personaggio e promuoverlo a variabile.
  4. Convertire il get della variabile `As BP Bot 2` in **Validated Get** (tasto destro → `Convert to Validated Get`) e collegarlo all'evento `Blueprint Update Animation`.
  5. All'interno del ramo validato, ricollegare le variabili necessarie (Velocity, Character Movement, ecc.).
  6. Se le variabili non esistono, crearle con `Create Variable` nominandole opportunamente (es. `Velocity`).
- Per mantenere ordine nel codice, creare **commenti** (premere `C` e cliccare) attorno ai blocchi logici (es. `Check Actual Velocity`, `Check Is Falling`, `Check Direction`).

10.2.5 Blend Space e animazioni Lean

- Creare un nuovo **Blend Space 2D** chiamato `BS_Bot2` (tasto destro → Animation → Blend Space, senza suffisso 1D).
- Configurare gli assi:
  - Asse orizzontale: `Lean`, con range $[-10, 10]$ (inclinazione sinistra/destra).
  - Asse verticale: `Ground Speed`, con range $[0, 500]$ (velocità di camminata).
- Regolare la **velocità massima di camminata** nel Character Movement: `Max Walk Speed = 500`.
- Posizionare le animazioni nel Blend Space:
  - A sinistra (-10) sul valore massimo di Ground Speed (500): `Bot_Run_Lean_Left`.
  - A destra ($+10$) a 500: `Bot_Run_Lean_Right`.
  - Al centro a 250: `Bot_Walk`.
  - Formare un triangolo che copra l'area di movimento.
- **Nota**: l'animazione di Idle non viene inserita nel Blend Space; sarà gestita nell'Animation Blueprint per differenziare lo stato di fermo (Idle) da quello in movimento dopo un salto.

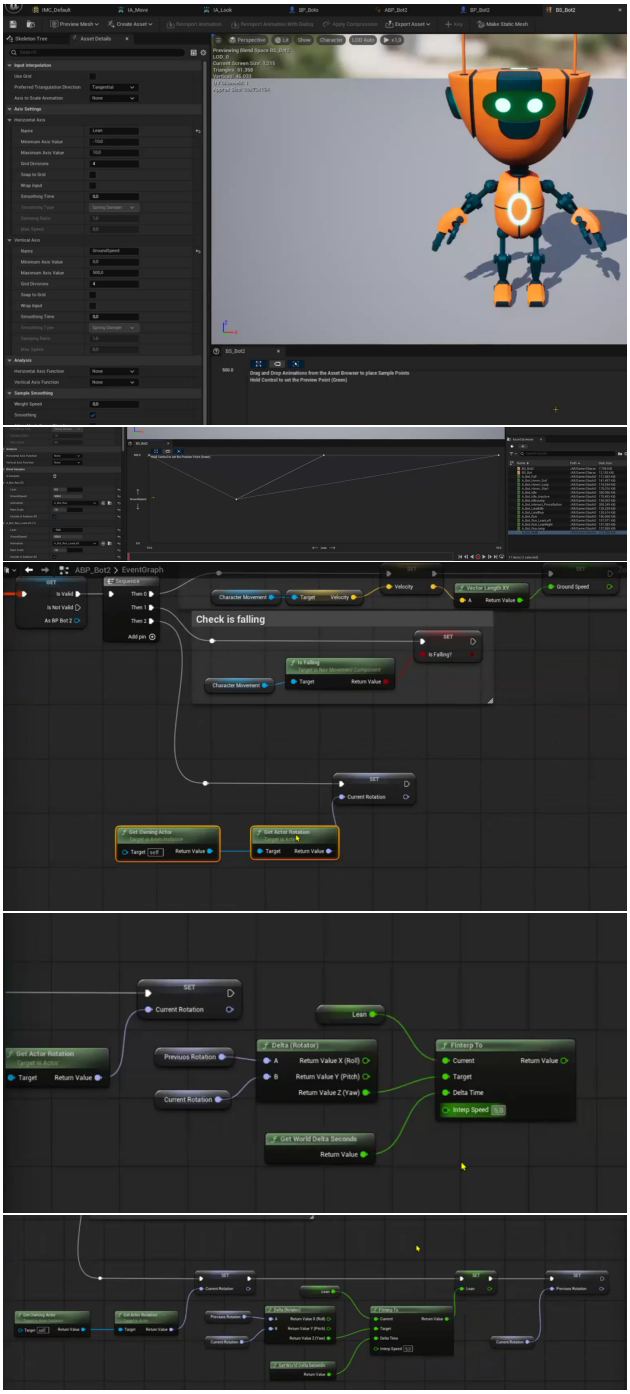

Check actual velocity
Check is falling
Check direction

10.2.6 Calcolo della Lean nell'Animation Blueprint

- Eliminare il blocco `Check Direction` usato in precedenza.
- Ottenere la rotazione corrente del personaggio: `Get Actor Rotation` dal `Get Owning Actor`.
- Promuoverla a variabile `Current Rotation` (tipo `Rotator`).
- Creare una variabile `Previous Rotation` (tipo `Rotator`) per memorizzare la rotazione del frame precedente.
- Calcolare la differenza tra le due rotazioni con **Delta (Rotator)**:
  - Ingresso A: `Previous Rotation`.
  - Ingresso B: `Current Rotation`.
- Dal risultato, estrarre solo la componente **Yaw** (asse Z, rotazione orizzontale) con `Split Rotator` → `Yaw`.
- Interpolare la Yaw attuale verso 0 usando **FInterp To**:
  - `Current`: Yaw attuale.
  - `Target`: 0.
  - `Delta Time`: `World Delta Seconds` (da `Event Blueprint Update Animation`).
  - `Interp Speed`: 5 (valore regolabile).
- Promuovere l'output di FInterp To a una variabile `Lean` (tipo `float`).
- Aggiornare `Previous Rotation` con il valore di `Current Rotation` alla fine di ogni frame:
  - Assegnare `Current Rotation` a `Previous Rotation`.
- **Spiegazione**: ad ogni frame, la differenza di Yaw viene gradualmente riportata a zero, producendo un valore di inclinazione ($Lean$) compreso tra -10 e $+10$ circa, che nel Blend Space controlla l'animazione di camminata inclinata. Inizialmente $Lean = 0$, quindi nessuna inclinazione.
- Collegare infine la variabile `Lean` all'ingresso `Lean` del nodo Blend Space.

10.2.7 Note e chiarimenti

- **Perché non si copia tutto dall'ABP precedente?** Il nuovo character BP richiede referenze aggiornate. Copiando solo la logica di calcolo (event graph) si risparmia tempo, ma le variabili vanno ricollegate.
- **Modificare la velocità massima**: se si cambia `Max Walk Speed` (es. a 300), occorre aggiornare anche il `Maximum Axis Value` del Blend Space (a 300) e riposizionare le animazioni di conseguenza, altrimenti la corsa parte in momenti errati. Non esiste un collegamento automatico diretto tra le due proprietà.
- **Validated Get**: equivale a un get con controllo `Is Valid` implicito, snellendo il grafo.


ABP_Bot2 > EventGraph
Check is falling
Is Falling
Character Movement
Velocity
Vector Length XY
Ground Speed
Current Rotation
Get Actor Rotation
Previous Rotation
Delta (Rotator)
Return Value X (Roll)
Return Value Y (Pitch)
Return Value Z (Yaw)
Lean
FInterp To
Current
Target
Delta Time
Interp Speed
Get World Delta Seconds
Return Value

10.2.8 State Machine della Locomozione (Anim Graph)

Stati e animazioni

- Creare una **State Machine** denominata `Locomotion` (e per precauzione anche uno slot `Default Slot`) nell'Anim Graph.
- All'interno della State Machine, definire i seguenti stati:
  1. **Idle** – animazione di attesa in loop (`Bot_Idle`).
  2. **Walk** – Blend Space `BS_Bot2` (controllato da `Lean` e `Ground Speed`).
  3. **Jump Idle** – animazione di salto da fermo (`Bot_Jump_Idle`).
  4. **Jump Run** – animazione di salto in movimento (`Bot_Jump_Run`).
  5. **Fall** – animazione di caduta in loop (`Bot_Fall`).
  6. **Land Idle** – animazione di atterraggio da fermo (`Bot_Land_Idle`).
  7. **Land Run** – animazione di atterraggio in movimento (`Bot_Land_Run`).
- Disposizione consigliata degli stati: in alto Idle e Walk, al centro Jump Idle e Jump Run, sotto Fall e infine Land Idle e Land Run, con transizioni che seguono il flusso logico del movimento.

Condizioni di transizione

- **Idle ↔ Walk**
  - `Idle` → `Walk`: `Ground Speed > 0`.
  - `Walk` → `Idle`: `Ground Speed` $\leq$ `0`.
- **Da Idle/Walk a Salto**
  - `Idle` → `Jump Idle`: `IsFalling == true` **AND** `Ground Speed` $\leq$ `0`.
  - `Walk` → `Jump Run`: `IsFalling == true` **AND** `Ground Speed > 0`.
- **Da salto a caduta**
  - `Jump Idle` → `Fall`: transizione automatica (nessuna condizione).
  - `Jump Run` → `Fall`: transizione automatica.
- **Da caduta ad atterraggio**
  - `Fall` → `Land Idle`: `IsFalling == false` **AND** `Receive Input == false`.
  - `Fall` → `Land Run`: `IsFalling == false` **AND** `Receive Input == true`.
- **Ritorno allo stato di movimento**
  - `Land Idle` → `Idle`: transizione automatica.
  - `Land Run` → `Walk`: transizione automatica.

Rilevamento degli input (`Receive Input`)

Per decidere se il personaggio sta ricevendo comandi di movimento durante la caduta (e quindi passare a `Land Run` o `Land Idle`), si utilizza l'input vector:

1. Ottenere il `Character Movement` del personaggio (già referenziato nell'Animation Blueprint).
2. Prendere il nodo `Get Last Input Vector` dal componente `CharacterMovement`.
3. Calcolare la lunghezza del vettore con `Vector Length`.

4. Verificare se la lunghezza è maggiore di 0 (`> 0`).
5. Promuovere il risultato booleano a variabile, chiamandola `Receive Input`.

`Receive Input` sarà vero se l'utente sta premendo un qualsiasi tasto di movimento (WASD o analogico), falso altrimenti.

Note e accorgimenti

- La variabile `IsFalling` è ottenuta dal `Character Movement` e indica se il personaggio è in volo/ricaduta (non a terra).
- Per testare il valore di `IsFalling` si può temporaneamente inserire un `Print String` nel Blueprint del personaggio.
- È necessario aver configurato una telecamera (Spring Arm + Camera) e gli input nel nuovo Character Blueprint affinché il movimento e le animazioni funzionino correttamente durante il test.
- In una precedente lezione si era usata la `Velocity Z` al posto di `IsFalling` per evitare che una semplice caduta da un dislivello attivasse l'animazione di salto. Qualora si riscontrasse tale problema, è possibile sostituire la condizione `IsFalling` con `Velocity Z > 0` (per il salto) e gestire la caduta con `Velocity Z < 0`.

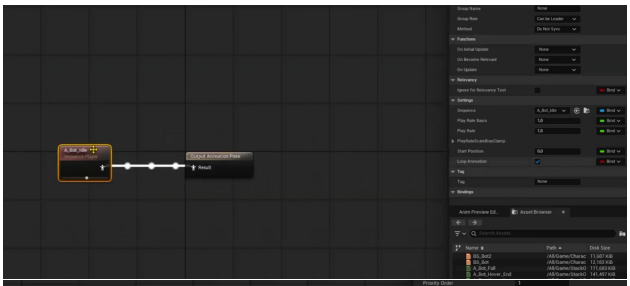
10.3 Testing e correzione del personaggio

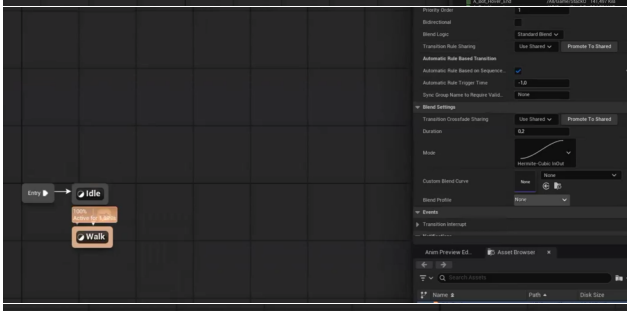
10.3.1 Preparazione del Character Blueprint per il test

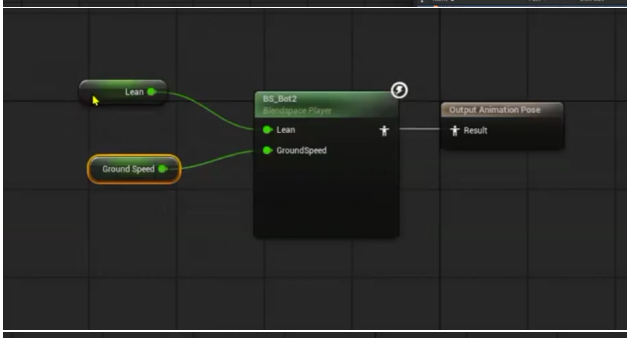
- Nel `BP_Bot2`, aggiungere i componenti **Spring Arm** e **Camera**.
- Azzerare la posizione di Spring Arm e Camera (`Location = (0,0,0)`).
- Impostare un `Target Offset` sullo Spring Arm per alzare leggermente la telecamera (es. `Z = 60`) e, volendo, ruotare di qualche grado la Camera per una visuale migliore (es. `Y = 5`).
- Abilitare `Use Pawn Control Rotation` sullo Spring Arm.
- Copiare dal vecchio Character (`BP_Night`) le componenti di input e di movimento (es. `Control Rotation`, `Orient Rotation to Movement`) e incollarle nel nuovo `BP_Bot2`.
- Impostare il **Game Mode** di default: `World Settings` → `Default Pawn Class` → `BP_Bot2`.

10.3.2 Correzione degli errori comuni

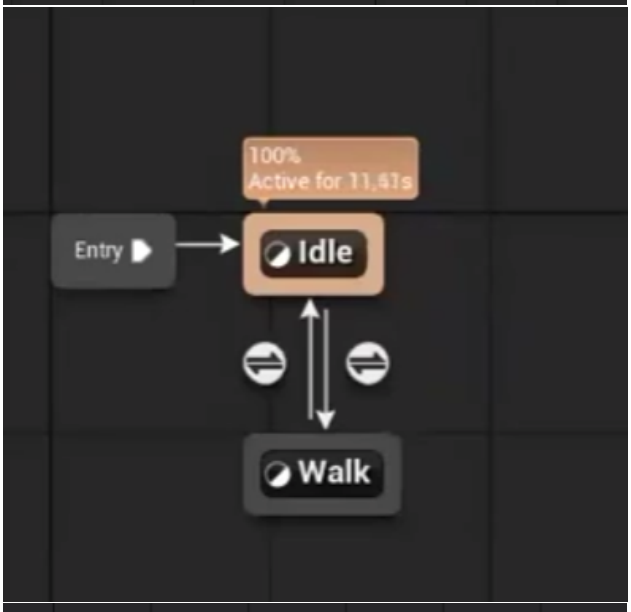
- **Personaggio in T-Pose**: verificare che nell'Animation Blueprint l'output della state machine `Locomotion` sia effettivamente collegato al `Result` dell'Anim Graph.
- **Mancata esecuzione delle animazioni**: assicurarsi che il nodo `Event Blueprint Update Animation` sia collegato al `Validated Get` della variabile del personaggio (`As BP Bot 2`).
- **Riferimenti alle variabili errati**:
  - Usare il `Character Movement` promosso a variabile (es. `CharacterMovement0`) per tutti i calcoli (Velocity, Is Falling).
  - Eliminare variabili duplicate (es. vecchie `Velocity`, `NewVar`, `Lean 0`, `Previous Rotation` non collegate).
  - Controllare che la `Ground Speed` sia calcolata come `Vector Length` della `Velocity` (solo XY, non Z).

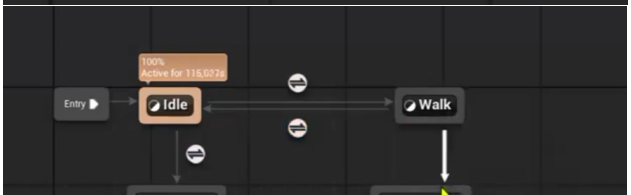


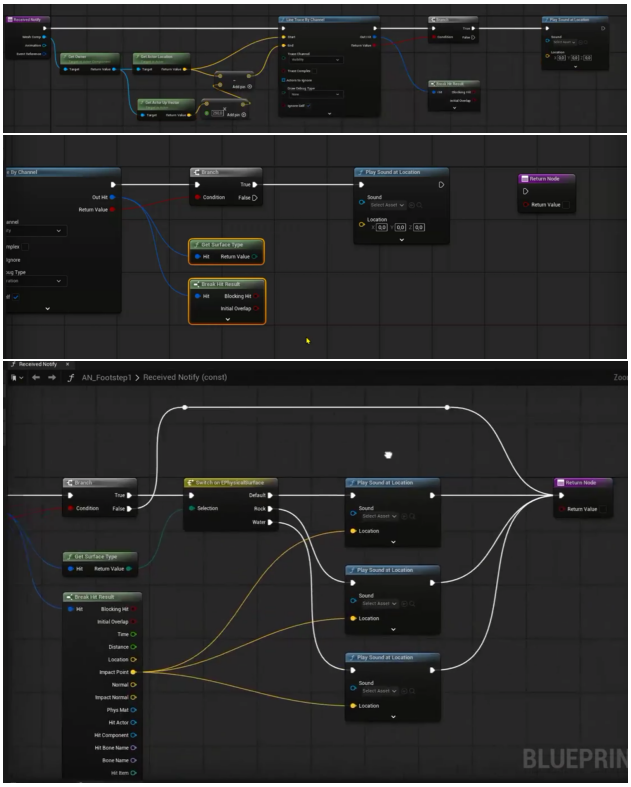

Entry
Idle
Walk


Lean
Ground Speed
BS_Bot2
BlendSpace Player
Lean
GroundSpeed
Output Animation Pose
Result

Ground Speed
Result
Can Enter Transition
3.0
No entries match the search text


100%
Active for 11,61s
Entry
Idle
Walk


100%
Entry
Idle
Walk



- **Transizioni mancanti**: tutte le transizioni automatiche (es. `Jump` → `Fall`, `Land` → `Idle/Walk`) devono essere impostate su `Automatic Rule Based` (selezionare la freccia di transizione e spuntare l'opzione nei dettagli).
- **Inclinazione (Lean)**: regolare `Interp Speed` a 1 per un movimento più fluido; l'inclinazione dipende dalla differenza di `Yaw` tra `Current Rotation` e `Previous Rotation`. Se il personaggio non si inclina, verificare la logica di calcolo e che il valore `Lean` arrivi effettivamente al Blend Space.
- **Orientamento durante il movimento**: se il personaggio ruota in modo innaturale, provare a disattivare temporaneamente `Use Controller Rotation Yaw` sul `BP_Bot` e/o regolare `Orient Rotation to Movement`.
- **Personaggio sospeso o fluttuante**: regolare la posizione verticale della Mesh (es. `Location Z = -90`) finché i piedi toccano il suolo.

10.4 Suoni dei passi con Anim Notify e Physic Material

10.4.1 Creazione dell'Anim Notify

1. Creare una cartella `Notify` nel *Content Browser*.
2. Creare una **Blueprint Class** di tipo `Anim Notify` (cercare `Anim Notify` in *All Classes*).
3. Aprire il nuovo Anim Notify e sovrascrivere la funzione `Received Notify`: *Functions* → *Override* → `Received Notify`.

10.4.2 Logica del Line Trace per il rilevamento del terreno

All'interno del `Received Notify`:

- Ottenere il proprietario dell'animazione: `Get Owner`.
- Ottenere la posizione dell'attore: `Get Actor Location` (punto di partenza del trace).
- Calcolare il punto di arrivo del trace verso il basso:
  - Prendere il vettore `su` globale: `Get Up Vector`.
  - Moltiplicarlo per una distanza (es. `250`) tramite `Multiply (Float)` (tasto destro sul pin inferiore del nodo Multiply e scegliere `To Float`).
  - Sottrarre il risultato alla posizione dell'attore: `Start` = `Actor Location`, `End` = `Actor Location` - (`Up Vector` × `250`).
- Eseguire un **Line Trace by Channel**:
  - Collegare `Start` e `End`.
  - Abilitare il *Debug Draw* per visualizzarlo (opzione `Draw Debug Type = Persistent`, `Duration = 2` secondi).
- Diramare l'esecuzione in base al risultato (`Return Value` booleano):
  - Se ha colpito qualcosa, ottenere il `Hit Result` tramite `Break Hit Result`.
  - Determinare il tipo di superficie colpita: `Get Surface Type` (dal `Hit Result` → `Phys Material`).
  - In base al tipo di superficie (es. `PM_Rock`, `PM_Water`, `Default`), eseguire `Play Sound at Location` con il suono corrispondente.

10.4.3 Note

- I suoni da utilizzare (acqua, roccia, default) sono forniti nella cartella `Footstep Sound` del materiale didattico.
- Il collegamento per moltiplicare un vettore per uno scalare richiede di convertire il float nel nodo `Multiply`: cliccare sul pin inferiore, selezionare `To Float` per ottenere il pallino verde.

10.4.4 Inserimento dell'Anim Notify nelle animazioni

- Aprire le animazioni di camminata dal Blend Space: `Bot_Run`, `Bot_Run_Lean_Left`, `Bot_Run_Lean_Right`.
- In ciascuna animazione, individuare i fotogrammi in cui i piedi toccano terra:
  - Piede destro: frame **3**.
  - Piede sinistro: frame **13**.
- Aggiungere i **Sync Marker** per sincronizzare i passi:
  - Tasto destro sulla timeline delle notifiche → `Add Sync Marker`.
  - Chiamare `R` il marker al frame 3 (piede destro) e `L` quello al frame 13 (piede sinistro).
- Creare una traccia per i suoni: tasto `Track` → `Insert Notify Track`, nominarla `Sound`.
- Posizionare l'Anim Notify `AnimFootstep` sulla traccia `Sound` esattamente in corrispondenza di entrambi i marker `R` ed `L` (tasto destro → `Add Notify` → `AnimFootstep` o copia-incolla del nodo).
- Ripetere l'operazione per tutte e tre le animazioni (`Run`, `Lean Left`, `Lean Right`).

10.4.5 Creazione e assegnazione dei Physic Material

1. Creare una cartella `PhysicMaterial` nel *Content Browser*.
2. Creare i materiali fisici: tasto destro → `Physics` → `Physical Material`.
   - `PM_Rock`: aprire il materiale, impostare `Surface Type` → `Rock`.
   - `PM_Water`: aprire il materiale, impostare `Surface Type` → `Water`.
3. Assegnare i Physic Material alle superfici:
   - Selezionare una *Static Mesh* (es. un cubo piatto usato come piattaforma).
   - Nel pannello *Details*, scorrere fino a `Physic Material Override`.
   - Impostare `PM_Rock` per la superficie rocciosa e `PM_Water` per quella acquatica.

10.4.6 Debug e verifica

- Per visualizzare il Line Trace durante il test: nel nodo `Line Trace By Channel`, impostare `Draw Debug Type` → `For Duration`.
- Avviare il gioco e camminare sulle diverse superfici per verificare che i suoni corrispondano (default, roccia, acqua).
- Se il suono non viene riprodotto, controllare:
  - La corretta creazione e assegnazione dell'Anim Notify `AnimFootstep`.
  - La presenza e la configurazione dei *Sound Cue* nel grafo del Notify.
  - L'effettiva applicazione del *Physic Material Override* sulle mesh.

10.4.7 Problema residuo e prossimi argomenti

- L'inclinazione del personaggio (`Lean`) presenta ancora un comportamento anomalo (es. ruota su sé stesso). Verrà analizzato e risolto nella prossima lezione.
- La prossima lezione tratterà la creazione dell'interfaccia utente (**UI** e **Menu**).

Chapter 11

Sviluppo dell'HUD in Unreal Engine: Vita, Stamina e Logiche di Gioco

11.1 Risoluzione dei problemi del personaggio

11.1.1 Input non funzionante (personaggio fermo)

- **Causa**: mancava il caricamento del *mapping context* degli input.
- **Soluzione**: copiare dal vecchio character (`BP_Night`) il nodo che carica gli `Input Mapping Context` (tipicamente `Add Mapping Context` con l'asset `IMC_Default`) e inserirlo nel `BeginPlay` del nuovo `BP_Bot`.
- Il mapping context è indispensabile per *dare la mappa* ai comandi; senza di esso il personaggio non riceve input.

11.1.2 Problemi con l'Animation Blueprint

- Verificare che nel `BP_Bot` l'`Animation Class` sia impostata su `ABP_Bot` (selezionare la Mesh → `Animation Class`).
- **Interpolation Speed** del nodo `FInterp To` per la `Lean`: ridurre a `1` per un movimento più fluido.
- **Debug della Ground Speed**: inserire un `Print String` dopo il calcolo della `Ground Speed` per verificarne il valore a runtime.
  - Se non stampa nulla, il nodo non viene eseguito → il personaggio non è quello corretto (verificare `World Settings` → `Default Pawn Class`).
  - Con `Ground Speed` forzata a 300 tramite `Default Value` nel Blend Space, l'animazione di walk parte, quindi i dati non fluiscono correttamente.
- **Breakpoint** (tasto `F9`) sul nodo `Ground Speed` per ispezionare l'esecuzione; se non si ferma, l'Animation Blueprint non è collegata al personaggio.
- Possibile errore nell'`Initialize Animation`: rifare il cast al `BP_Bot` e il `Validated Get` per garantire che le variabili del personaggio vengano lette.
- Il problema sarà analizzato più approfonditamente in pausa perché non bloccante per la lezione del giorno.

11.1.3 Correzione dell'interazione con la torretta

1. **Aggiungere il tag** `Player`:
   - Aprire il `BP_Bot`, dettagli → `Actor` → `Advanced` → `Tags`.
   - Aggiungere un nuovo tag con il nome esattamente uguale a quello usato nella torretta per identificare il giocatore (es. *`Player`* con la P maiuscola).
2. **Migliorare il target della torretta**:
   - Nel blueprint della torretta, sostituire il nodo che prende il target con `Get Player Pawn` (ottenendo direttamente il personaggio del giocatore) invece di un riferimento generico.
3. **Abilitare la collisione corretta sulla Mesh**:
   - Nel `BP_Bot`, selezionare la `Skeletal Mesh`.
   - Nella sezione `Collision`, impostare `Collision Preset` su `Custom` e spuntare i canali `DetectionTurret` e `ProjectileTrace` su `Block` (gli stessi usati dalla torretta per il rilevamento).
4. **Nota**: il personaggio (Bot) è più basso del precedente Knight; se necessario, abbassare leggermente il *pivot* della torretta per allineare la linea di tiro.

11.2 Introduzione all'User Interface (UI)

11.2.1 Definizione e componenti

- **UI** (User Interface): tutto ciò che sullo schermo fornisce *informazioni* al giocatore tramite elementi grafici.
- Esempi: barra della vita, barra della stamina/mana, quantità di monete, indicatori di danno (schermo rosso), menu, pulsanti.
- L'HUD (Head-Up Display) è la parte di UI visibile durante il gameplay.
- Ruolo del **UI/UX Designer**: progettare come trasmettere informazioni in modo efficace e intuitivo.

11.2.2 Obiettivi della lezione

- Creare due **Progress Bar**:
  - Barra della **vita** (rossa).
  - Barra della **stamina** (verde).
- La barra della vita diminuisce quando il personaggio viene colpito dalla torretta; se arriva a zero, il livello viene ricaricato (morte).
- La barra della stamina si consuma tenendo premuto `Shift` (corsa) e si ricarica gradualmente nel tempo quando non si corre.
- La comunicazione tra i valori numerici (vita, stamina) del `BP_Bot` e i widget grafici avverrà tramite **Event Dispatcher**.

11.2.3 Utilizzo degli Event Dispatcher

- **Event Dispatcher**: meccanismo per *chiamare* e *reagire* a eventi tra diversi Blueprint.
- Flusso previsto:
  1. Nel `BP_Bot` si gestiranno le variabili `Health` e `Stamina`, con relative logiche (danno, consumo, ricarica).
  2. Ogni volta che un valore cambia, il `BP_Bot` lancerà un Event Dispatcher (es. `OnHealthChanged`, `OnStaminaChanged`).
  3. Il Widget della UI (creato a parte) si *binderà* (subscribe) a questi dispatcher per aggiornare graficamente le barre.
- Questo approccio mantiene separata la logica del personaggio dalla rappresentazione visiva.

11.2.4 Prossimi passi

- Creare il Widget per la UI (Health Bar e Stamina Bar).
- Implementare nel `BP_Bot` le variabili e le funzioni per la vita (danno subito, morte) e per la stamina (consumo in corsa, ricarica).
- Collegare tutto tramite Event Dispatcher.

11.3 Implementazione del danno e della vita del personaggio

11.3.1 Configurazione del proiettile per infliggere danno

- Aprire il blueprint del proiettile (es. `Projectile1`).
- Sostituire il nodo `Multi Sphere Trace By Channel` con un singolo `Sphere Trace By Channel` (per evitare array).
- Dal pin `Out Hit` del trace, usare `Break Hit Result` per ottenere i dati di collisione.
- Utilizzare il nodo `Apply Damage`:
  - `Damaged Actor` ← `Hit Actor` (dal Break).
  - `Base Damage` ← valore del danno (es. `1`).
- Collegare `Get Actor Location` del proiettile a `Start` del trace; raggio (`Radius`) `20`; canale di collisione `ProjectileTrace` (impostato su `Block`).
- Verifica del passaggio del danno: nel personaggio, l'evento `Event Any Damage` stamperà il danno ricevuto (debug).

11.3.2 Variabili e logica della vita nel Character Blueprint

- Aggiungere al `BP_Bot2` le variabili:
  - `CurrentHealth` (tipo `Float`, default `10`).
  - `MaxHealth` (tipo `Float`, default `10`).
- Nell'evento `Event Any Damage`:
  1. Sottrarre il `Damage` ricevuto a `CurrentHealth`: `CurrentHealth = CurrentHealth - Damage`.
  2. Clampare il risultato tra `0` e `MaxHealth` (nodo `FClamp`) per evitare valori negativi.
  3. Aggiornare `CurrentHealth` con il valore clampato.

11.3.3 Creazione e utilizzo dell'Event Dispatcher per la vita

- Creare un **Event Dispatcher** nel `BP_Bot2`:
  - Nome: `e_UpdateHealth`.
  - Aggiungere due input di tipo `Float`: `CurrentHealth` e `MaxHealth`.
- Dopo aver aggiornato `CurrentHealth`, **chiamare** l'Event Dispatcher `e_UpdateHealth` passando i valori attuali di `CurrentHealth` e `MaxHealth`.
- **Gestione della morte**: con un nodo `Branch`, se `CurrentHealth` $\leq$ `0`, ricaricare il livello corrente.
  - Ottenere il nome del livello attuale: `Get Current Level Name`.
  - Riaprirlo con `Open Level (by Name)` collegando il `Return Value` al `Level Name`.
- Flusso completo: Danno $\rightarrow$ Sottrazione e clamp $\rightarrow$ Chiamata a `e_UpdateHealth` $\rightarrow$ (se vita $\leq$ 0) Reload livello.

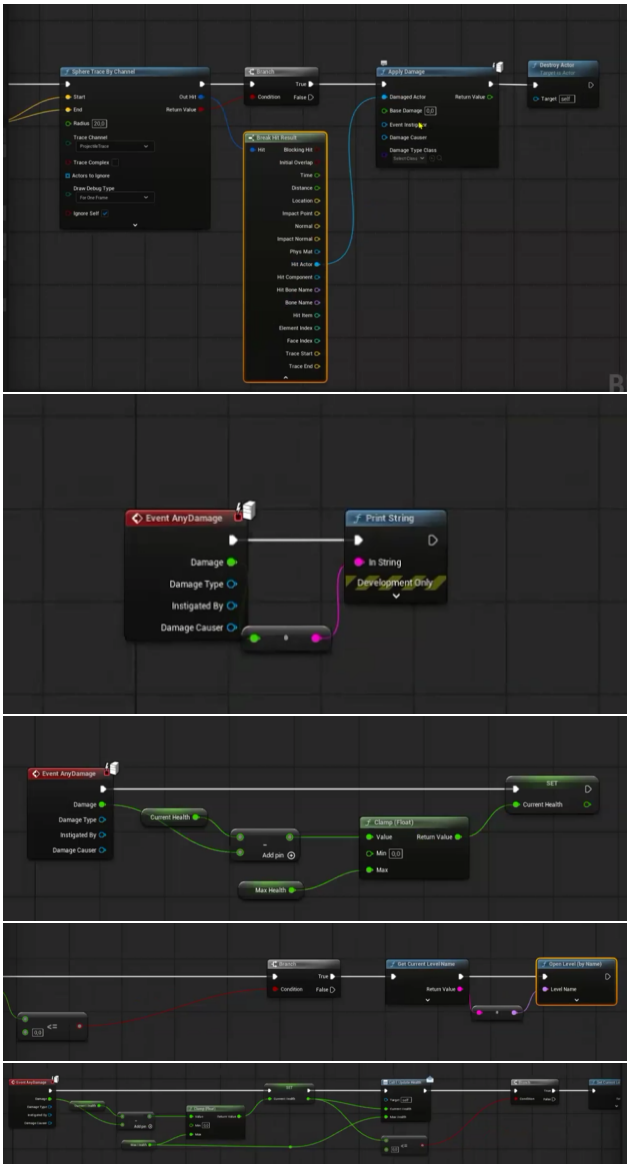
11.4 Creazione del Widget per la UI

11.4.1 Creazione del Widget Blueprint e design della barra della vita

- Creare una cartella `UI` nel Content Browser.
- Creare un **Widget Blueprint**: tasto destro $\rightarrow$ `User Interface` $\rightarrow$ `Widget Blueprint`, nominarlo `UB_HUD_1`.
- Il Widget Blueprint si compone di due schede principali:
  - **Designer**: spazio grafico dove si trascinano gli elementi.
  - **Graph**: logica e funzionalità (eventi, bindings).
- Nel *Designer*:
  1. Aggiungere un `Canvas Panel` (se non presente) alla gerarchia.
  2. Trascinare una `Progress Bar` dalla palette all'interno del `Canvas Panel`.
  3. Posizionarla in basso al centro: selezionare la barra, andare su `Anchors` e scegliere *basso al centro*.
  4. Ridimensionare a piacere.
  5. Impostare il colore: nella proprietà `Fill Color and Opacity`, selezionare un rosso intenso (es. `R=1, G=0, B=0, A=1`).
  6. Lasciare il valore `Percent` a `1` (verrà aggiornato dinamicamente).

11.4.2 Logica del Widget e collegamento con il personaggio

- Passare alla scheda `Graph` del Widget.
- Creare un `Custom Event` chiamato `Initialize Health Bar`.
  - Aggiungere un input di tipo `BP_Bot2` (Object Reference) per passare il riferimento al giocatore.
- L'idea: dal `BeginPlay` del `BP_Bot2` creeremo il widget, e alla sua inizializzazione gli passeremo il riferimento al personaggio stesso (il `Self`).
- Il widget dovrà **bindarsi** all'Event Dispatcher `e_UpdateHealth` del personaggio per aggiornare la `Progress Bar` ogni volta che la vita cambia.


Event AnyDamage
Damage
Damage Type
Instigated By
Damage Causer
Print String
In String
Development Only
Current Health
Add pin
Clamp (Float)
Value
Return Value
Min
Max
Max Health
SET
Branch
Condition
True
False
Get Current Level Name
Open Level (by Name)
Level Name

11.4.3 Prossimi passi (preannunciati)

- Nel `BeginPlay` del `BP_Bot2`:
  1. Creare il widget tramite `Create Widget` (classe `UB_HUD_1`).
  2. Aggiungerlo alla viewport con `Add to Viewport`.
  3. Chiamare l'evento `Initialize Health Bar` del widget passando `Self`.
- All'interno di `Initialize Health Bar`:
  1. Ottenere il riferimento al `BP_Bot2` dall'input.
  2. Eseguire il **Bind Event** all'Event Dispatcher `e_UpdateHealth` del personaggio.
  3. Nel callback del bind, calcolare il rapporto `CurrentHealth / MaxHealth` e assegnarlo a `Progress Bar` → `Percent`.
  4. Inizializzare subito il valore della barra con i valori attuali di vita.

11.5 Visualizzazione e aggiornamento della barra della vita

11.5.1 Creazione e visualizzazione del Widget nel Character Blueprint

- Nel `BP_Bot2`, all'evento `Event BeginPlay`:
  1. **Creare** il widget con `Create Widget`:
     - `Class` → `UB_HUD_1` (il Widget Blueprint creato).
  2. **Promuovere** il valore di ritorno a variabile (es. `As HUD`) per future comunicazioni con la UI.
  3. **Aggiungere** il widget alla viewport: nodo `Add to Viewport` collegato alla variabile `As HUD`.
  4. **Inizializzare** la barra della vita: dal nodo `As HUD` chiamare il `Custom Event Initialize Health Bar` e passare come `Player Ref` un riferimento al personaggio stesso (`Get Reference to Self`).

11.5.2 Binding dell'Event Dispatcher e aggiornamento grafico

All'interno del `Graph` del Widget `UB_HUD_1`:

- Nell'evento `Initialize Health Bar`:
  1. Utilizzare l'input `Player Ref` per accedere al `BP_Bot2`.
  2. **Bindare** l'Event Dispatcher: dal pin `Player Ref` trascinare e cercare `Bind Event to e_UpdateHealth`.
  3. Dal pin `Event` del Bind creare un `Custom Event`: si genererà automaticamente un nodo con i parametri `CurrentHealth` e `MaxHealth` (ereditati dall'Event Dispatcher). Rinominarlo ad es. `Update Health Bar`.
- Nell'evento `Update Health Bar`:
  1. Ottenere un riferimento alla `Progress Bar` della vita:
     - Nel `Designer`, selezionare la `Progress Bar`, rinominarla (es. `ProgressBar_Health`) e assicurarsi che l'opzione `Is Variable` sia spuntata.
     - Nel `Graph`, trascinare la variabile `ProgressBar_Health` dal pannello `Variables`.
  2. Impostare il valore della barra: nodo `Set Percent`.
  3. Calcolare la percentuale: `CurrentHealth` ÷ `MaxHealth` (nodo `Divide`).
  4. Collegare il risultato al pin `In Percent` del `Set Percent`.
- **Nota**: se la barra non si aggiorna, verificare che nel `BeginPlay` del `BP_Bot2` sia effettivamente chiamato `Initialize Health Bar` con il corretto riferimento `Self`. A scopo di debug, inserire un `Print String` dopo il `Set Percent` o un `Breakpoint` sull'evento `Update Health Bar`.

11.5.3 Allineamento preciso della Progress Bar

- Nel `Designer`, con la `Progress Bar` selezionata:
  - Impostare `Position X` e `Position Y` a `0`.
  - Impostare `Alignment` a `(0.5, 0.5)` per centrarla rispetto all'ancora.
  - Regolare `Anchors` (es. basso al centro) per il posizionamento globale nel `Canvas Panel`.
  - Regolare `Size` (es. `X = 640`, `Y = 30`) per dimensioni uniformi.

11.6 Preparazione della barra della stamina

11.6.1 Design nel Widget Blueprint

- Nel `Designer` di `UB_HUD_1`:
  1. Duplicare la `ProgressBar_Health` (`Ctrl+C`, `Ctrl+V`).
  2. Rinominare la copia in `ProgressBar_Stamina`.
  3. Posizionarla sotto la barra della vita (es. `Alignment = (0.5, 0.5)`, `Position Y = 60`).
  4. Cambiare il colore: `Fill Color and Opacity` → verde (es. `R=0, G=1, B=0`).
- Nel `Graph`:
  1. Creare un nuovo `Custom Event` chiamato `Initialize Stamina Bar` con un input `Player Ref` di tipo `BP_Bot2`.
  2. (La logica di binding e aggiornamento verrà completata dopo aver creato l'Event Dispatcher per la stamina nel personaggio).

11.7 Implementazione della corsa (Stamina)

11.7.1 Configurazione dell'Input Action per la corsa

- **Creare** `IA_Run` (se non già presente):
  - Nella cartella degli input, duplicare un'azione esistente (es. `IA_Jump`), rinominarla in `IA_Run`.
  - Il tipo di valore (`Digital`) è lo stesso di `IA_Jump`.
- **Aggiungere** `IA_Run` **all'Input Mapping Context**:
  - Aprire l'`IMC` in uso.
  - Aggiungere una nuova riga, selezionare `IA_Run`.
  - Associare il tasto `Left Shift` (cliccare sul menù a tendina della tastiera e premere Shift).

11.7.2 Logica della velocità e della stamina nel Character Blueprint

- Impostare la **velocità di camminata** di default:
  - Nel `BP_Bot2`, selezionare il componente `CharacterMovement`.
  - Nel pannello `Details`, categoria `Walking`, impostare `Max Walk Speed = 300`.
- **Gestione dell'input** `IA_Run`:
  1. Aggiungere il nodo dell'evento `IA_Run` (dall'Enhanced Input).
  2. Utilizzare i pin `Started` e `Completed`:
     - `Started` (Shift premuto): `Set Max Walk Speed` → `600` (velocità di corsa).

  - `Completed` (Shift rilasciato): `Set Max Walk Speed` → `300` (torna a camminare).
3. In entrambi i nodi `Set Max Walk Speed`, il `Target` deve essere il `CharacterMovement` del `BP_Bot2`.

- **Nota**: la gestione del consumo e della ricarica della stamina (variabile, Event Dispatcher, aggiornamento barra) sarà affrontata nel proseguimento della lezione.

11.7.3 Problemi tecnici segnalati

- **Widget non appare/non si aggiorna**:
  - Verificare che nel `Designer` la `Progress Bar` abbia il flag `Is Variable` attivo.
  - Controllare che il `Custom Event Initialize Health Bar` venga effettivamente chiamato dal `BeginPlay` con un riferimento `Self` valido.
  - Usare `Print String` o `Breakpoint` per tracciare l'esecuzione.
- **La torretta non spara**:
  - Verificare nuovamente le collisioni della `Skeletal Mesh` del personaggio (`Collision Preset Custom` con i canali `DetectionTurret` e `ProjectileTrace` impostati su `Block`).
  - Assicurarsi che il tag `Player` sia presente e scritto correttamente (uguale a quello cercato dalla torretta).
- **Surriscaldamento portatile**: si consiglia l'uso di basi di raffreddamento con ventole per evitare throttling termico e cali di performance.

11.8 Gestione della User Interface (UI) e del Personaggio

11.8.1 Creazione e visualizzazione del Widget HUD

- Nel `BP_Bot2`, all'evento `Event BeginPlay`:
  1. **Creare** il widget con `Create Widget`:
     - `Class` → `UB_HUD_1` (il Widget Blueprint creato in precedenza).
  2. **Promuovere** il valore di ritorno a variabile (es. `As HUD`) per future comunicazioni.
  3. **Aggiungere** il widget alla viewport: nodo `Add to Viewport` collegato alla variabile `As HUD`.
  4. **Inizializzare** la barra della vita: dal nodo `As HUD` chiamare l'evento `Initialize Health Bar` e passare come `Player Ref` un riferimento al personaggio stesso (`Get Reference to Self`).

11.8.2 Design e logica della barra della vita nel Widget

Design (scheda Designer)

- Aggiungere un `Canvas Panel` se non presente.
- Trascinare una `Progress Bar`, posizionarla in basso al centro tramite `Anchors` → *basso al centro*.
- Allineamento preciso:
  - `Position X = 0`, `Position Y = 0`.
  - `Alignment = (0.5, 0.5)` per centrarla rispetto all'ancora.
  - Dimensione consigliata: `X = 640`, `Y = 30` (regolabile).
- Rinominare la barra in `ProgressBar_Health` (F2) e assicurarsi che `Is Variable` sia attivo (dettaglio a destra).
- Impostare il colore rosso: `Fill Color and Opacity` → selezionare un rosso intenso.

Logica di aggiornamento (scheda Graph)

- `Custom Event Initialize Health Bar` con input `Player Ref` (tipo `BP_Bot2`).
- All'interno dell'evento:
  1. Dal pin `Player Ref` chiamare `Bind Event to e_UpdateHealth` (Event Dispatcher creato nel personaggio).
  2. Dal pin `Event` del Bind, creare un `Custom Event` (si genererà automaticamente con i parametri `CurrentHealth` e `MaxHealth`), rinominarlo ad es. `Update Health Bar`.
  3. Trascinare la variabile `ProgressBar_Health` nel grafo.
  4. Usare `Set Percent` sulla barra.
  5. Calcolare la percentuale: `CurrentHealth` ÷ `MaxHealth` e collegare il risultato a `In Percent`.
- **Nota**: La mancata inizializzazione nel `BeginPlay` (dimenticato il nodo `Initialize Health Bar`) causa la mancata attivazione del binding; inserire un `Print String` dopo il `Set Percent` o un `Breakpoint` per debug.

11.8.3 Preparazione della barra della stamina

- Nel `Designer` di `UB_HUD_1`:
  - Duplicare la `ProgressBar_Health` (`Ctrl+C`, `Ctrl+V`), rinominarla in `ProgressBar_Stamina`.
  - Posizionarla sotto la barra della vita (es. `Position Y = 60`, stesso allineamento).
  - Cambiare colore in verde.
- Nel `Graph`, creare un `Custom Event Initialize Stamina Bar` con input `Player Ref` (logica di binding da completare dopo la creazione dell'Event Dispatcher della stamina).

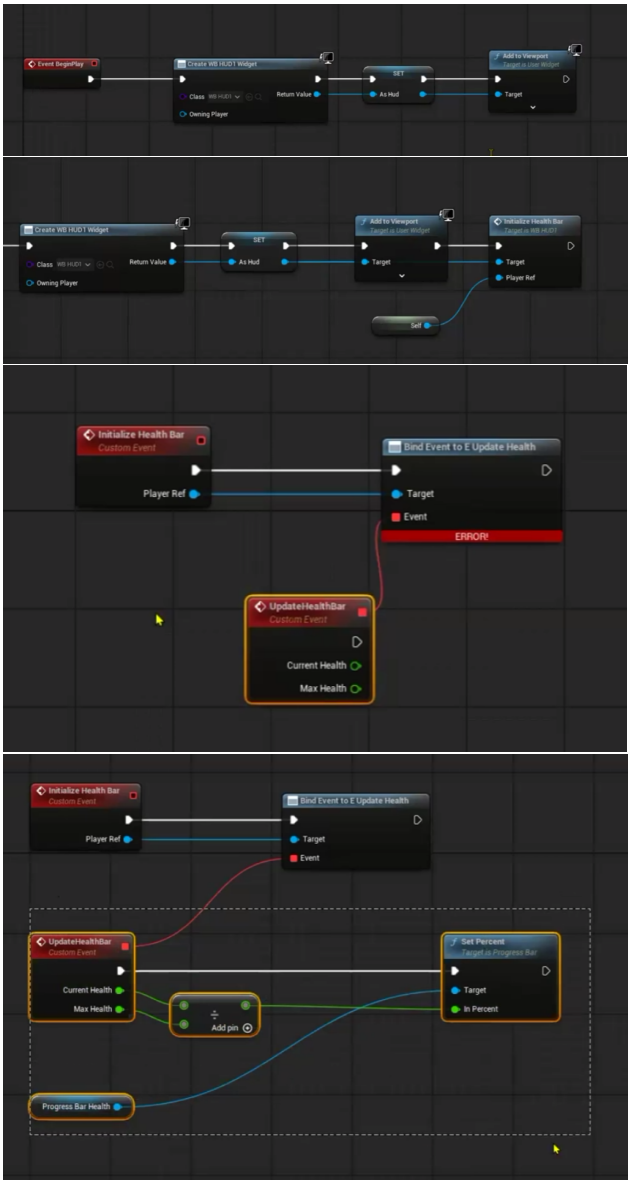
11.9 Implementazione della corsa e della velocità

- **Input Action** `IA_Run`: Se non esiste, duplicare `IA_Jump` e rinominarla. Aggiungerla all'`IMC` associando il tasto `Left Shift`.
- Nel `BP_Bot2`:
  - Impostare la velocità di camminata di default: selezionare `CharacterMovement` → `Max Walk Speed = 300`.
  - Nodo evento `IA_Run`:
    - `Started` (Shift premuto) → `Set Max Walk Speed = 600` (corsa).
    - `Completed` (Shift rilasciato) → `Set Max Walk Speed = 300` (camminata).
  - Target di entrambi i `Set`: il componente `CharacterMovement`.
- **Nota**: La gestione della stamina (consumo graduale e recupero tramite timer) verrà affrontata in seguito.

11.10 Correzioni e troubleshooting

11.10.1 Problemi con la torretta

- **La torretta non spara / non rileva il giocatore**:
  - **Tag**: Verificare che nel `BP_Bot` sia presente il tag `Player` (identico a quello cercato dalla torretta) in `Actor` → `Advanced` → `Tags`.


Event BeginPlay
Create WB HUD1 Widget
Class WB HUD1
Owning Player
Return Value
SET
As Hud
Add to Viewport
Target is User Widget
Target
Initialize Health Bar
Target is WB HUD1
Player Ref
Self
Initialize Health Bar
Custom Event
Player Ref
Bind Event to E Update Health
Target
Event
ERROR!
UpdateHealthBar
Custom Event
Current Health
Max Health
Add pin
Set Percent
Target is Progress Bar
In Percent
Progress Bar Health

- **Collisioni sulla mesh del personaggio**: `Collision Preset` → `Custom`, spuntare `DetectionTurret` e `ProjectileTrace` su `Block`.
- **Target della torretta**: Sostituire il nodo `Player Target` con un `Get Player Pawn` per ottenere direttamente il giocatore.
- **Tracciamento**: Nel `Move Turret` usare `Find Look at Rotation` invece di `Find Relative Look at Rotation`, e collegare il risultato splittato alla `Target Z`.
- **Spawn proiettili**: Verificare che i componenti `ProjectileSpawn1` e `ProjectileSpawn2` siano posizionati all'uscita delle canne e che il proiettile abbia `Sphere Trace Radius = 20` (non zero).
- **Collisione proiettile-camera**: Nel proiettile, sulla sfera di collisione, impostare `Collision` → `Ignore Camera`.

11.10.2 Problemi con l'Animation Blueprint

- **Personaggio in T-pose o non animato**:
  - Verificare che l'`Animation Class` del `BP_Bot` sia `ABP_Bot`.
  - Controllare nell'ABP che il `Validated Get` si riferisca al `BP_Bot` corretto (non a un altro attore).
  - In `Event Initialize Animation`, rifare il cast e il `Validated Get` con l'esatta variabile del personaggio.
- **Transizione Idle → Walk non avviene**:
  - La condizione deve essere `Ground Speed > 0`; per il ritorno `Walk` → `Idle` usare `Ground Speed <= 0` (minore o uguale per evitare micro-valori).
- **Inclinazione (Lean) assente o errata**:
  - Nel `Blend Space` ridurre il range dell'asse `Lean` da $[-10, 10]$ a $[-1, 1]$ e spostare di conseguenza i keyframe delle animazioni `Lean Left/Right` a -1 e $+1$.
  - Verificare che il nodo `FInterp To` abbia `Interp Speed = 1`.
  - Se il movimento è invertito (sinistra/destra scambiati), invertire le animazioni nel Blend Space.
- **Rotazione innaturale durante la corsa**: regolare le impostazioni di rotazione nel `BP_Bot` (`Use Controller Rotation Yaw`, `Orient Rotation to Movement`) o nello `Spring Arm` (`Use Pawn Control Rotation`).

11.10.3 Altri problemi

- **La barra della vita non si aggiorna visivamente**:
  - Controllare che nel `BP_Bot` il `BeginPlay` chiami effettivamente `Initialize Health Bar` con `Self`.
  - Verificare che nel Widget la `Progress Bar` abbia il flag `Is Variable` attivo e che il nodo `Set Percent` sia collegato al risultato della divisione.
  - Usare `Print String` o `Breakpoint` sull'evento `Update Health Bar` per tracciare l'esecuzione.
- **Personaggio bloccato dopo essere stato colpito**: probabile crash o loop dovuto a collisioni multiple; verificare la logica di distruzione del proiettile e che non ci siano chiamate a `Reload Level` prima del previsto.

11.11 Correzioni e completamento della logica di danno e UI

11.11.1 Proiettile: applicazione del danno e autodistruzione

- Nel `BP_Projectile`, dopo la collisione (`Sphere Trace`), il nodo `Apply Damage` deve essere usato al posto di `Destroy Actor` sul target colpito:
  - `Damaged Actor` ← `Hit Actor` dal `Break Hit Result`.
  - `Base Damage` ← `1`.
- Subito dopo `Apply Damage`, distruggere il proiettile stesso con `Destroy Actor` (collegato al pin di esecuzione successivo).

11.11.2 Correzione della barra della vita e della morte

- Nel `BP_Bot2`, verificare le variabili:
  - `CurrentHealth` e `MaxHealth` devono essere entrambi impostati a `10` (o il valore desiderato).
  - Eliminare eventuali variabili duplicate (es. `NewVar`) non utilizzate.
- Nell'Event Graph:
  - Il flusso di esecuzione dopo `Event Any Damage`:
    1. `CurrentHealth = CurrentHealth - Damage`.
    2. `Clamp` tra `0` e `MaxHealth`.
    3. Impostare `CurrentHealth` con il nuovo valore.
    4. Chiamare l'Event Dispatcher `e_UpdateHealth` con `CurrentHealth` e `MaxHealth`.
  - Dopo la chiamata al dispatcher, un `Branch` controlla se `CurrentHealth <= 0`: in tal caso, ricaricare il livello.
- Nel `BP_Bot2` → `Event BeginPlay`:
  - Assicurarsi che il nodo `Initialize Health Bar` del widget sia **collegato** al flusso di esecuzione dopo `Add to Viewport` e prima di altri nodi (era staccato).
  - Il riferimento passato al widget è `Get Reference to Self`.
- Nel Widget `UB_HUD_1`, all'interno dell'evento `Update Health Bar`:
  - `Set Percent` della `ProgressBar_Health` deve essere collegato al risultato di `CurrentHealth / MaxHealth`.
  - Assicurarsi che la variabile `ProgressBar_Health` abbia il flag `Is Variable` attivo.

11.11.3 Ripristino della torretta

- **Event Tick**: aggiungere un nodo `Event Tick` nel grafico della torretta; controllare che il `Player Target` sia valido tramite `IsValid`; se valido, chiamare la funzione `Shoot`.
- **Fire rate**: regolare il valore della variabile `FireRate` a circa `0.06` per una frequenza di fuoco adeguata.
- **Velocità del proiettile**: nel `BP_Projectile` → componente `ProjectileMovement` → impostare `Initial Speed` e `Max Speed` a `1000` ciascuno.
- **Collisioni e rilevamento**:
  - La torretta deve usare `Get Player Pawn` per il target, non un riferimento generico.
  - La mesh del giocatore (`BP_Bot2`) deve avere il canale di collisione `DetectionTurret` impostato su `Block` e il tag `Player`.

11.12 Implementazione del sistema di Stamina

11.12.1 Variabili e timer principali

- Aggiungere nel `BP_Bot2`:
  - `CurrentStamina` (Float, default = 100).
  - `MaxStamina` (Float, default = 100).
  - `TimeStamina` (Float, tempo tra un tick e l'altro del timer, default = 0.05).
- I timer sono creati con `Set Timer by Event`:
  - **Timer di consumo**: attivato quando si inizia lo sprint (`IA_Run Started`).
  - **Timer di recupero**: attivato quando si termina lo sprint (`IA_Run Completed`) o quando la stamina è esaurita.
  - Entrambi i timer sono in `Looping` e con `Max Once per Frame` disattivo.
  - Salvare i return value di ciascun `Set Timer by Event` in variabili (`TimerDeductionStamina`, `TimerRegainStamina`) per poterli fermare.

11.12.2 Logica di consumo della stamina

- **Custom Event** `StaminaDeduction` (chiamato dal timer):
  1. `CurrentStamina = CurrentStamina - 1`.
  2. `Clamp` il risultato tra `0` e `MaxStamina`.
  3. Aggiornare `CurrentStamina`.
  4. **Arresto anticipato**: se `CurrentStamina <= 0` **oppure** il personaggio sta cadendo (`CharacterMovement` → `Is Falling`), eseguire lo stop dello sprint (vedi `StopRunning`).
- La condizione *sta cadendo* serve ad evitare il consumo di stamina mentre si è in volo, anche se il tasto Shift rimane premuto.

11.12.3 Logica di recupero della stamina

- **Custom Event** `RegainStamina` (chiamato dal secondo timer):
  1. `CurrentStamina = CurrentStamina + 1`.
  2. `Clamp` tra `0` e `MaxStamina`.
  3. Aggiornare `CurrentStamina`.
  4. **Arresto al massimo**: se `CurrentStamina >= MaxStamina`, fermare il timer di recupero con `Clear and Invalidate Timer by Handle` su `TimerRegainStamina`.
- Il timer di recupero deve essere fermato anche se il giocatore ricomincia a sprintare: nell'evento `IA_Run Started`, fermare `TimerRegainStamina` e far ripartire il timer di consumo.

11.12.4 Gestione dell'input e stop forzato della corsa

- Creare un `Custom Event StopRunning` che:
  - Imposta la velocità di movimento a `300` (camminata).
  - Ferma il timer di consumo (`Clear and Invalidate Timer by Handle` su `TimerDeductionStamina`).
  - Avvia il timer di recupero.
- Collegare `StopRunning`:
  - Al pin `Completed` di `IA_Run` (quando il giocatore rilascia Shift).
  - Alla logica di consumo, quando `CurrentStamina <= 0` o il personaggio sta cadendo.

11.12.5 Note aggiuntive

- **Animazioni di corsa**: come per il movimento base, assicurarsi che l'Animation Blueprint passi alla corretta animazione in base alla velocità (300 camminata, 600 corsa). Eventuali piccoli scatti possono essere risolti con un `Blend In/Out` nelle transizioni di stato.
- **Compito facoltativo**: creare un semplice sistema di porte con l'uso di Timeline (da consegnare per la prossima lezione).

11.13 Debugging del sistema di stamina e corsa

11.13.1 Correzione dell'aggiornamento della barra della stamina

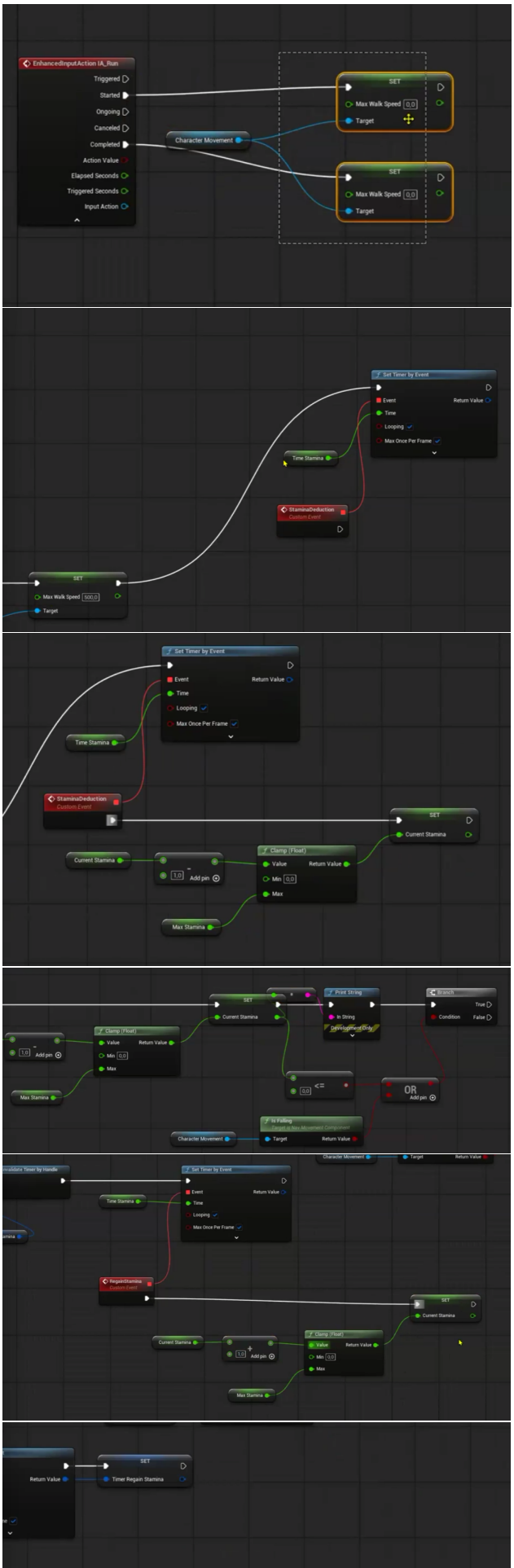
- Nel `BP_Bot2`, nei nodi `Call e_UpdateStamina` (sia in `StaminaDeduction` che in `RegainStamina`) è necessario collegare esplicitamente i valori `CurrentStamina` e `MaxStamina` ai pin di input dell'Event Dispatcher.
- Verificare che nel Widget `UB_HUD_1`:
  - L'evento `Initialize Stamina Bar` riceva effettivamente il `Player Ref` (aggiungere l'input nel Custom Event se mancante).
  - Il `Bind Event to e_UpdateStamina` sia collegato e generi un `Custom Event` (es. `Update Stamina Bar`).
  - La `ProgressBar_Stamina` abbia il flag `Is Variable` attivo e venga impostata con `Set Percent` usando il rapporto `CurrentStamina / MaxStamina`.
- Se la barra della stamina appare subito vuota (grigia), controllare che i valori `CurrentStamina` e `MaxStamina` siano entrambi inizializzati a 100 e che il collegamento dei parametri nel `Call` sia corretto.

11.13.2 Risoluzione del problema della corsa (Shift non funzionante)

- **Sintomo**: premendo Shift il personaggio non aumentava la velocità, continuava a camminare a 300.
- **Causa**: era presente un nodo `Set Max Walk Speed = 300` scollegato dall'input `IA_Run` che interferiva, oppure un doppione dell'evento `IA_Run` con valori errati.
- **Correzione**:
  1. Eliminare ogni nodo `Set Max Walk Speed` non direttamente collegato ai pin `Started` e `Completed` di `IA_Run`.
  2. Assicurarsi che `IA_Run` abbia i trigger corretti: nell'Input Action, impostare `Trigger` → `Pressed` e `Released` (non `Down`).
  3. I due `Set Max Walk Speed` devono essere:
     - `Started` → `600`.
     - `Completed` → `300` (o chiamata a `StopRunning`).
- Dopo la correzione, il personaggio torna a scattare correttamente.

11.13.3 Consumo della stamina durante la caduta

- **Problema**: alcuni studenti notavano consumo di stamina anche mentre il personaggio è in volo (se Shift rimane premuto).
- **Verifica**: nella logica di `StaminaDeduction`, subito dopo il clamp, il `Branch` che controlla l'arresto deve usare un `OR` tra `CurrentStamina <= 0` e `Is Falling` (ottenuto dal `CharacterMovement`).
- Se il `StopRunning` viene chiamato correttamente, la stamina non si consuma in caduta. Controllare che l'evento `StopRunning` arresti il timer di consumo e imposti la velocità a 300.


EnhancedInputAction IA_Run
Triggered
Started
Ongoing
Canceled
Completed
Action Value
Elapsed Seconds
Triggered Seconds
Input Action
Character Movement
SET
Max Walk Speed
Target
Set Timer by Event
Event
Time
Looping
Max Once Per Frame
Return Value
Time Stamina
StaminaDeduction
Custom Event
Clamp (Float)
Value
Min
Max
Current Stamina
Max Stamina
Print String
In String
Development Only
Branch
Condition
True
False
OR
Add pin
RegainStamina
Timer Regain Stamina

11.14 Compito per casa: Porta automatica con Timeline

- **Obiettivo**: creare un asset di porta che si apre quando il giocatore entra in un volume di trigger e si chiude quando ne esce.
- **Asset necessari** (già presenti nel progetto):
  - `SM_Modular_Wall_Door` (telaio/frame).
  - `SM_Door` (anta della porta).
  - Un duplicato di `SM_Door` ruotato di 180° per la seconda anta (se a doppio battente).
- **Indicazioni di massima**:
  1. Creare un nuovo `Blueprint Actor` (es. `BP_Door`).
  2. Aggiungere i componenti `Static Mesh` per telaio e ante.
  3. Aggiungere un `Box Collision` come volume di trigger.
  4. Utilizzare una **Timeline** per animare la rotazione delle ante (da chiusa a aperta e viceversa).
  5. Logica: `On Begin Overlap` → `Play` (apri); `On End Overlap` → `Reverse` (chiudi).
  6. Gestire più giocatori? Per semplicità, non richiesto.
- **Nota**: è un compito **facoltativo**, senza valutazione, per esercitarsi con le Timeline già affrontate a lezione. In caso di difficoltà, verrà mostrato in aula la prossima volta.

11.15 Conclusioni e consigli

- La prossima lezione verterà sulle **interazioni** (es. tasto Interact, oggetti interagibili) e completerà il modulo tecnico prima di passare al level design con Davide.
- Le registrazioni delle lezioni restano disponibili fino a settembre per permettere il recupero.
- Per chi è interessato alla programmazione o al game design, si consiglia di esercitarsi nella prototipazione di meccaniche: *le idee sono belle nella nostra testa, ma bisogna vedere se funzionano concretamente in un ambiente di gioco.* Avere una buona manualità tecnica è un valore aggiunto nel settore.

Chapter 12

Sviluppo del Menu di Gioco e Interazioni tramite Blueprint Interface e Component

12.1 Introduzione e organizzazione del corso

12.1.1 Recupero delle lezioni

- Lo studente Lorenzo chiede se sia meglio recuperare le lezioni precedenti o seguire quella corrente. L'insegnante consiglia di seguire la diretta, porre domande su eventuali dubbi e poi recuperare le lezioni vecchie per rimettersi in pari.
- Per l'esercizio odierno è sufficiente avere un personaggio che si muove e guarda intorno.
- Le lezioni registrate rimangono disponibili fino a settembre.

12.1.2 Supporto e comunicazione

- Anche dopo la fine del corso, è possibile contattare l'insegnante tramite Discord per qualsiasi domanda o chiarimento, sia in diretta che in differita.
- È importante postare i compiti nell'apposito canale `Introduzione a Unreal` per evitare di mischiare i messaggi.

12.1.3 Esame e percorso didattico

- Il corso prevede un esame finale che consiste nella realizzazione di un progetto da inserire nel proprio portfolio.
- Il percorso di Game Designer prevede:
  - **Primo anno**: introduzione a Unreal Engine e Blueprint, con focus sulla prototipazione di meccaniche.
  - **Secondo anno**: progetto più strutturato su Unreal (`Big One`).
  - **Terzo anno**: `Biggest One`, eventuale evoluzione del progetto precedente, con l'obiettivo di pubblicarlo su Steam.
- **Consiglio di design**: `less is more`. Un design più snello è più leggibile per il giocatore, che impiegherà circa il triplo del tempo dello sviluppatore per comprendere le meccaniche.

12.2 Pianificazione della lezione odierna

- La lezione è l'ultima tenuta da questo insegnante; le successive saranno con Davide e riguarderanno il Level Design.
- Argomenti del giorno:
  1. Completare la gestione della stamina (sistemare un bug per cui la barra scende solo al rilascio del tasto).
  2. Sistema di interazione (argomento più lungo, modulare, basato su telecamera).
  3. Creazione del menu principale (argomento più breve, scelto per primo).
- La scelta dell'ordine (prima il corto, poi il lungo) è stata decisa per gestire al meglio la pausa.

12.3 Creazione del menu principale

12.3.1 Creazione del Widget Blueprint

- Creare un nuovo **Widget Blueprint**: tasto destro → `User Interface` → `Widget Blueprint` → `User Widget`. Nominarlo `VB_Menu`.
- All'interno del Widget, aggiungere un `Canvas Panel` per adattare il contenuto allo schermo.

12.3.2 Preparazione del livello vuoto per il menu

- Creare una nuova mappa: `File` → `New Level` → `Empty Level`.
- L'Empty Level è completamente privo di luci, skybox o altri elementi predefiniti.
- Aggiungere un elemento 3D di sfondo per creare un effetto diorama: trascinare un `Cube` nella scena e ridimensionarlo come un pavimento.
- Applicare un materiale di base (es. `Grid`) al cubo.
- Salvare la mappa: `Ctrl+S` → `M_Menu`.
- Verificare che il cubo non sia impostato come `Hidden in Game`.

12.3.3 Design dei pulsanti nel Widget

- Nel `Designer` del Widget, dalla palette `Common` trascinare due `Button` all'interno del `Canvas Panel`.
- Impostare l'ancoraggio (`Anchors`) al centro per entrambi i pulsanti.
- Configurare il primo pulsante:
  - Rinominarlo in `Button_Start_Game` (F2).
  - `Size`: X = 300, Y = 80.
  - `Alignment`: 0.5, 0.5.
  - Posizione: Y = 2.7 (o 0.5, 1.5) a seconda della regolazione fine; l'insegnante ha mostrato di azzerare i valori e spostare solo l'altezza.
- Configurare il secondo pulsante:
  - Copiare e incollare il primo, rinominarlo in `Button_Quit_Game`.
  - Posizionarlo più in basso, modificando l'allineamento verticale (es. Y = -0.5 o -0.3).
- Se il pulsante copiato non appare, controllare la gerarchia (`Hierarchy`); eventualmente crearne uno nuovo.

12.4 Creazione del Menu Principale

12.4.1 Preparazione del livello vuoto

- Creare una nuova mappa vuota: `File` → `New Level` → `Empty Level`.
- L'Empty Level è completamente privo di luci, skybox o altri elementi predefiniti; lo sfondo risulterà nero.
- Aggiungere un elemento 3D di sfondo: trascinare un `Cube` nella scena, ridimensionarlo a piacere per creare un pavimento.
- Applicare un materiale di base (es. `Grid`) al cubo.
- Salvare la mappa come `M_Menu` nella cartella `Maps`.
- Assicurarsi che il cubo non sia impostato su `Hidden in Game`.

12.4.2 Widget Blueprint del Menu

- Creare un nuovo Widget Blueprint: tasto destro → `User Interface` → `Widget Blueprint` → `User Widget`. Rinominarlo `VB_Menu`.
- Nel Designer, aggiungere un `Canvas Panel` come contenitore principale.

Design dei pulsanti e del testo

- Dalla palette `Common`, trascinare due `Button` all'interno del Canvas Panel.
- Per ogni pulsante, impostare l'ancoraggio al centro (`Anchors` → centrato) e l'`Alignment` a `(0.5, 0.5)`.
- Rinominare i pulsanti (F2): `Button_Start_Game` e `Button_Quit_Game`.
- Posizionarli verticalmente uno sopra l'altro modificando l'allineamento verticale (es. Y = 1.5 per il primo, Y = 0.5 o -0.3 per il secondo).
- Aggiungere il testo direttamente nei pulsanti: dalla palette `Common` trascinare un `Text` e rilasciarlo sopra il pulsante nella gerarchia; il testo si ancorerà automaticamente al bottone.
- Modificare il contenuto del testo: selezionare il `Text Block` → `Details` → `Content` → `Text`. Inserire `Play Game` per il primo pulsante e `Quit` per il secondo.

Stile dei pulsanti

- Selezionare un pulsante e, nei `Details`, cercare la sezione `Style`.
- Sono presenti tre stati: `Normal`, `Hovered`, `Pressed`.
- Per cambiare colore di base: `Normal` → `Brush` → `Tint`, scegliere un colore (es. blu).
- Per lo stato `Hovered` (mouse sopra) scegliere un colore diverso (es. arancione); la preview non lo mostrerà, ma in esecuzione sì.
- Per lo stato `Pressed` (cliccato) scegliere un altro colore (es. rosso).
- Per copiare esattamente un colore da un pulsante all'altro: cliccare sulla barra del colore, copiare il codice esadecimale `sRGB` o `Linear` (es. `FFA500`) e incollarlo nell'altro stato.

Immagine di sfondo con opacità

- Trascinare un `Image` dal pannello `Common` all'interno del Canvas Panel.
- Assegnare un'immagine o un colore tramite `Brush` → `Tint`; per renderla semitrasparente, agire su `Brush` → `Color and Opacity` → `A` (Alpha), es. 0.2.
- Per mettere l'immagine dietro ai pulsanti, usare la `Hierarchy`: trascinare l'elemento `Image` sopra i pulsanti in modo che compaia una linea blu tra di essi; in questo modo l'immagine sarà visualmente sotto.

12.4.3 Logica del Menu nel Level Blueprint

Visualizzazione del mouse e modalità input

- Aprire il Level Blueprint: pulsante `Blueprints` → `Open Level Blueprint`.
- All'evento `Event BeginPlay`:
  1. `Create Widget` → classe `VB_Menu`.
  2. `Add to Viewport` per mostrarlo.
  3. `Get Player Controller` → `Set Show Mouse Cursor` (spuntare).
  4. `Set Input Mode UI Only` (da `Get Player Controller`) → `In Widget to Focus` collegare il valore di ritorno del `Create Widget` (trascinando dal nodo).

Impostazione della camera di menu (Set View Target with Blend)

- Trascinare un `Camera Actor` nella scena e posizionarla per inquadrare il diorama.
- Aggiungere una `Directional Light` per illuminare la scena.
- Nel Level Blueprint, dopo l'input mode, per impostare la camera fissa:
  1. `Get Player Controller` → `Set View Target with Blend`.
  2. `New View Target` → creare un riferimento al Camera Actor: selezionarlo nella viewport, poi nel Level Blueprint tasto destro → `Create Reference to CameraActor`.
  3. Collegare il riferimento a `New View Target`; `Blend Time` può essere lasciato a 0.
- Nota: inizialmente l'insegnante aveva usato `Get Player Camera Manager`, che non è corretto; il metodo giusto è tramite il `Player Controller`.

12.4.4 Funzionalità dei pulsanti nel Widget Graph

Pulsante Start Game

- Nel Widget `VB_Menu`, passare alla scheda `Graph`.
- Aggiungere l'evento `OnClicked` del pulsante `Button_Start_Game`.
- Prima di cambiare livello, ripristinare l'input per il gioco:
  1. `Get Player Controller` → `Set Input Mode Game Only`.
- `Open Level (by Name)`: inserire il nome esatto della mappa di gioco (es. `ThirdPersonMap` o la propria mappa principale).
- Collegare l'esecuzione da `OnClicked` al `Set Input Mode Game Only` e poi a `Open Level`.

Pulsante Quit Game

- Aggiungere l'evento `OnClicked` del pulsante `Button_Quit_Game`.
- Usare il nodo `Quit Game` (fornito da Unreal) per uscire dall'applicazione.

12.4.5 Risultato finale

- Il menu mostra due pulsanti (Play Game, Quit) con stili hover/pressed, un'immagine di sfondo semi-trasparente, il cursore del mouse visibile e nessun controllo del personaggio.
- Cliccando su Play Game, il gioco carica il livello di gioco principale; cliccando su Quit, l'applicazione si chiude.
- È possibile abbellire la scena di sfondo con modelli 3D (es. personaggio, porte) per un effetto diorama.

12.5 Componente per il Tracciamento dell'Interazione

12.5.1 Creazione del Blueprint Component (Scene Component)

- Creare un nuovo Blueprint di tipo **Scene Component**: `Content Browser` → `Blueprint Class` → `Scene Component`. Rinominarlo `BPC_RayInteract`.
- Un Scene Component, a differenza di un Actor Component, possiede un `Transform` (posizione, rotazione, scala) e può essere spostato fisicamente nell'editor.
- Il componente verrà agganciato al personaggio per generare un raggio (trace) verso la telecamera.
- All'interno del Componente, creare una nuova **Funzione**: pannello `My Blueprint` → `Functions` → `+`. Nominarla `LookForInteractions`.
- La funzione conterrà la logica del tracciamento.

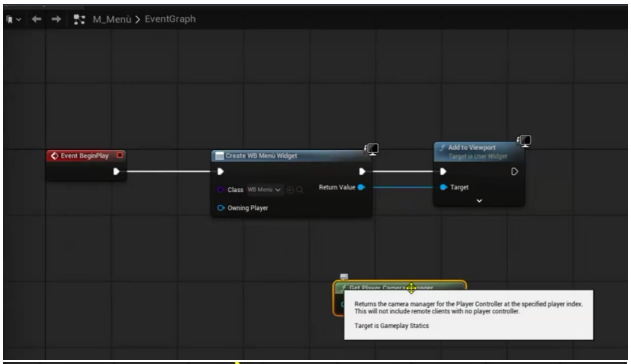
12.5.2 Logica della funzione `LookForInteractions`

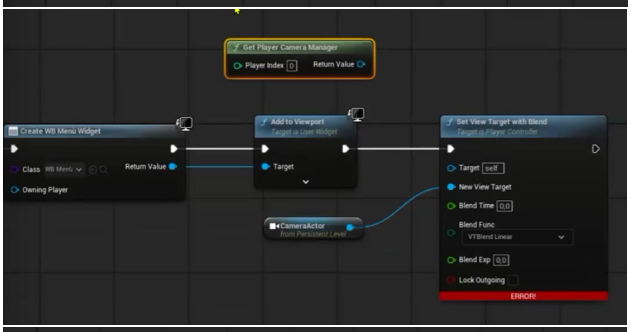
Parametri del trace

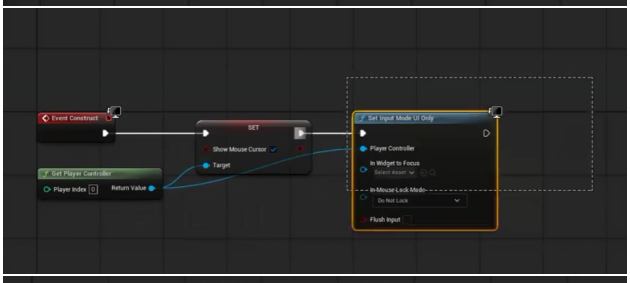
- **Sphere Trace By Channel**: nodo per il tracciamento.
- `Radius`: impostare a `5`.
- `Start`: `Get World Location` del `SceneComponent` stesso (posizione del componente agganciato, es. testa del personaggio).
- `End`: da calcolare come `Start` + (`Forward Vector` × `DistanceTrace`).

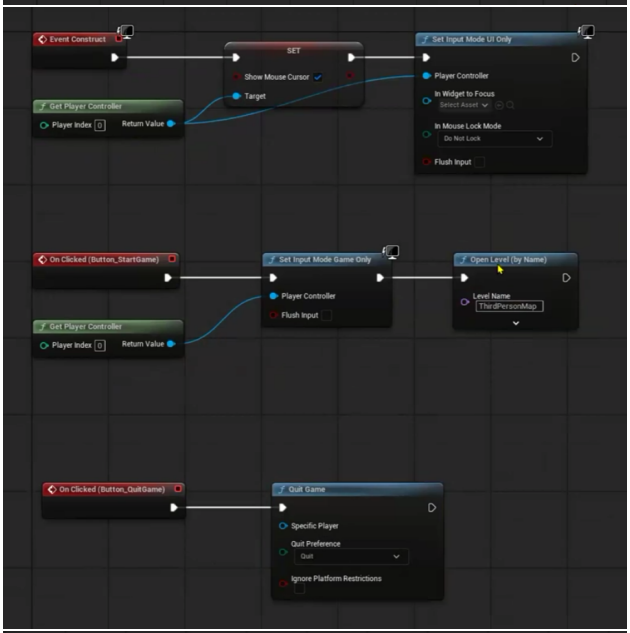
Ottenere il Forward Vector corretto (Select)

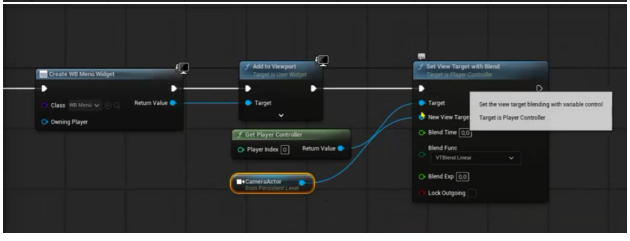
- Per decidere se usare il `Forward Vector` della **telecamera** o del componente stesso, si usa un nodo `Select`.
- **Problema tecnico**: il nodo `Select` determina il tipo di dato dal primo input collegato. La telecamera (`CameraComponent`) e il riferimento al componente stesso (`Self`) sono entrambi `SceneComponent`, ma per garantire la compatibilità occorre creare una variabile di tipo `SceneComponent` da usare come primo input.
- **Soluzione**:


M_Menù > EventGraph
Event BeginPlay
Create WB Menu Widget
Class
Owning Player
Return Value
Add to Viewport
Target
Returns the camera manager for the Player Controller at the specified player index.
This will not include remote clients with no player controller.
Target is Gameplay Statics


Get Player Camera Manager
Player Index 0
Return Value
Create WB Menu Widget
Class
Owning Player
Add to Viewport
Target
Set View Target with Blend
Target
self
New View Target
Blend Time
Blend Func
Blend Exp
Lock Outgoing
ERROR!
CameraActor


Event Construct
SET
Show Mouse Cursor
Target
Get Player Controller
Player Index 0
Return Value
Set Input Mode UI Only
Player Controller
In Widget to Focus
In Mouse Lock Mode
Do Not Lock
Flush Input


Event Construct
SET
Show Mouse Cursor
Target
Get Player Controller
Player Index 0
Return Value
Set Input Mode UI Only
Player Controller
In Widget to Focus
In Mouse Lock Mode
Do Not Lock
Flush Input
On Clicked (Button_StartGame)
Set Input Mode Game Only
Player Controller
Flush Input
Open Level (by Name)
Level Name
ThirdPersonMap
Get Player Controller
Player Index 0
Return Value
On Clicked (Button_QuitGame)
Quit Game
Specific Player
Quit Preference
Quit
Ignore Platform Restrictions


Create WB Menu Widget
Class
Owning Player
Return Value
Add to Viewport
Target
Get Player Controller
Player Index 0
Return Value
CameraActor
Set View Target with Blend
Target
New View Target
Blend Time
Blend Func
Blend Exp
Lock Outgoing
Set the view target blending with variable control
Target is Player Controller

1. Creare una variabile di tipo `SceneComponent` (nominata ad es. `Sin`) nel pannello `Variables` del Componente.
2. Collegare questa variabile al primo input del `Select` (pin `Option 0`).
3. Sostituire il riferimento alla variabile con `Self` (`Get Reference to Self`) per l'opzione di fallback.
4. Collegare il `CameraComponent` al secondo input (`Option 1`).

- Il selettore sceglie l'opzione in base a un booleano: `IsValid` del `CameraComponent`.
  - Se la telecamera esiste (`IsValid = True`) → usa il `CameraComponent`.
  - Altrimenti → usa `Self`.
- Dal `Return Value` del `Select` si ottiene il `Forward Vector` con il nodo `Get Forward Vector`.

Calcolo del punto di arrivo (End)

- Moltiplicare il `Forward Vector` per una distanza: `Multiply` (tasto destro sul pin inferiore → `To Float`) × `DistanceTrace`.
- Creare la variabile `DistanceTrace` (Float, default 300) promuovendo il valore `300` a variabile.
- Aggiungere il risultato della moltiplicazione a `Start` tramite `Add` (Vector + Vector).
- Collegare il risultato dell'`Add` al pin `End` dello `Sphere Trace By Channel`.

12.5.3 Inizializzazione del componente (Begin Play)

- Nell'`Event Graph` del Componente, evento `Event BeginPlay`.
- Recuperare il proprietario del componente: `Get Owner`.
- Dal proprietario, ottenere il componente telecamera: `Get Component by Class` → `CameraComponent`.
- Promuovere il risultato a variabile, nominarla `CameraComponent`, per poterla usare nella funzione `LookForInteractions`.
- Collegare il flusso di esecuzione da `BeginPlay` al `Set` della variabile `CameraComponent`.

12.6 Sistema di Interazione

12.6.1 Blueprint Interface `BPI_InteractionRay`

- Le **Blueprint Interface** permettono a Blueprint diversi di comunicare tramite funzioni condivise, senza bisogno di cast specifici. Funzionano come un `intermediario`.
- Per creare l'interfaccia: `Content Browser` → `Blueprint` → `Blueprint Interface`. Rinominarla `BPI_InteractionRay`.
- All'interno, aggiungere tre **funzioni** (pulsante `+`):
  1. `InteractionStart`
  2. `InteractionEnd`
  3. `Interaction`
- L'interfaccia è un grafo di sola lettura (`ReadOnly`), non si scrive codice al suo interno; le funzioni verranno implementate nei Blueprint che adotteranno l'interfaccia.

12.6.2 Componente `BPC_RayInteract`: logica di tracciamento e comunicazione

Variabile Interactable e confronto

- Nel Componente `BPC_RayInteract`, creare una variabile di tipo `Actor` (Object Reference) chiamata `Interactable`. Servirà a memorizzare l'oggetto con cui si sta attualmente interagendo.
- Dopo il `Break Hit Result` dello `Sphere Trace`:
  1. Ottenere l'`Hit Actor`.
  2. Verificare se l'`Hit Actor` **implementa l'interfaccia** `BPI_InteractionRay` usando il nodo `Does Implement Interface`.
  3. Se la condizione è vera, proseguire.

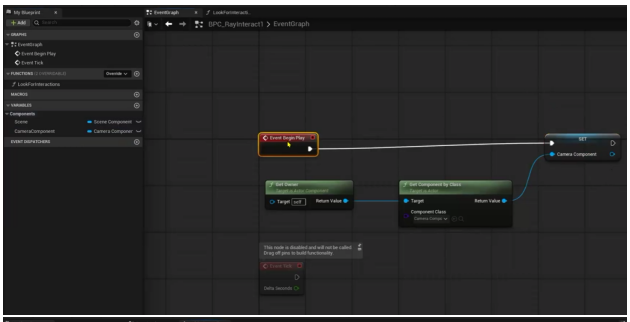
Gestione dell'inizio e della fine dell'interazione

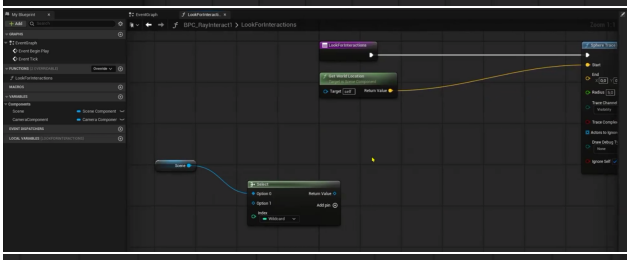
- Confrontare l'`Hit Actor` con la variabile `Interactable` usando l'operatore **Not Equal** (`!=`).
- **Se l'attore è diverso** dal precedente:
  - Controllare se `Interactable` è valido tramite `Validated Get` (o `IsValid`).
  - Se è valido (cioè si stava già interagendo con un altro oggetto), chiamare il messaggio di interfaccia `InteractionEnd` sul vecchio `Interactable`.
  - Impostare il nuovo `Interactable` con `Set Interactable` (passando `Hit Actor`).
  - Chiamare il messaggio di interfaccia `InteractionStart` sul nuovo `Interactable`.
- **Se l'attore è uguale** al precedente, non si esegue alcuna operazione (evita chiamate ripetute).
- **Nota tecnica**: se `Interactable` è stato creato come `SceneComponent`, il collegamento con `Hit Actor` (tipo `Actor`) fallirà. Basta modificare il tipo della variabile `Interactable` in `Actor`.

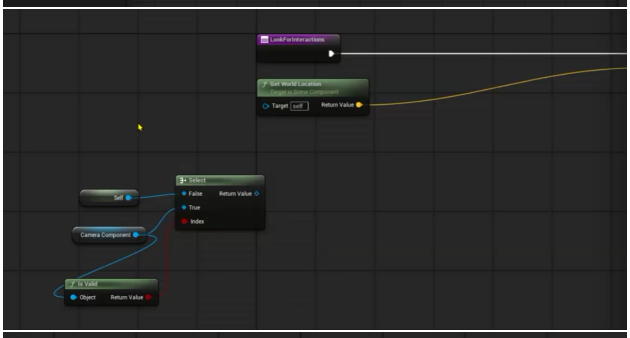
12.6.3 Completamento del Componente `BPC_RayInteract`

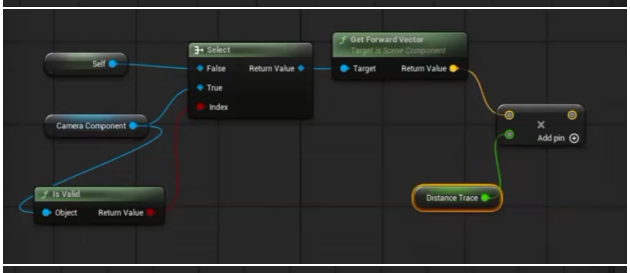
Gestione della fine dell'interazione (nessun oggetto valido)

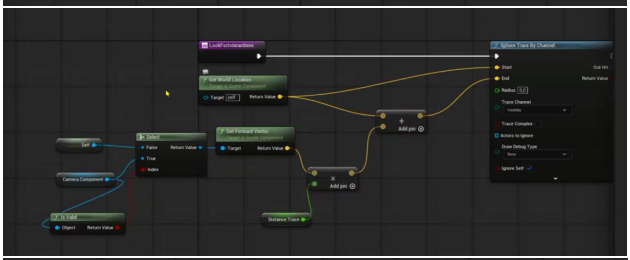
- **Scenario**: il giocatore sposta la visuale su un oggetto non interagibile (es. muro) o su nulla; in quel caso il trace non restituisce un attore con l'interfaccia, oppure non restituisce proprio alcun attore.
- **Cosa fare**: terminare l'interazione in corso e svuotare la variabile `Interactable`.
- **Procedura**:
  1. Prendere la variabile `Interactable` e convertirla in `Validated Get`.
  2. Collegare l'esecuzione di questo ramo sia al caso `non ha colpito nulla` (`Return Value = False` dello `Sphere Trace`), sia al caso `l'attore colpito non implementa l'interfaccia` (`Does Implement Interface = False`).
  3. Se `Interactable` è valido:
     - Chiamare il messaggio di interfaccia `InteractionEnd` su `Interactable`.
     - Svuotare la variabile: `Set Interactable = null` (semplicemente non collegare nulla al pin di input, oppure usare un nodo `Make Null`).
  4. Se `Interactable` non è valido (già vuoto), svuotarla comunque per sicurezza, senza chiamare `InteractionEnd`.
- Questo meccanismo garantisce che quando il giocatore distoglie lo sguardo da un oggetto interagibile, l'evento di fine interazione venga notificato e la variabile sia pronta per un nuovo oggetto.

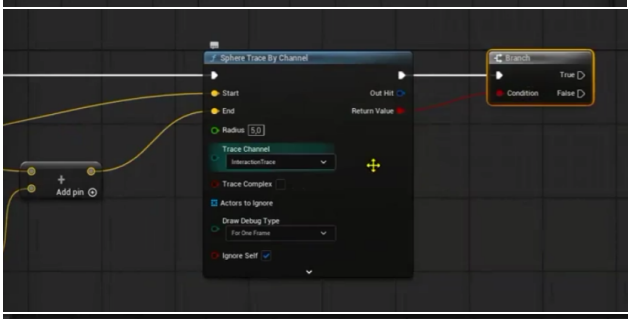





LookForInteractions
Get World Location
Target self
Return Value
Select
False
True
Index
Self
Camera Component
Is Valid
Object
Return Value


Self
Select
False
True
Index
Return Value
Get Forward Vector
Target is Scene Component
Target
Return Value
Camera Component
Add pin
Is Valid
Object
Return Value
Distance Trace




Sphere Trace By Channel
Start
End
Radius 5.0
Trace Channel
InteractionTrace
Trace Complex
Actors to Ignore
Draw Debug Type
For One Frame
Ignore Self
Out Hit
Return Value
Branch
True
Condition
False
Add pin

Out Hit
Return Value
Branch
True
Condition
False
SET
Hit Actor
Break Hit Result
Hit
Blocking Hit
Initial Overlap
Time
Distance
Location

Esecuzione su ogni frame

- Per far funzionare il tracciamento in tempo reale, la funzione `LookForInteractions` deve essere chiamata ad ogni frame.
- Nell'`Event Graph` del Componente, aggiungere il nodo `Event Tick` e collegarlo all'esecuzione di `LookForInteractions`.
- Ora ad ogni frame il componente esegue il trace e aggiorna lo stato dell'interazione.

12.6.4 Creazione dell'Actor interagibile di test

Blueprint Actor `BP_InteractableTest`

- Creare un nuovo `Actor`: `Content Browser` → `Blueprint Class` → `Actor`. Rinominarlo `BP_InteractableTest`.
- Aggiungere un componente `Static Mesh` (es. un `Cube`) e, opzionalmente, un componente `Widget` per indicare visivamente l'interagibilità.

Implementazione dell'interfaccia

- Aprire `BP_InteractableTest` e cliccare su `Class Settings` (in alto a destra).
- Nella sezione `Interfaces` → `Implemented Interfaces`, cliccare `Add` e selezionare `BPI_InteractionRay`.
- Una volta aggiunta l'interfaccia, nel pannello `My Blueprint` sotto `Interfaces` compariranno le tre funzioni: `InteractionStart`, `InteractionEnd`, `Interaction`.
- **Override delle funzioni**:
  - Cliccare con il tasto destro su ciascuna funzione e selezionare `Implement Event`.
  - Per ognuna, collegare un `Print String` con un messaggio identificativo (es. `Inizio Interazione`, `Fine Interazione`, `Interagendo`) e impostare il `Text Color`:
    - `InteractionStart`: verde.
    - `InteractionEnd`: rosso.
    - `Interaction`: giallo/arancione.

Collisione per il canale `InteractionTrace`

- Selezionare il componente `Static Mesh` del `BP_InteractableTest`.
- Nei `Details`, sotto `Collision`, impostare `Collision Preset` su `Custom`.
- Nella lista dei canali, trovare `InteractionTrace` e impostarlo su `Block`.
- Questo permette allo `Sphere Trace By Channel` del componente `BPC_RayInteract` di rilevare l'oggetto.

Test del tracciamento

- Posizionare un'istanza di `BP_InteractableTest` nel livello.
- Avviare il gioco e guardare nella direzione dell'oggetto; si dovrebbero vedere i messaggi di inizio e fine interazione a schermo, che indicano che il sistema di tracciamento e l'interfaccia funzionano correttamente.

12.6.5 Input per l'interazione: IA_Interact

- **Creazione dell'Input Action**: nella cartella degli input, duplicare un'azione esistente (es. IA_Run), rinominarla in IA_Interact. Verificare che abbia i trigger Pressed e Released.
- **Aggiunta all'IMC**: aprire l'IMC_Default, aggiungere una nuova riga, selezionare IA_Interact e assegnare il tasto desiderato (es. E).
- **Nel Character Blueprint** (BP_Bot2): dovrebbe ora essere disponibile l'evento IA_Interact.

12.6.6 Logica di interazione nel Character Blueprint

Ottenere l'Interactable e chiamare Interaction

- Nell'evento IA_Interact (probabilmente sul pin Started):
  1. Usare un nodo DoOnce per evitare che l'interazione venga attivata più volte durante l'animazione.
  2. Ottenere il componente BPC_RayInteract dal personaggio (Get Component by Class o trascinando il componente dalla lista dei componenti nel grafo).
  3. Dal componente, ottenere la variabile Interactable (Get Interactable) e convertirla in Validated Get.
  4. Se Interactable è valido, chiamare il messaggio di interfaccia Interaction su Interactable.

Rotazione verso l'oggetto

- Sempre nel ramo valido, calcolare la rotazione per guardare l'oggetto:
  1. Ottenere la posizione del personaggio: Get Actor Location (target Self).
  2. Ottenere la posizione dell'Interactable: Get Actor Location su Interactable.
  3. Usare Find Look at Rotation (Start = posizione personaggio, Target = posizione Interactable).
  4. Split del risultato e della rotazione attuale del personaggio (Get Actor Rotation → Break Rotator).
  5. Prendere solo il valore Yaw (asse Z) dalla rotazione calcolata e combinarlo con i valori di Pitch e Roll della rotazione attuale (che restano invariati) usando un Make Rotator.
  6. Applicare la nuova rotazione con Set Actor Rotation.

Esecuzione del Montage e gestione input

- **Creazione del Montage**: nel Content Browser, trovare l'animazione ABot_Interact_PressButton, tasto destro → Create Anim Montage. Rinominarlo AM_Bot_Interact.
- **Play Montage**:
  - Usare il nodo Play Montage con target la Skeletal Mesh del personaggio.
  - Selezionare il montaggio creato.
- **Variabile Interacting**:
  - Creare una variabile booleana Interacting nel BP_Bot2.
  - All'inizio del Play Montage, impostare Interacting = True.
  - All'evento On Completed (o On Blend Out) del montaggio, impostare Interacting = False.
- **Disabilitazione dell'input durante l'animazione**:
  - Subito dopo il Play Montage, con Get Player Controller:
    * Disable Input sul Player Controller.

    * `Stop Movement Immediately`.
  - Al completamento del montaggio, con `Get Player Controller` → `Enable Input`.

- **Reset del DoOnce**: al completamento del montaggio, collegare il flusso di esecuzione al `Reset` del `DoOnce` per permettere una nuova interazione.

12.7 Completamento del Sistema di Interazione

12.7.1 Reset del DoOnce e completamento dell'interazione

- Dopo aver eseguito il montage e disattivato/riattivato l'input, è necessario resettare il nodo `DoOnce` per permettere una nuova interazione.

- Al termine del montage (evento `On Completed`), collegare il flusso di esecuzione al pin `Reset` del `DoOnce`.

- Risultato: il personaggio può ora eseguire l'animazione di interazione ogni volta che il tasto viene premuto e l'oggetto è nuovamente valido.

12.8 Widget dell'interazione (press prompt)

12.8.1 Creazione del Widget `VB_InteractUI`

- Creare un nuovo **Widget Blueprint** nella cartella `UI`: `User Interface` → `Widget Blueprint` → rinominarlo `VB_InteractUI`.

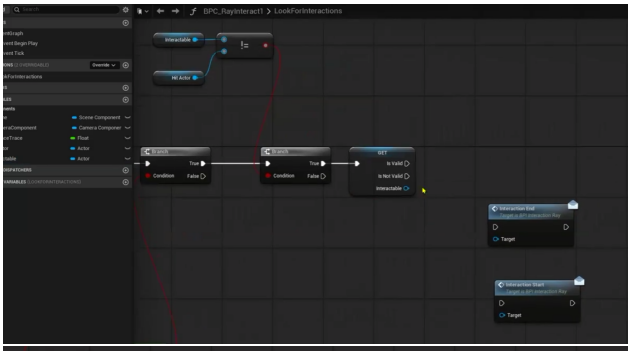
- Nel `Designer`:

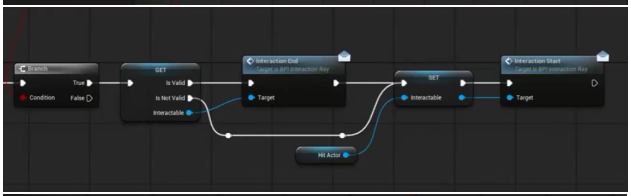
  1. Aggiungere un **Border** (dal pannello `Common`).
  2. Impostare l'opacità del border esterno (alpha) a 0 per renderlo trasparente. Ridimensionarlo a circa 96x96.
  3. Aggiungere un secondo **Border** all'interno del primo.
  4. Nel border interno, impostare il `Brush` → `Tint` su nero e l'alpha a 1.
  5. Sempre nel border interno, sotto `Brush` → `Image`, scegliere uno dei pannelli disponibili (es. `Panel_5`) dagli asset forniti.
  6. Allineare i border e impostare il `Padding` a 0 per un perfetto incastro.

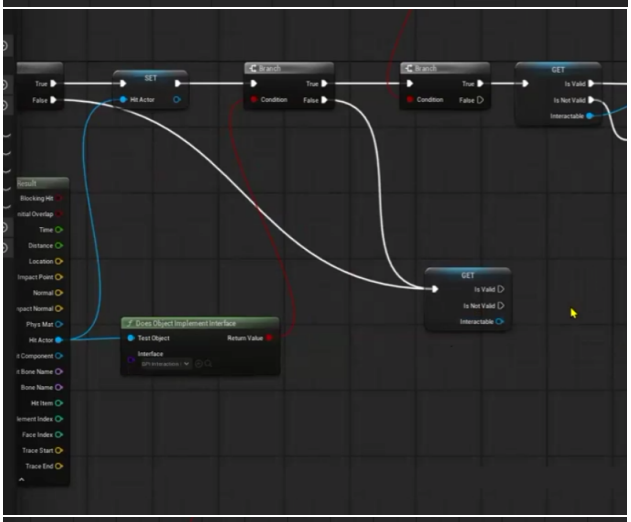
- **Testo del tasto**:

  1. Trascinare un `Text` all'interno del border più interno.
  2. Impostare il testo su `E` (o il tasto scelto per l'interazione).
  3. Centrare il testo: `Horizontal Alignment` e `Vertical Alignment` → `Center`.
  4. Aumentare la dimensione del font (es. 35-45) per renderlo ben visibile.

- **Nota**: se il widget mostra contenuti errati all'apertura, ricrearlo senza usare l'anteprima automatica; nominarlo in modo univoco.







This node is disabled and will not be called.
Drag off pins to build functionality
Event BeginPlay
Event InteractionStart
From BPI Interaction Ray
Print String
In String
Inizio Interazione
Print to Screen
Print to Log
Text Color
Duration
2.0
Key
None
Development Only
Event InteractionEnd
From BPI Interaction Ray
Print String
In String

12.8.2 Visualizzazione e nascondimento del Widget nell'Actor Interactable

- In `BP_InteractableTest`, aggiungere un componente `Widget`.
- Configurare il componente Widget:
  - `Widget Class` → `VB_InteractUI`.
  - `Space` → `Screen` (per renderlo un elemento 2D ancorato allo schermo, non al mondo 3D).
  - `Draw Size` → 100x100 (o leggermente più piccolo).
- Logica di visibilità nell'Event Graph:
  - `Event BeginPlay`: `Set Hidden in Game` → `True` (nascosto di default).
  - `InteractionStart`: `Set Hidden in Game` → `False` (mostra il prompt).
  - `InteractionEnd`: `Set Hidden in Game` → `True` (nasconde il prompt).
  - (Opzionale) `Interaction`: per evitare conflitti con il delay, si può lasciare senza modifiche o gestire un breve delay prima di nascondere il widget.
- Test: avvicinando la telecamera all'oggetto, appare il prompt `E`; distogliendo lo sguardo, scompare.

12.9 Esempio di interazione: Attivazione/Disattivazione di una Luce

12.9.1 Preparazione dell'Actor Interactable con Luce

- In `BP_InteractableTest`, aggiungere un riferimento a una `Point Light` (o altra luce) presente nella scena.
- Per rendere l'esempio modulare, si può creare un nuovo Actor `BP_Activables` con una `Point Light` e le funzioni `Activation` e `Deactivation`.
- Nell'evento `Interaction` (quando il giocatore preme il tasto):
  1. Utilizzare un nodo `Flip Flop`. Questo nodo alterna l'esecuzione tra due rami (A e B) ad ogni chiamata.
  2. Ramo A: chiamare `Activation` sulla luce (accendi).
  3. Ramo B: chiamare `Deactivation` sulla luce (spegni).
  4. Collegare il flusso di esecuzione di `Interaction` al `Flip Flop`.
- Ora, ogni pressione del tasto interazione mentre si guarda l'oggetto accenderà o spegnerà la luce alternativamente.

12.10 Conclusioni del corso e saluti

12.10.1 Riepilogo e utilizzo del sistema di interazione

- Il sistema realizzato è modulare: il componente `BPC_RayInteract` gestisce il tracciamento e la comunicazione tramite interfaccia, mentre l'attore interagibile definisce il proprio comportamento specifico.
- È possibile estendere il sistema a qualsiasi tipo di interazione: pulsanti, ascensori, torrette, luci, porte, ecc. Basta aggiungere l'interfaccia all'attore e implementare le funzioni `InteractionStart`, `InteractionEnd`, `Interaction`.
- Il widget di prompt fornisce un feedback visivo immediato al giocatore.

BP_InteractableTest > EventGraph
Event BeginPlay
Set Hidden in Game
Target
New Hidden
Propagate to Children
Widget
Event InteractionStart
Print String
In String
Inizio Interazione
Print to Screen
Print to Log
Text Color
Duration
Key
None
Development Only
Event InteractionEnd
Fine Interazione
Flip Flop
Activate
Deactivate
Light
Interagendo
BLUEPRINT

12.10.2 Feedback e supporto futuro

- Il docente chiede feedback sul corso e si dichiara disponibile anche dopo la fine delle lezioni per domande o assistenza sui progetti degli studenti.
- Per qualsiasi problema o dubbio, gli studenti possono scrivere sul server Discord del corso, dove sia docenti che compagni potranno rispondere.
- Viene incoraggiato l'uso delle validazioni (`IsValid`, `Validated Get`) per una programmazione robusta.

12.10.3 Saluti finali

- Il docente ringrazia gli studenti per l'attenzione e la partecipazione, augurando buon proseguimento con i loro progetti su Unreal Engine.
- Le lezioni successive saranno tenute da Davide e riguarderanno il Level Design.

Chapter 13

Fondamenti di Level Design in Unreal Engine: Dal Concept al Blackout

13.1 Introduzione e panoramica del corso

13.1.1 Saluti e comunicazioni iniziali

- Il docente (Davide) chiede agli studenti come si stanno trovando con Unreal Engine.
- Viene menzionato il problema del surriscaldamento del computer durante l'uso di Unreal e lo streaming; si consiglia l'uso di basi di raffreddamento.
- Si chiede se qualcuno abbia sperimentato con l'engine al di fuori delle lezioni precedenti.

13.1.2 Struttura delle lezioni successive

- Le lezioni restanti (quattro) si concentreranno sul **flusso di lavoro (workflow)** per la creazione di livelli.
- Argomenti previsti:
  - Teoria di base del level design.
  - **Block out** con *Geometry Brush*.
  - Cenni al *Modeling Mode* per le mesh statiche (possibile rinvio).
  - Uso delle modalità **Landscape** e **Foliage** per esterni/interni.
  - Fondamenti di **illuminazione** e **post-produzione**.
  - **Dressing** (arricchimento estetico dei livelli).
  - Esercitazione pratica finale con creazione di livelli, feedback reciproco e condivisione degli eseguibili su Discord.

13.2 Principi fondamentali del Level Design

13.2.1 Ispirazione: realtà vs altri videogiochi

- **La fonte principale deve essere la realtà.**
  - È originale e **non filtrata**.
  - Un gioco come *Spider-Man* ha adattato la mappa di Manhattan al proprio *game design* (traversal veloce, edifici alti), scartando le aree non funzionali.

  - Se il nostro gioco ha esigenze di gameplay diverse (es. rapine in interni), la mappa di Spider-Man sarebbe irrilevante; meglio ispirarsi direttamente alla realtà.

- **Conseguenza: il level design deve essere guidato dal Game Design.**
  - Le meccaniche di gioco (movimento, combattimento, interazione) determinano la struttura del livello.
  - Esempio: in *Assassin's Creed Brotherhood* l'acqua permette di aggirare le aree sorvegliate, sfruttando le nuove meccaniche.

13.2.2 Documentazione e Layout

- **Level Design Document**: documento che descrive la struttura del livello in modo chiaro, comprensibile a tutto il team ma non eccessivamente restrittivo per chi lo realizza.
- **Layout / Piantina**:
  - Fase preliminare al block out, fondamentale per prevenire errori.
  - Consiste nel disegnare una mappa con percorsi, punti di interesse, ostacoli, ecc.
  - Esempio di pianta con legenda: spawn point, zone di combattimento, punti di raccolta oggetti, scale.
  - Esempio di mappa in stile Metroidvania: percorsi lineari, aree bloccate da potenziamenti, shortcut da aprire, stanze del boss, checkpoint.
  - L'obiettivo è **strutturare l'esperienza del giocatore**, guidandolo sui binari desiderati.

13.2.3 La Gym (Test Level)

- Livello dedicato al test delle **capacità del personaggio**.
- Serve a misurare parametri come:
  - Altezza e lunghezza del salto.
  - Velocità di camminata e corsa.
  - Distanza massima raggiungibile con corsa+salto.
  - Dimensioni delle porte e passaggi.
- Permette di calibrare le meccaniche di gioco (es. modificare i parametri del personaggio) e di progettare ostacoli e percorsi di conseguenza.
- Ogni gioco richiede una gym costruita su misura per le proprie esigenze di gameplay.

13.3 Principi di Level Design

13.3.1 La Gym (Test Level) – continuazione

Dimensioni delle porte e telecamera

- Nei giochi in terza persona si tendono a evitare porte di altezza realistica (es. 2 metri) perché il character (spesso alto circa 1,80 m) e la telecamera alle spalle rischiano di **clippare** con l'architrave, creando un fastidioso effetto di abbassamento/rialzo improvviso della visuale.
- Per questo motivo, in giochi come *Uncharted* o *The Last of Us*, le porte sono spesso "falsamente" più alte del normale, per garantire un passaggio fluido alla telecamera.
- La gym serve anche a testare e definire queste misure "di comodo" in base alle proprie meccaniche.

Parametri da testare nella gym

Oltre all'altezza del salto, la gym permette di misurare e calibrare molti altri valori in base al gameplay desiderato:

- **Lunghezza del salto**: distanza orizzontale massima.
- **Distanza di corsa sul muro** (se presente): es. massimo di 12 metri prima che la meccanica diventi rischiosa.
- **Angolo di camminabilità**: inclinazione massima delle superfici su cui il personaggio può camminare o arrampicarsi (es. 45°, 60°).
- **Altezza del personaggio accovacciato**: per progettare passaggi stretti.

L'importanza di ciascun parametro dipende dal genere di gioco: in un platform saranno cruciali salto e corsa; in un puzzle potrebbero esserlo meno, mentre sarà più rilevante il raggio di interazione con leve o porte.

13.4 Il Block Out

13.4.1 Definizione e scopo

- Il **block out** (o *grey boxing*) è la fase successiva alla creazione del layout 2D.
- Consiste nel ricostruire il livello all'interno del motore di gioco **utilizzando esclusivamente geometrie semplici** (cubi, cilindri, piani), cercando di essere il più fedeli possibile al risultato finale immaginato.
- **Obiettivi principali:**
  - Testare il **gameplay** (movimento, combattimento, interazioni) all'interno dello spazio progettato, prima di dedicare risorse alla parte estetica.
  - Valutare il **flow** e il **pacing**: la camminata è troppo lunga o troppo breve? L'arena è abbastanza grande? La narrazione ambientale funziona?
  - Posizionare **landmark** (punti di riferimento visivi) e **oggetti di interesse** (forzieri, leve, nemici) per guidare il giocatore e testare la leggibilità dello spazio.

13.4.2 Esempio: Case Study del Block Out di God of War (2018)

- Un utente ha ricreato come *ipotesi* il block out della prima area di *God of War* (2018).
- Elementi mostrati nel video:
  - Uscita dalla casa di Kratos: presenza di un portale con un simbolo.
  - Illuminazione di base per suggerire l'atmosfera.
  - Landmark come un albero caduto, un doppio albero a formare un arco, una scalinata.
  - Assenza di Atreus e di meccaniche di combattimento non ancora implementate (es. salto gestito poi con trigger automatici).
  - Posizionamento concettuale di ricompense (un forziere) per premiare l'esplorazione.
  - Oggetti distruttibili (es. tronco da spaccare con l'ascia) segnalati come future interazioni.
  - Uso di **bandiere** come elementi guida per il percorso del giocatore.
  - Definizione dell'arena di combattimento prima ancora di avere il combat system definitivo.
- Il block out permette quindi di "prevedere" e strutturare l'esperienza di gioco prima di passare alla fase di *dressing* (arricchimento estetico).

13.5 Utilizzo pratico della Gym e del Block Out in Unreal Engine

13.5.1 La griglia come strumento di misura

- Per prendere misure precise, è fondamentale utilizzare materiali con **griglia** (es. il materiale `Grid` fornito) sulle geometrie di test.
- La griglia fornisce un riferimento metrico immediato: ad esempio, ogni quadrato grande può corrispondere a 1 metro, ogni quadretto a 20 cm.
- Le impostazioni di **snap** (aggancio alla griglia) permettono di spostare gli oggetti con incrementi fissi (es. 10 cm), garantendo precisione nelle misurazioni.

13.5.2 Misurazione dello sprint

- Si posiziona il personaggio su una superficie piana con griglia e lo si fa scattare con la stamina piena fino all'esaurimento.
- Si misura la distanza percorsa contando i metri sulla griglia.
- **Esempio pratico**: il robottino di test ha percorso 26 metri in sprint con una barra di stamina completa.
- **Utilità di questa misura**:
  - In un **inseguimento**: il percorso tra un ostacolo e l'altro (o tra il punto di partenza e un rifugio) non può superare la distanza massima di sprint, altrimenti il giocatore viene raggiunto/non raggiunge il bersaglio.
  - In un **horror** o **action game**: serve a calibrare la distanza tra "zone sicure" o tra un power-up di stamina e il successivo, per mantenere la tensione o il ritmo desiderato.

13.5.3 Test delle altezze di salto e valutazione del *feeling*

- Si creano una serie di piattaforme/cubi ad altezze crescenti (es. 1 m, 1,40 m, 1,60 m, 1,80 m, 2 m) per testare il salto massimo del personaggio.
- **Valutazione qualitativa del salto**:
  - **Salto troppo basso (es. 1 m)**: il personaggio sembra "scivolare" sopra l'ostacolo (a causa della capsula di collisione arrotondata), il movimento non è soddisfacente né chiaro. **Pessima comunicazione**.
  - **Salto al limite (es. 1,40 m o oltre)**: il bordo della piattaforma arriva circa a metà altezza del personaggio. È chiaro quando un salto è possibile e quando non lo è. **Buona comunicazione e feeling**.
- Queste osservazioni portano a modificare i parametri del `CharacterMovement` per ottenere il comportamento desiderato.

13.5.4 Modifica dei parametri del Character Movement

- Per cambiare il feeling e le capacità del personaggio, si agisce sui parametri del componente `CharacterMovement` all'interno del Blueprint del personaggio.
- Alcuni parametri modificabili:
  - **Accelerazione**: quanto rapidamente il personaggio raggiunge la velocità massima.
  - **Gravità**: quanto è forte la forza di gravità che agisce sul personaggio (influenza la traiettoria e l'altezza del salto).

  - **Velocità di camminata/corsa** (`Max Walk Speed`).
  - **Altezza del salto** (`Jump Z Velocity`).

- La gym serve proprio a testare "a vivo" l'effetto di queste modifiche per trovare il bilanciamento perfetto per il proprio gioco.

13.6 Lezione di Level Design – Davide (seconda parte)

13.6.1 Struttura e obiettivi delle lezioni

- Le lezioni si concentreranno sul **flusso di lavoro (workflow)** per la creazione di livelli in Unreal Engine.

- Argomenti previsti:

  - Teoria di base del level design.
  - **Block out** e uso delle *Geometry Brush*.
  - Cenni al *Modeling Mode* (se il tempo lo consente).
  - Modalità **Landscape** e **Foliage** per esterni/interni.
  - Fondamenti di **illuminazione** e **post-produzione**.
  - **Dressing** (arricchimento estetico dei livelli).
  - Esercitazione pratica finale: creazione di livelli, feedback reciproco e condivisione degli eseguibili su Discord.

13.6.2 Principi di Level Design

Realtà vs altri videogiochi come fonte

- La fonte principale per la creazione di un livello deve essere la **realtà**, perché è originale e non filtrata dalle scelte di game design di altri titoli.

- Esempio: *Spider-Man* ha adattato Manhattan al proprio gameplay (swinging tra palazzi), eliminando le zone poco funzionali. Se il nostro gioco ha meccaniche diverse, quel layout non è utile.

- **Il level design deve essere guidato dal Game Design.** Le meccaniche di gioco determinano la struttura del livello.

Documentazione e Layout

- Prima di costruire in engine si realizza un **Layout / piantina** (2D) per prevenire errori e strutturare il percorso del giocatore.

- Il layout può includere: spawn point, zone di combattimento, punti di raccolta oggetti, aree inaccessibili senza determinati power-up, shortcut.

- Esempio: mappa in stile Metroidvania con percorsi bloccati, potenziamenti, stanze del boss, checkpoint.

- Un **Level Design Document** descrive il livello in modo chiaro per tutto il team, senza essere troppo restrittivo.

La Gym (Test Level)

- Livello dedicato a testare e misurare le **capacità del personaggio**.
- Parametri da testare:
  - Altezza e lunghezza del salto (da fermo e in corsa).
  - Velocità di camminata e corsa.
  - Distanza massima di sprint con la barra della stamina.
  - Dimensioni delle porte (per evitare clipping della telecamera in terza persona).
  - Altezza massima di uno scalino scavalcabile automaticamente (`Max Step Height`).
  - Inclinazione massima delle superfici camminabili (`Walkable Floor Angle`).
- La gym permette di calibrare i parametri del `CharacterMovement` e di progettare ostacoli e percorsi coerenti con le meccaniche.

13.6.3 Il Block Out

Definizione e scopo

- Il **block out** (o *grey boxing*) consiste nel ricostruire il livello in engine usando solo **geometrie semplici** (cubi, piani, cilindri), senza curare l'estetica.
- **Obiettivi:**
  - Testare il **gameplay** (movimento, interazioni, combattimento) nello spazio progettato.
  - Valutare **flow** e **pacing**: la camminata è troppo lunga? L'arena è abbastanza grande?
  - Posizionare **landmark** e oggetti di interesse (forzieri, leve, nemici) per guidare il giocatore e testare la leggibilità dello spazio.

Esempio: Case Study God of War (2018)

- Ricostruzione ipotetica del block out iniziale:
  - Uscita dalla casa: portale con simbolo, illuminazione di base.
  - Landmark: albero caduto, doppio albero ad arco, scalinata.
  - Assenza di Atreus e di meccaniche non ancora implementate (salto gestito con trigger).
  - Posizionamento concettuale di ricompense (forziere) per l'esplorazione.
  - Oggetti distruttibili (tronco) segnalati come future interazioni.
  - Uso di **bandiere** come elementi guida per il percorso.
  - Definizione dell'arena di combattimento prima del combat system definitivo.

13.6.4 Lavorare con le misure: griglia e snap

- Utilizzare materiali con **griglia** sulle superfici di test per avere un riferimento metrico immediato (es. 1 quadrato = 1 metro).
- Le impostazioni di **snap** permettono di spostare gli oggetti a incrementi fissi (es. 10 cm), garantendo precisione.
- **Esempio di misurazione dello sprint**: posizionare il personaggio, correre fino a esaurimento stamina, contare i metri sulla griglia (es. 26 metri).
- **Utilità**: in un inseguimento, la distanza tra ostacoli non può superare quella massima di sprint.

13.6.5 Test e regolazione dei parametri di movimento

Valutazione del salto

- Creare piattaforme ad altezze crescenti (1 m, 1,40 m, 1,60 m, ...).
- Osservare il **feeling**: un salto su un ostacolo troppo basso (es. 1 m) appare come uno "scivolamento" a causa della capsula di collisione (**pessima comunicazione**). Un salto al limite (bordo a metà altezza personaggio) è chiaro e soddisfacente (**buona comunicazione**).

Modifica dei parametri del CharacterMovement

- Nel Blueprint del personaggio, selezionare il componente `CharacterMovement`. Parametri chiave:
  - **Gravità**: `Gravity Scale` (es. per simulare la Luna).
  - **Massa**: `Mass` (influenza il momentum).
  - **Camminata**: `Max Step Height` (altezza massima di un gradino scavalcabile), `Walkable Floor Angle`, `Max Walk Speed`.
  - **Salto**: `Jump Z Velocity` (forza verticale applicata al salto; default 420 cm/s, valori più alti aumentano l'altezza).
  - **Controllo aereo**: `Air Control` (quanto il giocatore può muoversi in volo).
- Registrare le misure ottenute dopo le modifiche (es. Walk Jump = 4 m, Sprint Jump = 6 m) per progettare i salti del livello.

13.6.6 Geometry Brush

Introduzione e tipi

- Le **Geometry Brush** sono attori utilizzati principalmente per il block out. Permettono di creare forme semplici e modificarle parametricamente.
- Tipi disponibili: `Box`, `Cone`, `Cylinder`, `Sphere`, `Linear Stairs`, `Curved Stairs`, `Spiral Stairs`.
- A differenza delle *Static Mesh*, le brush hanno **facce** editabili singolarmente (materiali, UV), ma sono più pesanti in termini di performance.

Proprietà e impostazioni comuni

- **Brush Settings** (nel pannello Details):
  - **Brush Type**: `Additive` (aggiunge volume) o `Subtractive` (scava buchi nelle brush additive).
  - **Hollow**: rende la brush cava, con un determinato `Wall Thickness` (spessore pareti).
  - **Tessellation**: suddivide la superficie in triangoli, utile per editing successivo.
- È preferibile modificare le dimensioni tramite i **parametri** (X, Y, Z, Raggio, Altezza) piuttosto che con lo scaling diretto, per mantenere le proporzioni delle facce.

Creazione rapida di ambienti

- Combinando brush additive e sottrattive si possono creare stanze, finestre, aperture sul soffitto (es. per cadere da un livello all'altro).
- **Ottimizzazione**: una volta soddisfatti della forma, convertire le brush in una **Static Mesh** (tasto destro → `Convert to Static Mesh`) per ridurre il carico di memoria.
- Salvare gli asset di block out in una cartella dedicata (es. `BlockOutAssets`).

Parametri specifici per tipo

- **Cone** e **Cylinder**: Altezza (`Z`), Raggio esterno, Raggio interno (se `Hollow`), Numero di lati (`Sides`).
- **Sphere**: Raggio, Numero di lati (tessellation).
- **Scale**:
  - **Lineari**: Lunghezza scalino (`Step Length`), Altezza scalino (`Step Height`), Larghezza (`Step Width`), Numero scalini (`Num Steps`), Altezza aggiuntiva primo scalino (`Add to First Step`).
  - **Curve/Spirale**: Parametri aggiuntivi come Raggio interno (`Inner Radius`), Angolo di curva (`Angle of Curve`, default 90°), `Counterclockwise` per invertire la direzione.

13.7 Illuminazione in Unreal Engine

13.7.1 Tipi di luce disponibili

- **Directional Light**: simula la luce solare. La sua posizione e scala non influenzano l'illuminazione; ciò che conta è esclusivamente la sua **rotazione**.
- **Skylight**: luce ambientale che cattura e diffonde la luce proveniente dal cielo e dall'ambiente circostante.
- **Point Light**: emette luce in tutte le direzioni da un singolo punto, come una lampadina.
- **Spotlight**: emette luce in un cono, con una direzione e un angolo di apertura definiti.
- **Rect Light**: emette luce da una superficie rettangolare, utile per simulare luci al neon o riflessi su schermi.

13.7.2 Il parametro Mobility

Tutte le luci possiedono il parametro `Mobility`, che determina come la luce viene calcolata e come interagisce con la scena e i personaggi. I tre stati sono:

- **Static** (Statica):
  - La luce viene "cucinata" (*baked*) direttamente sulle texture della scena durante la fase di build dell'illuminazione.
  - È il tipo di luce più **leggero** in termini di performance.
  - **Non può essere spostata** a runtime: se si muove l'attore, la luce rimane impressa dove era stata calcolata.
  - **Non interagisce** con i personaggi o altri oggetti in movimento (non proietta ombre dinamiche).
  - **Utilizzo ideale**: illuminazione d'atmosfera fissa in aree dove non è prevista interazione con il giocatore, come zone inaccessibili o sfondi lontani.
- **Stationary** (Stazionaria):
  - Luce che non può essere spostata dopo la build (se mossa, l'illuminazione si "rompe" e deve essere ricalcolata).
  - **Interagisce** con i personaggi e gli oggetti mobili, proiettando ombre dinamiche corrette.
  - **Utilizzo ideale**: la maggior parte delle luci in un livello (lampioni, lampade, fari), dove la posizione è fissa ma si vuole che il giocatore vi interagisca passandoci davanti.
- **Movable** (Mobile):
  - Luce completamente **dinamica**.

  - Può essere **spostata** a runtime in qualsiasi momento e l'illuminazione si aggiornerà di conseguenza.
  - **Interagisce** sempre con personaggi e oggetti.
  - **Utilizzo ideale**: torce, luci su veicoli, luci che si accendono/spengono o che fanno parte di meccaniche di gioco.
  - **Nota**: è il tipo di luce più costoso in termini di performance, quindi va usata con parsimonia.

13.7.3 Comportamento di Directional Light e Skylight

- La `Directional Light` e la `Skylight` sono luci "ambientali" che riguardano l'intera scena.
- **Directional Light**: spostare o scalare l'attore non produce alcun effetto visibile. L'unica trasformazione che influisce sull'illuminazione è la **rotazione**, che simula il movimento del sole.
- **Skylight**: cattura il colore del cielo e dell'orizzonte per diffonderlo uniformemente, contribuendo all'illuminazione ambientale globale.

13.7.4 Point Light e Spotlight: parametri e utilizzo

Confronto prestazionale

- **Spotlight** è generalmente più leggera di una **Point Light**.
- **Motivo**: la Point Light emette in tutte le direzioni (360°), costringendo il motore a calcolare i rimbalzi su tutte le superfici circostanti. La Spotlight, avendo un cono d'illuminazione direzionale, limita il numero di calcoli.
- In scene affollate, una Point Light va posizionata con attenzione perché colpisce indiscriminatamente ogni oggetto nel suo raggio, compresi quelli alle spalle.

Parametri chiave

- **Intensità**: potenza luminosa.
- **Light Color**: colore della luce (utile copiare il codice esadecimale `sRGB` per uniformare più luci).
- **Attenuation Radius**:
  - **Point Light**: è il raggio della sfera entro cui la luce ha effetto. Anche con intensità bassa, tutti gli oggetti all'interno del raggio vengono calcolati per ombre e illuminazione.
  - **Spotlight**: è la lunghezza del cono luminoso.
- **Spotlight** aggiuntivi:
  - **Outer Cone Angle**: ampiezza del cono luminoso esterno.
  - **Inner Cone Angle**: ampiezza del cono interno, dove la luce è più intensa e meno attenuata (simula l'effetto "torcia").
- **Source Radius / Length**: permette di allargare la sorgente luminosa (es. da puntiforme a capsula/linea), distribuendo la luce in modo più omogeneo (utile per simulare luci al neon o grandi lampade).
- **Soft Source Radius**: gestisce l'attenuazione morbida ai bordi della sorgente.

Luci con "X rossa" (limite di overlapping)

- Quando troppe luci si sovrappongono (raggi di attenuazione compenetranti), l'engine può rinunciare a calcolare le ombre per alcune di esse, mostrando una **X rossa** sull'icona della luce.
- La luce continua a illuminare, ma non proietta ombre.
- **Soluzioni**:
  - Disabilitare manualmente il `Cast Shadow` sulla luce in questione.
  - Ridurre l'`Attenuation Radius` per limitare la sovrapposizione.
- **Buona pratica**: usare poche luci significative piuttosto che molte luci piccole. Ad esempio, per un lungo corridoio con una fila di luci, è meglio utilizzare poche sorgenti allungate (`Source Length`) che tante Point Light.

13.7.5 Interazione tra luci e Post Process (cenni)

- Alcuni effetti visivi legati alla luce (es. **abbagliamento guardando direttamente la sorgente**, **God Rays** / fasci volumetrici) non possono essere ottenuti unicamente con le impostazioni delle luci.
- Questi effetti richiedono l'uso del **Post Process Volume**, che modifica il modo in cui la telecamera percepisce la scena illuminata.
- Esempi di effetti gestiti dal Post Process:
  - **Bloom**: bagliore diffuso attorno alle sorgenti luminose.
  - **Lens Flare**: riflessi interni alla lente della telecamera.
  - **God Rays** (raggi crepuscolari): fasci di luce volumetrica che attraversano l'ambiente (es. luce che entra da una finestra).
- La post-produzione è complementare all'illuminazione e sarà approfondita quando si lavorerà su una scena più completa, non solo geometrica.

13.8 Lezione di Level Design – Davide

13.8.1 Introduzione e organizzazione

- Il docente Davide prende il posto di Igor per le ultime quattro lezioni, focalizzate sul **flusso di lavoro** per la creazione di livelli in Unreal Engine.
- **Argomenti previsti**:
  - Teoria di base del level design.
  - **Block out** e *Geometry Brush*.
  - Cenni al *Modeling Mode* (se possibile).
  - Modalità **Landscape** e **Foliage** per esterni/interni.
  - Fondamenti di **illuminazione** e **post-produzione**.
  - **Dressing** (arricchimento estetico).
  - Esercitazione pratica finale: creazione di livelli con feedback reciproco.
- Incoraggiamento a interrompere con domande e a condividere i propri esperimenti sul server Discord.

13.8.2 Principi di Level Design

Ispirazione: realtà vs altri videogiochi

- La fonte principale per un livello deve essere la **realtà**, perché originale e non filtrata da scelte di game design altrui.
- Esempio: *Spider-Man* ha modellato Manhattan per il traversal rapido (edifici alti); se il nostro gioco ha altre meccaniche, quel layout non è utile.
- **Il level design deve essere guidato dal Game Design.** Le meccaniche (movimento, interazioni, combattimento) determinano la struttura del livello.
- Esempio: in *Assassin's Creed Brotherhood*, l'acqua permette di aggirare le guardie, sfruttando nuove meccaniche.

Layout e documentazione

- **Layout (piantina 2D)**: si disegna prima di iniziare il block out in engine, per prevenire errori e definire il percorso del giocatore.
- Può includere: spawn point, zone di combattimento, punti di raccolta oggetti, aree bloccate da power-up, scorciatoie, stanze del boss, checkpoint.
- Esempio: mappa in stile Metroidvania con percorsi guidati.
- **Level Design Document**: descrizione chiara del livello, sufficientemente dettagliata da essere eseguibile da altri, ma non troppo restrittiva.

La Gym (Test Level)

- Livello dedicato a testare e misurare le **capacità del personaggio**, fondamentale per calibrare il gameplay.
- Parametri da testare:
  - Altezza e lunghezza del salto (da fermo e in corsa).
  - Velocità di camminata e corsa, distanza massima di sprint con la stamina.
  - Altezza massima di uno scalino scavalcabile (`Max Step Height`).
  - Inclinazione massima delle superfici camminabili (`Walkable Floor Angle`).
  - Dimensioni delle porte (per evitare clipping della telecamera in terza persona: nei giochi con visuale alle spalle, le porte sono spesso "falsamente" più alte del reale).
- La gym permette di calibrare i parametri del `CharacterMovement` e progettare ostacoli coerenti.

13.8.3 Il Block Out

- Il **block out** (o *grey boxing*) è la ricostruzione del livello in engine usando solo **geometrie semplici** (cubi, piani, cilindri), senza curare l'estetica.
- **Obiettivi**:
  - Testare il **gameplay** nello spazio progettato (movimento, interazioni, combattimento).
  - Valutare **flow** e **pacing**: durata delle camminate, dimensioni delle arene, chiarezza dei percorsi.
  - Posizionare **landmark** (punti di riferimento) e oggetti di interesse (forzieri, leve, nemici) per guidare il giocatore e testare la leggibilità dello spazio.
- Esempio citato: ricostruzione ipotetica del block out di *God of War* (2018), con uso di luci di base, bandiere come guida, elementi distruttibili segnalati, arene di combattimento predefinite prima ancora del combat system.

13.8.4 Lavorare con le misure: griglia e snap

- Per misurare è fondamentale usare materiali con **griglia** (es. materiale `Grid`) e le impostazioni di **snap** (aggancio alla griglia, es. incrementi di 10 cm).
- Esempio: misurazione della distanza di sprint: si posiziona il personaggio, si corre fino a esaurire la stamina, si contano i metri sulla griglia (es. 26 m).
- **Utilità**: in un inseguimento, la distanza tra ostacoli non può superare la distanza massima di sprint.

13.8.5 Test e regolazione dei parametri del Character Movement

Test del salto e feeling

- Si creano piattaforme ad altezze crescenti (1 m, 1.40 m, 1.60 m, ...).
- **Valutazione qualitativa**:
  - Salto su ostacolo troppo basso (es. 1 m): il personaggio "scivola" sopra a causa della capsula di collisione arrotondata. **Pessima comunicazione**.
  - Salto al limite (bordo a metà altezza del personaggio): chiaro e soddisfacente. **Buona comunicazione**.

Parametri principali del `CharacterMovement`

- **Gravità**: `Gravity Scale` (es. per simulare la Luna).
- **Massa**: `Mass` (influenza il momentum).
- **Camminata**:
  - `Max Step Height`: altezza massima di un gradino scavalcabile automaticamente.
  - `Walkable Floor Angle`: inclinazione massima delle superfici.
  - `Max Walk Speed`: velocità di camminata/corsa.
- **Salto**:
  - `Jump Z Velocity`: forza verticale applicata al salto (default 420 cm/s). Aumentandola si ottiene un salto più alto.
  - `Air Control`: controllo del personaggio in volo (es. 0 = nessun controllo, 1 = controllo totale).
- Annotare le misure ottenute (es. Walk Jump = 4 m, Sprint Jump = 6 m) per progettare i salti del livello.

13.8.6 Geometry Brush

Tipi e caratteristiche

- Le **Geometry Brush** sono attori per il block out: `Box`, `Cone`, `Cylinder`, `Sphere`, `Linear/Curved/Spiral Stairs`.
- Si trovano nel pannello `Place Actors` → `Geometry`.
- Rispetto alle Static Mesh, sono più **malleabili** (hanno facce editabili singolarmente) ma più **pesanti** in termini di performance.
- È preferibile modificarne le dimensioni tramite i **parametri** nei `Brush Settings` piuttosto che con lo scaling diretto, per mantenere le proporzioni delle facce.

Impostazioni comuni (`Brush Settings`)

- **Brush Type**:
  - `Additive`: aggiunge volume.
  - `Subtractive`: scava buchi nelle brush additive (es. per finestre, porte, aperture).
- **Hollow**: rende la brush cava con uno spessore di parete (`Wall Thickness`).
- **Tessellation**: suddivide la superficie in triangoli per editing più fine.

Creazione rapida di ambienti

- Combinando brush additive e sottrattive si creano rapidamente stanze, corridoi, aperture.
- Per ottimizzare, una volta soddisfatti della forma, convertire la selezione in una **Static Mesh** (tasto destro → `Convert to Static Mesh`) e salvarla in una cartella dedicata (es. `BlockOutAssets`).

Parametri specifici per tipo

- **Box**: X, Y, Z.
- **Cone / Cylinder**: Altezza (Z), Raggio esterno, Raggio interno (se Hollow), Numero di lati (`Sides`).
- **Sphere**: Raggio, Numero di lati.
- **Scale**:
  - **Lineari**: `Step Length`, `Step Height`, `Step Width`, `Num Steps`, `Add to First Step`.
  - **Curve / Spirale**: `Inner Radius`, `Angle of Curve` (default 90°), `Counterclockwise`, numero di scalini per 360°, possibilità di rendere soffitto/pavimento inclinati.

Brush Editing Mode

- Attivando la modalità `Brush Editing` (dal menu `Selection Mode`) si possono manipolare direttamente vertici, spigoli e facce della brush.
- Strumento `Split`: taglia la geometria creando nuovi vertici e spigoli per editing più complesso.
- Attenzione: la modifica è temporanea; una volta riconvertita o deselezionata, la brush torna alla forma parametrica originale.

13.9 Illuminazione in Unreal Engine

13.9.1 Tipi di luci disponibili

- **Directional Light**: simula la luce solare. La posizione e scala sono irrilevanti; conta solo la **rotazione**.
- **Skylight**: luce ambientale che diffonde il colore del cielo e dell'ambiente.
- **Point Light**: emette in tutte le direzioni (360°) da un punto, come una lampadina.
- **Spotlight**: emette in un cono direzionale, come una torcia o un faretto.
- **Rect Light**: emette da una superficie rettangolare, utile per insegne al neon, riflessi su schermi TV, luci industriali.

13.9.2 Il parametro `Mobility`

- **Static** (Statica):
  - Luce "cucinata" (*baked*) sulle texture durante la build.
  - **Molto leggera**, non può essere spostata, non interagisce con i personaggi (non proietta ombre dinamiche).
  - Uso: illuminazione d'atmosfera fissa in aree senza interazione (sfondi, zone inaccessibili).
- **Stationary** (Stazionaria):
  - Posizione fissa dopo la build (se spostata, va ricalcolata).
  - **Interagisce** con i personaggi (ombre dinamiche).
  - Uso: la maggior parte delle luci fisse (lampioni, lampade da soffitto, torce a parete).
- **Movable** (Mobile):
  - Completamente **dinamica**: può essere spostata a runtime, interagisce sempre.
  - Uso: torce impugnate, fari di veicoli, luci di meccaniche di gioco.
  - **Più costosa** in termini di performance; usare con parsimonia.

13.9.3 Comportamento delle luci ambientali

- **Directional Light**: la rotazione determina l'angolo del sole. Non vanno mai usate due directional nella stessa scena.
- **Skylight**: contribuisce al mood e all'occlusione ambientale. Anche qui, una sola per scena.

13.9.4 Point Light e Spotlight: parametri e buone pratiche

Confronto prestazionale

- La **Spotlight** è generalmente più leggera della Point Light, perché il cono limita i rimbalzi da calcolare rispetto all'emissione a 360°.

Parametri chiave

- **Intensità**: potenza luminosa.
- **Light Color**: colore della luce (copiare il codice `sRGB` per uniformare più luci).
- **Attenuation Radius**:
  - Point Light: raggio della sfera di influenza.
  - Spotlight: lunghezza del cono.
- **Spotlight** aggiuntivi:
  - **Outer Cone Angle**: ampiezza del cono esterno.
  - **Inner Cone Angle**: ampiezza del cono interno (luce piena, senza attenuazione).
- **Source Radius / Length**: allarga la sorgente (utile per luci al neon o grandi lampade).
- **Soft Source Radius**: sfuma i bordi della sorgente.

Luci con "X rossa" (limite di overlapping)

- Se troppe luci si sovrappongono, l'engine rinuncia a calcolare le ombre per alcune, mostrando una **X rossa**.
- **Soluzioni**: ridurre il raggio di attenuazione, disabilitare manualmente il `Cast Shadow` su luci poco importanti, o usare meno luci ma più grandi.
- In un corridoio, meglio usare una sorgente allungata che tante Point Light.

13.9.5 Rect Light e riflessi

- **Rect Light**: emette da un rettangolo. Parametro `Barn Door Angle` simula ostacoli ai lati per fasci di luce più tagliati.
- Utile per simulare insegne al neon, schermi, luci industriali.
- Per i **riflessi** su superfici lucide (metalli, specchi, pozzanghere), si usano i **Reflection Capture** (Box, Sphere), che vanno buildati per catturare l'ambiente.
- I materiali devono avere `Specular` e `Roughness` impostate per essere riflettenti.

13.9.6 Post Process (cenni)

- Il **Post Process Volume** modifica la resa visiva della telecamera (non la luce in sé).
- Effetti ottenibili: **Bloom** (bagliore), **Lens Flare** (riflessi lente), **God Rays** (raggi volumetrici), **aberrazione cromatica**, **profondità di campo**, **distorsione**.
- La post-produzione sarà approfondita quando si lavorerà su una scena con dressing, non solo geometrie.

13.9.7 Build dell'illuminazione

- Luci `Static` e `Stationary` richiedono il **build** per essere calcolate correttamente; altrimenti rimangono in preview.
- La qualità del build (Preview, Medium, High, Production) determina precisione e tempi.
- In Unreal 4, la mancata build generava la scritta "Lighting needs to be rebuilt" e icone `Preview`. In Unreal 5, il sistema è più immediato.
- Si può buildare selettivamente (solo luci, riflessi, visibilità) o l'intero livello.

13.10 Assegnazione per la prossima lezione

- **Gym personale**: testare i parametri del proprio robottino (salto, sprint, velocità, controllo aereo, pendenza massima, ecc.) e segnare i valori.
- **Modifica parametri del** `CharacterMovement`: adattare il personaggio al proprio gameplay (es. renderlo più lento/veloce, salto più alto, gravità differente).
- **Block out**: creare un piccolo livello con le Geometry Brush (e Static Mesh) che contenga **da 3 a 5 collezionabili**, ognuno raggiungibile in modo diverso (salti, enigmi, schivare la torretta, uso di ascensori, ecc.).
- **Obiettivo**: iniziare a pensare a come i parametri di movimento influenzano l'esperienza e a progettare piccole sfide per il giocatore.
- Condividere i progressi su Discord; la prossima lezione si commenteranno i lavori e si proseguirà con nuovi argomenti.

Chapter 14

Creazione di Paesaggi in Unreal Engine: Strumenti di Scultura e Materiali Landscape

14.1 Discussione iniziale e aggiornamenti

Federico ha realizzato una *gym* ma non un livello, perché sentiva la mancanza di meccaniche divertenti. Aveva idee per zone con boost di velocità o salto, ma non ha avuto il tempo di programmarle.
L'istruttore propone di concludere il corso con un'ultima lezione in cui lavorare insieme su qualcosa, fornendo eventuali scadenze, limiti o idee per stimolare la produzione. L'obiettivo è raccogliere eseguibili sul server Discord per far provare a tutti i lavori dei colleghi.

14.2 Riepilogo della lezione precedente

14.2.1 Documento di layout

Prima di costruire, si prepara un documento in cui si descrivono:

- i punti di interesse;
- lo spawn del personaggio;
- le sfide e la loro collocazione.

14.2.2 Blockout

Il **blockout** è una mappa realizzata con sole geometrie, utile per:

- testare gli spazi e le meccaniche di gameplay ancora in prototipazione;
- verificare se gli ambienti vanno modificati (più spazio/meno spazio);
- posizionare landmark, elementi interagibili (catene, casse, ...) e prevedere *feedback* positivi per l'esplorazione o negativi (hard gate come muri, soft gate come nemici ostici) per guidare il giocatore.

Esempio mostrato: il livello iniziale di *God of War* (2018), dove il blockout ricostruisce l'ambiente e segnala le aree interattive ancor prima di avere codice o asset definitivi.

14.2.3 Gym: definizione e scopo

Una **gym** è un ambiente di test in cui si creano ostacoli fittizi per misurare le capacità del personaggio:

- distanza di corsa prima di esaurire la stamina;
- altezza e lunghezza del salto;
- pendenza massima superabile, controllo in aria, ecc.

Sulla base dei risultati si decide il *feeling* desiderato e si costruiscono i livelli rispettando le possibilità reali del PG (es. se salta 5 m, non si progettano ostacoli da 7 m, a meno che non siano raggiungibili con power-up successivi).

14.3 Analisi della gym di Federico

14.3.1 Test con torretta e coperture

Federico ha misurato l'altezza minima di un ostacolo che impedisce alla torretta di colpire il personaggio. Questo dato è fondamentale per progettare sezioni *stealth*: un muro di altezza sufficiente può bloccare i colpi (o il rilevamento), permettendo di definire spazi sicuri.

14.3.2 Piattaforme mobili e strumenti di debug

Avendo una piattaforma che si muove fino a un endpoint, ha:

- sfruttato il movimento obliquo;
- aggiunto un hitbox visibile solo in editor per regolare l'arrivo;
- creato una funzione richiamabile dall'editor che disegna una linea dal punto iniziale a quello finale per rendere immediatamente comprensibile il percorso.

14.3.3 Gestione della telecamera

Durante i test ha notato fastidio quando la telecamera urta ostacoli o compie movimenti bruschi. Ha regolato l'altezza di un passaggio per migliorare il comfort. Queste regolazioni si fanno intervenendo sul `CameraBoom` (braccio della telecamera) nel Character, modificando ad esempio la velocità di spostamento o la morbidezza del movimento. Sapere che un varco di 2 m d'altezza può creare problemi di camera permette al level designer di evitare quelle dimensioni o di accettarle solo dove il movimento lento della camera è tollerabile.

14.3.4 Esperimenti sul salto

Ha costruito una serie di ostacoli a diverse altezze (1 m, 2 m, 3 m, 4 m) e larghezze. Le pedane quadrate hanno lato 1 m.

- Il salto a 1,60 m risultava sgradevole perché il personaggio, una volta raggiunta la sommità, scivolava via; questa informazione dice al level designer di non usare dislivelli di 1,60 m con le attuali impostazioni (a meno di volerlo come elemento di difficoltà).
- Un salto di 2 m è chiaramente fuori portata, quindi può essere usato come *hard gate* verticale.
- Ha provato un salto con un'accelerazione forzata (slancio) notando comportamenti anomali, indice della necessità di bilanciare accelerazione e altezza di salto.

Ogni quadrato rappresenta 1 m, quindi le misure sono immediatamente utilizzabili per la costruzione degli spazi.

14.4 Principi di level design emersi

14.4.1 Progettare a partire dalle capacità del personaggio

Le metriche raccolte in gym diventano le regole base: non si progetta un ostacolo che richiede un salto di 7 m se il personaggio ne fa 5, a meno che non sia previsto un potenziamento successivo. Conoscere l'altezza esatta di un salto "comodo" consente di creare percorsi fluidi, mentre l'altezza "fastidiosa" può essere usata per sfidare il giocatore o per bloccare temporaneamente un passaggio.

14.4.2 Less is more e versatilità delle meccaniche

Non serve accumulare molte meccaniche. Anche il solo salto, se ben progettato, ha generato migliaia di giochi diversi. Conviene avere poche meccaniche ma interessanti, applicabili in modi molteplici. Aggiungere trigger box per boost temporanei (velocità, salto) è semplice e può arricchire un livello senza complicarlo eccessivamente. L'importante è che il risultato sia soddisfacente per *almeno un tipo di giocatore*, anche con mezzi minimi.

14.5 Aspetti organizzativi

- L'ultima lezione si terrà martedì prossimo; l'istruttore spera di vedere i lavori dei partecipanti e fornire commenti in tempo reale su aspetti di design (guidance, affordance, costruzione degli spazi).
- Le registrazioni delle lezioni rimangono disponibili indicativamente fino a settembre, quando inizieranno i nuovi corsi IVE. Chi non ha seguito le lezioni precedenti può comunque partecipare: gli strumenti mostrati sono in parte indipendenti dagli asset creati in precedenza.
- L'obiettivo finale è raccogliere file eseguibili su Discord per permettere a tutti di provare i progetti realizzati.

14.6 Risultati della gym di Federico (continuazione)

14.6.1 Metriche dei salti e leggibilità degli ostacoli

Un dislivello di **1,80 m** è inutilizzabile: il salto risulta innaturale (*snappy*) e il giocatore non capisce se l'ostacolo è superabile. L'obiettivo è rendere immediatamente riconoscibile un muro che il personaggio può scavalcare: il giocatore non deve provare a caso. Per questo Federico ha costruito una serie di altezze da **1,40 m** a **2 m**.

- A 1,40 m il personaggio sale senza problemi.
- A 2 m sbatte contro il muro e non può salire.
- Le altezze intermedie indicano il limite preciso: 1,80 m è sconsigliabile perché il movimento non è chiaro.

Ha poi testato **pendenze** di 45° e 50° (probabilmente eccessive) e ha misurato il **salto in camminata** (massimo 3 unità) e il **salto in corsa** (massimo 6 unità). Ha creato anche un **blocco letale** che uccide il personaggio al contatto.

14.6.2 Esposizione di parametri in editor

Per rendere più comoda la modifica delle piattaforme mobili, ha esposto direttamente nell'editor le variabili di posizione (coordinate x e y dell'endpoint), semplificando la regolazione senza dover entrare nei blueprint.

14.7 Concetti di design emersi dalla gym

14.7.1 Forgiveness e bilanciamento dei salti

Il **forgiveness** (indulgenza) tiene conto che i giocatori non eseguono salti perfetti come un tool assisted.

- Nei **salti in corsa** si può ridurre leggermente la distanza richiesta rispetto al massimo teorico, in modo che il giocatore riesca quasi sempre.
- Nei **salti in camminata** si può essere più rigorosi, perché sono meno frequenti e spesso non vengono usati per passaggi critici.

Un salto "al pelo" (incerto) è adatto solo a giochi **hardcore platform** con *trial and error* e frustrazione intenzionale (es. *Getting Over It*, *Jump King*). Il designer può anche intervenire sulle **collisioni in volo** (restringere l'hitbox in aria) per aumentare la *mastery*, analogamente a certe tecniche di *parkour* in Minecraft. Tutte queste scelte si valutano quando si ha il livello davanti, per decidere l'esperienza desiderata.

14.7.2 Valore delle informazioni raccolte nella gym

Avere dati precisi su distanze, altezze, tolleranza delle torrette, ecc. permette di progettare livelli con:

- piattaforme mobili e salti precisi;
- zone da attraversare evitando il fuoco nemico;
- sezioni a gravità modificata.

14.7.3 Pivot degli asset e trucchi di editor

I cubi del pacchetto fornito hanno il pivot al centro, mentre alcuni cubi personalizzati lo avevano su un angolo. Non è possibile modificare il pivot degli *engine asset* di default. Soluzione: creare una mesh cubo e modificarne il pivot con la **Modeling Mode**, salvandola nel Content Browser.

14.8 Livello di Filippo

14.8.1 Struttura generale

Filippo ha realizzato direttamente un livello (senza gym) sfruttando gli strumenti visti nelle lezioni precedenti. Obiettivo: raccogliere **5 collezionabili** per aprire una porta e completare il livello. Ha lavorato con interazioni, gravità alterata, torrette, piattaforme, teletrasporti e luci.

14.8.2 Caratteristiche delle sezioni

- **Zona a gravità ridotta**: salti più alti, collezionabile in alto.
- **Platform con torrette**: percorso orizzontale e verticale; se colpiti si viene riportati al checkpoint.
- **Sequenza di porte**: solo la sequenza corretta permette di proseguire; quella sbagliata teletrasporta all'inizio (meccanica "classica" di memory game con luci).
- **Integrazione del menù**: ha collegato il menù creato in lezioni precedenti e ha aggiunto una schermata di vittoria.

14.8.3 Aspetti di level design osservabili

- **Primi salti con rischio basso**: nella progettazione platform è buona norma iniziare con salti *low stake* (se sbagli, cadi e puoi risalire). Qui il teletrasporto al checkpoint dopo un errore è "poco grazioso" ma funzionale perché non comporta perdita di progressi significativa.
- **Peso dell'errore**: nei livelli di esercizio il costo dell'errore è minimo; in un gioco completo si bilancerebbe diversamente.
- **Memorizzazione dello stato**: la gestione delle monete raccolte e di quelle già inserite nella porta è stata la parte più complessa a livello di blueprint.

14.9 Lezione da trarre: *less is more* e apprendimento attivo

Filippo ha utilizzato una meccanica per livello, dimostrando che con poco si può costruire qualcosa di funzionante. Il valore dell'esercizio non è solo il risultato, ma **imparare a informarsi** quando si incontra un ostacolo tecnico. Lo sviluppatore più efficace non è chi conosce più cose, ma chi sa **adattarsi e trovare rapidamente soluzioni**. L'insegnante fornirà oggi le basi di **Landscape Mode** (e forse Foliage Mode), incoraggiando poi l'approfondimento autonomo.

14.10 Landscape Mode

14.10.1 Cos'è il Landscape

Il **Landscape** è un attore costituito da un piano che può essere modificato manualmente tramite *terraformazione*, simile a quella presente nei gestionali di parchi a tema. Viene usato per costruire ampi spazi aperti (*open world*) senza dover posizionare ogni singolo elemento a terra; si lavora con un pennello che alza, abbassa e modella il terreno.

Una limitazione importante: il piano non può mai essere ripiegato su sé stesso. Per realizzare grotte o zone sotterranee occorrono quindi un secondo Landscape, *brush* modellate o *mesh* dedicate. Vedremo più avanti la tecnica della *Visibility Mask*.

14.10.2 Creazione del Landscape

Entrando in **Landscape Mode** appare un'anteprima (un lenzuolo verde) che mostra come sarà il terreno dopo il `Create`.

Componenti e sezioni

Il Landscape è suddiviso in **Component** (chunk) e **Section**.

- Il numero di *Section per Component* si imposta con l'apposito parametro (default 2×2).
- Se portato a 1×1, ogni Component coincide con una Section.

Risoluzione e numero di Component

- **Risoluzione delle sezioni** (quads per section): default 63×63. Valori più bassi (31×31, 7×7) producono un terreno più spigoloso quando scolpito, ma possono andare bene per piccole porzioni.
- **Numero di Component** lungo X e Y: si può creare un Landscape rettangolare (es. molto lungo e poco largo). Default 8×8.
- **Location, Rotation, Scale**: al momento si lasciano invariate.
- **Materiale**: si assegna un materiale che sarà applicato al Landscape; lo tratteremo più tardi.

- **Fill World**: crea un Landscape gigante che riempie tutto il mondo; utile solo per ambienti enormi.

Alla pressione di `Create` vengono generati i Component, visibili nel *World Outliner* come oggetti separati.

14.10.3 Gestione (Manage)

La sotto-finestra **Manage** permette di:

- Creare un nuovo Landscape.
- Importare un Landscape da un altro livello.
- Selezionare/deselezionare Component specifici (Shift + Click).
- Aggiungere o rimuovere Component se il Landscape è troppo piccolo o grande.
- Modificare il Landscape con le *Spline* (argomento che sarà trattato più avanti).

Sono inoltre disponibili i **Layer**: funzionano come i layer di Photoshop; si può bloccare un layer per proteggerlo da modifiche e lavorare solo su layer separati.

14.10.4 Il pennello (Brush)

Parametri fondamentali

Il pennello è uno strumento a forma di cerchio (di default) che modifica il terreno.

- **Brush Size**: dimensione del pennello (in Component, ad es. 2×2, 4×4).
- **Falloff**: distanza fra il bordo interno (pieno) e quello esterno del cerchio.
  - Falloff = 0: bordo netto, l'effetto viene applicato in modo uniforme su tutta l'area.
  - Falloff = 1: solo la punta centrale è piena, il resto è sfumato.
- **Strength**: intensità dell'effetto. Valori bassi (0,1) applicano il cambiamento lentamente; a 1 è immediato.

Forme di Falloff

Oltre al falloff liscio standard, sono disponibili altre curve:

- **Smooth**: morbido, crea colline dolci.
- **Sharp / Linear**: salita lineare, geometrica, adatta a forme nette.
- **Spherical**: effetto bombato, con una cima arrotondata e discesa morbida.
- **Tip (convesso)**: opposto allo Spherical, la base è morbida e la parte superiore quasi lineare, utile per forme aguzze.

Tipi di pennello

- **Circle (Simple)**: cerchio di default.
- **Alpha Brush**: usa una texture come forma del pennello. La texture interpreta i valori dei pixel: gli 1 (bianco) vengono sollevati, gli 0 (nero) no. È possibile scegliere il canale della texture (R, G, B) da utilizzare per il pennello. Utile con texture di *noise* per creare terreni accidentati (zigrinati) che da lontano risultano piacevoli. L'opzione `Stroke` ruota il pennello lungo la traiettoria (disattivabile).
- **Pattern Brush**: estende l'Alpha Brush permettendo di modificare ulteriormente la texture applicata:

  - **Scale**: ingrandisce o rimpicciolisce il pattern.
  - **Rotation**: ruota il pattern.
  - **Pan U/V**: sposta la texture orizzontalmente/verticalmente.

  Esempio: con una texture a forma di volto si può modellare un terreno che riproduce fedelmente quel volto.

- **Component Brush** (raramente usato): permette di alzare/abbassare interi Component, utile per creare zone a quote nettamente separate (es. area superiore e inferiore distinte).

14.10.5 Visualizzazione della risoluzione poligonale

I quadratini visibili durante la scultura mostrano la **densità della mesh** del Landscape.

- Un terreno ad alta risoluzione (più quads) rimane compatto e regolare quando deformato.
- Con poca risoluzione i poligoni si allungano verticalmente, causando **stretching** del materiale e perdita di qualità visiva.

La scelta della risoluzione è quindi un compromesso tra definizione e peso.

14.10.6 Creare cavità (Visibility Mask)

Non si può scavare un buco direttamente con gli strumenti di sculpt: il Landscape non ammette geometria ripiegata. Per ottenere grotte o passaggi sotterranei occorre la **Visibility Mask**, che si controlla tramite il materiale del Landscape aggiungendo nodi specifici. Una volta mascherata la zona, il terreno diventa invisibile e attraversabile, e al di sotto si possono collocare mesh o brush per costruire la caverna.

14.10.7 Pennello Visibility e creazione di buche

Il **Visibility Painting** permette di scavare buchi nel Landscape, ma non è disponibile con gli strumenti di sculpt diretti. Il pennello rosso (Visibility) non ha effetto finché il materiale assegnato al Landscape non include il nodo `Landscape Visibility Mask`. Una volta configurato il materiale, si potrà dipingere con questo pennello per rendere invisibili e attraversabili porzioni di terreno, creando grotte o passaggi sotterranei.

14.10.8 Sculpt, Erase e Smooth

Sculpt (Scolpire)

- **Clic sinistro**: alza il terreno.
- **Shift + clic sinistro**: scava, creando depressioni.

Erase (Cancella)

Rimuove qualsiasi modifica (sia verso l'alto che verso il basso) riportando il Landscape alla forma originale.

Smooth (Ammorbidire)

Crea una transizione graduale (una "media") tra le altezze all'interno del pennello: alza le parti più basse e abbassa le parti più alte.

- Essenziale per evitare lo **stretching** del materiale: poligoni troppo allungati produrranno texture deformate; lo smooth riproporziona i quadratini.
- **Strength**: intensità dell'effetto.
- **Dettaglio (Smooth precision)**: valori alti rimuovono molti dettagli (pialla tutto), valori bassi producono modifiche più sottili e precise.

14.10.9 Flatten (Appiattire)

Consente di appiattire il terreno all'altezza del punto cliccato (indicato dalla croce).

- **Modalità**:
  - **Both**: alza le parti sotto e abbassa quelle sopra.
  - **Raise**: alza soltanto, non abbassa mai (utile per preservare creste).
  - **Lower**: abbassa soltanto, non alza mai.
- **Use Slope Flatten**: prosegue il piano inclinato esistente, scavando o alzando lungo la diagonale. Utile per spiazzi montani in pendenza.
- **Applicazioni**:
  - Creare plateau, terrazzamenti (es. risaie, cascate a gradoni).
  - Scavare canyon mantenendo le pareti laterali.
  - Cancellare riportando tutto alla quota base del Landscape (usato come "gomma").

14.10.10 Rampe (Ramp Tool)

Non è un pennello ma un sistema a due punti: si selezionano punto A e punto B, si regolano i parametri e si clicca `Add Ramp` per creare un collegamento rettilineo di terreno.

- **Parametri**: larghezza della rampa e falloff (a 0 nessuna sfumatura, a 1 solo sfumatura).
- **Usi tipici**: rampe per salire su alture, dighe in crepacci, piste per veicoli/skateboard.
- **Limitazioni**: il Landscape non può sovrapporsi, quindi non si possono creare ponti sospesi (usare mesh). Rampe troppo sottili vengono "distrutte" dai LOD (si veda sotto).

14.10.11 LOD (Level of Detail) e ottimizzazione

I **LOD** sono versioni a risoluzione decrescente di una mesh (o del Landscape) che l'engine sostituisce in base alla distanza dalla camera, per alleggerire il carico computazionale.

- Ogni oggetto può avere più LOD (es. 10 000, 5 000, 2 000 poligoni) attivati a distanze configurabili.
- Sul Landscape i poligoni vengono semplificati con la distanza. Questo processo è invisibile su superfici ampie, ma su rampe molto sottili (pochi poligoni) provoca artefatti visivi e distorsioni (il terreno sembra animarsi).
- **Esempio**: *Pokémon Scarlatto/Violetto* e *Leggende Arceus* mostrano un Landscape che "si muove" perché i LOD scattano troppo presto, a causa di un'ottimizzazione eccessiva richiesta da un codice poco efficiente.

Regola pratica: dare sempre sufficiente risoluzione al Landscape ed evitare forme estremamente sottili o spigoli pronunciati che verrebbero degradati dai LOD.

14.10.12 Erosion e Hydro

Strumenti di simulazione, usati raramente dall'istruttore per il limitato controllo che offrono.

Erosion (Erosione)

Simula l'erosione di vento e pioggia creando micro-rilievi e asperità (l'opposto dello Smooth).

- **Parametri**:
  - **Strength**: forza dell'effetto.
  - **Iterations**: numero di passaggi.
  - **Threshold**: differenza minima di altezza tra punto A e punto B affinché l'erosione venga applicata. Un threshold basso (es. 17) coinvolge molte più aree; uno alto risparmia solo i dislivelli marcati.
  - **Noise Module/Scale**: aggiungono rumore al pattern erosivo.
- Può rompere le texture se troppo aggressivo; per questo si consiglia di tenerlo basso.

Hydro (Idro-erosione)

Pensato per generare forme simili a fondali marini o erosione causata dall'acqua. Funziona in modo analogo a Erosion, ma con ancora meno controllo. L'istruttore tende a non utilizzarlo.

14.11 Materiale del Landscape e pittura

14.11.1 Creazione e assegnazione dei layer

Il **Landscape** può essere dipinto con un materiale che utilizza *layer* multipli. Il materiale deve essere progettato appositamente per supportare il **Landscape Layer Blend**.

Il pacchetto `Stagobot` fornisce già un materiale predisposto: `M_Landscape`, che include quattro layer.

- Applicando il materiale al Landscape, questo appare nero perché nessun layer è stato ancora assegnato.
- Premendo `Create Layers from Assigned Materials` (opzione introdotta con Unreal Engine 5), l'editor crea automaticamente i layer definiti nel materiale e li rende disponibili per la pittura.
- Il Landscape rimane nero finché non si dipingono attivamente i layer.

14.11.2 Struttura del materiale: Material Function e nodo di output

Aprendo il materiale `M_Landscape` si osservano:

- **Material Function** dedicate ai singoli strati: `MF_Rocks`, `MF_Pebbles`, `MF_Sand`, `MF_Grass`, ecc.
- Ciascuna funzione utilizza il nodo `Absolute World Position` per ottenere texture allineate alla posizione globale (*world aligned*), evitando lo stretching.
- All'interno di ogni funzione, un nodo **SetMaterialAttributes** trasmette le proprietà del materiale (Base Color, Roughness, Normal, ecc.) verso l'uscita. Questi slot sono configurabili: si possono aggiungere campi per Metallic, Specular, ecc.
- Le uscite delle varie funzioni confluiscono tutte verso un unico nodo: **Landscape Layer Blend**.

14.11.3 Nodo Landscape Layer Blend

Il nodo `Landscape Layer Blend` è il cuore del materiale per Landscape.

- È un nodo vuoto, analogo a un array, in cui si aggiungono elementi tramite il pannello dei dettagli.
- Per ogni elemento si specifica:

  - **Nome del layer**: deve corrispondere esattamente al nome del layer di pittura (es. `Grass`, `Rock`, `Sand`). Scriverlo correttamente è fondamentale:
    * Un nome sbagliato o mancante impedisce all'editor di trovare il layer, rendendo impossibile la pittura.
    * Altri nodi, come `Landscape Layer Sample`, richiedono il nome esatto per leggere le informazioni del layer.
  - L'ingresso corrispondente (Base Color, Roughness, Normal, ecc.) a cui collegare l'uscita della rispettiva Material Function.

- Il nodo mescola i vari strati in base alle *weight maps* dipinte sul Landscape.

14.11.4 Altri nodi utili

- **Landscape Layer Sample**: permette di campionare il peso di un layer in un dato punto, utile per effetti che dipendono dal materiale locale (es. transizioni).
- **Landscape Visibility Mask**: nodo necessario per abilitare la **Visibility Mask**, con cui si possono creare buchi (caverne) nel terreno. Va collegato all'apposito canale nel materiale.

14.11.5 Pittura dei layer

Una volta configurati i layer, si torna alla **Paint Mode** del Landscape.

- Il pannello `Paint` offre strumenti analoghi a quelli di sculpt: **Paint** (applica il layer), **Smooth** (morbida le transizioni), **Flatten** (appiattisce la distribuzione), **Noise** (introduce variazioni casuali).
- La `Tool Strength` e il tipo di `Brush` funzionano come per lo sculpt; il pennello applica il materiale attivo solo dove il layer è selezionato.

14.12 Configurazione del materiale del Landscape e spawn automatico di foliage

14.12.1 Struttura del materiale `M_Landscape`

- Il materiale utilizza **Material Function** separate per ogni strato (`MF_Rocks`, `MF_Pebbles`, `MF_Sand`, `MF_Grass`), tutte costruite sul nodo `Absolute World Position` per ottenere texture allineate al mondo (*world aligned*).
- Ogni funzione impiega il nodo `SetMaterialAttributes` per passare in modo compatto le proprietà (Base Color, Roughness, Normal) al nodo successivo.
- L'opzione `Use Material Attributes` (ceckata) comprime tutti gli attributi in un unico canale, semplificando il grafo: il nodo finale principale del materiale riceve direttamente questo insieme di attributi anziché doverli collegare uno per uno ai singoli input (Base Color, Roughness, Normal, ecc.).

14.12.2 Nodo `Landscape Layer Blend`

- È il nodo centrale che riceve le uscite delle Material Function e le miscela in base ai **layer** di pittura.
- Si configura come un array: per ogni elemento si specifica il **nome esatto del layer** (es. `Grass`, `Rock`) e gli ingressi corrispondenti (Base Color, Roughness, Normal).
- Scrivere il nome corretto è cruciale: l'editor lo usa per associare il layer di pittura; un nome errato impedisce al materiale di funzionare correttamente.

14.12.3 Creazione dei layer nel Landscape

- Applicando il materiale `M_Landscape` al Landscape e premendo `Create Layers from Assigned Materials`, Unreal Engine 5 genera automaticamente i **layer di pittura** (Rock, Pebbles, Sand, Grass) così come definiti nel `Landscape Layer Blend`.
- Il Landscape inizialmente appare nero perché nessun layer è stato ancora dipinto.
- Dipingendo con i pennelli della modalità `Paint`, si applicano i materiali corrispondenti a ciascun layer.

14.12.4 Spawn automatico di erba tramite `Landscape Grass Output`

Il nodo e il suo funzionamento

- Il materiale di esempio contiene una sezione separata (dopo il nodo principale) dedicata allo spawn di **foliage** sul layer `Grass`.
- I nodi chiave sono:
  - `Landscape Layer Sample`: campiona il peso di un layer in un punto. Qui è impostato con il nome esatto `"Grass"`.
  - Una serie di operazioni matematiche (condizionali, moltiplicazioni) per decidere *dove* e *con quale densità* far apparire l'erba.
  - `Landscape Grass Output`: nodo di output che, a differenza del materiale principale, ha il solo scopo di **spawnare istanze di foliage** direttamente sul terreno durante il rendering.
- `Landscape Grass Output` non influenza il colore o la superficie, ma genera geometria (erba, cespugli) solo sulle aree dove il layer di riferimento è attivo e le condizioni sono soddisfatte.
- Se il nome passato al `Landscape Layer Sample` non corrisponde esattamente a un layer esistente (es. minuscole/maiuscole errate), l'erba non comparirà.

Configurazione del `Grass Type`

- Per ogni tipo di foliage si crea un asset `Landscape Grass Type` (click destro → Foliage → Landscape Grass Type).
- All'interno si aggiungono uno o più **Grass Varieties**, ciascuna delle quali specifica:
  - **Grass Mesh**: la Static Mesh da istanziare (es. un ciuffo d'erba, una roccia).
  - **Densità di spawn** (`Grass Density`): numero di istanze per unità di area (da regolare per evitare sovraccarichi).
  - **Scaling Uniforme** o `Scale X/Y/Z`: intervallo di variazione casuale della scala per ogni istanza.
  - Altri parametri come `Align to Surface`, `Random Rotation`, ecc.
- Il `Grass Type` viene poi collegato all'ingresso `Grass Type` del nodo `Landscape Grass Output` nel materiale.
- **Nota**: il nodo si chiama `Landscape Grass Output` indipendentemente dal tipo di foliage; il `Grass Type` determina *cosa* viene spawnato.

Abilitazione delle Virtual Texture

- L'erba generata in questo modo (almeno nella configurazione di esempio) richiede che nel progetto siano attive le **Virtual Texture**.
- Per abilitarle: `Project Settings` → `Engine` → `Rendering` → `Virtual Textures` → spuntare `Enable Virtual Texture Support`.
- Senza questa opzione, il foliage apparirà **nero** (privo di illuminazione) o non verrà renderizzato correttamente.

Esperimento con altri layer (roccia)

- È possibile associare un `Landscape Grass Output` anche ad altri layer, ad esempio `Pebbles`, copiando la struttura e usando un `Landscape Layer Sample` con nome `"Pebbles"` e un `Grass Type` contenente mesh di rocce.
- Durante la prova con un asset di roccia (`rockflats`) si sono verificati problemi di performance e materiali errati (riflettività inaspettata). Ciò evidenzia che:
  - Le mesh utilizzate devono avere materiali appropriati e scalature contenute.
  - È necessario regolare attentamente i parametri di densità e scaling per evitare scene sovraccariche o visivamente incoerenti.
- Il metodo resta valido: il `Landscape Grass Output` è un nodo generico per spawnare qualsiasi `Grass Type` sul layer campionato, non solo erba.

14.13 Creazione di un materiale personalizzato per il Landscape

14.13.1 Rimozione del materiale preesistente e pulizia dei layer

- Scollegare il materiale `M_Landscape` dal Landscape: nella scheda `Landscape`, rimuovere il materiale assegnato.
- I layer presenti (Rock, Pebbles, Sand, Grass) diventano puri layer senza informazioni; si possono eliminare tutti.
- Le informazioni originali restano comunque salvate nei `Landscape Layer Info` esistenti.

14.13.2 Creazione di un nuovo materiale

Creare un nuovo materiale (`M_Landscape_Test`) e prepararlo per il Landscape.

Nodo `Landscape Layer Blend` personalizzato

- Aggiungere il nodo `Landscape Layer Blend` e definire tre layer:
  - `Grass`
  - `Sand`
  - `Rock`
- Assegnare un nome esatto a ciascun layer; senza nome, l'editor non li riconosce.
- Collegare il nodo al `Base Color` e alla `Normal` del nodo materiale principale.

Assegnazione delle texture ai layer

- Per ogni layer, utilizzare un `Texture Sample` e collegare l'uscita `RGB` al corrispondente ingresso `Base Color` del `Landscape Layer Blend`.
- Scegliere texture piastrellabili per erba, sabbia e roccia (es. `T_Grass`, `T_Sand`, `T_Rock_Tileable`).
- Per la `Normal`, collegare le relative normal map (se presenti) agli ingressi `Normal` del Blend.
- Nel materiale di esempio, il colore dell'erba può essere modificato a piacere (es. da rosso scuro a rosso chiaro, o addirittura blu).

Controllo della scala delle texture

- Aggiungere un nodo `Texture Coordinate` e collegarlo all'ingresso `UVs` di ogni `Texture Sample`.
- Modificare i valori di `UTiling` e `VTiling` nel nodo `Texture Coordinate` per scalare tutte le texture contemporaneamente (es. $0{,}2 \times 0{,}2$ per ingrandirle).
- Per differenziare la scala tra i materiali, inserire un nodo `Multiply` o modificare separatamente i tiling nei singoli `Texture Sample`, oppure usare più nodi `Texture Coordinate` con valori diversi.

14.13.3 Creazione dei Layer Info e pittura

- Applicare il nuovo materiale al Landscape.
- Nella modalità `Paint`, il layer aggiuntivo (es. `Test`) mostra l'errore: `This layer has no layer info assigned yet`.
- Cliccare il pulsante `+` → `Create Layer Info` → `Weight Blended Layer (Normal)`.
- Scegliere il percorso di salvataggio (es. `Stagobot/Environment/Landscape/Layers`) per creare il nuovo `TestLayerInfo`.
- Il layer diventa dipingibile: il pennello applica la texture associata al layer scelto (nell'esempio, il layer `Pebbles`).

14.13.4 Strumenti di pittura: Smooth, Flatten, Noise

- **Smooth**: ammorbidisce la transizione tra due layer, fondendo gradualmente i materiali (es. da sola roccia a metà roccia/sabbia fino a sola sabbia).
- **Flatten**: appiattisce tutta l'area del pennello su un unico layer attivo (es. copre tutto con sabbia).
- **Noise**: introduce variazioni casuali prelevando porzioni dai layer circostanti, creando un effetto di "disturbo". Utile per micro-variazioni, ma generalmente si preferisce dipingere a mano.

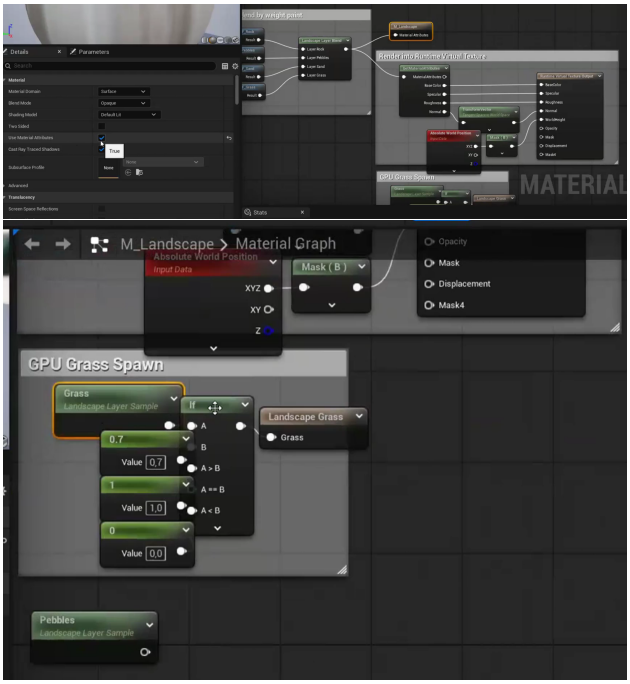
14.13.5 Abilitazione della Visibility Mask per scavare buchi

Impostazione del materiale in modalità Masked

- Per attivare l'`Opacity Mask` (e quindi il pin `Opacity Mask` nel nodo principale), cambiare la **Blend Mode** del materiale da `Opaque` a `Masked`.
- Questo fa comparire l'ingresso `Opacity Mask`; ad esso va collegato il nodo `Landscape Visibility Mask`.

Nodo `Landscape Visibility Mask` e utilizzo

- Il nodo `Landscape Visibility Mask` non ha parametri: va semplicemente collegato all'ingresso `Opacity Mask`.
- Ora nella modalità `Paint` del Landscape lo strumento `Visibility` diventa funzionante: dipingendo con esso si rimuove completamente il terreno (buco effettivo, senza collisioni).
- Il personaggio, se cammina sopra la zona resa invisibile, cade attraverso il buco.
- Lo stretching dei poligoni del Landscape influisce sulla precisione del buco: se i quadrati sono molto allungati, il buco risulta espanso e impreciso; meglio scavare in zone con poligoni regolari.
- Questa tecnica, unita a mesh che simulano un soffitto e pareti rocciose, permette di creare grotte.



14.14 Note finali e argomenti per la prossima lezione

14.14.1 Automateriale basato sulla pendenza

- È possibile creare un materiale che reagisce alla pendenza del Landscape: ad esempio, di default c'è sabbia, ma sui pendii ripidi compare automaticamente roccia.
- Richiede di inserire nel materiale nodi che calcolano l'inclinazione della superficie (*slope*) e, in base ad essa, fondono i layer.
- Argomento rinviato alla prossima lezione, insieme al Foliage Mode.

14.14.2 Misurare l'inclinazione del Landscape per il gameplay

- Non esiste uno strumento che mostri in tempo reale l'angolo di pendenza in un punto specifico, ma il comportamento del personaggio sulle pendenze del Landscape è identico a quello sui blockout: se il personaggio può salire pendenze fino all'80%, lo farà anche sul terreno scolpito; oltre tale soglia, scivolerà o non salirà.
- A livello di materiali/blueprint esistono nodi come `Get Slope` o funzioni per campionare la pendenza in un punto (es. per far spawnare qualcosa solo su pendenze specifiche), ma non c'è un visualizzatore diretto.
- L'istruttore verificherà se esiste un metodo più immediato e lo comunicherà la prossima lezione.

14.14.3 Compiti per casa e conclusione

- Continuare a esercitarsi con i blockout oppure creare un piccolo paesaggio con il Landscape, applicando i layer e sperimentando con la pittura.
- Prossima lezione: automateriale, Foliage Mode e ulteriori approfondimenti sul dressing e le luci.

Chapter 15

Auto-Materiali e Foliage in Unreal Engine: Tecniche per la Creazione di Ambienti Naturali

15.1 Materiale automatico per il Landscape

15.1.1 Introduzione all'automateriale

- Il docente mostra un esempio di **automateriale** applicato a un Landscape, che gestisce automaticamente la transizione tra diversi tipi di terreno (sabbia, roccia, erba) in base alle caratteristiche della superficie (altezza, pendenza).
- L'automateriale permette di non dover dipingere manualmente ogni layer, ma di definire regole che l'engine applica in tempo reale mentre si scolpisce il terreno.
- Nell'esempio, il materiale `M_Autolandscape_Base` presenta due parametri esposti:
  - **Bias**: determina la preferenza tra un materiale e l'altro (es. a quale altezza/pendenza inizia la transizione).
  - **Sharpness**: controlla quanto è netta la transizione tra i due materiali.
- Modificando questi parametri nell'istanza del materiale, si può regolare l'aspetto del paesaggio senza rientrare nell'editor del materiale.

15.1.2 Struttura del materiale `M_Autolandscape_Base`

- Il materiale utilizza due funzioni predefinite per i materiali del terreno: `MF_Sand` e `MF_Rock`.
- Queste funzioni forniscono già tutti gli attributi necessari: `Base Color`, `Roughness` e `Normal`.
- All'interno del grafo del materiale, gli attributi in uscita dalle funzioni vengono scomposti tramite il nodo **BreakMaterialAttributes**, che separa i singoli canali (colore base, roughness, normale) rendendoli disponibili separatamente.
- **Nota**: la scomposizione non è strettamente necessaria se si utilizzano direttamente i nodi `MakeMaterialAttributes`, ma aiuta a visualizzare e comprendere il flusso dei dati.

15.1.3 Il nodo LERP come strumento di interpolazione

- L'interpolazione tra i due materiali (sabbia e roccia) viene realizzata con il nodo **LERP** (Linear Interpolation).
- **Chiarimento sul funzionamento del LERP**:
  - Il LERP non lavora solo con le immagini, ma può interpolare qualsiasi tipo di valore (numerico, vettoriale, colore).
  - Riceve tre ingressi: **A** (valore di partenza), **B** (valore di arrivo) e **Alpha** (fattore di interpolazione).
  - `Alpha` è un valore compreso tra 0 e 1: se `Alpha = 0`, l'uscita è uguale ad `A`; se `Alpha = 1`, l'uscita è uguale a `B`; per valori intermedi si ottiene una miscela proporzionale.
  - In un caso tipico, l'`Alpha` può provenire da una texture in scala di grigi (nero = 0, bianco = 1), ma può anche essere il risultato di un calcolo matematico.

15.1.4 Il criterio di interpolazione (Alpha)

- In questo materiale, il fattore `Alpha` non proviene da una texture, ma da un **calcolo basato sulla posizione e sull'orientamento del Landscape rispetto al mondo**.
- Il nodo utilizzato (di cui non viene specificato il nome esatto, ma riconducibile a funzioni come `World Aligned Blend` o calcoli basati su `Vertex Normal`) analizza la relazione tra la superficie del terreno e un vettore di riferimento (tipicamente l'asse Z verticale).
- In sostanza:
  - Sulle superfici **pianeggianti** (normale allineata con l'asse Z), il valore di `Alpha` è tale da favorire il materiale **sabbia**.
  - Sulle superfici **in pendenza** (normale molto inclinata rispetto all'asse Z), il valore di `Alpha` cambia e favorisce il materiale **roccia**.
- I parametri `Bias` e `Sharpness` modificano rispettivamente la soglia a cui avviene la transizione e la sua nettezza.

15.1.5 Creazione ed esposizione dei parametri

- All'interno del materiale, è possibile creare parametri esposti (modificabili dall'esterno, ad esempio nell'istanza del materiale) in due modi:
  1. Creare direttamente un nodo `Scalar Parameter` o `Vector Parameter` e collegarlo al valore desiderato.
  2. Creare una costante (`Constant`) e successivamente convertirla in parametro tramite l'opzione `Convert to Parameter` (tasto destro sul nodo).
- I parametri esposti compaiono poi nel pannello `Details` dell'istanza del materiale, permettendo regolazioni rapide senza riaprire il Material Editor.

15.2 Miglioramento dell'automateriale

15.2.1 Aggiunta di un terzo layer: erba, sabbia e roccia

- Partendo dal materiale a due layer (sabbia e roccia), si può aggiungere un terzo layer (erba) per creare un paesaggio più vario.
- Invece di usare più nodi `LERP` separati, si utilizza il nodo **BlendMaterialAttributes**, che combina automaticamente tutti gli attributi (Base Color, Roughness, Normal, ecc.) di due materiali in un unico passaggio.

- Il criterio di blend (Alpha) viene calcolato separatamente per ogni transizione:
  - **Roccia vs (Sabbia/Erba)**: l'Alpha è basato sulla **pendenza**, come visto nel materiale base.
  - **Sabbia vs Erba**: l'Alpha è basato sull'**altezza** (coordinata Z).

15.2.2 Calcolo dell'Alpha basato sull'altezza

- Per determinare dove applicare la sabbia e dove l'erba in base all'altezza, si esegue il seguente calcolo:
  1. **Absolute World Position**: si estrae la coordinata Z (altezza) del punto in cui il materiale viene renderizzato.
  2. **Subtract**: si sottrae un valore di riferimento, il `Blend Height`, alla coordinata Z.
     - `Z - BlendHeight`: sopra `BlendHeight` il risultato è positivo, sotto è negativo.
  3. **Clamp (Min=0, Max=BlendWidth)**: si limita il risultato tra 0 e un valore `BlendWidth`. Questo definisce una "fascia" di transizione alta `BlendWidth` che inizia esattamente a `BlendHeight`.
  4. **Divide per BlendWidth**: si riconverte il range $[0, \text{BlendWidth}]$ in un range normalizzato $[0, 1]$.
  5. **Clamp (Min=0, Max=1)**: si assicura che l'Alpha finale sia esattamente compreso tra 0 e 1, pronto per essere usato nel nodo di blend.
- **Parametri esposti**:
  - **Blend Height**: l'altezza esatta (in unità Unreal/cm) a cui **inizia** la transizione da sabbia a erba. Sotto questa altezza c'è solo sabbia; sopra inizia a comparire l'erba.
  - **Blend Width**: l'**ampiezza** della zona di transizione. Un valore basso crea un passaggio netto; un valore alto crea una sfumatura graduale.

15.2.3 Dimostrazione dell'effetto

- Modificando i parametri in tempo reale nell'istanza del materiale si può vedere:
  - Aumentando **Blend Height**, la transizione si sposta più in alto.
  - Aumentando **Blend Width**, la fascia di transizione diventa più ampia e sfumata; diminuendola, il passaggio diventa più netto.
- L'uso di una rampa ripida nel landscape aiuta a visualizzare chiaramente l'effetto della transizione verticale, dove la pendenza è costante e l'unica variabile è l'altezza.

15.3 Discussione e organizzazione

15.3.1 Recupero lezioni e argomenti futuri

- Le lezioni registrate rimangono disponibili fino a settembre, permettendo a chi è rimasto indietro di recuperare.
- Per l'ultima lezione si valuta se fare un'esercitazione pratica o approfondire un argomento specifico, come gli effetti visivi (VFX) con **Niagara** (il tool di Unreal per la creazione di effetti particellari).
- Il docente chiede un parere: Niagara è complesso e in tre ore si potrebbe vedere solo una base, ma potrebbe essere interessante.
- Viene confermato che gli argomenti trattati (in particolare per chi seguirà il corso di Game Design) verranno ripresi e approfonditi durante l'Accademia, sia dal punto di vista teorico che pratico.

15.4 Automateriale avanzato per il Landscape

15.4.1 Riepilogo e obiettivo

- Nella lezione precedente è stato creato un **automateriale** a due layer (sabbia e roccia) la cui transizione è guidata dalla **pendenza** del terreno.
- L'obiettivo di oggi è aggiungere un terzo layer (**erba**) e gestire la transizione in base all'**altezza**, automatizzando ulteriormente il materiale.

15.4.2 Materiale base: pendenza e altezza

- Il materiale mostrato (`M_Autolandscape_Base`) utilizza due parametri esposti nell'istanza: `Bias` e `Sharpness`, che controllano soglia e nitidezza della transizione sabbia/roccia.
- La transizione è guidata da un calcolo che confronta la normale del vertice con l'asse Z (vettore "su"): le superfici pianeggianti ricevono sabbia, quelle in pendenza roccia.
- Per mostrare il funzionamento, il docente modifica il vettore di riferimento (da Z a Y o X), facendo sì che la sabbia appaia solo sulle superfici orientate verso quel nuovo asse.

15.4.3 Interpolazione e attributi del materiale

- L'interpolazione tra due materiali usa il nodo **LERP** (Linear Interpolation).
- Il LERP fonde due ingressi (A e B) in base a un valore `Alpha` (0–1). In questo caso, l'Alpha non proviene da una texture ma da un calcolo matematico basato sulla posizione e orientamento della superficie.
- Per combinare tutti gli attributi (Base Color, Roughness, Normal, ecc.) in modo compatto, si usa il nodo **BlendMaterialAttributes** al posto di molteplici LERP separati. Allo stesso modo, `MakeMaterialAttributes` raggruppa gli attributi in un unico output.

15.4.4 Aggiunta del terzo layer: erba

- Si introduce un terzo materiale (erba) e lo si fonde con il blend sabbia/roccia.
- **Transizione sabbia/erba**: è guidata dall'**altezza** (coordinata Z assoluta).
- **Calcolo dell'Alpha per l'altezza**:
  1. `Absolute World Position` → si estrae la componente Z.
  2. `Subtract`: `Z - BlendHeight`. Il valore `BlendHeight` è un parametro esposto che definisce l'altezza esatta a cui **inizia** la transizione.
  3. `Clamp` (Min=0, Max=`BlendWidth`): si limita il risultato in una fascia di ampiezza `BlendWidth`.
  4. `Divide` per `BlendWidth`: si normalizza il valore nell'intervallo $[0, 1]$.
  5. `Clamp` finale (0–1): si ottiene l'Alpha pronto per il nodo di blend.
- **Parametri esposti nell'istanza**:
  - **Blend Height**: altezza di inizio transizione (sotto = sabbia, sopra = inizia l'erba).
  - **Blend Width**: ampiezza della fascia di transizione (valori bassi = cambio netto, valori alti = sfumatura ampia).
- Il docente dimostra l'effetto modificando questi parametri in tempo reale e osservando il cambiamento sulla rampa del landscape.

15.4.5 Combinare automateriale e layer dipingibili

- Problema: un materiale completamente automatico non permette di usare il **Landscape Painting** per aggiungere manualmente zone di un materiale specifico, né di usare il **Landscape Grass Output** per far spawnare il foliage (erba) solo su determinati layer.
- Soluzione: combinare i layer automatici con i layer dipingibili tramite il nodo **Landscape Layer Blend**.
- Il grafo finale unisce:
  - Il **layer automatico** (che fonde sabbia, erba e roccia in base a pendenza e altezza).
  - I **layer manuali** (Grass, Rock, Sand) su cui l'utente può dipingere.
- **Gestione del Grass Output per il foliage**:
  - Per far spawnare l'erba automaticamente sull'erba (senza farla apparire su sabbia o roccia), si costruisce una maschera.
  - La maschera è il risultato di una **somma** e una **sottrazione**:
    1. **Somma**: si prende il layer `Grass` (dipingibile) e lo si somma ai punti in cui **non** c'è roccia (usando l'Alpha della pendenza).
    2. **Sottrazione**: dal risultato si sottrae l'Alpha dell'altezza (che identifica la sabbia), così da escludere le zone sabbiose.
  - Il nodo `OneMinus` può invertire un valore (1 - x) per adattare la logica al risultato desiderato.
- Il risultato è un landscape dove la transizione sabbia/erba/roccia è automatica, ma è possibile dipingere manualmente per correggere o aggiungere aree specifiche, e l'erba (foliage) spawna correttamente solo dove previsto.

15.5 Correzioni e approfondimenti sull'illuminazione

15.5.1 Lumen e illuminazione dinamica

- Nella lezione precedente si era parlato di luci `Static`, `Stationary` e `Movable` e della necessità di buildare l'illuminazione.
- **Correzione**: quelle logiche di baking valgono se **non** si utilizza **Lumen**, il sistema di Global Illumination dinamica predefinito in Unreal Engine 5.
- Per attivare/disattivare Lumen: `Project Settings` → `Rendering` → `Global Illumination` e `Reflections`, impostare su `None` per tornare al baking tradizionale.
- Con Lumen attivo (default in UE5), non è necessario buildare le luci statiche/stazionarie, e la gestione è più immediata. Lumen può essere disattivato per piattaforme poco performanti (mobile, PC di fascia bassa).

15.5.2 God Rays (Light Shaft) con Directional Light

- Nella lezione precedente si era detto che i *God Rays* (raggi volumetrici) richiedono il Post Process.
- **Metodo rapido**: è possibile ottenerli direttamente dalla **Directional Light** regolando i parametri `Light Shaft Occlusion`, `Light Shaft Bloom` e `Brightness` nei dettagli della luce.
- Questa soluzione è più immediata e non richiede la configurazione di un Post Process Volume.

15.6 Modalità Foliage

15.6.1 Introduzione e concetto

- La **Foliage Mode** è una modalità di Unreal Engine che permette di posizionare e randomizzare grandi quantità di mesh (alberi, cespugli, erba, rocce) in modo rapido e non distruttivo, senza doverle piazzare manualmente una a una.
- A differenza del piazzamento manuale (che crea singoli attori nel livello), il Foliage Mode raggruppa tutte le istanze posizionate in un unico `Foliage Actor`, ottimizzando le performance e semplificando la gestione.
- Per accedervi, basta selezionare la modalità `Foliage` dal menu a tendina delle modalità di editing.

15.6.2 Static Mesh Foliage vs. Static Mesh

- Nel Content Browser, alcune mesh sono già configurate come **Static Mesh Foliage** (evidenziate in verde). Queste hanno una serie di parametri preimpostati per essere utilizzate direttamente nel Foliage Mode (es. impostazioni di LOD, culling, collisioni).
- Le **Static Mesh** normali (colore standard) possono comunque essere usate nel Foliage Mode. Trascinandole nella finestra del Foliage, Unreal chiederà di creare un nuovo asset di tipo `StaticMeshFoliage`, convertendole automaticamente.
- La differenza principale è che la Static Mesh Foliage parte già con una configurazione ottimizzata per il piazzamento di massa, mentre la Static Mesh normale va configurata da zero al momento della conversione.

15.6.3 Gestione del Foliage generato dal Landscape Grass Output

- L'erba generata automaticamente dal materiale del Landscape (tramite `Landscape Grass Output`) viene gestita dal `Landscape Grass Type`.
- Selezionando il Landscape e andando nel pannello `Details` → `Landscape` → `Landscape Layers`, si trovano i parametri del Grass Type.
- **Parametri principali del Grass Type**:
  - **Densità** (`Density`): controlla quanto è fitto il manto d'erba. Valori più bassi diradano l'erba.
  - **Grid placement / Jitter**: determinano se l'erba è posizionata su una griglia regolare e quanta randomicità c'è nello spostamento. Un Jitter alto crea un aspetto più naturale e irregolare.
  - **Distanza di culling** (`Start/End Culling Distance`): distanza dalla telecamera entro cui il foliage viene renderizzato. Utile per ottimizzare le performance in scene grandi.
  - **LOD**: livello di dettaglio da usare per il foliage. Un valore di -1 usa la mesh completa; valori più alti usano versioni a minor numero di poligoni per ottimizzare a distanza.
  - **Scale** (`Scale Min/Max`): intervallo di scala casuale applicato a ogni istanza (es. Min=0.8, Max=1.1).
  - **Random Rotation**: se attivo, ogni istanza ha una rotazione casuale sull'asse Z.
  - **Align to Surface**: se attivo, l'istanza si orienta in base alla normale della superficie sottostante. Fondamentale per l'erba su pendii; per gli alberi può essere disattivato se si vuole che crescano sempre verticali.
  - **Receive Decals**: determina se il foliage può ricevere decal (es. schizzi di sangue, segni). Disabilitarlo può migliorare le performance.
  - **Cast Dynamic Shadows**: se il foliage proietta ombre dinamiche. Disabilitarlo fa risparmiare molte risorse, ma può peggiorare l'aspetto visivo.

15.6.4 Utilizzo pratico del Foliage Mode

- Dopo aver aggiunto i tipi di foliage desiderati nella finestra del Foliage Mode, si può iniziare a "dipingere" o "riempire" aree del Landscape.
- Gli strumenti di pittura permettono di regolare la densità del pennello e di aggiungere/rimuovere istanze in modo selettivo, proprio come si farebbe con un pennello di texture, ma piazzando oggetti 3D.
- Questo metodo è ideale per creare foreste, macchie di cespugli, zone rocciose, o per rifinire i bordi delle aree generate automaticamente dal Landscape Material.

15.7 Modalità Foliage

15.7.1 Gestione dei parametri di piazzamento

Densità e Raggio (Radius)

- **Densità (`Density`)**: determina il numero di istanze piazzate in un'area di riferimento (1000×1000 unità). Valori più alti producono un manto più fitto.
- **Raggio (`Radius`)**: impone una distanza minima tra le singole istanze. Un valore di 0 permette alle mesh di compenetrarsi; aumentandolo si forza una spaziatura fissa tra gli oggetti.
- **Interazione tra Densità e Raggio**:
  - Il raggio ha la **priorità** sulla densità: se il raggio è alto, anche con una densità elevata il numero di istanze sarà limitato dalla distanza minima imposta.
  - In pratica, l'engine calcola in anticipo il numero massimo di oggetti che possono essere inseriti nell'area del pennello rispettando il raggio. Cliccando più volte sulla stessa zona non si aggiungono nuove istanze se lo spazio è già saturo.
  - Esempio: con `Densità = 1000` e `Raggio = 1000`, vengono piazzate pochissime istanze (es. 3) perché la distanza minima è molto ampia. Riducendo il raggio, il numero di istanze aumenta.
- Il raggio è particolarmente utile per oggetti grandi (alberi, cespugli) per evitare compenetrazioni, mentre per l'erba si possono usare valori bassi o nulli per un manto uniforme.

Scala (`Scale`)

- La scala viene applicata in modo casuale a ogni istanza entro un intervallo definito (`Min` e `Max`).
- Di default la scala è **uniforme** (stesso valore per X, Y, Z), ma è possibile sbloccare gli assi per scalare in modo non uniforme (utile per effetti particolari, ma generalmente sconsigliato perché distorce le mesh).
- Un intervallo di scala (es. 0.8–1.5) crea un aspetto più naturale e variegato.

Offset verticale (`Z Offset`)

- Permette di spostare casualmente le istanze lungo l'asse Z entro un intervallo (`Min`, `Max`).
- Utile per simulare oggetti sospesi o fluttuanti (es. pietre che levitano), ma con l'erba è poco usato. Può essere impiegato per composizioni artistiche o scene statiche.

Allineamento alla normale (`Align to Normal` e `Max Angle`)

- `Align to Normal`: se attivo, ogni istanza si orienta seguendo la normale della superficie sottostante. Fondamentale per l'erba su pendii.
- `Max Angle`: l'angolo massimo di inclinazione che la mesh può assumere per adattarsi alla pendenza.

Filtro per pendenza del terreno (`Ground Slope Angle`)

- Definisce l'intervallo di angoli (minimo e massimo) della superficie su cui il foliage può essere piazzato.
- Esempio: impostando un range di 0°–45°, il foliage viene posizionato solo sulle superfici con pendenza inferiore o uguale a 45°, evitando pareti verticali.
- Utile per evitare che l'erba cresca su pareti rocciose o per confinare certi tipi di vegetazione solo in zone pianeggianti.

Filtro per altezza (`Height`)

- Simile al filtro per pendenza, ma basato sull'altitudine (coordinata Z assoluta).
- Permette di stabilire una fascia altimetrica (es. da 0 a 1000 unità) entro cui il foliage viene piazzato. Al di fuori di questo range, il foliage non verrà posizionato.
- Esempio: si può fare in modo che un certo tipo di albero appaia solo sopra una certa quota.

Mobilità (`Mobility`)

- Specifica se le istanze di foliage sono `Static` (ferme, non si muoveranno mai) o `Movable` (possono essere spostate a runtime).
- Ai fini del foliage per il dressing, si lascia tipicamente su `Static` per ottimizzare le performance.

15.7.2 Attenzione alle performance

- Piazzare un numero molto elevato di istanze in una singola area (es. densità altissima su un brush grande) può causare rallentamenti o crash, specialmente su macchine non performanti.
- È consigliabile aumentare la densità gradualmente e monitorare il numero di istanze generate.
- L'utilizzo del raggio e dei filtri aiuta a mantenere il controllo sul numero di oggetti.

15.8 Correzioni e approfondimenti sull'illuminazione

15.8.1 Lumen e Global Illumination

- La distinzione fra luci `Static`, `Stationary` e `Movable` e la necessità di *baking* valgono soltanto quando **non** si utilizza **Lumen**, il sistema di illuminazione globale dinamica predefinito in Unreal Engine 5.
- Per disattivare Lumen e tornare al sistema di illuminazione tradizionale con baking:
  - `Project Settings` → `Rendering` → `Global Illumination` e `Reflections`, impostare su `None`.
- Disabilitare Lumen può essere utile per progetti destinati a piattaforme poco performanti (mobile, PC di fascia bassa).

15.8.2 God Rays (Light Shaft) tramite Directional Light

- **Metodo rapido**: è possibile ottenere l'effetto *God Rays* (raggi volumetrici) direttamente dalla `Directional Light`, senza utilizzare un `Post Process Volume`.
- Parametri da regolare nella `Directional Light` (sezione `Light` → `Light Shaft`):
  - **Light Shaft Occlusion**: controlla l'effetto di occlusione dei raggi.
  - **Light Shaft Bloom**: intensità del bagliore dei raggi.
  - **Brightness**: luminosità complessiva dell'effetto.

  - **Threshold**: soglia di luminosità della scena oltre la quale l'effetto viene generato. Valori più bassi rendono i raggi visibili più facilmente.

- Ruotando la `Directional Light` e regolando questi parametri si può simulare l'ingresso di raggi di sole da finestre o aperture, creando atmosfere suggestive (es. effetto desertico, controluce).

- **Alternativa per interni**: Unreal mette a disposizione asset predefiniti come i `SM_Beam`, mesh statiche a forma di cono con materiale emissivo, che simulano raggi volumetrici senza richiedere calcoli complessi. Si consiglia di copiare questi asset nel proprio Content Browser prima di utilizzarli.

15.9 Gestione del Foliage generato dal Landscape

- L'erba generata automaticamente dal materiale del Landscape (tramite `Landscape Grass Output`) è controllata dal **Grass Type** associato al layer.

- Per modificare i parametri, selezionare il Landscape, andare in `Details` → `Landscape` → `Landscape Layers`, e aprire il Grass Type del layer desiderato.

- **Parametri principali del Grass Type**:

  - **Densità** (`Density`): numero di istanze per area. Valori più alti creano un manto più fitto.
  - **Grid Placement / Jitter**: se attivo, le istanze sono posizionate su una griglia; `Jitter` aggiunge casualità allo spostamento, creando un aspetto più naturale.
  - **Distanza di culling** (`Start/End Culling Distance`): distanza dalla telecamera entro cui il foliage viene renderizzato. Utile per ottimizzare.
  - **LOD**: livello di dettaglio; -1 usa la mesh piena, valori superiori usano versioni semplificate.
  - **Scale** (`Min/Max`): intervallo di scala casuale (es. 0.8–1.1). Mantenere l'uniformità per non distorcere le mesh.
  - **Random Rotation**: rotazione casuale sull'asse Z.
  - **Align to Surface**: le istanze si orientano in base alla normale del terreno. Fondamentale per erba su pendii; può essere disattivato per alberi che devono crescere dritti.
  - **Receive Decals**: se il foliage riceve proiezioni di decal. Disabilitare per migliorare le performance.
  - **Cast Dynamic Shadows**: se il foliage proietta ombre dinamiche. Molto impattante sulle performance; va usato con cautela.

15.10 Modalità Foliage (Pittura manuale)

15.10.1 Concetti fondamentali

- La **Foliage Mode** permette di posizionare e randomizzare grandi quantità di mesh statiche (alberi, cespugli, erba, rocce, detriti) utilizzando un sistema a pennello, senza dover piazzare manualmente ogni singolo attore.

- Tutte le istanze posizionate con il Foliage Mode vengono raggruppate in un unico `Foliage Actor`, ottimizzando le performance rispetto a centinaia di `StaticMeshActor` separati.

- Per accedere alla modalità, selezionare `Foliage Mode` dal menu a tendina delle modalità di editing.

15.10.2 Tipi di mesh utilizzabili

- **Static Mesh Foliage** (verdi nel Content Browser): asset già preconfigurati per l'uso nel Foliage Mode, con parametri di default ottimizzati (LOD, culling, collisioni).

- **Static Mesh** normali: possono essere utilizzate trascinandole nella finestra del Foliage Mode; Unreal chiederà di creare un nuovo asset `StaticMeshFoliage`, convertendole automaticamente.

15.10.3 Strumenti di pittura

- **Paint**: aggiunge istanze nell'area del pennello.
- **Erase**: rimuove istanze (tenendo premuto `Shift` o selezionando lo strumento).
- **Selezione dei tipi di foliage**: tramite i checkbox si decide quali tipi di mesh vengono piazzati o cancellati contemporaneamente.
- **Brush Size**: regola la dimensione del pennello.
- **Paint Density** e **Erase Density**: moltiplicatori temporanei della densità per aggiungere o rimuovere rispettivamente più o meno oggetti durante la sessione di pittura (es. densità 0.5 per diradare, 2 per infittire).
- **Single Instance Mode**: permette di posizionare una singola istanza alla volta, utile per dettagli precisi dopo la stesura generale, mantenendo le istanze all'interno del `Foliage Actor`.

15.10.4 Parametri di piazzamento per tipo di foliage

Ogni tipo di foliage aggiunto alla palette ha parametri indipendenti che controllano la distribuzione e l'aspetto delle istanze.

Densità e Raggio (`Density` e `Radius`)

- **Densità**: numero di istanze per unità di area (riferimento: 1000×1000 unità). Valori più alti aumentano il numero di oggetti.
- **Raggio**: distanza minima tra le singole istanze. Impedisce la compenetrazione.
- **Priorità del Raggio**: il raggio ha la priorità sulla densità. Se il raggio è alto, anche con densità elevata, verranno piazzate poche istanze perché lo spazio è saturato dalla distanza minima.
- Consiglio: per l'erba usare raggio basso o nullo e densità media; per alberi e cespugli usare un raggio adeguato alla loro dimensione per evitare sovrapposizioni.

Scala (`Scale`)

- `Scale Min/Max`: intervallo di scala casuale applicato ad ogni istanza. L'opzione `Uniform` fa sì che la scala sia la stessa su tutti gli assi (consigliato).
- Un intervallo ampio (es. 0.8–1.5) crea un aspetto più naturale e variegato.

Offset verticale (`Z Offset`)

- `Z Offset Min/Max`: sposta casualmente le istanze lungo l'asse Z.
- Utile per effetti particolari (es. rocce fluttuanti) o per correggere l'affondamento di mesh nel terreno. Generalmente non si usa per l'erba.

Allineamento e pendenza

- **Align to Normal**: se attivo, ogni istanza si orienta in base alla normale della superficie sottostante.
- **Max Angle**: angolo massimo di inclinazione rispetto alla normale.
- **Ground Slope Angle** (`Min/Max`): intervallo di pendenza del terreno entro cui il foliage può essere piazzato. Utile per impedire la crescita di erba su pareti rocciose (es. `Min=0, Max=45`).

Filtro per altezza (`Height`)

- `Height Min/Max`: intervallo di altitudine (coordinata Z) in cui il foliage viene posizionato. Consente di confinare certi tipi di vegetazione a fasce altimetriche specifiche (es. alberi solo sopra i 500 metri).

Mobilità e Culling

- **Mobility**: `Static` (più performante, per oggetti fissi) o `Movable`.
- **Cull Distance** (`Min/Max`): distanza dalla telecamera entro cui il foliage viene renderizzato. Impostare `0` disabilita il culling e lascia le istanze sempre visibili (può essere molto pesante).

Collisioni

- **Collision Presets**: permette di impostare il comportamento delle collisioni per tutte le istanze di quel tipo di foliage (es. `BlockAll`, `NoCollision`).
- **Attenzione**: se la Static Mesh originale **non ha collisioni** (o le ha errate), modificare il preset nel Foliage Mode non avrà effetto. È necessario aprire la mesh e generare/aggiustare le collisioni (es. `Use Complex Collision as Simple`, aggiungere una capsula, ecc.).
- Esempio pratico: un albero con collisione mal posizionata può essere corretto ruotando le primitive di collisione nell'editor della mesh, oppure usando `Complex Collision as Simple` (più pesante ma più fedele).
- Per oggetti decorativi (erba, piccoli cespugli) si usa tipicamente `NoCollision` per non ostacolare il personaggio.

15.10.5 Gestione manuale delle istanze

- **Selezione**: con lo strumento `Select` si possono selezionare singole istanze o gruppi di istanze (trascinando una regione).
- Dalla selezione è possibile **spostare**, **ruotare**, **scalare** o **eliminare** (`Remove`) le istanze selezionate, proprio come normali attori, pur rimanendo all'interno del `Foliage Actor`.
- Questo flusso di lavoro unisce la velocità del piazzamento a pennello con la precisione del posizionamento manuale.

15.10.6 Consigli sull'uso e performance

- Piazzare quantità enormi di istanze in un colpo solo (es. densità 10000 su brush grande) può causare rallentamenti o crash, specialmente su macchine non potenti. Aumentare la densità gradualmente.
- Il Foliage Mode non è limitato alla vegetazione: può essere usato per qualsiasi mesh ripetibile in gran numero (detriti, sassi, cartacce, rottami) per arricchire rapidamente una scena.
- Alcuni materiali (come quelli *World Aligned*) possono cambiare aspetto se l'oggetto viene spostato o ruotato, perché il loro calcolo si basa sulla posizione/orientamento nel mondo. Tenere presente questo comportamento quando si ruotano istanze.

15.11 Assegnazione per la prossima lezione

- **Obiettivo**: creare un piccolo livello di esplorazione utilizzando **Landscape** e **Foliage Mode**.
- **Indicazioni**:
  - Realizzare un Landscape di dimensioni contenute (es. `31×31` componenti, per avere spazio sufficiente senza esagerare).

  - Scolpire il terreno con alture, avvallamenti e, volendo, piccole grotte o caverne.
  - Utilizzare il **materiale automatico** visto a lezione (o uno proprio) per gestire la transizione tra i materiali in base a pendenza e altezza.
  - Popolare il paesaggio con il **Foliage Mode**, posizionando erba, cespugli e alberi in modo da creare un ambiente naturale curato.
  - Inserire alcuni **collezionabili** (es. 2-3) sparsi per la mappa, in punti che invoglino il giocatore a esplorare.
  - Opzionale: aggiungere piattaforme mobili o altri elementi interattivi per permettere spostamenti verticali o raggiungere zone sopraelevate.

- **Consegna**:
  - Poiché molti non potranno partecipare all'ultima lezione, si consiglia di registrare un breve video del personaggio che si muove nell'ambiente creato e caricarlo su Discord (o su un drive con link).
  - In alternativa, durante l'ultima lezione si vedrà il **packaging**, per creare un eseguibile giocabile da condividere.

15.12 Introduzione al Post-Process Volume

15.12.1 Cos'è e dove si trova

- Il **Post Process Volume** è un attore di tipo volume che applica effetti visivi alla "lente" della telecamera, modificando la resa grafica di tutto ciò che viene visto al suo interno.
- Si trova in: `Place Actors` → `Volumes` → `Post Process Volume`.
- A differenza di un `Camera Actor`, che può avere i propri effetti di post-process validi solo per quella telecamera, il Post Process Volume agisce quando il giocatore (o la telecamera attiva) si trova **all'interno del suo volume**.

15.12.2 Proprietà fondamentale: `Infinite Extent (Unbound)`

- Di default, gli effetti sono confinati all'area del volume (rappresentata da un box).
- Spuntando l'opzione `Infinite Extent (Unbound)`, il volume estende i suoi effetti all'infinito, coprendo l'intero mondo di gioco. Questo permette di avere un unico Post Process Volume che definisce l'aspetto globale della scena, senza doverlo ridimensionare enormemente.

15.12.3 Utilizzo di più volumi e transizioni

- È possibile posizionare più Post Process Volume nella scena.
- Quando il giocatore passa da un volume all'altro, gli effetti di post-process si **interpolano** gradualmente, creando transizioni morbide. Questo è utile, ad esempio, per simulare l'adattamento dell'occhio quando si passa da un ambiente esterno molto luminoso a un interno buio, o per cambiare la correzione colore tra zone diverse.
- **Attenzione**: se due volumi si sovrappongono, i loro effetti si sommano, il che può portare a risultati inaspettati e difficili da controllare. In generale, è meglio evitare sovrapposizioni o gestirle con molta cautela.

15.12.4 Esempi di effetti e lenti

- Il Post Process Volume contiene decine di parametri suddivisi in categorie (`Lens`, `Bloom`, `Color Grading`, `Depth of Field`, ecc.).
- **Bloom**: gestisce il bagliore attorno alle sorgenti luminose. È lo stesso effetto regolabile anche dalla `Directional Light` per i *God Rays*, ma qui è applicato globalmente.
- **Chromatic Aberration**: effetto di sfasamento dei colori ai bordi dell'immagine, spesso usato per simulare lenti di scarsa qualità o per dare un aspetto "vintage". Può risultare fastidioso se troppo accentuato.
- Molti altri effetti (vignettatura, grana, correzione colore, mappatura tonale) saranno approfonditi in seguito.

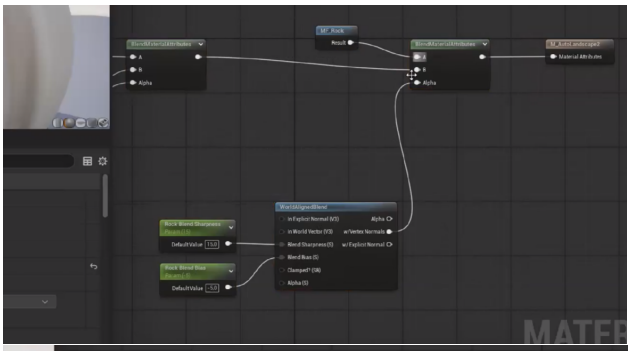
15.13 Domande e approfondimenti

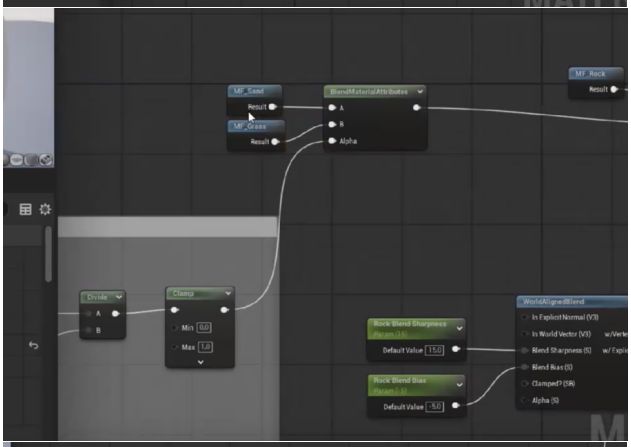
15.13.1 God Rays e finestre colorate

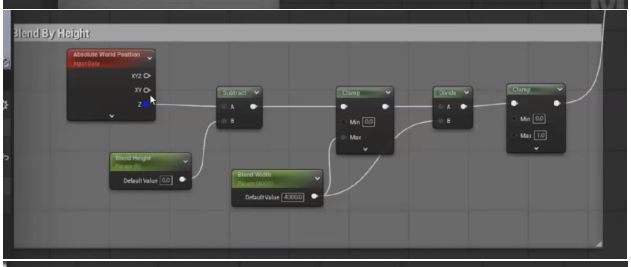
- **Domanda**: per realizzare l'effetto di una vetrata colorata che proietta luce colorata all'interno, si usa la post-produzione o le luci?
- **Risposta**: come spesso accade nel level design, la soluzione migliore è una **combinazione di tecniche**.
  - **Materiale**: la vetrata deve essere una mesh con un materiale trasparente e colorato, che filtri la luce visivamente.
  - **Luce**: si può posizionare una `Spotlight` o una `Rect Light` dietro la vetrata, con il colore desiderato, in modo che la luce proiettata nella stanza sia effettivamente colorata.
  - **Post-Process**: si può aggiungere un `Post Process Volume` all'interno della stanza per enfatizzare l'effetto, ad esempio modificando il *Color Grading* per rendere l'atmosfera più calda e satura, oppure regolando il *Bloom* per dare maggiore impatto ai raggi luminosi.
- Il level design è spesso un lavoro di "trucchi" e stratagemmi per ottenere un risultato visivo credibile senza appesantire eccessivamente l'engine con calcoli fisicamente accurati.

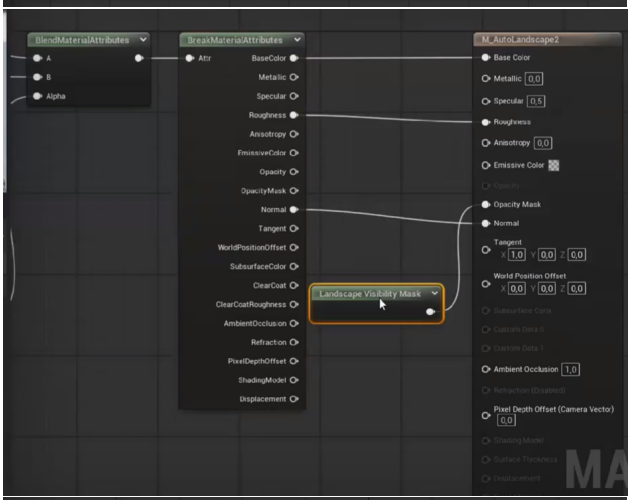
15.13.2 Conclusioni e saluti

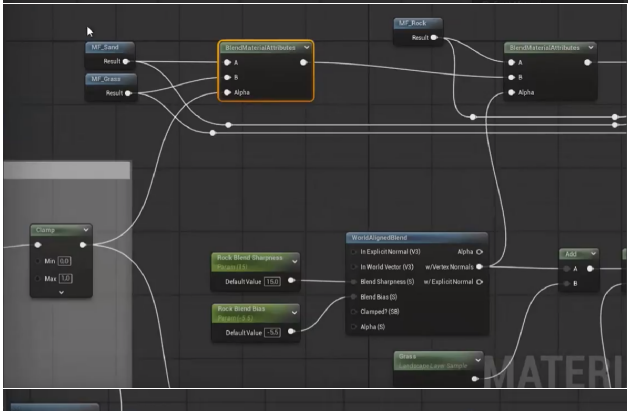
- Il docente ricorda di utilizzare il server Discord per qualsiasi domanda o per condividere i progressi.
- La prossima lezione, l'ultima, tratterà il **packaging** (creazione di eseguibili) e concluderà il corso.
- Augura a tutti un buon fine settimana e una buona serata.




MF_Sand
MF_Grass
MF_Rock
BlendMaterialAttributes
Result
A
B
Alpha
Divide
Clamp
Min 0,0
Max 1,0
Rock Blend Sharpness
Default Value 15,0
Rock Blend Bias
Default Value -5,0
WorldAlignedBlend


Blend By Height
Absolute World Position
XYZ
XY
Z
Subtract
Clamp
Divide
Blend Height
Default Value 0,0
Blend Width
Default Value 400000


BlendMaterialAttributes
A
B
Alpha
BreakMaterialAttributes
Attr
BaseColor
Metallic
Specular
Roughness
Anisotropy
FmissiveColor
Opacity
OpacityMask
Normal
Tangent
WorldPositionOffset
SubsurfaceColor
ClearCoat
ClearCoatRoughness
AmbientOcclusion
Refraction
PixelDepthOffset
ShadingModel
Displacement
M_AutoLandscape2
Base Color
Metallic 0,0
Specular 0,5
Roughness
Anisotropy 0,0
Emissive Color
Opacity Mask
Normal
Tangent
World Position Offset
Landscape Visibility Mask
Ambient Occlusion 1,0
Pixel Depth Offset (Camera Vector)
0,0


MF_Sand
MF_Grass
MF_Rock
BlendMaterialAttributes
Result
A
B
Alpha
Clamp
Min 0,0
Max 1,0
Rock Blend Sharpness
Default Value 15,0
Rock Blend Bias
Default Value -5,5
WorldAlignedBlend
Add

Chapter 16

Introduzione al Game Audio: Ruoli, Tecniche ed Evoluzione del Suono nei Videogiochi

16.1 Introduzione alla masterclass di Game Audio

- Docente: Davide Pensato, fondatore di `DP Studios` (studio verticale specializzato in audio per videogiochi dal 2008).
- La masterclass ha un taglio teorico-narrativo: non un workshop pratico, ma una panoramica sul ruolo, gli strumenti e il flusso di lavoro del Game Audio, con possibilità di vedere qualche strumento.
- Importanza dello scambio diretto, anche in lezione online: viene chiesto a ogni partecipante di presentarsi brevemente (nome, percorso, obiettivi, rapporto con musica/sound design, gioco preferito per l'audio).

16.2 Percorso ed esperienza del docente

16.2.1 Formazione e ingresso nell'industria

- Laurea in Ingegneria delle Telecomunicazioni (dopo un passaggio da Gestionale), unendo la passione tecnico-scientifica a quella musicale (chitarra, canto, studio del suono).
- Primo impiego come Audio Designer in **Ubisoft Milano** (2000), ottenuto dopo aver lasciato il curriculum al Laboratorio di Informatica Musicale dell'Università Statale di Milano, che lo indirizzò verso una posizione aperta.
- Ha lavorato in Ubisoft per circa 8 anni e mezzo, contribuendo a progetti come:
  - `Rayman` (Game Boy Color) – esperienza con l'audio a 8 bit.
  - `Rayman` (Game Boy Advance) – prime forme di audio digitale su portatile.
  - `Tomb Raider: The Prophecy` (GBA).
  - Titoli per PC, Xbox, PlayStation 2 e Xbox 360 (es. `Splinter Cell: Double Agent`).
- Dopo l'esperienza in Ubisoft, diventa libero professionista e fonda `DP Studios`.

16.2.2 Consigli su branding e presenza online

- Scegliere con cura se proporsi con un **marchio/studio** o con il **nome e cognome**: entrambe le strade sono valide, ma è fondamentale curare l'identità visiva.
- Elementi consigliati:
  - Logo e palette di colori coerenti.
  - Sito web personale/dello studio con portfolio dei lavori (galleria organizzata).
  - Profilo LinkedIn curato.
  - In alternativa o in aggiunta, un `Linktree` ben impostato (con logo e colori coordinati).
- Motivazione: quando si incontrano team di sviluppo (es. a eventi come *First Playable*) o si inviano candidature, avere un riferimento online chiaro e professionale è indispensabile per il follow-up.

16.2.3 Specializzazione verticale e portfolio

- `DP Studios` si occupa **esclusivamente** di audio per videogiochi.
- Scelta deliberata: non mescolare nel portfolio dello studio lavori eterogenei (cinema, pubblicità, installazioni) perché è difficile eccellere in tutto; eventuali lavori extra vengono tenuti separati (portfolio personale del collaboratore o altro brand).
- Il portfolio dello studio conta numerosi progetti; il team è composto da professionisti con partita IVA, coordinati in base ai periodi di lavoro.

16.3 Il ruolo dell'audio nel videogioco: lineare vs interattivo

16.3.1 Il problema dell'audio lineare applicato ai giochi

- L'approccio `cinematografico` (musica e suoni lineari, non reattivi) è comune ma inadeguato perché il videogioco è un **medium interattivo** per definizione.
- La semplice riproduzione di un file audio statico (es. un brano musicale) crea problemi quali:
  - **Disallineamento dinamico**: un giocatore che resta fermo in un'area sente la musica avanzare in modo scollegato dall'azione.
  - **Ripetitività**: il loop di un brano di 2-3 minuti può diventare fastidioso se ascoltato decine di volte.
- L'interattività obbliga a ripensare il flusso audio: non più una timeline fissa, ma un sistema che reagisce alle azioni del giocatore.

16.3.2 Parallelismo con il cinema e differenza fondamentale

- Il sonoro nel cinema lineare si è evoluto come **rafforzativo**: accompagna l'immagine, enfatizza le emozioni, ma non può cambiare lo svolgimento della storia.
- Nel videogioco, l'audio deve **partecipare attivamente** al gameplay e alla narrativa.
- Esempi di funzioni specifiche dell'audio interattivo:
  - **Gameplay**: segnalare un pericolo imminente, indicare la direzione di un nemico, confermare un'azione (pick-up, porta che si apre), fornire conteggio alla rovescia o allarmi.
  - **Narrativa**: sottolineare un colpo di scena, cambiare tono in base alle scelte del giocatore, segnalare un cambiamento di stato emotivo del personaggio.

16.3.3 Flusso di lavoro tipico del Game Audio

- Il lavoro non si limita alla creazione dei contenuti sonori, ma include fasi fondamentali di collaborazione con il team di sviluppo:

  1. **Analisi del design document** (quando presente) o confronto diretto con game designer e director per capire il tono, lo stile, le meccaniche.
  2. **Realizzazione** di bozze, mock-up e vere e proprie *vertical slice* audio.
  3. **Integrazione** nel motore di gioco: l'audio deve essere collegato alle variabili di gioco (stato del personaggio, posizione, eventi di collisione, condizioni ambientali). Questo richiede una stretta sinergia con i programmatori.
  4. **Test e debug** direttamente in-game: verificare che i suoni si attivino correttamente, che i volumi siano bilanciati, che non ci siano conflitti di priorità o problemi di performance.

- **Strumenti di integrazione**: vengono citati middleware audio (es. `FMOD`, `Wwise`) e sistemi built-in degli engine (es. Unreal Engine, Unity). Il docente mostra un esempio concreto di integrazione in Unreal Engine (area delle collisioni, eventi, `Sound Cue`, `Audio Component`) per far vedere come i suoni siano collegati a eventi di gioco.

16.4 Conclusioni e prossimi argomenti

- L'audio nei videogiochi è un campo che richiede competenze ibride: sensibilità musicale/sonora, conoscenze tecniche (elaborazione del segnale, formati, compressione) e capacità di programmazione/scripting di base per l'implementazione.
- Non serve essere virtuosi di uno strumento: è più importante la capacità di creare l'atmosfera giusta e risolvere problemi di design con il suono.
- **Consiglio pratico**: iniziare a fare esperienza partecipando a *game jam* (es. Global Game Jam) o proponendosi a team indie, anche gratuitamente all'inizio, per costruire un portfolio di progetti reali.
- Il docente rimane a disposizione per domande e per condividere riferimenti (libri, community online, siti di annunci per game audio).

16.5 Masterclass di Game Audio – Davide Pensato (DP Studios)

16.5.1 Introduzione e presentazione del docente

- **Davide Pensato**, fondatore di `DP Studios`, studio specializzato esclusivamente in *Game Audio* dal 2008.
- La masterclass ha taglio teorico-narrativo: panoramica su cos'è l'audio per videogiochi, ruoli, competenze, pipeline produttiva e integrazione.
- Importanza dello scambio diretto: giro di presentazioni dei partecipanti (nome, formazione, obiettivi, rapporto con musica/sound design, gioco preferito per l'audio).

16.5.2 Il percorso del docente

- Formazione: Ingegneria delle Telecomunicazioni (dopo passaggio da Gestionale), unendo passione tecnico-scientifica e musicale (chitarra, canto, studio del suono).
- Primo impiego come **Audio Designer in Ubisoft Milano** (2000), ottenuto lasciando il curriculum al Laboratorio di Informatica Musicale dell'Università Statale di Milano.
- 8 anni e mezzo in Ubisoft, lavorando a progetti come:

  - `Rayman` (Game Boy Color) – audio 8 bit.
  - `Rayman` (Game Boy Advance).
  - `Tomb Raider: The Prophecy`.
  - `Splinter Cell: Double Agent` (Xbox 360) e altri titoli per PC, Xbox, PlayStation 2.
- Passaggio a libero professionista e fondazione di `DP Studios`.

16.5.3 Consigli su branding e presenza online

- Scegliere tra marchio/studio o nome e cognome; in ogni caso curare l'identità visiva.
- Elementi fondamentali:
  - Logo e palette di colori coerenti.
  - Sito web/portfolio ben organizzato.
  - Profilo LinkedIn curato.
  - Eventuale `Linktree` con logo e colori coordinati.
- Motivazione: negli incontri professionali (es. eventi come *First Playable*) e nel follow-up, un riferimento online chiaro è indispensabile.

16.5.4 Il portfolio di DP Studios e la specializzazione

- DP Studios si occupa **esclusivamente** di audio per videogiochi; non mescola nel portfolio lavori eterogenei (cinema, pubblicità) per mantenere focus e qualità.
- I progetti in showcase sono ormai tutti di studi italiani, segnale della crescita dell'industria italiana.

16.5.5 Contrazione del settore gaming

- Il settore è in contrazione globale, ma non dovuta all'IA come minaccia diretta al lavoro.
- Cause principali:
  1. **Bolla post-Covid**: durante la pandemia boom del gaming → assunzioni massicce; finita l'emergenza, ridimensionamento e licenziamenti (es. Epic Games).
  2. **Spostamento investimenti**: i capitali si stanno dirigendo verso aziende di intelligenza artificiale (OpenAI, Anthropic), riducendo i fondi per il gaming.
  3. **Nuovi modelli di business**: maggiore rischio per titoli AA; crescono i giochi `triple I` (indie di alta qualità con team 10 persone).

16.5.6 L'importanza dell'audio nel videogioco

- Citazione di Steven Spielberg: `Il suono e la musica comunicano più della metà dell'esperienza in un film, ancora più di quello che vedi`.
- Nel videogioco (medium interattivo) questa affermazione è ancora più vera: l'audio permette di percepire ciò che non è visibile, influenzando le scelte del giocatore.
- Tre pilastri dell'audio nei giochi:
  1. **Potere evocativo** – suscitare emozioni, rafforzare situazioni, anticipare eventi.
  2. **Caratterizzazione** – definire ambienti, personaggi, stati d'animo.
  3. **Supporto al gameplay** – fornire informazioni funzionali al gioco (posizione nemici, pericoli, stati del personaggio, feedback azioni).

16.5.7 Esempi video

Gears of War – Trailer `Mad World`

- Scena di guerra sonorizzata non con musica epica/tensiva, ma con un brano malinconico (*Mad World*).
- Effetto: spostamento emotivo totale dall'azione adrenalinica alla tristezza, all'introspezione, all'inevitabilità della morte.
- Dimostra il potere evocativo della musica nel ridefinire la percezione di una scena.

Assassin's Creed – Gameplay

- Cinematica iniziale con musica di forte tensione.
- Nel momento in cui il controllo passa al giocatore, la musica si `apre`, liberando la tensione e invitando all'azione.
- Più avanti, entrando in una zona di combattimento (trigger), un layer ritmico si aggiunge in fade-in, aumentando progressivamente l'intensità.
- Mostra come la musica possa adattarsi in tempo reale agli stati di gioco (esplorazione → azione).

Manhunt (Rockstar Games)

- Gioco citato per l'uso del solo sound design (assenza di musica) nel creare atmosfera emotiva e tensione.
- Il lead game designer era Christian Cantamessa (ex Ubisoft).
- Il contesto narrativo (condannato a morte costretto a uccidere in una città fantasma) è amplificato dall'audio ambientale e dai suoni diegetici, dimostrando che anche senza musica si può veicolare un forte impatto emotivo.

16.5.8 Ruoli e competenze nel Game Audio

- Le figure professionali spaziano dal **Sound Designer** (creazione suoni, effetti, ambienti) al **Compositore** (musiche interattive), fino al **Tecnico dell'implementazione** (scripting, middleware, engine).
- Serve un mix di sensibilità artistica e competenze tecniche (elaborazione del segnale, formati audio, programmazione di base).
- Non è indispensabile essere virtuosi di uno strumento; conta la capacità di creare l'atmosfera giusta e risolvere problemi di design con il suono.

16.5.9 Pipeline di produzione e integrazione

1. **Analisi del design document** o briefing con game designer/director per comprendere tono, stile, meccaniche.
2. **Creazione** di bozze, mock-up e vertical slice audio.
3. **Integrazione** nel motore di gioco (Unreal, Unity) o tramite middleware (FMOD, Wwise): collegare suoni a variabili di gioco, eventi, stati.
4. **Test e debug** direttamente in-game: bilanciamento volumi, priorità, performance.
5. La collaborazione stretta con i programmatori è fondamentale.

16.5.10 Consigli pratici per iniziare

- Partecipare a **game jam** (es. Global Game Jam) per costruire un portfolio di progetti reali.
- Proporsi a team indie, anche inizialmente a titolo gratuito, per fare esperienza e creare contatti.
- Riferimenti utili: community online, libri, siti di annunci specializzati in game audio.

16.6 Approfondimenti sul potere evocativo: esempi di audio oppressivo

16.6.1 Manhunt (Rockstar Games)

- Il lead game designer Christian Cantamessa (ex Ubisoft) ha dato al dipartimento audio l'input di rendere il gioco il più ansiogeno e opprimente possibile.
- Caratteristiche audio:
  - Voce off e tutorial sempre filtrati con effetto radio, per alienazione.
  - Nessuna musica durante il gameplay: solo nelle cutscene, con brani punk rock ruvidi.
  - Ambiente sonoro ibrido: suoni di città fantasma mescolati a elementi tonali ambigui (non chiaramente musicali).
  - Effetti come il battito cardiaco che aumentano la tensione.
  - Anche senza musica, il solo sound design comunica oppressione, inquietudine e paura.

16.6.2 Altri esempi citati

- **Alien Isolation**: suoni ambientali continui (l'alieno che si muove nei condotti) creano paura costante, senza bisogno di jump scare.
- **Silent Hill**: il suono di Pyramid Head genera panico immediato; il giocatore impara ad associarlo all'apparizione e lo sente spesso quando il mostro non è visibile.
- **Scanner Sombre**: scansione digitale dell'ambiente; i rumori si deformano in base alla vicinanza di qualcosa, generando ansia anche senza incontrare mai un mostro.
- **Jumpscare vs atmosfera**: molti trovano più spaventoso un sound design capace di rievocare paure reali e sensazioni di tensione costante piuttosto che spaventi improvvisi.

16.7 La caratterizzazione

16.7.1 Definizione e complessità

- In un videogioco la caratterizzazione è **in tempo reale** e va intesa come **simulazione**: ogni oggetto interagibile può trovarsi in stati diversi (es. integro, danneggiato, distrutto) e deve emettere suoni corrispondenti.
- Non si tratta di un semplice loop di sottofondo, ma di un sistema che lancia suoni in base alle condizioni di gioco, variandoli man mano che il giocatore si muove o interagisce con l'ambiente.

16.7.2 Esempio: Hitman (missione a Kinshasa)

- L'ambiente urbano africano è quasi privo di movimento visivo, ma il suono lo popola: clacson, voci lontane, creando l'idea di un villaggio.
- Caratterizzazione dei passi: cambio di superficie (marciapiede, asfalto, ghiaia, legno) corrisponde a suoni diversi.
- Ambiente interno: riverbero differente, assenza dei suoni esterni.
- Oggetti: colpire una ringhiera metallica o una televisione accesa produce suoni specifici; se la TV viene distrutta, la musichetta cessa e rimane il rumore bianco.
- In questo tipo di giochi non c'è quasi mai musica in-game; l'immersione è affidata al sound design dettagliato (armi, armature, passi, porte, ambienti, riverberi).

16.7.3 Esempio: Assassin's Creed (con musica diegetica)

- Oltre ai passi e ai suoni ambientali, sono presenti dialoghi che partono casualmente dalle coppie di personaggi, rafforzando la sensazione di folla.
- Suoni di piccioni, fuochi d'artificio, guardie, rumori di arrampicata: ogni elemento contribuisce a contestualizzare epoca e luogo.
- La musica diegetica (es. suonatori nella piazza) contribuisce all'immersione.
- L'obiettivo del sound design è proprio **far sentire immersi** nel mondo di gioco.

16.8 Supporto al gameplay

16.8.1 Guida e informazione tramite l'audio

- L'audio è fondamentale per guidare il giocatore: i suoni forniscono informazioni cruciali sullo stato del gioco e dell'ambiente.
- **Giochi stealth**: ascoltare i passi dei nemici, capire quando si fermano, ripartono, permette di dedurre i percorsi di pattuglia e decidere quando muoversi.
- **Camera di sorveglianza**: sfruttando l'audio stereo, il giocatore muove la telecamera per individuare la direzione del suono, capire dove si trova la minaccia e danneggiarla.
- **Indicatore di combattimento**: la musica può segnalare se si è ancora in fase di combattimento (es. nei giochi a orde) o se tutti i nemici sono stati eliminati, permettendo al giocatore di non guardarsi più intorno.

16.8.2 Sincronizzazione musicale con le meccaniche

- **Esempio della finishing move**: durante l'animazione finale, il motore di gioco sa già che il nemico è morto e comunica l'informazione al motore audio.
- Il compositore ha così diversi secondi di tempo (es. 5 secondi) per concludere la frase musicale, inserire una transizione e preparare la musica successiva perfettamente a tempo.
- La musica interattiva si basa su questi `anticipi` forniti dal motore di gioco (dati, statistiche, eventi) per reagire in modo fluido.

16.8.3 Sfruttamento intelligente della musica da parte del giocatore

- **Esempio di stealth**: un giocatore apre di proposito una porta, facendo scattare l'allarme e la musica di tensione. Attira la guardia, la neutralizza, la porta via.
- Quando la musica di allerta cala, capisce che non c'erano altre guardie allertate nella zona, perché normalmente un sistema di allarme coinvolgerebbe più nemici.
- Avendo `testato` il livello di minaccia attraverso la musica, il giocatore può poi attraversare il corridoio senza ulteriori precauzioni.
- Questo mostra come la struttura musicale possa diventare uno strumento di gameplay direttamente sfruttabile dal giocatore.

16.9 Conclusioni parziali

- Abbiamo analizzato i tre pilastri dell'audio nei videogiochi:
  1. **Potere evocativo** (emozioni, tensione, paura).
  2. **Caratterizzazione** (immersione, simulazione di ambienti e oggetti).
  3. **Supporto al gameplay** (informazioni, guida, anticipazione di eventi).
- La musica e il sound design possono agire singolarmente o in combinazione per raggiungere questi obiettivi, rendendo il videogioco un'esperienza interattiva completa.

16.10 Storia dell'audio nei videogiochi

16.10.1 Gli albori: anni '80

- Il videogioco diventa forma di intrattenimento di massa; nascono publisher, generi (RPG, adventure, platformer, racing).
- Le console e i PC montano **chip audio** dedicati: piccoli sintetizzatori con un numero limitato di voci (es. 4 voci: due onde quadre, un'onda custom, un rumore bianco).
- Musica ed effetti sonori vengono creati interamente in sintesi, scrivendo direttamente i registri del chip in linguaggio macchina o assembly.
- I team sono minuscoli (2-3 persone); spesso il **programmatore** è anche compositore e sound designer.
- Costi elevatissimi dei kit di sviluppo (es. 20 milioni di lire) limitavano l'accesso.

16.10.2 Il Voice Stealing

- Con solo 4 voci disponibili, se un effetto sonoro (es. distruzione di un blocco) usava la stessa voce di uno strumento musicale (es. hi-hat), la musica perdeva momentaneamente quella parte.
- Esempio classico: in alcuni giochi, colpendo un mattone, spariva l'hi-hat o addirittura la linea melodica, per poi riapparire subito dopo.
- **Voce stealing** = `rubare` una voce: dare priorità a un suono interrompendone un altro.
- Questa tecnica è ancora attuale: oggi si limita il numero di istanze contemporanee (es. massimo 3 passi) per evitare di saturare la polifonia (es. 64 voci su console), zittendo i suoni più vecchi per far posto ai nuovi, sfruttando la psicoacustica.

16.10.3 La nascita del Tracker

- La composizione musicale richiedeva di scrivere `partiture` in esadecimale (byte, registri, pattern), un lavoro complesso e macchinoso.
- **Karsten Obarski** crea su Commodore Amiga **Ultimate Soundtracker**: un editor con tracce a scorrimento verticale (pattern di 64/128 righe) dove inserire note, ottave, strumenti e modificatori (volume, pitch bend, vibrato).
- Il tracker semplifica la scrittura musicale, esportando poi i dati nel formato richiesto dal chip sonoro.
- Il tracker è considerato il `padre` dei sequencer e delle moderne DAW (Digital Audio Workstation).
- Esempio mostrato con **MilkyTracker** (software free): interfaccia a pattern verticali, riproduzione in tempo reale.

16.10.4 Creazione dei suoni

- Per generare timbri ed effetti, si usavano editor che permettevano di manipolare in tempo reale i parametri del chip sonoro (duty cycle, sweep, inviluppo).
- Esempio mostrato con l'emulatore **No$GBA**: modificando i registri del Game Boy Advance si poteva ascoltare immediatamente il risultato e salvare i `preset` per le varie voci musicali o per gli effetti sonori (es. bonus pick-up).

16.11 Il supporto al gameplay tramite l'audio (conclusione)

16.11.1 Sfruttamento della musica da parte del giocatore

- **Esempio in Splinter Cell**: un giocatore, invece di avanzare furtivamente, apre di proposito una porta, facendo scattare la musica di allarme.
- Attira la guardia, la neutralizza e la porta fuori dalla zona di allerta.
- Quando la musica torna calma, deduce che non ci sono altre guardie allertate (se ce ne fossero, la musica resterebbe in tensione).
- Il giocatore `usa` la musica come strumento per testare il livello di minaccia e poi attraversare la zona senza rischi.
- Questo comportamento non era stato previsto esplicitamente dagli sviluppatori, ma dimostra come l'audio possa diventare una meccanica di gameplay emergente.

16.11.2 Il passaggio al digitale e il CD (anni '90)

- Arrivano le **schede audio con wavetable**: piccole memorie RAM in cui memorizzare campioni sonori (inizialmente 64 KB, poi 128 KB, fino a qualche MB).
- Si afferma lo standard **MIDI** per pilotare i campioni: un file MIDI contiene le note, mentre i suoni degli strumenti sono forniti da campioni (chitarra, batteria, archi, ecc.).
- I giochi iniziano a usare **musica e suoni campionati** anziché puramente sintetici: ad esempio, le colonne sonore di Nintendo 64 (Zelda, Banjo-Kazooie) sono MIDI che suonano sample, con sonorità più realistiche ma ancora dal timbro `finto` tipico dell'epoca.
- Nasce la figura del **Game Audio Engineer**: un professionista che fa da ponte tra lo studio di produzione creativa e il team di programmazione, gestendo l'integrazione del MIDI e dei sample.

- **PlayStation** (1995) introduce il **CD come supporto**: permette di includere tracce audio di qualità CD (orchestra, band), superando i limiti dei sample e offrendo audio stereo di alto livello.
- Anche con il CD, alcune console mantengono capacità di sintesi (chip sonoro) per alcuni suoni, creando un **ibrido** tra sintesi e digitale.

16.11.3 Il presente (2000–oggi)

- L'audio diventa completamente **digitale**: campionamento a 44 kHz, 16 bit (o superiore).
- Aumentano RAM e processori dedicati, permettendo:
  - **Audio spaziale** (3D).
  - **Effetti in tempo reale**: riverbero, equalizzazione, distorsione applicati dinamicamente.
  - **Audio adattivo**: transizioni tra strati musicali, randomizzazione, sincronizzazione con eventi di gioco.
- La qualità audio raggiunge quella cinematografica, allineandosi all'evoluzione grafica.
- Aumentano i budget e i team audio diventano più strutturati.

16.11.4 I ruoli nel team audio

- **Audio Director**: responsabile della visione sonora del gioco. Definisce stile creativo e approccio tecnico, coordina il team audio, collabora con game designer e programmatori, sceglie compositori e sound designer. Spesso proviene dal sound design.
- **Sound Designer**: crea effetti sonori, ambienti, suoni di interfaccia. Lavora con registrazioni sul campo, foley, librerie audio. Deve tenere conto delle limitazioni tecniche (lunghezza suoni, memoria).
- **Technical Sound Designer**: figura con competenze più tecniche, mette le mani nell'engine, crea script, ottimizza l'integrazione.
- **Compositore**: scrive le musiche. Può limitarsi all'aspetto creativo o, come alcuni, creare direttamente eventi interattivi in middleware (es. FMOD) fornendo non un file lineare ma la logica interattiva.
- In team piccoli o produzioni indie, spesso i ruoli si sovrappongono: una sola persona può fare da sound designer, compositore e integratore.

16.11.5 Limitazioni e buone pratiche nella produzione audio

- **Lunghezza dei suoni**: per risparmiare RAM e non sovraccaricare il sistema di streaming, i suoni, specialmente i loop ambientali, devono essere corti.
  - Standard: loop di 2-5 secondi per suoni semplici (ronzii, macchinari), massimo 10 secondi per suoni più complessi.
  - In middleware come FMOD, i suoni oltre i 5 secondi vengono automaticamente messi in streaming, rischiando di saturare la banda di streaming.
- **Code e silenzi**: eliminare le code inutili (es. la risonanza lunga di un colpo di martello) perché in un mix denso non sono percepibili e occupano memoria.
- **Esempio pratico**: un rintocco di campana di 27 secondi con coda lunghissima causava blocchi; tagliato a 6 secondi ha liberato circa 50-60 MB di RAM e risolto il problema di performance.
- **Suoni asciutti e riverbero**: tutti gli effetti sonori vanno creati **asciutti** (senza riverbero), perché il riverbero viene applicato in tempo reale dall'engine in base all'ambiente (reverb zone).
- Questo approccio permette di avere un unico suono `secco` e lasciare che il motore lo contestualizzi, riducendo il numero di varianti da produrre e risparmiando memoria.

16.12 Storia dell'audio nei videogiochi (continuazione)

16.12.1 Il passaggio al digitale e il CD (anni '90)

- **Schede audio con wavetable**: memorie RAM in cui memorizzare campioni sonori (inizialmente 64 KB, poi 128 KB, fino a qualche MB). Consentono di avere suoni campionati invece che puramente sintetici.
- **MIDI**: standard di computer music che permette di separare partitura (note) e suoni (campioni). Un brano MIDI pilota i campioni caricati nella wavetable.
- Esempi: le colonne sonore di Nintendo 64 (*Zelda*, *Banjo-Kazooie*) sono MIDI che suonano sample, ottenendo sonorità più realistiche ma ancora dal timbro `finto` tipico dell'epoca.
- Nasce il **Game Audio Engineer**: figura-ponte tra lo studio di produzione creativa (compositori, sound designer) e il team di programmazione, gestisce l'integrazione di MIDI e sample.
- **PlayStation** (1995) introduce il **CD come supporto**: permette di includere tracce audio di qualità CD (orchestrali, band), superando i limiti dei sample. Ad esempio, musica orchestrale per i menu, brani di Iron Maiden.
- Nonostante il CD, alcune console mantengono il chip sonoro per suoni sintetici, creando un **ibrido** tra sintesi e digitale.
- **Evoluzione della programmazione**: dal linguaggio macchina si passa a C++ e ai primi script, semplificando l'integrazione audio. I sound designer potevano scrivere piccole routine con funzioni come `play music`, `play sound`, `stop sound`, anche se il processo di build e test restava lento.

16.12.2 Il presente (2000–oggi)

- **Audio completamente digitale**: campionamento a 44 kHz, 16 bit (o superiore).
- Aumento di RAM e processori dedicati all'audio permette:
  - **Audio spaziale** (3D).
  - **Effetti in tempo reale**: riverbero, equalizzazione, distorsione, delay.
  - **Audio adattivo**: transizioni tra strati musicali, randomizzazione, sincronizzazione con eventi di gioco.
- La qualità audio raggiunge quella cinematografica, allineandosi all'evoluzione grafica.
- Aumentano i budget e i team audio diventano più strutturati.

16.13 I ruoli nel team audio

16.13.1 Audio Director

- Responsabile della visione sonora complessiva del gioco.
- Definisce stile creativo e approccio tecnico.
- Coordina il team audio, collabora con game designer e programmatori.
- Sceglie compositori e sound designer in base alle esigenze del progetto.
- Spesso proviene dal sound design; può mettere le mani nell'integrazione, ma è colui che decide le strategie.

16.13.2 Sound Designer

- Crea effetti sonori, ambienti, suoni di interfaccia, armi, passi, ecc.
- Lavora con registrazioni sul campo (field recording), foley, librerie audio.
- Può specializzarsi come **Technical Sound Designer** se ha competenze più tecniche e lavora direttamente nell'engine.
- **Buone pratiche specifiche per il gaming**:
  - **Suoni asciutti**: vanno creati senza riverbero, perché il riverbero viene applicato in tempo reale dall'engine in base all'ambiente (reverb zone). Questo evita di avere più varianti dello stesso suono e risparmia memoria.
  - **Nessun silenzio in testa**: il suono deve attaccare subito (transiente iniziale). Se un passo ha del silenzio prima del colpo, si avverte un ritardo innaturale. L'engine ha già strumenti per aggiungere delay random se necessario.
  - **Lunghezza contenuta**: loop di suoni ambientali devono essere corti (2-5 sec per suoni semplici, max 10 sec). In middleware come FMOD, i suoni oltre i 5 sec vengono messi in streaming automaticamente, rischiando di saturare la banda. Code troppo lunghe vanno tagliate (es. rintocco di campana di 27 sec ridotto a 6 sec per liberare 50-60 MB di RAM e far ripartire il gioco).
  - **Qualità compressa**: non sempre è possibile usare 48 kHz/24 bit; spesso si deve scendere a compromessi per memoria e performance.
- **Strumentazione**: field recording richiede registratori portatili, microfoni direzionali e ambientali; foley richiede studi attrezzati con superfici e oggetti vari.

16.13.3 Compositore

- Scrive le musiche del gioco. Può limitarsi all'aspetto creativo o anche creare eventi interattivi in middleware (es. FMOD).
- **Peculiarità del comporre per videogiochi**:
  - **Loop**: il 90% delle musiche è in loop. La durata tipica è 1–2 minuti; raramente si superano i 3 minuti per non appesantire lo streaming e lo spazio occupato. Il loop deve essere vario ma omogeneo, in modo che non si percepisca la ripetizione.
  - **Musica interattiva**: non si compone un brano lineare ma un sistema che si adatta a diverse situazioni (esplorazione, combattimento, tensione). Richiede transizioni fluide tra strati o sezioni, spesso sfruttando i dati forniti dal motore di gioco.
  - **Micro-loop e randomizzazione**: per variare la musica si possono creare segmenti brevi (es. linea di basso di 30 sec) che vengono selezionati casualmente ad ogni ciclo.
  - **Stili multipli**: il compositore deve spesso padroneggiare generi molto diversi (es. foresta amazzonica, piramidi, ghiacciai) e riarrangiare lo stesso tema in stili differenti, il tutto in tempi ristretti.
  - **Immedesimazione**: molti compositori si immergono nel mood del livello ascoltando in loop il genere di riferimento per interiorizzarne schemi e pattern (es. il compositore di *Rayman M* si creava atmosfere a tema con candele e cappelli da pirata).

16.13.4 Dialogues Director e doppiaggio

- Figura che supervisiona il doppiaggio: organizza script, testi, brief per i doppiatori (personaggio, età, stile di voce), gestisce le consegne in più lingue.
- Il doppiaggio è quasi un dipartimento a sé: coinvolge scrittori, doppiatori, direttori, fonici, editor (pulizia e taglio dei file), post-produzione (livellamento).

- **Intelligenza Artificiale nel doppiaggio**:
  - I test mostrano risultati buoni, ma ancora carenti nell'aspetto recitativo.
  - Per team indie a budget zero può essere una soluzione, ma ci sono resistenze da parte dei doppiatori professionisti che temono l'uso non autorizzato della propria voce.
  - Uso ibrido ipotetico: clonare la voce per generare piccole correzioni o varianti di una frase, con il consenso del doppiatore, potrebbe ottimizzare tempi e costi.
  - Alcuni doppiatori hanno già messo a disposizione la propria voce clonata su piattaforme come Eleven Labs, accettando il cambiamento del mercato.

16.13.5 Audio Programmer

- Figura tecnica che realizza o integra il **sound engine** (o middleware come FMOD, Wwise).
- **Compiti principali**:
  - Fornire linee guida e limitazioni tecniche ai sound designer (cosa è possibile sulla piattaforma).
  - Sviluppare tool interni per l'audio.
  - Gestire la comunicazione tra audio team e programmatori: deve conoscere la terminologia audio per comprendere le richieste dei creativi e tradurle in specifiche tecniche.
  - Essere consapevole che le soluzioni tecniche hanno ricadute sulla produzione audio (es. aspettare la fine di una battuta musicale prima di cambiare traccia).
  - Definire formati di consegna chiari per evitare che i contenuti arrivino in formati errati e non più modificabili (es. mancanza di stem per la musica interattiva).
- Un errore nella comunicazione delle specifiche (es. non richiedere gli stem al compositore) può compromettere la possibilità di avere musica adattiva.

16.14 Il processo di sviluppo e il ruolo dell'audio

16.14.1 L'importanza dell'implementazione

- **Citazione di Alexander Brandon** (audio director storico): `I contenuti audio (musica, effetti, doppiaggio) sono solo la metà del lavoro; l'altra metà è come li implementi.` Anche con i migliori contenuti, un'implementazione scadente produce un audio brutto.
- Per questo è cruciale capire come l'audio si inserisce nelle diverse fasi di sviluppo di un videogioco.

16.14.2 Pre-produzione

- **Team di sviluppo**: si forma un *core team* (game designer, programmatore, artista, a volte producer/creative director). Si definisce il game design e si realizza un **prototipo** per valutarne la bontà e la vendibilità.
- Nei grandi publisher (es. Ubisoft) si finanziano più prototipi in parallelo, scartando la maggior parte; nei team indie il prototipo serve per cercare un publisher o validare l'idea sul mercato.
- **Ruolo dell'audio in pre-produzione** (fondamentale e spesso trascurato):
  1. **Audio Design**: definire regole e categorizzazioni sonore in parallelo al game design. Esempi: tipologie di passi (pesante, medio, leggero), superfici (neve, erba, ghiaia), sonorità delle armi, stili musicali.
  2. **Scelta delle funzionalità tecniche**: decidere se servono musica interattiva, riverbero in tempo reale, multistreaming, e su questa base scegliere la tecnologia (engine nativo vs middleware come Wwise o FMOD).

3. **Benchmark e test**: approfittare del prototipo per provare diverse soluzioni (es. creare due cartelle, una per Wwise e una per FMOD) e confrontarle.
4. **Lista degli asset**: iniziare a quantificare il fabbisogno (numero di passi, superfici, suoni di armi, musiche per menu, livelli, credits) per **definire il budget** da presentare al publisher.

- La pre-produzione può durare poche settimane o alcuni mesi; è il momento in cui si pongono le basi per non trovarsi impreparati in produzione.

16.14.3 Produzione

- Il team si allarga: arrivano level designer, texture artist, animatori, environment artist, ecc. Si creano i livelli, le animazioni, gli ambienti.

- **Attività del team audio**:
  - **Creazione dei contenuti**: compositore produce musiche, sound designer effetti sonori, si realizza il doppiaggio.
  - **Integrazione continua**: i contenuti vengono inseriti nel gioco, testati e raffinati iterativamente.
  - **Modifiche richieste dall'integrazione**:
    * Esempio musica: può servire un pezzo di transizione tra due aree non previsto inizialmente.
    * Esempio sound design (caso reale): il suono dell'armatura, inizialmente fornito come unico sample, si rivela innaturale perché il personaggio ha movimenti a velocità variabile. Si chiede al sound designer di scomporlo in **tre layer separati** (metallo, catenelle, pelle/tessuto). In base alla velocità del movimento, il motore di gioco mixa i tre layer in proporzioni diverse (es. movimento lento: più pelle; movimento brusco: più metallo).
    * Esempio dialoghi: possono mancare frasi, richiedere un mixaggio migliore o l'implementazione del *sidechain* (abbassare la musica quando parte un dialogo).

- Questa fase è cruciale perché è dove si concretizza la qualità percepita: le scelte di design aiutano, ma le soluzioni finali si trovano provando e adattando.

- La produzione si conclude con la **versione Beta**: il gioco deve contenere tutte le feature, anche se alcune potrebbero essere ancora buggate.

16.14.4 Post-produzione: Beta, Candidate Master, Gold Master

- Dopo la Beta, inizia un periodo di **testing** (1-2 mesi) per sistemare i bug e arrivare alla **Candidate Master** (candidata a diventare la versione finale).

- La Candidate Master viene inviata ai produttori di console (Sony, Nintendo, Microsoft) per la certificazione; se non conforme (es. tempi di caricamento, nomenclature errate), viene rimandata indietro. Quando approvata, diventa **Gold Master** (pronta per il rilascio).

- **Attività audio nella post-produzione**:

1. **Audio Mixing**: mixaggio in tempo reale del gioco. Si collega il middleware (FMOD/Wwise) alla sessione di gioco e si regolano i volumi mentre si gioca, sentendo il risultato immediato. Può durare da pochi giorni a settimane e coinvolgere più persone su giochi grandi.
2. **Audio Test**: test specifici condotti dal team audio (spesso in remoto o in orari tranquilli). Si simula una sessione di gioco reale: menu, opzioni, cinematiche iniziali, walkthrough completo, senza usare cheat o saltare parti. L'obiettivo è verificare che tutto l'audio funzioni correttamente.

16.15 Categorie di suoni nel videogioco

16.15.1 Musica

- **Source Music** (musica diegetica): prodotta da una sorgente presente nel gioco (es. musicista di strada in Assassin's Creed). Viene integrata come un effetto sonoro: il volume cala con la distanza, è localizzata spazialmente. Ha scopo di caratterizzazione ambientale.
- **Musica lineare**: usata in contesti non interattivi come **menu**, **credits**, **cutscene**. È composta come un brano tradizionale (con intro, sviluppo, fine), senza necessità di loop o adattamenti. Può anche essere una canzone vera e propria.
- **Musica non lineare / interattiva / adattiva**: è la `padrona` del videogioco. Si adatta in tempo reale a ciò che accade nel gioco, cambiando strati, intensità, o sezioni in base al **mood** o agli eventi. Esempio mostrato: un sistema musicale basato sul mood che reagisce alle azioni del giocatore, garantendo transizioni fluide e coerenti con lo stato emotivo della scena.

16.15.2 Effetti sonori (SFX)

- Funzione principale **descrittiva**: comunicano cosa sta succedendo (spari, passi, porte, menu click).
- Prodotti dal sound designer e implementati per reagire agli eventi di gioco.

16.15.3 Dialoghi (Voices)

- Funzione **narrativa** e di **caratterizzazione**: trasmettono la storia, le personalità, le istruzioni.
- Gestiti dal Dialogues Director e integrati con attenzione al mix (es. sidechain).

16.15.4 Ambience (Ambienti)

- Funzione **descrittiva/narrativa**: creano l'atmosfera, il senso di luogo (vento, uccelli, riverbero, suoni di una città).
- Anche se sono registrati come gli SFX, vengono spesso implementati in modo diverso: loop continui, sistemi generativi, riverbero in tempo reale.

16.16 Musica non lineare (adattiva/interattiva)

16.16.1 Tipologie di reattività

- La musica adattiva reagisce principalmente a due fattori:
  1. **Action-driven** – basata su eventi discreti: inizia il combattimento → parte la musica di battaglia; finisce il combattimento → stop. Esempio già visto: la *finishing move* in cui il nemico è già morto e la musica ha il tempo di concludere la frase.
  2. **Mood-driven** – basata su stati continui o a soglia: vita del giocatore, tensione, vicinanza all'obiettivo, grado di copertura, distanza e orientamento dei nemici. I valori possono essere dinamici o suddivisi in step (es. 0, 1, 2, 3).

16.16.2 Esempio: Mod di Far Cry (musica mood-driven)

- Un utente ha creato un sistema che raccoglie diversi parametri di gioco (copertura nell'erba, distanza dei nemici, direzione dello sguardo dei nemici, rumore prodotto) e li condensa in un **cursore bianco** che rappresenta il livello di minaccia/tensione.
- Il cursore si muove in un range continuo; al superamento di determinate soglie cambia lo stato musicale:

  - Livello < 2: **Neutral** (verde).
  - Livello $2 \rightarrow 5$: **Suspense** (giallo).
  - Livello > 5: **Threat** (rosso).
- Il compositore ha creato diversi `mattoncini` (clip) per ogni stato e delle clip di transizione tra uno stato e l'altro.
- Durante l'esecuzione, a metà di ogni clip il sistema sceglie la clip successiva in base al valore attuale del cursore. Se lo stato è cambiato, parte una transizione; altrimenti prosegue con un'altra clip dello stesso stato.
- Il risultato è una **colonna sonora generata dinamicamente**: se il giocatore rigiocasse la stessa sezione con comportamenti diversi, ascolterebbe una soundtrack diversa, composta dagli stessi mattoncini ma concatenati diversamente.
- Questo meccanismo è un esempio di **interattività orizzontale** (passaggio tra clip successive). L'interattività **verticale** consiste invece nel mixare livelli di volume di tracce sovrapposte (es. alzare un layer di archi quando aumenta la tensione).

16.17 Effetti sonori (SFX)

- **Definizione**: suoni associati a tutto ciò che *succede* nel gioco. Funzione descrittiva.
- **Metodi di implementazione**:
  - **Animazioni dei personaggi**: i suoni sono agganciati a specifici fotogrammi dell'animazione (es. passo, colpo di spada). Quando l'animazione raggiunge quel frame, il motore lancia il suono.
  - **Oggetti di gioco**: associati a stati e script (es. oggetto distrutto, acceso/spento, esploso). Il codice chiama l'evento sonoro corrispondente.
  - **Eventi di gioco**: spesso attivati tramite *trigger* volumetrici. Il giocatore attraversa un volume invisibile e scatena un suono (es. allarme, dialogo ambientale).
  - **Interfaccia utente (UI)**: suoni collegati agli input del giocatore (click, hover, notifiche).
- **Produzione**: registrazioni sul campo, foley, librerie audio. Stesse buone pratiche già viste (asciutti, senza silenzio in testa, code ridotte).

16.18 Suoni d'ambiente (Ambience)

- **Definizione**: suoni di tutto ciò che *non succede*, il contesto sonoro costante. Se il giocatore sta fermo, sente l'ambiente.
- **Differenza con gli SFX**: anche se prodotti dal sound designer allo stesso modo, vengono implementati diversamente perché devono creare un tappeto sonoro persistente.
- **Approccio tradizionale**: loop molto lunghi (1 minuto o più) di registrazioni ambientali (traffico, giungla, stanza).
- **Approccio moderno**:
  - Un **loop di fondo** molto omogeneo e corto (10–15 secondi) che fornisce il `letto` sonoro dell'ambiente (es. rumore generico di città, vento, fruscio di foresta).
  - Su questo loop vengono lanciati continuamente **suoni one-shot randomici** (clacson, frenate, accelerate, cinguettii, versi animali) con volumi e pitch leggermente variati.
  - Gli strumenti utilizzabili sono middleware come FMOD e Wwise, ma anche `Metasounds` in Unreal Engine o script in Unity.
  - Vantaggio: l'ambiente risulta **dinamico e mai ripetitivo**, anche se il giocatore resta fermo a lungo.

16.19 Dialoghi

16.19.1 Tipologie

- **Dialoghi lineari**: durante cutscene o con un narratore. C'è un testo fisso, registrato in studio, che viene riprodotto dall'inizio alla fine.
- **Dialoghi non lineari**: tipici dei sistemi a scelta multipla o delle telecronache sportive. Una macchina a stati seleziona la frase successiva in base alle scelte del giocatore o agli eventi di gioco (domanda-risposta, rami di conversazione).

16.19.2 Criticità e gestione

- **Localizzazione**: gestire più lingue (testi e audio). I middleware moderni aiutano a organizzare automaticamente le risorse per lingua, ma la produzione deve comunque coordinare script, traduttori e doppiatori per ogni idioma.
- **Numero elevato di file**: un sistema di dialoghi ramificati può generare centinaia o migliaia di clip audio, da organizzare e nominare con rigore.
- **Uso di placeholder**: i testi e i dialoghi vengono spesso rivisti e aggiunti durante lo sviluppo. Registrare subito i doppiatori professionisti è rischioso: se cambiano le frasi, bisogna riconvocarli (con costi e problemi di disponibilità).
  - **Metodo tradizionale**: far registrare le bozze a colleghi o amici, in modo grezzo, per riempire il gioco e testare il funzionamento dei dialoghi. Alla fine del progetto, quando i testi sono congelati, si fanno registrare i doppiatori veri e si sostituiscono i file.
  - **Oggi con IA**: si possono generare tutti i dialoghi temporanei con strumenti come Eleven Labs, inserirli nel gioco, e solo a testo definitivo sostituirli con le registrazioni reali. Questo permette di avere placeholder di buona qualità senza impegnare attori.
- La registrazione definitiva va fatta `il più tardi possibile, ma non troppo`: abbastanza avanti per avere i testi finali, ma non così a ridosso della consegna da non avere tempo di integrare e mixare.

16.20 Domande e risposte

16.20.1 Sincronizzazione labiale e adattamento lingue

- **Problema**: in giochi con doppiaggio in più lingue, le animazioni facciali (apertura della bocca) sono spesso realizzate su una sola lingua (es. inglese). Nelle altre lingue, la durata e la sincronia labiale possono non corrispondere.
- Esistono **strumenti software** (anche da oltre dieci anni) che analizzano l'audio e adattano automaticamente l'apertura della bocca del personaggio.
- Dal lato doppiaggio, si usa la tecnica del **time constraint**: il doppiatore deve rispettare la durata della battuta originale (es. inglese 3 secondi → italiano 3 secondi). È una limitazione impegnativa, ma necessaria per evitare sfasamenti tra audio e video.

16.20.2 Interazione in tempo reale (esempio televisione e passi)

- **Spiegazione del sistema di rilevamento**:
  - Dal personaggio (o dalla telecamera) parte un **raycast**, un raggio invisibile che colpisce gli oggetti nella scena.

  - Quando il giocatore spara, il codice verifica quale oggetto era colpito dal raggio in quel momento. Se è la televisione, viene eseguito lo script `distruggi televisione` che cambia lo stato dell'oggetto e attiva il suono corrispondente (TV che scoppia, rumore bianco).
  - Per i passi, il personaggio emette un raggio verso il basso che a intervalli regolari (o in corrispondenza del frame dell'animazione) legge il materiale della superficie sottostante (erba, cemento, legno). In base al parametro restituito, viene riprodotto il suono del passo corretto.

- Tutto avviene tramite codice che intercetta le collisioni (invisibili) e restituisce l'informazione necessaria per attivare l'audio giusto in tempo reale.

16.21 Conclusioni e contatti

- Il docente ringrazia e si dichiara disponibile a rimanere in contatto tramite **LinkedIn** (Davide Pensato).
- **Eventi di settore consigliati**:
  - **First Playable** (Firenze) – evento business, richiede biglietto.
  - **Svilupparty** (Bologna) – gratuito, si tiene a metà maggio.
  - **Milan Games Week** (Milano) – zona *Indie Dungeon* con team indipendenti.
  - **Cotoletta 8-bit** (Milano) – evento annuale cresciuto molto.
  - **IGDA** (International Game Development Association) – capitoli locali in varie città italiane (Milano-Torino, Sardegna, Campania, Bologna) con eventi periodici.
  - **Milano Indies** (Cologno Monzese, studi Mediaset) – ogni due mesi; di recente sono nati anche *Firenze Indies* e *Bologna Indies*.
- Questi eventi sono utili per incontrare team indie e professionisti del settore, anche di aziende più grandi.
- Possibilità futura di un workshop di approfondimento.